微网与外部电网电力调控策略

摘　要

本文研究含光伏和储能的居民微网购电调度问题。随着预测信息和电价信息逐步揭示，购电计划的可调整范围也随时间变化。为保持四个问题的建模口径一致，本文以 10 min 为时间步，在同一能量平衡与储能状态模型上依次处理确定性调度、日前购电锁定、日内预测更新和动态电价四种情形。计算中先将功率换算为区间电量，并将 144 个运行区间与 145 个 SOC 边界状态分开编号。

针对问题一，在负荷、光伏和分时电价均已知的条件下建立单日线性规划模型，联合确定外部购电、储能充放电和 SOC。最优方案的正常购电量为 59 482.699 kWh，购电费用为 35 126.949 元，且 $E_0 = E_{144} = 6000\,\mathrm{kWh}$。从调度轨迹看，储能主要在低价时段充电，在高价及负荷高峰时段放电。

针对问题二，0:00 形成的正常购电计划在当日内保持不变，后续运行只能利用逐步获得的实际信息进行储能补救，供电仍不足时再进行紧急购电。本文采用严格因果的预测与历史残差构造非预见场景，并以 1 月作为预测和 SOC 暖启动期、2 月 1 日至 12 月 31 日作为统一统计期。334 个统计日内，正常购电量为 20 789.350 MWh，紧急购电量为 1 031.043 MWh，正常购电与紧急购电合计运行费用为 1 668.727 万元。

针对问题三，在问题二的购电锁定规则上增加 6:00、12:00 和 18:00 三次日内预测更新；已执行区间不再改动，只重新安排尚未执行的购电与储能计划。调整费用按最终有效计划相对 0:00 原计划的净变化一次性结算。统计期内最终正常购电量为 20 335.635 MWh，正常购电结算费用较问题二下降 1.121%，但紧急购电费用增加 6.267%，总运行费用反而增加 0.609%。这表明日内信息更新改变了常规采购结构，但在当前预测误差和补救规则下，并未降低总体运行成本与尾部缺电风险。

针对问题四，将固定分时电价进一步替换为动态电价，并保持未来真实价格在决策时不可知。Q4-2 采用日初一次决策，Q4-3 允许在日内更新未来未执行计划。Q4-3 的正常购电结算费用较 Q4-2 下降 1.027%，但紧急购电费用增加 4.135%，使总运行费用增加 0.165%。由此可见，在本题给定的信息条件下，降低常规采购成本与控制紧急购电风险并非同一目标，需要同时以总运行费用和可靠性指标评价。

关键词：微网调度；线性规划；多阶段非预见决策；滚动预测；动态电价；储能

一、 问题重述

1.1 问题背景

居民微网由用户负荷、光伏、储能和外部电网共同供能。光伏出力具有明显的日内波动，储能则通过跨时段充放电把不同时间段的供需和电价联系起来，因此购电计划不能逐时独立决定，而需要同时考虑能量平衡、SOC 演化与价格信号。随着可再生能源占比提高，预测误差又使微网调度由确定性优化扩展到随机优化、鲁棒优化和滚动时域控制等不确定性决策框架 [1, 4]。住宅储能研究表明，将短期预测与滚动优化结合，可以根据负荷、光伏和电价的新信息修正后续计划 [5]；面向微网的不确定性调度研究则进一步关注经济性与运行风险之间的权衡 [6, 7]。本文据题目给定的信息时序，将四个问题统一到同一储能能量平衡模型下，并逐步增加预测误差、日内更新和动态电价条件。

1.2 问题要求

根据题目条件，需要完成以下四项工作：

1. 在负荷、光伏和分时电价已知时，求得满足储能运行约束的最低成本购电方案；
2. 在 0:00 正常购电计划锁定后，模拟预测误差下的实际运行，并统计紧急购电等风险指标；
3. 利用 6:00、12:00、18:00 到达的新预测修正尚未执行的计划，同时按题目规则计算调整费用；
4. 在动态电价条件下分别考察日初一次决策和日内滚动修正两种策略，比较其购电成本与运行风险。

二、 问题分析

2.1 问题一分析

问题一给出了全天负荷、光伏预测和分时电价，可以作为确定性单日调度处理。若没有储能，各时段购电只需覆盖当时的净负荷；加入储能后，低价时段的充电会改变后续可放电量，各时段因而产生跨时段联系。题目中的购电费用、能量平衡、SOC 转移以及容量和功率限制均可写成线性关系，所以采用连续线性规划求解，并把该模型作为后续各问共用的物理基础。

2.2 问题二分析

问题二首先改变的是购电计划的**可调整性**：0:00 一旦确定全天计划，正常购电量在当日内不再修改。这样，实际负荷和光伏与预测之间的偏差不能靠重新购买正常电消除，只能由储能和紧急购电吸收。建模时必须保证两点：一是日前购电量在所有可能场景中相同；二是在场景尚未分叉以前，储能等运行决策也应相同。基于这一要求，采用多阶段场景树表达预测误差，并施加非预见约束；其思想与不确定性下的风险调整调度和鲁棒日内调度研究保持一致 [6, 7]。实际仿真仍按时间顺序推进，在每个区间只使用

已经获得的信息决定储能动作，随后再用紧急购电或弃光闭合真实能量平衡。

2.3 问题三分析

问题三没有改变微网的物理结构，而是增加了日内信息。0:00 的全天计划仍是基准；6:00、12:00 和 18:00 获得新的光伏预测后，只允许调整尚未执行的区间，已经发生的购电和储能动作保持不变。因此，滚动模型需要同时记录当前 SOC、冻结历史决策，并重新求解剩余时段；这一处理直接借鉴了滚动优化与 MPC 在住宅/微电网调度中的时序重求解逻辑 [4, 5]。由于修改购电计划会产生额外费用，结果评价还必须把原计划费用、计划调整费用和最终正常购电结算费用分开计算。

2.4 问题四分析

问题四将固定分时价改为动态电价，价格也成为需要预测的量。0:00 决策只能依据此前已经观察到的价格；日内更新时可以利用当天已经发生的价格修正未来预测，但尚未发生的真实价格不能进入当前优化。Q4-2 因而保持问题二的日初锁定方式，Q4-3 保持问题三的日内更新方式。两者共用相同的物理约束，便于把差异集中到“价格信息何时到达、购电计划何时能够修改”这两个方面。

2.5 总体建模思路

上述四问沿着“确定性调度–预测误差–日内信息更新–动态电价”的顺序展开。图1给出数据、模型和输出之间的关系；后续各问在同一储能模型上逐步增加新的信息条件，而不重复改变基础物理约束。

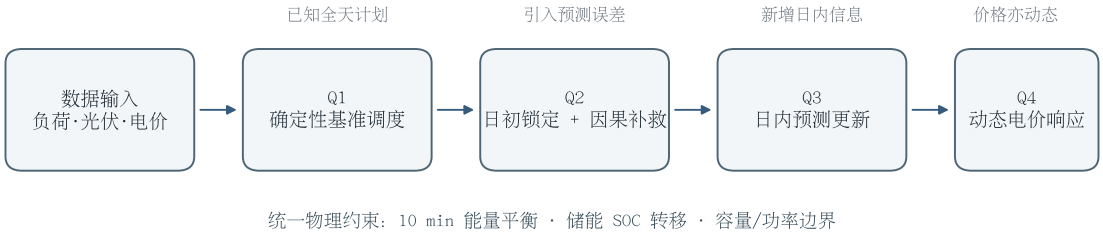


图 1 四个问题的建模递进关系

三、 模型假设

为使模型与题目给出的数据结构和实际调度过程一致，作如下假设。

1. **10 min 区间内功率取给定值或预测平均值**。因此区间电量由功率乘以 $\Delta t = 1/6\,\mathrm{h}$ 得到。这一处理与附件数据的 10 min 分辨率一致。
2. **储能参数在研究期内保持不变**。充、放电效率均为 0.9，可用电量范围为 1200–10800 kWh，最大充放电功率为 5000 kW。该假设用于刻画题目给定储能设备的基本运行边界；未额外引入电池寿命退化成本。
3. **不向外部电网反向售电**。当光伏和既定购电超过负荷及可充电能力时，采用弃光量

q^{curt} 或外部计划盈余量 q^{sur} 闭合能量平衡，而不把其作为售电收益。

4. **问题二至问题四严格遵循信息可用性**。任一决策时刻只能使用当前之前已经揭示的信息和由历史数据构造的预测，未来真实负荷、光伏和动态电价均不可提前读取。该假设对应实际日前/日内调度的因果性要求。
5. **正式主模型采用连续线性规划**。代码中虽然保留了充放电互斥的可选开关，但正式结果未启用额外二进制变量，因此论文不将其描述为混合整数规划。
6. **跨日 SOC 连续**。问题二至问题四不在每天开始时重置储能状态，而采用前一日结束状态作为下一日初始状态；1 月用于历史和 SOC 暖启动，正式统计期为 2 月 1 日至 12 月 31 日。

四、 符号与变量说明

表 1 主要符号与变量

符号	含义	单位
$k = 0, \ldots, 143$	10 min 物理区间索引	–
$j = 0, \ldots, 144$	储能边界状态索引	–
Δt	单个物理区间长度，$\Delta t = 1/6$	h
L_k	第 k 区间负荷电量	kWh
G_k^{pv}	第 k 区间可用光伏电量	kWh
c_k	第 k 区间正常购电电价	元/kWh
q_k^{buy}	正常购电量	kWh
$q_k^{\mathrm{ch}}, q_k^{\mathrm{dis}}$	储能充电量、放电量	kWh
q_k^{emg}	紧急购电量	kWh
q_k^{curt}	光伏弃电量	kWh
q_k^{sur}	既定购电形成的外部计划盈余量	kWh
E_j	第 j 个边界时刻储能电量	kWh
η_c, η_d	充、放电效率，均为 0.9	–
m_{emg}	紧急购电价格倍率，基准值为 5	–
E_{cap}	储能额定容量，基准值为 12000	kWh
$E_{\min}, E_{\max}$	储能下、上界，分别为 1200, 10800	kWh
$P_{\max}$	最大充/放电功率，5000	kW
ρ_n	场景树节点 n 的概率权重	–
G_k^0, G_k^F	Q3/Q4-3 中 0:00 原计划与最终有效正常购电量	kWh
dG_k^+, dG_k^-	最终计划相对原计划的上调量、下调量	kWh
C_{plan}	按 0:00 原计划计算的正常购电费用	元
C_{adj}	购电计划调整费用	元
C_{reg}	调整后正常购电结算费用	元

五、 数据处理与特征分析

5.1 时间离散与右端点映射

一天划分为 144 个 10 min 物理区间，记

$$K = \{0, 1, \ldots, 143\}, \qquad \Delta t = \frac{1}{6}\ \text{h}.$$

附件 1、附件 2、附件 4 的时间标签采用区间右端点：表头 “0:10” 对应物理区间 $k = 0$，即 $[0{:}00, 0{:}10)$；最后的 “0:00+1” 对应 $k = 143$，即 $[23{:}50, 24{:}00)$。程序读取这些附件时按照 144 个业务槽位的列顺序建立 $k = 0, \ldots, 143$ 的物理索引，而不根据字符串时间重新反推区间。

对功率变量 P_k，统一转换为区间电量

$$q_k = P_k \Delta t = \frac{P_k}{6}. \tag{1}$$

因此，负荷、光伏、购电和充放电均有 144 个区间值；SOC 是边界状态，定义为

$$E_0, E_1, \ldots, E_{144},$$

共 145 个状态，其中 E_k 表示第 k 个区间开始时的储能电量，E_{k+1} 表示该区间执行后的储能电量。结果模板中的时间字符串只用于显示，回填程序按业务槽位顺序写入，避免 144/145 索引错位。

5.2 结果文件字段与时间槽位

附件 1、附件 2、附件 4 的原始时间标签采用区间右端点，而题目要求的结果应以真实物理区间表示。程序因此把第 k 个业务槽固定解释为

$$k \longleftrightarrow [k\Delta t, (k+1)\Delta t),$$

例如 $k = 0$ 对应 “0:00–0:10”，$k = 60$ 对应 “10:00–10:10”，$k = 143$ 对应 “23:50–24:00”。正式写出结果文件时，时间标签统一改写为上述物理区间，而不保留可能造成整体 10 min 误读的模板显示文字；数值仍严格按业务槽位顺序写入。由此，输入附件的右端点标签、模型物理索引和结果文件显示标签三者相互独立，避免 Excel 字符串反向污染模型索引。

结果工作簿与本文变量的对应关系如表2。其中 “调整购电量” 工作表存放的是滚动更新后的**最终有效计划** G_k^F，不是差额 $G_k^F - G_k^0$；调整差额仅在费用结算时计算。

表 2　结果工作簿字段与模型变量对应关系

工作表/字段	数学量	含义
计划购电量	G_k^0	当日 0:00 形成的原始正常购电计划
调整购电量	G_k^F	Q3/Q4-3 各次更新后最终对时段 k 生效的正常购电计划
紧急购电量	G_k^{emg}	实际运行中储能补救后仍未满足的供能缺口
充电量、放电量	$q_k^{\mathrm{ch}}, q_k^{\mathrm{dis}}$	储能在对应物理区间内的充、放电电量
储电量	E_k	物理区间边界上的储能状态

5.3　负荷、光伏与电价特征

从附件数据看，居民负荷在早晚和夜间保持较高水平，光伏则主要集中在白天。两者相减后形成净负荷，决定了外部电网与储能需要共同承担的供能压力；分时电价进一步决定了把电量从一个时段转移到另一个时段是否经济。图2将典型日负荷、光伏预测和分时电价并列展示，便于直接比较三类基础信息的日内变化。负荷给出需求节律，光伏形成白天的供给高峰，分时电价则提供跨时段转移电量的经济信号。

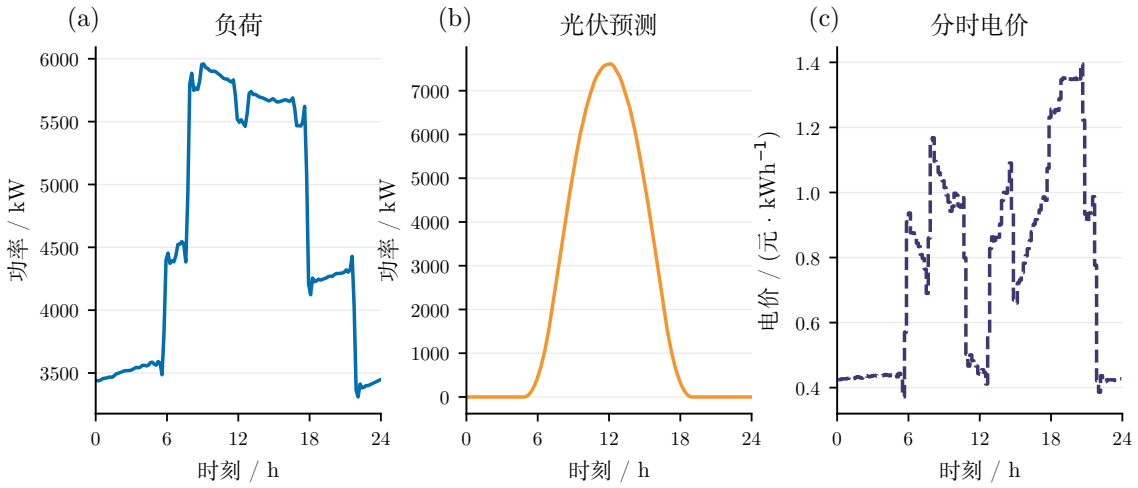


图 2　典型日负荷、光伏预测与分时电价

为避免单个典型日掩盖季节差异，进一步选取春分、夏至、秋分和冬至附近代表日比较实际净负荷 $L_k - G_k^{\mathrm{pv}}$。图3表明，夏季中午光伏出力可使净负荷降至零以下，而冬季白天仍维持较高的正净负荷；不同季节的晚间峰值也存在明显差异。因此，储能在全年运行中面对的并不是一个固定日型，后续随机模型和跨日 SOC 传递需要在全年尺度上评价。

5.4　因果预测与历史残差

问题二以后不能使用未来实际数据，因此负荷基准预测采用“7 日滞后同槽位”优先、否则使用历史同槽位扩展均值；光伏基准预测采用最近最多 7 个历史日的同槽位均值。历史残差库也严格采用样本外方式构造：对历史日期 s 的预测仅使用 $t < s$ 的数据，再以实际值减去当时预测值得到残差。由此构造的场景树不包含目标日未来真实轨迹。

图4给出 334 个正式统计日中“实际净负荷减日前因果预测净负荷”的逐时分位带。

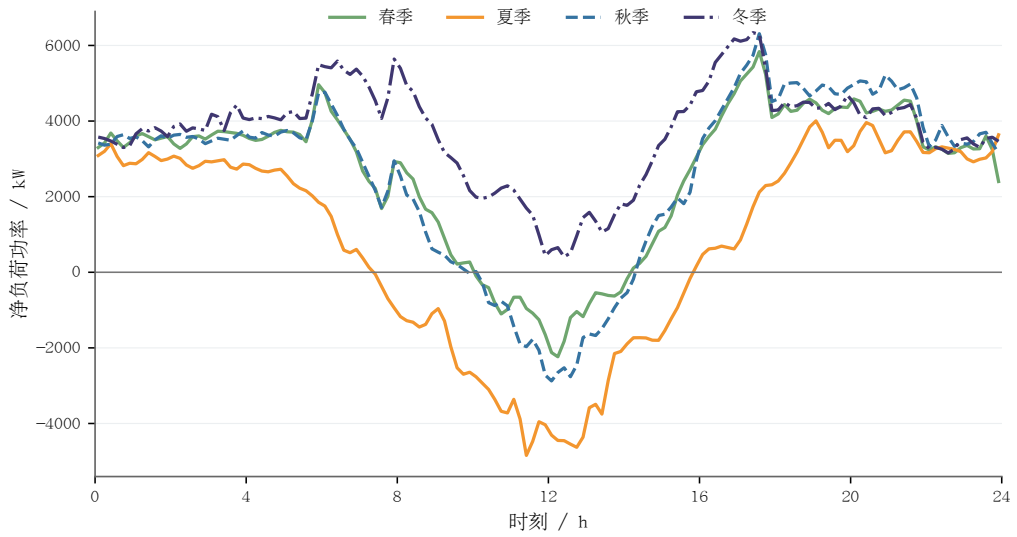


图 3　四季代表日的实际净负荷曲线

残差中位数整体接近零，但白天 8:00–16:00 的 10%–90% 区间显著变宽，说明光伏活跃时段的不确定性明显高于夜间。这一数据特征直接说明了问题二不能只依赖一条确定性预测轨迹，而需要把历史样本外残差引入场景生成。

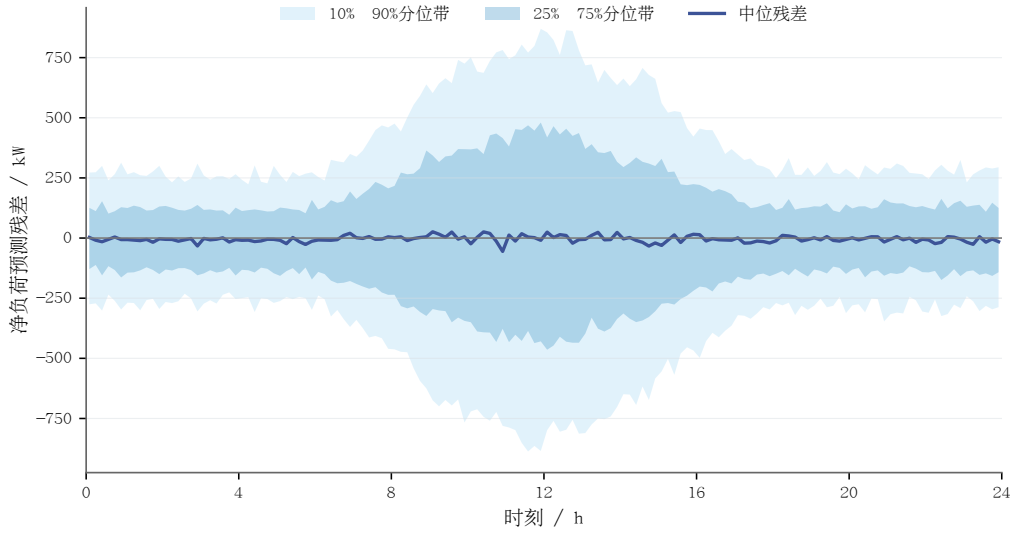


图 4　因果日前净负荷预测残差的日内分位带

问题三的附件 3 提供日内光伏预测更新。程序把各发布时刻的小时预测锚点映射为绝对时间，先在线性插值下得到 145 个 10 min 边界功率，再用梯形积分得到 144 个区间电量。6:00、12:00、18:00 更新时，对应时刻之前的真实数据已经发生，可以进入当前状态；未来区间仍只使用新预测而不读取真实值。

六、 问题一的建模与求解

6.1 模型建立

6.1.1 决策变量与目标函数

在确定性场景中，所有 $L_k, G_k^{\mathrm{pv}}, c_k$ 已知，决策变量为 $q_k^{\mathrm{buy}}, q_k^{\mathrm{ch}}, q_k^{\mathrm{dis}}, q_k^{\mathrm{curt}}$ 和 E_j。问题一不允许紧急购电和外部计划盈余，故 $q_k^{\mathrm{emg}} = q_k^{\mathrm{sur}} = 0$。目标是最小化全天正常购电费用：

$$\min C_1 = \sum_{k=0}^{143} c_k q_k^{\mathrm{buy}}. \tag{2}$$

该目标直接反映电网购电支出；储能通过改变 q_k^{buy} 的时序分布产生经济价值。

6.1.2 能量平衡与储能约束

每个 10 min 区间必须满足

$$q_k^{\mathrm{buy}} + G_k^{\mathrm{pv}} + q_k^{\mathrm{dis}} = L_k + q_k^{\mathrm{ch}} + q_k^{\mathrm{curt}}. \tag{3}$$

式(3)表示外部购电、光伏和储能放电共同覆盖用户负荷、储能充电以及不可利用光伏。

储能状态按

$$E_{k+1} = E_k + \eta_c q_k^{\mathrm{ch}} - \frac{q_k^{\mathrm{dis}}}{\eta_d}, \qquad k \in K, \tag{4}$$

演化，其中 $\eta_c = \eta_d = 0.9$。容量与充放电能力满足

$$1200 \le E_j \le 10800, \qquad j = 0, \ldots, 144, \tag{5}$$

$$0 \le q_k^{\mathrm{ch}}, q_k^{\mathrm{dis}} \le 5000 \times \frac{1}{6} = 833.333 \text{ kWh}. \tag{6}$$

问题一给定 $E_0 = 6000\,\mathrm{kWh}$，并约束

$$E_{144} = E_0, \tag{7}$$

避免通过透支日末储能获得虚假的单日成本下降。

6.2 求解方法

上述目标函数和约束均为线性式，因此可直接整理为标准线性规划，并调用 HiGHS 求解器 [2]。程序先按 144 个时段完成数据与单位转换，再建立购电、充放电和 SOC 变量，加入能量平衡及储能边界；求解后按原业务槽位回填结果，并以能量守恒和首末 SOC 进行复核。

6.3 结果分析

最终结果中，全天正常购电量为 $59\,482.699\,\mathrm{kWh}$，购电费用为 $35\,126.949$ 元；全天负荷电量为 $111\,024.808\,\mathrm{kWh}$，预测光伏电量为 $55\,482.836\,\mathrm{kWh}$。储能累计充电

20 740.666 kWh、累计放电 16 799.940 kWh，充放电量之差与 90% 双向效率下的损耗一致，且日初、日末 SOC 均为 6000 kWh。

题目指定时段的购电结果列于表3，储能在六个 4 h 区间内的充放电量及初末储电量列于表4。

表 3　问题一指定时段购电量及全天购电指标

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.0000	12:00–12:10	480.4124	14:00–14:10	0.0000
16:00–16:10	445.4317	18:00–18:10	531.8940	20:00–20:10	0.0000
全天购电量：59 482.6990 kWh			全天购电费：35 126.9486 元		

表 4　问题一指定区间储能充放电量及初末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	4500.0000	0.0000	4:00–8:00	833.3333	6365.8412
8:00–12:00	4787.9643	1702.9970	12:00–16:00	5286.0352	91.1014
16:00–20:00	0.0000	5780.1319	20:00–24:00	5333.3333	2859.8681
0:00 储电量：6000.0000 kWh			24:00 储电量：6000.0000 kWh		

图5中，储能的充放电与价格变化具有明显对应关系。低价区间内计划购电量上升并伴随充电；进入较高电价区间后，储能释放此前保存的电量，从而减少外部购电。图6进一步表明，SOC 全程保持在 1200–10800 kWh 内，并在日末回到初始水平。该结果验证了能量平衡、状态转移和价格响应三者在确定性模型中的一致性。

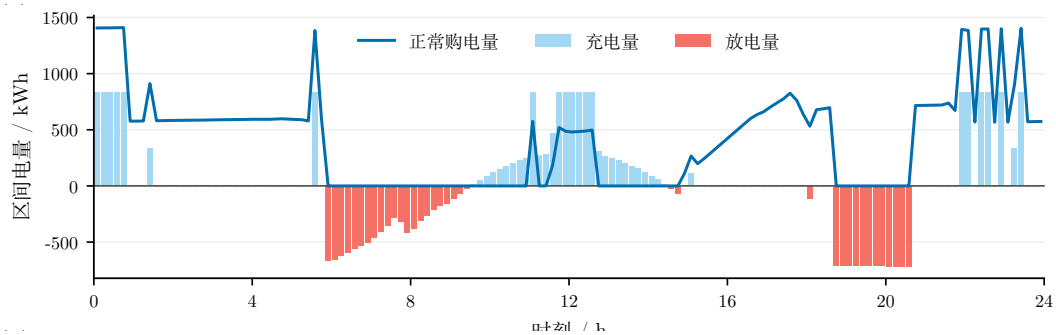


图 5　确定性场景下的正常购电与储能充放电

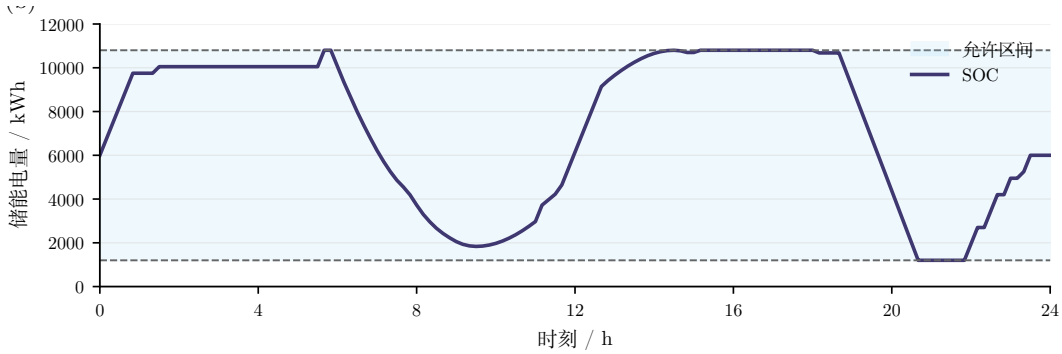


图 6　确定性场景下的储能 SOC 变化

七、 问题二的建模与求解

7.1 信息结构与非预见性

设日期为 d。在 0:00 时，决策者只能使用日期 $s < d$ 的历史数据构造当日负荷和光伏预测及残差场景；当日未来真实负荷和光伏不进入日前优化。生成的 $q_{d,k}^{\mathrm{buy}}$ 在 0:00 后全天锁定。运行至区间 k 时，控制器只使用已经观察到的前缀 $0,\dots,k-1$、当前 SOC 以及因果预测构造条件场景，并只执行场景树根节点的 $q_{d,k}^{\mathrm{ch}}, q_{d,k}^{\mathrm{dis}}$。之后当前区间真实负荷和光伏才揭示，再通过 $q^{\mathrm{emg}}, q^{\mathrm{curt}}, q^{\mathrm{sur}}$ 完成实际能量平衡。

按上述执行顺序，储能动作只能依据决策时已经掌握的信息，当前尚未发生的真实量和未来整日轨迹均不能用于求解。因此，本问对应多阶段非预见决策，而不是在知道完整真实轨迹后重新计算全天计划。随机规划中的非预见约束用于保证具有相同历史的场景在分叉前采用相同决策 [3]。需要说明的是，题目只明确规定正常购电计划在 0:00 制定，并未进一步规定当前 10 min 区间的负荷和光伏是否在储能指令下达前实时公布。本文采用较严格的口径：第 k 区间储能指令只使用 $0,\dots,k-1$ 已实现信息，L_k, G_k^{pv} 在指令确定后用于实际平衡结算。该口径保证因果性，但会比允许使用当前实时测量的控制方式更保守。图7概括了问题二的因果决策时序与非预见约束：0:00 时先依据当时可获得的预测信息与历史残差场景确定全天计划正常购电计划 $G_{0:143}^{0}$，该计划在当日内保持不变；对任一时段 k，储能充放电指令仅由信息集 $\mathcal{I}_k$ 决定，尚未实现的当前真实负荷 L_k、真实光伏 G_k^{pv} 以及未来真实量均不得进入决策；实际量实现后，系统按真实能量平衡结算，并以紧急购电 G_k^{emg} 补足储能无法覆盖的剩余缺口，从而更新到 E_{k+1}。

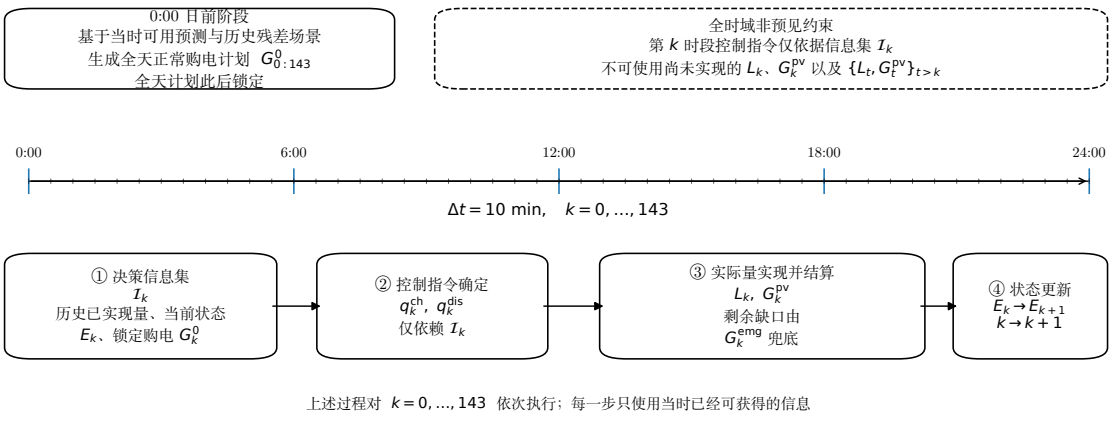


图 7 问题二的因果决策时序与非预见约束

7.2 多阶段场景树模型

设场景树节点集合为 $\mathcal{N}$，节点 n 所属物理时段为 $k(n)$，节点概率为 ρ_n。日前正常购电变量 q_k^{buy} 对所有场景共享，而 $q_n^{\mathrm{ch}}, q_n^{\mathrm{dis}}, q_n^{\mathrm{emg}}, q_n^{\mathrm{curt}}, q_n^{\mathrm{sur}}, E_n$ 按节点定义。节点能量平

衡为

$$q^{\mathrm{buy}}_{k(n)} + q^{\mathrm{emg}}_n + G^{\mathrm{pv}}_n + q^{\mathrm{dis}}_n = L_n + q^{\mathrm{ch}}_n + q^{\mathrm{curt}}_n + q^{\mathrm{sur}}_n. \tag{8}$$

若 $p(n)$ 为节点 n 的父节点，则其状态转移沿树传播：

$$E^{+}_n = E^{+}_{p(n)} + \eta_c q^{\mathrm{ch}}_n - \frac{q^{\mathrm{dis}}_n}{\eta_d}, \tag{9}$$

并受与问题一相同的容量和充放电边界约束。

紧急购电价格按正常价格的 5 倍计取，因此问题二以正常购电费用与场景期望紧急购电费用之和为目标：

$$\min C_2 = \sum_k c_k q^{\mathrm{buy}}_k + 5 \sum_{n \in \mathcal{N}} \rho_n c_{k(n)} q^{\mathrm{emg}}_n. \tag{10}$$

较高的紧急购电价格使其承担供能缺口的兜底作用，而不是成为常规能源来源。

7.3 实际运行的因果补救

对真实运行日，全天计划购电固定后，逐个 10 min 区间执行：

1. 根据历史前缀和当前 SOC 建立条件场景树；
2. 在尚不知道当前真实 L_k, G^{pv}_k 的情况下求解并只执行根节点储能动作；
3. 当前真实量揭示后，若供能不足则产生 q^{emg}_k，若供能过剩则优先形成充电、弃光或 q^{sur}_k；
4. 根据真实执行动作更新 E_{k+1}，进入下一时段。

日末 SOC 直接传递至下一日，不作人为复位。1 月只用于形成历史预测信息和 SOC 暖启动，正式统计从 2 月 1 日开始。

7.4 结果与风险分析

334 个正式统计日中，正常购电总量为 20 789.350 MWh，正常购电费用为 12 780 139.323 元（1 278.014 万元）。由逐日正式运行结果直接汇总，紧急购电费用为 3 907 129.593 元（390.713 万元），因此实际总运行费用为 16 687 268.915 元（1 668.727 万元）。紧急购电总量为 1 031.043 MWh，结果文件共记录 7239 个连续紧急购电区间块；每天紧急购电区间数中位数为 22、均值为 21.67，最大为 32。日紧急购电量均值为 3086.956 kWh，中位数为 2485.195 kWh，95% 分位数为 7301.910 kWh，最大为 16 754.717 kWh。

为了说明尾部风险如何形成，图8给出全年紧急购电量最大的 2025-06-01。该日全天计划在 0:00 后保持不变；随着实际净负荷与锁定购电之间的缺口持续出现，SOC 多次向下界靠近，紧急购电在午后和晚间明显增加。该剖面说明，紧急购电并非孤立的数值异常，而是“预测偏差持续累积–SOC 可用余量下降–剩余缺口外购”的运行结果。

图9进一步给出了全年日紧急购电量的经验累积分布，并在图中标注了 50%、90%

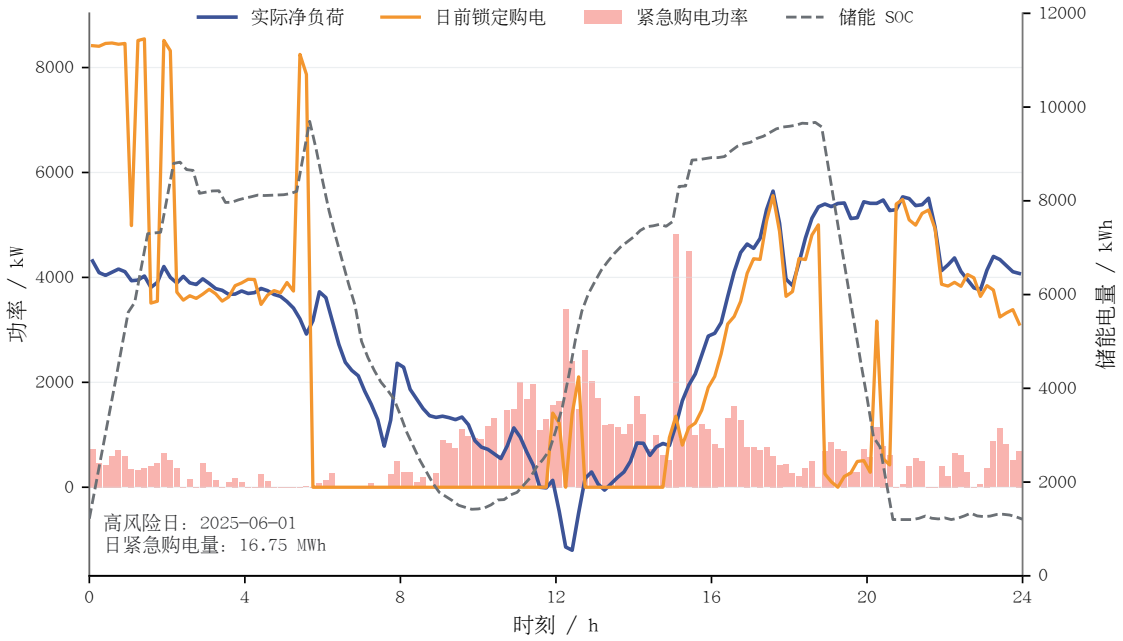


图 8　问题二全年最高风险日的购电缺口与 SOC 演化

与 95% 分位位置。多数日期对应较小缺口，但分布右端仍保留少量高风险日期，这与图8展示的“日初购电固定、储能容量有限”机制一致。紧急购电费用不依赖模板长表反推，而由程序保存的逐日 10 min 实际紧急购电序列与对应电价直接汇总。

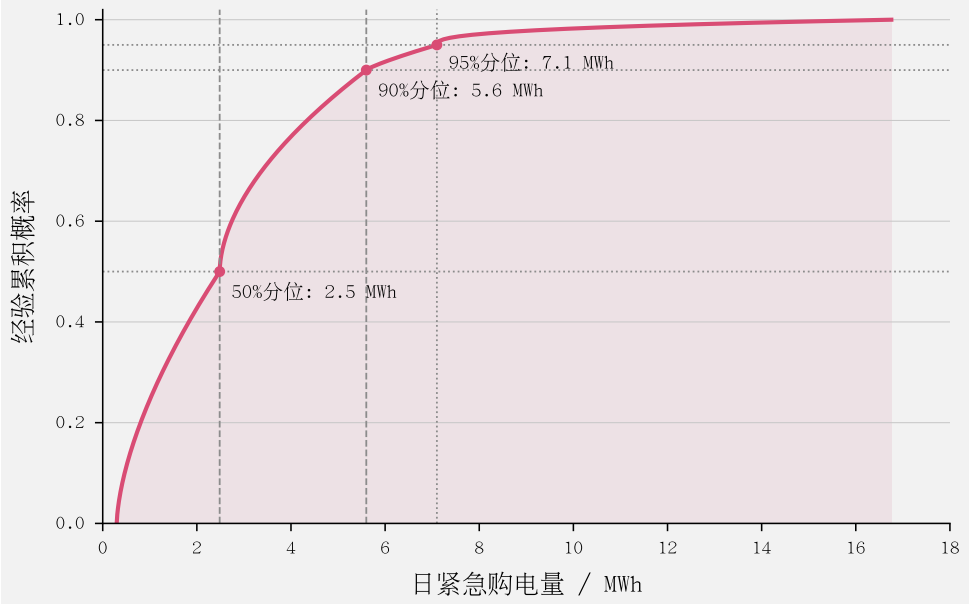


图 9　日紧急购电量的经验累积分布

八、　问题三的建模与求解

8.1　信息更新机制

问题三在 0:00、6:00、12:00、18:00 分别设置更新时点，对应区间索引 $r \in \{0, 36, 72, 108\}$。0:00 首先形成全天计划；此后每次更新仅允许修改 $k \geq r$ 的未执行区间，$k < r$ 的购电和储能决策全部冻结。6:00 等更新时点可以使用刚刚完成区间的

真实信息，例如 6:00 可使用 $k = 35$ 的实际光伏，但不得使用 $k \geq 36$ 的真实值。

附件 3 的新预测用于更新未来光伏中心轨迹；负荷仍采用问题二中的因果基准预测。图10以 2025-08-25 为例，展示 0:00、6:00、12:00、18:00 四个版本与实际光伏的关系。0:00 版本对午间峰值存在明显低估，随着日内新信息到达，后续预测逐步向当日实际轨迹修正；每一版本只在其发布时间之后参与未来决策。

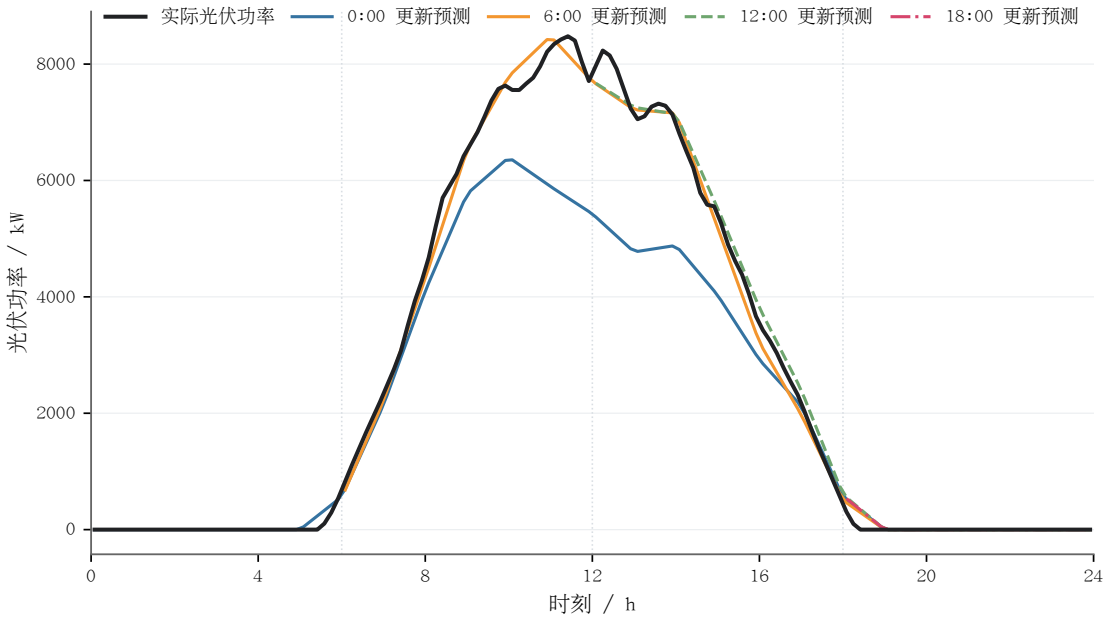


图 10　代表日光伏预测在 0:00、6:00、12:00、18:00 的滚动更新

为合理评价更新时点附近的 SOC，优化窗口采用 24 h 预览并允许跨午夜。跨日部分的购电变量仅存在于当前次优化内部，用于估计当前 SOC 的未来价值；求解结束后不写入次日的原始计划、计划版本簿或结果工作簿。次日正式全天计划仍在次日 0:00 根据当时可获得的信息重新生成，因此前一日 18:00 的预览不构成次日购电承诺。

8.2　滚动计划模型

在每个更新时点 r，模型冻结历史状态 E_r 和已执行区间，并对未来区间重新求解正常购电、充放电和必要的预测紧急购电。0:00 时目标与普通正常购电成本一致；对 $r > 0$ 的当前日区间，最终代码将调整代价写成相对 0:00 原计划的对称绝对偏差成本：

$$\min \sum_{k \geq r} c_k \left[q_k^{\mathrm{buy}} + \frac{1}{2} \left(dG_k^{+} + dG_k^{-} \right) \right] + 5 \sum_{k \geq r} c_k q_k^{\mathrm{emg}}, \tag{11}$$

其中 dG_k^{+}, dG_k^{-} 分别表示当前候选计划相对 0:00 原计划的上调与下调。实际运行的储能补救仍遵循问题二的因果执行顺序，并未把未来真实负荷/光伏用于当前动作。

8.3　调整费用结算

题目给出了计划上调与下调时的价格系数，但没有唯一规定多次滚动修改应“统一相对 0:00 原计划净结算”，还是“每次相对上一版计划逐次结算”。本文把前者作为主

模型的**关键结算假设**，并在第10.1节用逐次结算进行口径敏感性检验；因此下述公式是本文采用的建模解释，而不是把题面表述视为唯一可能的结算规则。

记 0:00 原正常购电计划为 G_k^0，一天中最后一次对区间 k 生效的最终计划为 G_k^F，则

$$dG_k^+ = \max(G_k^F - G_k^0, 0), \tag{12}$$

$$dG_k^- = \max(G_k^0 - G_k^F, 0). \tag{13}$$

正式主假设是：**只按最终计划相对 0:00 原计划的净变化进行一次性调整结算**，不把 6:00、12:00、18:00 的中间反复修改逐次累计收费。由代码实现，原计划购电费用、调整费用和调整后正常购电结算费用分别为

$$C_{\text{plan}} = \sum_k c_k G_k^0, \tag{14}$$

$$C_{\text{adj}} = \sum_k c_k \left(1.5 dG_k^+ - 0.5 dG_k^-\right), \tag{15}$$

$$C_{\text{reg}} = \sum_k c_k \left[G_k^F + 0.5(dG_k^+ + dG_k^-)\right]. \tag{16}$$

由于

$$G_k^F - G_k^0 = dG_k^+ - dG_k^-,$$

可直接推出

$$\boxed{C_{\text{reg}} = C_{\text{plan}} + C_{\text{adj}}}. \tag{17}$$

该恒等式在程序中被逐日检查，用于防止计划费用与调整费用口径不一致。特别地，当 $G_k^F < G_k^0$ 时，C_{adj} 在该时段为负值，表示未执行的下调部分按原交易电价的 50% 留作违约结算；等价地，实际执行的 G_k^F 按全价计费，而取消的 $G_k^0 - G_k^F$ 仅承担半价成本，并非“原计划全额费用之外再额外加收 50%”。

紧急购电费用另记为

$$C_{\text{emg}} = 5 \sum_k c_k G_k^{\text{emg}}, \tag{18}$$

总运行费用为 $C_{\text{total}} = C_{\text{reg}} + C_{\text{emg}}$；但本文的最终工作簿主要输出正常购电结算和紧急购电量，未据缺失字段自行补造全年 C_{emg} 数字。

8.4 结果分析

问题三 0:00 原计划正常购电量为 20 378.693 MWh，$C_{\text{plan}} = 12\,326\,551.146$ 元；日内滚动后最终正常购电量为 20 335.635 MWh，比原计划减少 43.059 MWh。全年累计上调 475.830 MWh、下调 518.889 MWh，绝对调整量为 994.720 MWh。调整后 $C_{\text{reg}} = 12\,636\,851.078$ 元，故 $C_{\text{adj}} = 310\,299.932$ 元，约占 C_{plan} 的 2.517%，与式(17)严格一致。

与问题二比较，问题三最终正常购电量减少 453.716 MWh，降幅 2.182%；按主结算假设计算的正常购电结算费用下降 143 288.245 元，降幅 1.121%。然而，Q3 的紧急购电费用为 4 151 978.796 元，比 Q2 增加 244 849.203 元（6.267%）；紧急购电总量由 1 031.043 MWh 上升到 1 060.206 MWh，增加 2.828%，连续紧急购电区间块数由 7239 增至 7495。由此 Q3 的实际总运行费用为 16 788 829.873 元（1 678.883 万元），反而比 Q2 高 101 560.958 元，即 0.609%。逐日配对中，Q3 只有 129/334 天的紧急购电量低于 Q2，占 38.62%。

因此，滚动信息更新确实改变了正常采购结构，但“正常购电结算费用下降”不能直接等价为“实际总成本下降”。在当前预测精度和储能补救规则下，正常采购节省被更高的紧急购电费用抵消。造成这一结果的一个可能原因是，滚动更新主要依据中心预测重新安排正常购电，而实际运行仍要面对预测误差的尾部；当计划下调与实际供能不足同时出现时，有限 SOC 不能完全吸收缺口。这里把这一解释视为对结果的机理说明，而不是已经验证的因果结论。图11给出代表日 0:00 原计划与最终有效计划的差异。

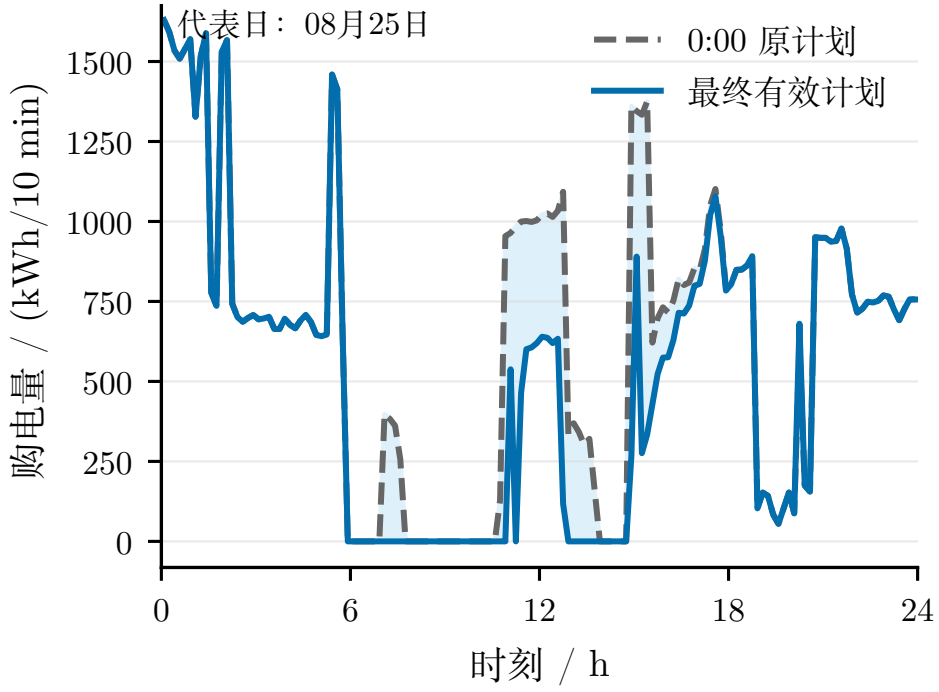


图 11 代表日 0:00 原计划与最终有效购电计划

代表日只能展示局部机制。为检验调整行为是否具有全年普遍性，将 334 个正式统计日的 $G^{F}_{d,k} - G^{0}_{d,k}$ 按日期和时刻排列得到图12。购电修正主要集中在白天光伏活跃区间，其中 8:00–16:00 最为明显；夜间调整接近零。不同月份的红蓝条带表明滚动更新并非个别日期现象，同时也说明上调和下调会交替出现，全年累计绝对调整量不能简单理解为单向增加或减少购电。

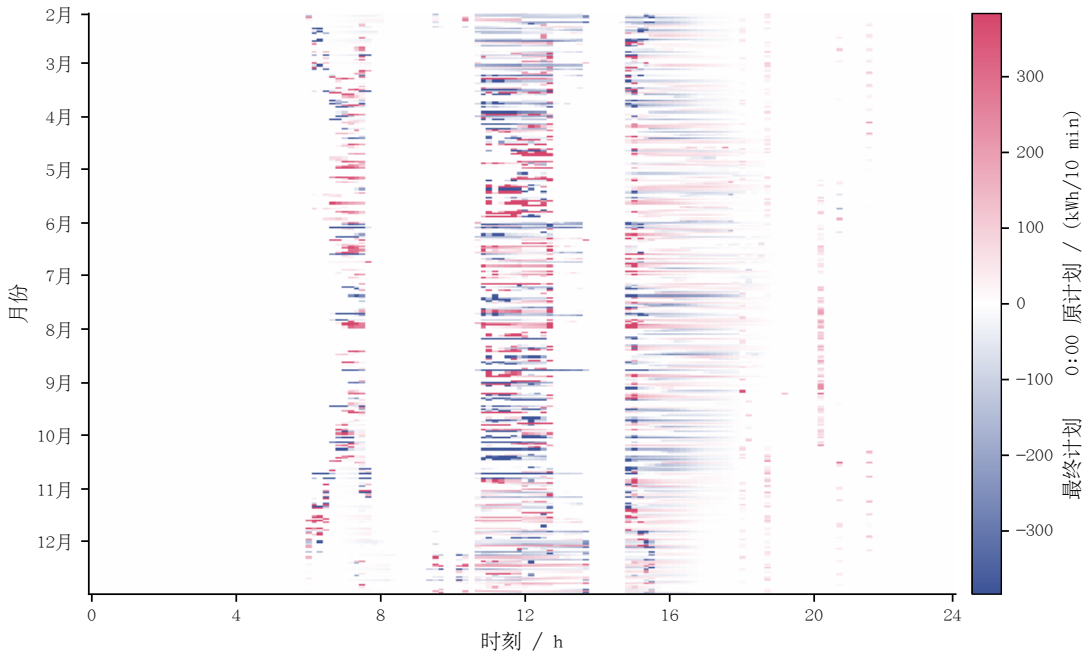


图 12 问题三全年最终购电计划相对 0:00 原计划的调整热力图

8.5 预报更新时点的讨论

题目给出的 6:00、12:00、18:00 三次新预报能够改变尚未执行区间的购电计划，但“更新次数增加”并不等价于“总费用必然下降”。本算例中，采用全部三次日内更新后，正常购电结算费用相对 Q2 下降 1.121%，而紧急购电费用上升 6.267%，最终总运行费用反而增加 0.609%。这说明新预报只有在其误差改善足以抵消调整费用及由 SOC 时序变化带来的风险时，才具有稳定的经济价值。

因此，在当前预测精度和补救规则下，**不建议仅为增加更新频率而额外引入题目给定时点之外的预报**。若能够获得新的高质量预报，应以“预测误差下降幅度–计划调整成本–紧急购电风险”的边际收益作为是否增加更新时间的判据，而不是固定地继续加密更新时间。题目给定的三个日内时点仍作为正式策略使用，但本文不对单个时点的边际价值作未经计算的定量宣称。

九、 问题四的建模与求解

9.1 动态电价信息结构

附件 4 给出真实动态电价，但真实未来价格在实际决策时不可提前获得。程序构造因果价格模块：对目标日 0:00 的价格预测只使用过去日期，优先选取最近 4 个同星期历史日并按槽位求均值；若没有同星期样本，则退化为最近 4 个历史日；历史不足时再使用附件 1 的固定分时电价作为回退。日内更新时，可使用当日已经发生的真实价格前缀计算相对日初基准的平均偏差，并将这一偏差用于修正尚未发生的预测价格。任何 k 及未来区间的真实价格都不会提前进入优化。

9.2 Q4-2 模型

Q4-2 继承问题二的信息结构：0:00 基于因果动态电价预测与负荷/光伏预测生成全天计划正常购电量，全天计划锁定；日内储能采用条件场景树因果补救。物理约束仍为式(8)及储能状态转移，区别仅在于目标中的 c_k 换为 0:00 可获得的因果动态电价预测。实际结算时再按附件 4 当天真实电价对已经执行的 G_k^0 和紧急购电进行事后计费，真实电价不反向影响此前决策。该处理采用较保守的信息时序：本文不假设当前区间真实动态价格在储能动作之前必然公布；若实际市场规则允许动作前获得当前实时价格，则可在保持未来价格未知的前提下进一步修正日内储能策略。

9.3 Q4-3 模型

Q4-3 继承问题三的信息结构：在 0:00、6:00、12:00、18:00 更新未来未执行区间的价格预测和光伏预测，已执行决策冻结；正常购电的调整仍按式(14)–(17)进行一次性最终差额结算。实际实时储能动作继续遵循因果补救，而不是使用未来真实动态价格进行全日重优化。因此，Q4-2 与 Q4-3 的差异主要反映“日内信息更新”在动态价格环境中的价值和代价。

9.4 结果与对比分析

Q4-2 正常购电量为 20 797.185 MWh,实际动态电价下正常购电费用为 13 496 495.713 元，即 1 349.650 万元；紧急购电费用为 4 055 466.646 元，总运行费用为 17 551 962.359 元（1 755.196 万元）。紧急购电量为 1 030.660 MWh，连续紧急购电区间块数为 7243。

Q4-3 的 0:00 原计划正常购电量为 20 381.980 MWh，$C_{\text{plan}} = 13\,015\,451.629$ 元；滚动后最终正常购电量为 20 347.222 MWh，$C_{\text{reg}} = 13\,357\,821.591$ 元，$C_{\text{adj}} = 342\,369.962$ 元，调整费约占原计划购电费用的 2.630%。全年累计上调 491.544 MWh、下调 526.302 MWh，绝对调整量为 1 017.845 MWh。

相较 Q4-2，Q4-3 最终正常购电量减少 449.964 MWh，下降 2.164%；按主结算假设计算的正常购电结算费用减少 138 674.122 元，下降 1.027%。与此同时，Q4-3 的紧急购电费用为 4 223 169.291 元，比 Q4-2 增加 167 702.645 元（4.135%）；紧急购电量由 1 030.660 MWh 增至 1 051.882 MWh，区间块数由 7243 增至 7507。Q4-3 的总运行费用为 17 580 990.882 元（1 758.099 万元），比 Q4-2 高 29 028.523 元，即 0.165%。逐日比较中，Q4-3 的紧急购电量低于 Q4-2 的日期为 136 天，占 334 个统计日的 40.72%。

由此可见，在当前模型下，日内信息更新降低了主结算口径下的正常购电费用，但更高的紧急购电费用抵消了这部分节省，使实际总运行费用略有上升。图13给出代表日价格响应；图14把正常购电与紧急购电费用置于同一尺度比较，可以直接看到 Q3 和 Q4-3 的常规采购节省被紧急购电增量抵消。

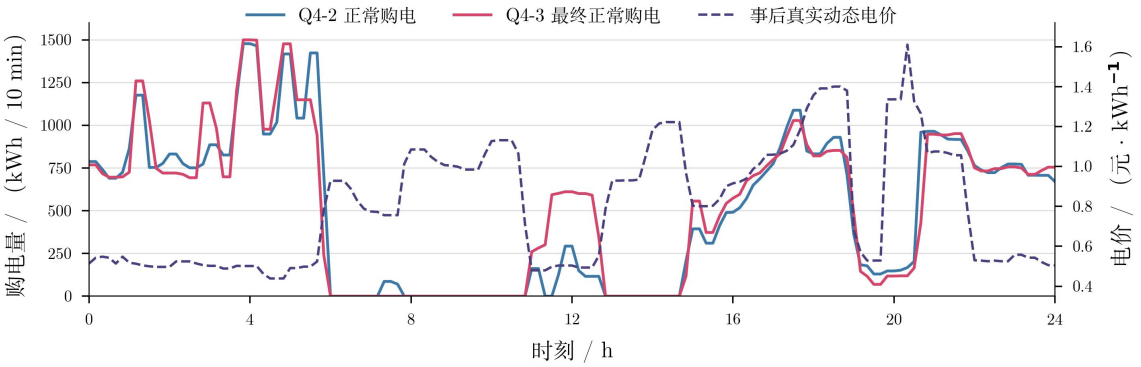


图 13　动态电价下 Q4-2 与 Q4-3 的代表日购电响应

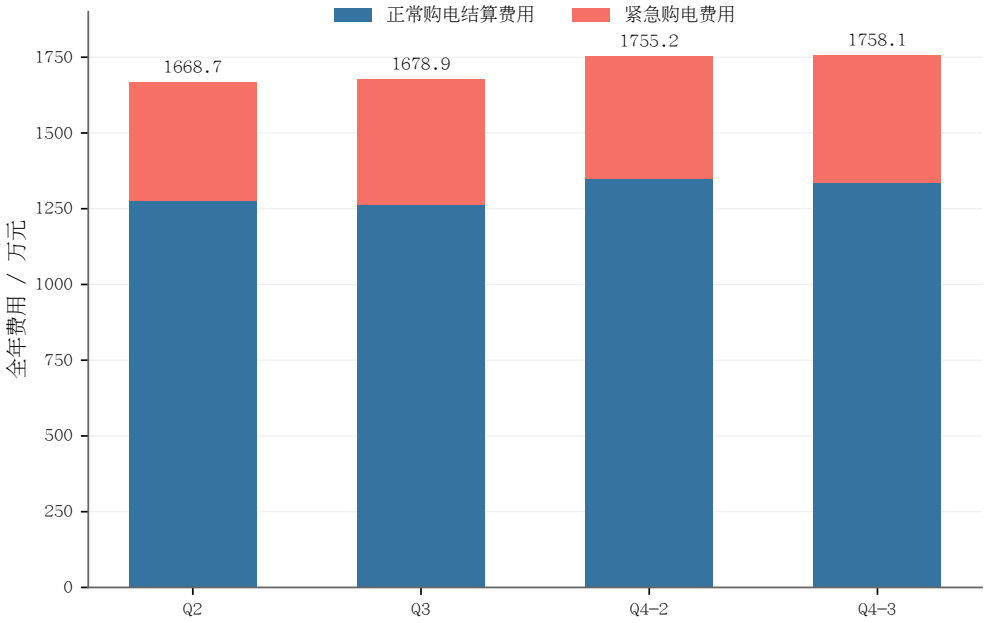


图 14　Q2–Q4 全年正常购电与紧急购电费用分解

表 5　四种正式策略的全年费用分解

方案	正常购电结算费用/万元	紧急购电费用/万元	总运行费用/万元
Q2	1278.014	390.713	1668.727
Q3	1263.685	415.198	1678.883
Q4-2	1349.650	405.547	1755.196
Q4-3	1335.782	422.317	1758.099

十、　稳健性分析

10.1　调整结算口径

问题三和 Q4-3 的调整费用存在两种可能的解释：其一，最终有效计划相对 0:00 原计划作一次净结算；其二，每次更新均相对上一版计划逐次结算。为检验结论是否依赖这一口径，固定正式求解已经产生的各版购电计划，仅改变结算方式。若第 j 次更新后

的有效计划为 $G_k^{(j)}$，上一版为 $G_k^{(j-1)}$，定义

$$\Delta_{j,k}^{+}=\max\left(G_k^{(j)}-G_k^{(j-1)},0\right),\qquad \Delta_{j,k}^{-}=\max\left(G_k^{(j-1)}-G_k^{(j)},0\right),$$

逐次结算费用为

$$C_{\mathrm{adj}}^{\mathrm{seq}}=\sum_{j}\sum_{k\geq r_j}c_k\left(1.5\Delta_{j,k}^{+}-0.5\Delta_{j,k}^{-}\right), \tag{19}$$

其中只对当次更新后尚未执行的区间计费。

表6给出两种口径的比较。Q3 的正常购电结算费用由净结算的 1263.685 万元增至逐次结算的 1280.070 万元；Q4-3 由 1335.782 万元增至 1353.337 万元。由于物理调度和紧急购电轨迹保持不变，对应总运行费用分别由 1678.883 万元增至 1695.268 万元、由 1758.099 万元增至 1775.654 万元。

表 6　固定计划版本下两种调整结算口径的费用比较

方案	净结算正常费用/万元	逐次结算正常费用/万元	净结算总费用/万元	逐次结算总费用/万元
Q3	1263.685	1280.070	1678.883	1695.268
Q4-3	1335.782	1353.337	1758.099	1775.654

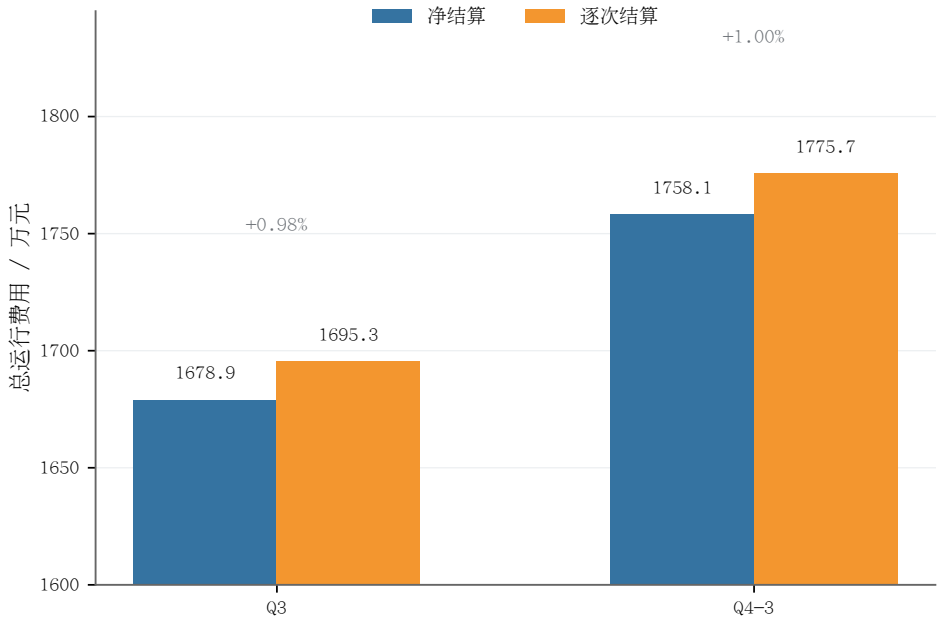


图 15　固定计划版本下净结算与逐次结算的总运行费用比较

在逐次结算下，Q3 与 Q4-3 的正常购电费用优势消失，但“滚动策略未降低实际总运行费用”这一判断没有反转。因此，正文关于 Q3/Q4-3 的经济性结论均限定在相应结算假设下，并以总运行费用而非单独的正常购电费用评价方案优劣。

十一、 模型特点与局限

11.1 模型特点

1. **四问物理内核统一**。全文均以 10 min 能量平衡、SOC 状态转移和容量/功率边界为基础，只改变信息结构，使 Q1–Q4 的递进关系清晰。
2. **信息边界明确**。Q2 日前计划只用历史预测和残差，实际补救按时间因果执行；Q3 只能更新未执行区间；Q4 的未来真实价格不进入当前决策。这避免了常见的未来信息泄露。
3. **模型采用连续线性结构**。主体为连续线性规划，可由 HiGHS 稳定求解；购电、充放电、SOC 和调整费用均有明确物理含义，费用恒等式可直接由结果复核。
4. **经济性与可靠性分开统计**。本文没有把正常购电费用下降等同于风险下降，而是同时报告紧急购电量、区间块数和尾部分位数，从而真实揭示策略权衡。

11.2 模型局限

1. 负荷、光伏和动态电价预测采用历史同槽位、同星期均值等因果基线，保证了信息边界，但预测精度仍有提升空间。
2. 正式模型没有计入电池循环老化成本，储能频繁充放电的长期经济代价未进入目标函数。
3. 主模型未设置充放电二进制互斥变量，而由效率损失与成本结构抑制无意义的同步充放电；若价格结构允许异常套利，应增加显式互斥约束。
4. Q3/Q4-3 的正常购电结算费用对调整计费口径敏感。按最终计划相对 0:00 原计划净结算时费用略低；按固定计划版本逐次结算时，这一优势消失。因此相关结论均限定于本文采用的结算解释。
5. 1 月用于暖启动而未进入正式汇总，因此本文统一统计期为 334 天，而不是直接使用全年 365 天。

十二、 结论

本文以统一的能量平衡和 SOC 约束贯穿四个问题，并随题目给出的信息条件逐步扩展模型。问题一中，确定性线性规划得到全天正常购电量 59 482.699 kWh、购电费用 35 126.949 元；储能在低价时段充电、高价时段放电，且日末 SOC 回到 6000 kWh，说明购电决策与储能状态转移能够保持一致。

问题二中，0:00 形成的正常购电计划在日内保持不变。334 个正式统计日内，正常购电费用为 1 278.014 万元，紧急购电费用为 390.713 万元，总运行费用为 1 668.727 万元；紧急购电量为 1 031.043 MWh，共记录 7239 个连续紧急购电区间。结果表明，在预测误差存在且全天计划不能修改时，有限 SOC 不能完全吸收所有供需偏差，因此需要由紧急购电承担剩余缺口。

问题三允许在 6:00、12:00、18:00 修正尚未执行的购电计划。按本文主结算假设，

Q3 的正常购电结算费用比 Q2 下降 1.121%，但紧急购电费用增加 6.267%，使总运行费用反而增加 0.609%。问题四进一步引入动态电价后，Q4-3 相对 Q4-2 的正常购电结算费用下降 1.027%，紧急购电费用增加 4.135%，总运行费用增加 0.165%。因此，在当前预测精度与储能补救规则下，日内信息更新改变了常规采购结构，却没有形成总费用和可靠性的同步改善。

结算口径比较进一步表明：固定既有计划版本时，若把多次调整改为逐次结算，Q3 与 Q4-3 的正常购电费用分别比净结算增加约 1.297% 和 1.314%，但“滚动策略未降低总运行费用”的判断不发生反转。因此，本题应同时报告正常购电、紧急购电和总运行费用，不能只凭正常购电费用评价策略优劣。结合 Q3 的结果，在当前预测精度和补救规则下也没有证据支持继续增加题目给定时点之外的预报更新；新增时点是否值得采用，应由预测误差改善带来的边际收益与计划调整成本共同决定。

AI 工具使用声明：本参赛队在竞赛过程中使用了 AI 工具，主要用于论文结构与语言润色、代码调试与审查、图表设计及排版辅助，详细使用情况见支撑材料。

参考文献

[1] 鲁卓欣, 徐潇源, 严正, 等. 不确定性环境下数据驱动的电力系统优化调度方法综述[J]. 电力系统自动化, 2020, 44(21): 172–183. DOI: 10.7500/AEPS20200202004.

[2] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142.

[3] BIRGE J R, LOUVEAUX F. Introduction to Stochastic Programming[M]. 2nd ed. New York: Springer, 2011.

[4] HU J, SHAN Y, GUERRERO J M, IOINOVICI A, CHAN K W, RODRIGUEZ J. Model predictive control of microgrids–An overview[J]. Renewable and Sustainable Energy Reviews, 2021, 136: 110422. DOI: 10.1016/j.rser.2020.110422.

[5] ABEDI S, KWON S. Rolling-horizon optimization integrated with recurrent neural network-driven forecasting for residential battery energy storage operations[J]. International Journal of Electrical Power & Energy Systems, 2023, 145: 108589. DOI: 10.1016/j.ijepes.2022.108589.

[6] YUAN Z P, LI P, LI Z L, XIA J. Data-Driven Risk-Adjusted Robust Energy Management for Microgrids Integrating Demand Response Aggregator and Renewable Energies[J]. IEEE Transactions on Smart Grid, 2023, 14(1): 365–377. DOI: 10.1109/TSG.2022.3193226.

[7] ZHAO Z, XU J, GUO J, NI Q, CHEN B, LAI L L. Robust Energy Management for Multi-Microgrids Based on Distributed Dynamic Tube Model Predictive Control[J]. IEEE Transactions on Smart Grid, 2024, 15(1): 203–217. DOI: 10.1109/TSG.2023.3290191.

[8] CHEN L, GAO L, XING S, CHEN Z, WANG W. Zero-carbon microgrid: Real-world cases, trends, challenges, and future research prospects[J]. Renewable and Sustainable Energy Reviews, 2024, 203: 114720. DOI: 10.1016/j.rser.2024.114720.

A 支撑材料文件列表

本论文对应的支撑材料包括：

1. `data/`：题目附件 1–4；
2. `results/`：Q1–Q4 的 5 个最终结果工作簿，即 `result1.xlsx`、`result2.xlsx`、`result3.xlsx`、`result4-2.xlsx` 与 `result4-3.xlsx`；
3. `code/`：全部可运行 Python 源程序；
4. `figures/`：论文使用的结果图；
5. `AI 工具使用详情.pdf`：AI 工具名称、用途、主要交互方式以及人工审查与核验说明。

为便于核对支撑材料与正文模型的对应关系，核心源程序按功能分组列于表7。

表 7 核心源程序及功能分组

分组	文件
基础与数据	`config.py`, `time_index.py`, `data_loader.py`, `data_checker.py`, `_excel_backend.py`
预测与场景	`forecast_model.py`, `forecast_parser.py`, `scenario_tree.py`
优化与滚动	`model_base.py`, `rolling_scheduler.py`, `question1_solver.py`–`question4_solver.py`
结果与作图	`run_all.py`, `result_writer.py`, `make_figures.py`, `visualize.py`
复核与诊断	`review_audit.py`, `analysis_cost_summary.py`, `analysis_settlement_sensitivity.py`
包与兼容层	`__init__.py`, `gurobipy.py`

B 复现入口与运行说明

支撑材料中的 `code/run_all.py` 为正式结果的一键复现入口。将附件 1–4 放入同一数据目录后，可执行

```
python code/run_all.py --data-dir ./data --output-dir ./results
```

程序依次读取附件、求解 Q1、全年 Q2、Q3、Q4-2 与 Q4-3，保持跨日 SOC 连续，并调用统一写出模块生成 `result1.xlsx`、`result2.xlsx`、`result3.xlsx`、`result4-2.xlsx` 和 `result4-3.xlsx`。同时保存 Q2–Q4 的中间 PeriodResult 供费用复核与结算口径分析使用。正式路径中不启用 Oracle、CVaR 或额外二进制互斥开关。

C 完整源程序

根据 2026 年论文格式规范，附录需包含建模所用的完整可运行源程序。本节将支撑材料 `code/` 中的全部 Python 源文件逐页排入论文，内容与电子支撑材料一致。

完整源程序索引

代码采用较大字号、自动换行和连续行号排版，以提高纸质版与电子版的可读性。

文件	代码行数
`__init__.py`	2
`config.py`	84
`time_index.py`	242
`data_loader.py`	453
`data_checker.py`	448
`forecast_parser.py`	345
`forecast_model.py`	514
`scenario_tree.py`	1011
`model_base.py`	647
`question1_solver.py`	884
`question2_solver.py`	1914
`question3_solver.py`	1850
`question4_solver.py`	1108
`rolling_scheduler.py`	389
`run_all.py`	112
`result_writer.py`	1742
`review_audit.py`	261
`analysis_cost_summary.py`	87
`analysis_settlement_sensitivity.py`	196
`gurobipy.py`	475
`_excel_backend.py`	292
`visualize.py`	984
`make_figures.py`	236
合计	**14276**

注：本附录只改变代码的排版，不改变源程序内容；电子支撑材料中的 `code/` 目录保留原始可运行文件。

文件：code/__init__.py

```
# -*- coding: utf-8 -*-
"""2026 国赛 C 题微网调控工程源码。"""
```

文件：code/config.py

```python
# -*- coding: utf-8 -*-
"""
config.py
C 题公共配置模块。

职责：
1. 统一附件路径、sheet 名称、字段名称；
2. 统一 2025 年日期范围与 10 分钟时间尺度；
3. 仅保存配置，不读取数据、不做预测、不建立优化模型。
"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Tuple


@dataclass(frozen=True)
class ProjectConfig:
    """C 题公共配置。"""

    # 数据目录：比赛工程中可改成 Path("data") 等。
    data_dir: Path = Path(".")

    # 原始附件文件名
    attachment1_name: str = " 附件 1.xlsx"
    attachment2_name: str = " 附件 2.xlsx"
    attachment3_name: str = " 附件 3.xlsx"
    attachment4_name: str = " 附件 4.xlsx"

    # Sheet 名称
    attachment1_sheet: str = "Sheet1"
    attachment2_load_sheet: str = " 小区负载"
    attachment2_pv_sheet: str = " 光伏发电实际功率"
    attachment3_sheet: str = "Sheet1"
    attachment4_sheet: str = "Sheet1"

    # 原始字段名称
    attachment1_time_col: str = " 时间"
    attachment1_price_col: str = " 电价"
    attachment1_load_col: str = " 小区负载"
    attachment1_pv_forecast_col: str = " 光伏发电预测功率"

    wide_date_col: str = " 日期\\时间"

    attachment3_date_col: str = " 日期"
    attachment3_issue_time_col: str = " 预报时刻"

    # 时间与日期
    year: int = 2025
    start_date: str = "2025-01-01"
    end_date: str = "2025-12-31"
    dt_minutes: int = 10
    intervals_per_day: int = 144
    state_nodes_per_day: int = 145

    # 附件 3
    issue_hours: Tuple[int, ...] = (0, 6, 12, 18)
    forecast_horizons: int = 24
    expected_attachment3_rows: int = 365 * 4

    @property
    def attachment1_path(self) -> Path:
        return self.data_dir / self.attachment1_name
```

```
66
67      @property
68      def attachment2_path(self) -> Path:
69          return self.data_dir / self.attachment2_name
70
71      @property
72      def attachment3_path(self) -> Path:
73          return self.data_dir / self.attachment3_name
74
75      @property
76      def attachment4_path(self) -> Path:
77          return self.data_dir / self.attachment4_name
78
79      @property
80      def attachment3_forecast_columns(self) -> Tuple[str, ...]:
81          return tuple(f" 预报 {i} 小时" for i in range(1, self.forecast_horizons + 1))
82
83
84  DEFAULT_CONFIG = ProjectConfig()
```

文件：code/time_index.py

```python
# -*- coding: utf-8 -*-
"""
time_index.py
全题唯一时间索引模块。

核心原则：
    k -> [tau_k, tau_{k+1}) -> source_right_endpoint_label=tau_{k+1}

附件 1/2/4 的 0:10, 0:20, ..., 0:00+1
统一解释为 10 分钟物理区间的右端点标签。

注意：
- paper_label 表示真实物理区间；
- template_display_label 作为正式结果文件的时间标签，统一采用真实物理区间；
- 官方空白模板若存在首槽 0:10-0:20 的显示偏移，写出时修正为 0:00-0:10 至 23:50-24:00。
"""

from __future__ import annotations

from datetime import time, datetime
from numbers import Real
from typing import Any

import pandas as pd

from config import DEFAULT_CONFIG, ProjectConfig



def _clock_label_from_minutes(total_minutes: int, plus_one_style: bool = False) -> str:
    """
    将从当天 0:00 起累计分钟数转换为显示标签。

    total_minutes:
        0~1439 -> 当天时刻
        1440 及以上 -> 次日时刻，plus_one_style=True 时输出 0:00+1 形式
    """
    day_offset, minute_in_day = divmod(int(total_minutes), 24 * 60)
    hour, minute = divmod(minute_in_day, 60)

    if plus_one_style and day_offset > 0:
        return f"{hour}:{minute:02d}+{day_offset}"

    if day_offset == 0:
        return f"{hour}:{minute:02d}"

    # 物理区间端点用于 paper_label 时，1440 分钟写成 24:00 更直观。
    if total_minutes == 24 * 60:
        return "24:00"

    return f"{24 * day_offset + hour}:{minute:02d}"


def _template_clock_label(total_minutes: int) -> str:
    """ 结果模板显示时刻：跨日后使用 0:00+1、0:10+1 形式。"""
    day_offset, minute_in_day = divmod(int(total_minutes), 24 * 60)
    hour, minute = divmod(minute_in_day, 60)
    if day_offset == 0:
        return f"{hour}:{minute:02d}"
    return f"{hour}:{minute:02d}+{day_offset}"


def build_time_map(config: ProjectConfig = DEFAULT_CONFIG) -> pd.DataFrame:
    """
    建立全题唯一 144 行 time_map。
```

```
66
67      至少包含:
68          k
69          interval_start
70          interval_end
71          source_right_endpoint_label
72          paper_label
73          template_slot
74          template_display_label
75
76      同时附加 start_minute/end_minute, 便于后续程序计算。
77      """
78      if config.dt_minutes != 10:
79          raise ValueError(" 当前赛题固定采用 10 分钟时间步, 不允许在 time_map 中静默改变。")
80      if config.intervals_per_day != 144:
81          raise ValueError(" 当前赛题每天必须恰好 144 个 10 分钟物理区间。")
82
83      rows = []
84      for k in range(config.intervals_per_day):
85          start_minute = k * config.dt_minutes
86          end_minute = (k + 1) * config.dt_minutes
87
88          interval_start = _clock_label_from_minutes(start_minute)
89          interval_end = _clock_label_from_minutes(end_minute)
90
91          source_label = (
92              "0:00+1"
93              if end_minute == 24 * 60
94              else _clock_label_from_minutes(end_minute)
95          )
96
97          # 论文/模型物理区间标签
98          paper_label = f"{interval_start}-{interval_end}"
99
100         # 正式结果文件统一使用真实物理区间标签。
101         # 例如 k=0 -> 0:00-0:10, k=60 -> 10:00-10:10。
102         # 这样可消除官方空白模板显示文字可能存在的整体 10 min 偏移。
103         template_display_label = paper_label
104
105         rows.append(
106             {
107                 "k": k,
108                 "start_minute": start_minute,
109                 "end_minute": end_minute,
110                 "interval_start": interval_start,
111                 "interval_end": interval_end,
112                 "source_right_endpoint_label": source_label,
113                 "paper_label": paper_label,
114                 "template_slot": k,
115                 "template_display_label": template_display_label,
116             }
117         )
118
119     time_map = pd.DataFrame(rows)
120
121     validate_time_map(time_map, config)
122     return time_map
123
124
125 def validate_time_map(
126     time_map: pd.DataFrame,
127     config: ProjectConfig = DEFAULT_CONFIG,
128 ) -> None:
129     """ 对唯一时间映射执行强制单元测试。"""
130     if len(time_map) != config.intervals_per_day:
131         raise AssertionError(
132             f"time_map 必须为 {config.intervals_per_day} 行, 实际 {len(time_map)} 行。"
133         )
134
```

```
    if list(time_map["k"]) != list(range(config.intervals_per_day)):
        raise AssertionError("time_map.k 必须严格为 0~143 且顺序唯一。")

    if time_map["template_slot"].tolist() != time_map["k"].tolist():
        raise AssertionError(" 必须满足 template_slot[k] = k。")

    expected = {
        0: ("0:00", "0:10", "0:10"),
        36: ("6:00", "6:10", "6:10"),
        60: ("10:00", "10:10", "10:10"),
        143: ("23:50", "24:00", "0:00+1"),
    }

    for k, (start, end, source) in expected.items():
        row = time_map.loc[time_map["k"] == k].iloc[0]
        assert row["interval_start"] == start, (
            f"k={k} interval_start 错误: {row['interval_start']} != {start}"
        )
        assert row["interval_end"] == end, (
            f"k={k} interval_end 错误: {row['interval_end']} != {end}"
        )
        assert row["source_right_endpoint_label"] == source, (
            f"k={k} source 错误: "
            f"{row['source_right_endpoint_label']} != {source}"
        )

    # 正式结果文件的关键显示槽位必须与真实物理区间一致。
    expected_template_labels = {
        0: "0:00-0:10",
        41: "6:50-7:00",
        60: "10:00-10:10",
        143: "23:50-24:00",
    }
    for k, expected_label in expected_template_labels.items():
        row = time_map.loc[time_map["k"] == k].iloc[0]
        actual_label = row["template_display_label"]
        if actual_label != expected_label:
            raise AssertionError(
                f"k={k} 正式结果标签错误: "
                f"{actual_label!r} != {expected_label!r}"
            )


def canonical_source_label(value: Any) -> str:
    """
    将 pandas 从 Excel 读出的时间表头/时间单元格统一成:
        0:10, 6:00, 10:00, ..., 0:00+1

    支持:
    - Excel 时间小数 (如 10min = 0.00694444)
    - datetime.time / Timestamp / datetime
    - '0:10' / '00:10:00'
    - 特殊末端字符串 '0:00+1'
    """
    if pd.isna(value):
        raise ValueError(" 发现空时间标签。")

    if isinstance(value, str):
        s = value.strip()
        if not s:
            raise ValueError(" 发现空字符串时间标签。")

        # 次日特殊标签
        normalized = s.replace(" +", "+")
        if normalized in {"0:00+1", "00:00+1", "0:00:00+1", "00:00:00+1"}:
            return "0:00+1"

        # 纯时刻字符串
        parts = normalized.split(":")
```

```
        if len(parts) >= 2 and parts[0].isdigit():
            hour = int(parts[0])
            minute_text = "".join(ch for ch in parts[1] if ch.isdigit())
            if minute_text:
                minute = int(minute_text[:2])
                if 0 <= hour <= 23 and 0 <= minute <= 59:
                    return f"{hour}:{minute:02d}"

        # pandas 解析兜底
        try:
            parsed = pd.to_datetime(s, errors="raise")
            return f"{parsed.hour}:{parsed.minute:02d}"
        except Exception as exc:
            raise ValueError(f" 无法识别时间标签: {value!r}") from exc

    if isinstance(value, time):
        return f"{value.hour}:{value.minute:02d}"

    if isinstance(value, (pd.Timestamp, datetime)):
        return f"{value.hour}:{value.minute:02d}"

    if isinstance(value, Real):
        # Excel 时间为一天的小数。使用四舍五入规避浮点误差。
        minute = int(round(float(value) * 24 * 60))
        minute %= 24 * 60
        hour, minute_in_hour = divmod(minute, 60)
        return f"{hour}:{minute_in_hour:02d}"

    raise TypeError(f" 不支持的时间标签类型: {type(value)!r}, value={value!r}")


def expected_source_labels(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> list[str]:
    """ 返回 144 个附件右端点标签的标准顺序。"""
    return build_time_map(config)["source_right_endpoint_label"].tolist()


TIME_MAP = build_time_map(DEFAULT_CONFIG)
```

文件：code/data_loader.py

```python
# -*- coding: utf-8 -*-
"""
data_loader.py
附件 1-4 公共数据读取模块。

原则：
1. 使用 pandas 读取，不写回原始附件；
2. 原始 DataFrame 始终保留；
3. 附件 1/2/4 只能通过 time_map 将右端点标签映射到模型 k；
4. 附件 3 保留原始宽表，不在此模块做插值、预测或 10 分钟化；
5. 日期统一为 pandas.Timestamp(normalized)。
"""

from __future__ import annotations

from dataclasses import dataclass
from datetime import date, datetime, time
from numbers import Real
from pathlib import Path
from typing import Dict, Optional

import pandas as pd

from config import DEFAULT_CONFIG, ProjectConfig
from time_index import (
    build_time_map,
    canonical_source_label,
    expected_source_labels,
)


@dataclass
class Attachment1Data:
    raw: pd.DataFrame
    aligned: pd.DataFrame


@dataclass
class Attachment2Data:
    load_raw: pd.DataFrame
    pv_raw: pd.DataFrame
    load_kw: pd.DataFrame
    pv_kw: pd.DataFrame


@dataclass
class Attachment3Data:
    raw: pd.DataFrame
    release_index: pd.DataFrame


@dataclass
class Attachment4Data:
    raw: pd.DataFrame
    price: pd.DataFrame


@dataclass
class DataBundle:
    time_map: pd.DataFrame
    attachment1: Attachment1Data
    attachment2: Attachment2Data
    attachment3: Attachment3Data
    attachment4: Attachment4Data

```

```
66
67  def _ensure_file(path: Path) -> None:
68      if not path.exists():
69          raise FileNotFoundError(f" 未找到附件: {path}")
70
71
72  def _parse_excel_date_value(value) -> pd.Timestamp:
73      """
74      解析附件 2/4 等日期单元格。
75
76      支持:
77      - Excel 日期序列号
78      - datetime/date/Timestamp
79      - 日期字符串
80      """
81      if pd.isna(value):
82          return pd.NaT
83
84      if isinstance(value, pd.Timestamp):
85          return value.normalize()
86
87      if isinstance(value, datetime):
88          return pd.Timestamp(value).normalize()
89
90      if isinstance(value, date):
91          return pd.Timestamp(value).normalize()
92
93      if isinstance(value, Real):
94          # Excel 1900 date system, pandas/openpyxl 常用兼容原点
95          return (
96              pd.Timestamp("1899-12-30")
97              + pd.to_timedelta(float(value), unit="D")
98          ).normalize()
99
100     s = str(value).strip()
101     if not s:
102         return pd.NaT
103
104     return pd.to_datetime(s, errors="raise").normalize()
105
106
107 def _parse_date_series(series: pd.Series) -> pd.Series:
108     parsed = series.map(_parse_excel_date_value)
109     return pd.to_datetime(parsed).dt.normalize()
110
111
112 def _validate_source_label_order(
113     labels,
114     config: ProjectConfig,
115     source_name: str,
116 ) -> None:
117     canonical = [canonical_source_label(x) for x in labels]
118     expected = expected_source_labels(config)
119
120     if len(canonical) != config.intervals_per_day:
121         raise ValueError(
122             f"{source_name} 时间标签数量应为 {config.intervals_per_day}, "
123             f" 实际为 {len(canonical)}。"
124         )
125
126     if canonical != expected:
127         mismatches = [
128             (i, got, exp)
129             for i, (got, exp) in enumerate(zip(canonical, expected))
130             if got != exp
131         ]
132         preview = mismatches[:10]
133         raise ValueError(
134             f"{source_name} 时间标签未按统一右端点顺序排列。"
```

```
            f" 前若干差异 ={preview}"
        )


def load_attachment1(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> Attachment1Data:
    """
    读取附件 1。

    aligned 为 144 行模型时段表:
        k
        interval_start/end
        source_right_endpoint_label
        price_yuan_per_kwh
        load_kw
        pv_forecast_kw
    """
    path = config.attachment1_path
    _ensure_file(path)

    raw = pd.read_excel(
        path,
        sheet_name=config.attachment1_sheet,
        dtype=None,
    )

    required = [
        config.attachment1_time_col,
        config.attachment1_price_col,
        config.attachment1_load_col,
        config.attachment1_pv_forecast_col,
    ]
    missing = [c for c in required if c not in raw.columns]
    if missing:
        raise ValueError(f" 附件 1 缺少字段: {missing}")

    if len(raw) != config.intervals_per_day:
        raise ValueError(
            f" 附件 1 数据行数应为 144, 实际为 {len(raw)}。"
        )

    _validate_source_label_order(
        raw[config.attachment1_time_col].tolist(),
        config,
        " 附件 1",
    )

    time_map = build_time_map(config)
    aligned = time_map.copy()

    aligned["price_yuan_per_kwh"] = pd.to_numeric(
        raw[config.attachment1_price_col], errors="raise"
    ).to_numpy()
    aligned["load_kw"] = pd.to_numeric(
        raw[config.attachment1_load_col], errors="raise"
    ).to_numpy()
    aligned["pv_forecast_kw"] = pd.to_numeric(
        raw[config.attachment1_pv_forecast_col], errors="raise"
    ).to_numpy()

    return Attachment1Data(raw=raw, aligned=aligned)


def _load_daily_wide_sheet(
    path: Path,
    sheet_name: str,
    config: ProjectConfig,
    source_name: str,
```

```
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """
    读取“日期 + 144 个右端点时间列”的宽表。

    返回:
        raw: 原始宽表
        model_grid: index=date, columns=0..143 的模型网格
    """
    _ensure_file(path)

    raw = pd.read_excel(path, sheet_name=sheet_name, dtype=None)

    if raw.shape[1] != config.intervals_per_day + 1:
        raise ValueError(
            f"{source_name} 应有 145 列 (日期 +144 时段), 实际 {raw.shape[1]} 列。"
        )

    date_col = raw.columns[0]
    if str(date_col) != config.wide_date_col:
        # 不静默改表头, 但允许 Excel 读取后出现等价字符串差异。
        if str(date_col).strip() != config.wide_date_col.strip():
            raise ValueError(
                f"{source_name} 首列应为 {config.wide_date_col!r}, "
                f" 实际为 {date_col!r}。"
            )

    time_columns = list(raw.columns[1:])
    _validate_source_label_order(time_columns, config, source_name)

    dates = _parse_date_series(raw.iloc[:, 0])
    values = raw.iloc[:, 1:].copy()
    values = values.apply(pd.to_numeric, errors="raise")

    model_grid = values.copy()
    model_grid.index = dates
    model_grid.index.name = "date"
    model_grid.columns = list(range(config.intervals_per_day))

    return raw, model_grid


def load_attachment2(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> Attachment2Data:
    """
    读取附件 2 两个 sheet。

    输出 load_kw / pv_kw:
        index: 统一日期 Timestamp
        columns: k=0..143
        values: kW
    """
    load_raw, load_kw = _load_daily_wide_sheet(
        config.attachment2_path,
        config.attachment2_load_sheet,
        config,
        " 附件 2-小区负载",
    )

    pv_raw, pv_kw = _load_daily_wide_sheet(
        config.attachment2_path,
        config.attachment2_pv_sheet,
        config,
        " 附件 2-光伏发电实际功率",
    )

    if not load_kw.index.equals(pv_kw.index):
        raise ValueError(" 附件 2 负荷与光伏日期索引不一致。")

```

```
    return Attachment2Data(
        load_raw=load_raw,
        pv_raw=pv_raw,
        load_kw=load_kw,
        pv_kw=pv_kw,
    )


def _canonical_issue_time(value) -> str:
    """ 将附件 3 预报时刻统一成 H:MM。"""
    if pd.isna(value):
        raise ValueError(" 附件 3 发现空预报时刻。")

    if isinstance(value, time):
        return f"{value.hour}:{value.minute:02d}"

    if isinstance(value, (pd.Timestamp, datetime)):
        return f"{value.hour}:{value.minute:02d}"

    if isinstance(value, Real):
        minute = int(round(float(value) * 24 * 60)) % (24 * 60)
        h, m = divmod(minute, 60)
        return f"{h}:{m:02d}"

    s = str(value).strip()
    parts = s.split(":")
    if len(parts) >= 2:
        h = int(parts[0])
        m = int("".join(ch for ch in parts[1] if ch.isdigit())[:2] or 0)
        return f"{h}:{m:02d}"

    raise ValueError(f" 无法解析附件 3 预报时刻: {value!r}")


def _is_attachment3_blank_date_marker(value) -> bool:
    """
    附件 3 中 6/12/18 时的日期原始单元格可能是:
    - NaN / 空字符串
    - 某些读取链路下暴露为 shared-string 索引文本 '28'

    '28' 仅在非 0:00 行作为已知空日期标记处理。
    原始 DataFrame 不会被修改。
    """
    if pd.isna(value):
        return True

    s = str(value).strip()
    return s == "" or s == "28"


def _build_attachment3_release_index(
    raw: pd.DataFrame,
    config: ProjectConfig,
) -> pd.DataFrame:
    """
    仅建立附件 3 发布记录索引, 不改动 raw, 不生成预测目标时刻, 不插值。
    """
    date_col = config.attachment3_date_col
    time_col = config.attachment3_issue_time_col

    required = [date_col, time_col, *config.attachment3_forecast_columns]
    missing = [c for c in required if c not in raw.columns]
    if missing:
        raise ValueError(f" 附件 3 缺少字段: {missing}")

    records = []
    current_date: Optional[pd.Timestamp] = None

    for pos, (_, row) in enumerate(raw.iterrows()):
```

```
        issue_time = _canonical_issue_time(row[time_col])
        raw_date = row[date_col]

        if issue_time == "0:00":
            if _is_attachment3_blank_date_marker(raw_date):
                raise ValueError(
                    f" 附件 3 第 {pos + 2} 行是 0:00 发布记录, 但日期为空/异常。"
                )
            current_date = _parse_excel_date_value(raw_date)
        else:
            if not _is_attachment3_blank_date_marker(raw_date):
                # 若非 0 点行显式写了日期, 也按其实际日期解析并校验。
                explicit_date = _parse_excel_date_value(raw_date)
                if current_date is not None and explicit_date != current_date:
                    raise ValueError(
                        f" 附件 3 第 {pos + 2} 行日期与当前自然日不一致: "
                        f"{explicit_date} != {current_date}"
                    )
                current_date = explicit_date

        if current_date is None or pd.isna(current_date):
            raise ValueError(
                f" 附件 3 第 {pos + 2} 行无法确定所属自然日。"
            )

        hour = int(issue_time.split(":")[0])
        issue_datetime = current_date + pd.Timedelta(hours=hour)

        records.append(
            {
                "raw_row_position": pos,
                "issue_date": current_date,
                "issue_time": issue_time,
                "issue_hour": hour,
                "issue_datetime": issue_datetime,
            }
        )

    release_index = pd.DataFrame(records)
    release_index["issue_date"] = pd.to_datetime(
        release_index["issue_date"]
    ).dt.normalize()

    return release_index


def load_attachment3(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> Attachment3Data:
    """
    读取附件 3 并保留原始宽表结构。

    raw:
        原始 1460×26 数据区, 不做 ffill、不插值、不转成长表。

    release_index:
        仅提供每一原始行对应的自然日/发布时间, 用于后续解析和校验。
    """
    path = config.attachment3_path
    _ensure_file(path)

    raw = pd.read_excel(
        path,
        sheet_name=config.attachment3_sheet,
        dtype=object,
    )

    release_index = _build_attachment3_release_index(raw, config)

```

```
    return Attachment3Data(
        raw=raw,
        release_index=release_index,
    )


def load_attachment4(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> Attachment4Data:
    """
    读取附件 4。

    price:
        index: 统一日期 Timestamp
        columns: k=0..143
        values: 元/kWh

    注意: 这里只做数据对齐, 不代表未来价格在决策时可知。
    信息可用性由后续问题 4 模块控制。
    """
    raw, price = _load_daily_wide_sheet(
        config.attachment4_path,
        config.attachment4_sheet,
        config,
        " 附件 4-实时电价",
    )

    return Attachment4Data(raw=raw, price=price)


def load_all(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> DataBundle:
    """ 一次性读取附件 1-4, 供后续模块复用。"""
    time_map = build_time_map(config)

    return DataBundle(
        time_map=time_map,
        attachment1=load_attachment1(config),
        attachment2=load_attachment2(config),
        attachment3=load_attachment3(config),
        attachment4=load_attachment4(config),
    )
```

文件：code/data_checker.py

```python
# -*- coding: utf-8 -*-
"""
data_checker.py
C 题附件 1-4 公共数据审计模块。

本模块只做：
- 数据长度检查
- 日期连续性检查
- 缺失值检查
- 时间索引/右端点映射检查
- 附件 3 结构检查

不删除、不插值、不修补原始数据，不做预测、不做优化。
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Iterable, List

import pandas as pd

from config import DEFAULT_CONFIG, ProjectConfig
from data_loader import (
    Attachment1Data,
    Attachment2Data,
    Attachment3Data,
    Attachment4Data,
    DataBundle,
    load_all,
)
from time_index import validate_time_map


@dataclass
class CheckItem:
    category: str
    item: str
    passed: bool
    detail: str


def _item(
    category: str,
    item: str,
    passed: bool,
    detail: str,
) -> CheckItem:
    return CheckItem(category, item, bool(passed), detail)


def _expected_dates(config: ProjectConfig) -> pd.DatetimeIndex:
    return pd.date_range(
        config.start_date,
        config.end_date,
        freq="D",
    )


def _check_date_index(
    index: pd.Index,
    source_name: str,
    config: ProjectConfig,
) -> list[CheckItem]:
    items = []
```

```
66
67      idx = pd.DatetimeIndex(index).normalize()
68      expected = _expected_dates(config)
69
70      items.append(
71          _item(
72              " 日期",
73              f"{source_name}-日期数量",
74              len(idx) == len(expected),
75              f" 实际 {len(idx)} 日，期望 {len(expected)} 日。",
76          )
77      )
78
79      items.append(
80          _item(
81              " 日期",
82              f"{source_name}-日期唯一",
83              idx.is_unique,
84              f" 重复日期数 ={int(idx.duplicated().sum())}",
85          )
86      )
87
88      same = len(idx) == len(expected) and idx.equals(expected)
89      items.append(
90          _item(
91              " 日期",
92              f"{source_name}-2025 全年连续",
93              same,
94              (
95                  f" 实际范围 ={idx.min().date() if len(idx) else None}"
96                  f"~{idx.max().date() if len(idx) else None}; "
97                  f" 期望 ={config.start_date}~{config.end_date}"
98              ),
99          )
100     )
101
102     return items
103
104
105 def check_time_map(
106     bundle: DataBundle,
107     config: ProjectConfig = DEFAULT_CONFIG,
108 ) -> list[CheckItem]:
109     items = []
110     try:
111         validate_time_map(bundle.time_map, config)
112         passed = True
113         detail = "144 行映射及 k=0/36/60/143 关键断言全部通过。"
114     except Exception as exc:
115         passed = False
116         detail = repr(exc)
117
118     items.append(
119         _item(
120             " 时间索引",
121             " 唯一 time_map",
122             passed,
123             detail,
124         )
125     )
126
127     tm = bundle.time_map.set_index("k")
128
129     expected = {
130         0: ("0:00", "0:10", "0:10"),
131         36: ("6:00", "6:10", "6:10"),
132         60: ("10:00", "10:10", "10:10"),
133         143: ("23:50", "24:00", "0:00+1"),
134     }
```

```
135
136     for k, triple in expected.items():
137         got = (
138             tm.loc[k, "interval_start"],
139             tm.loc[k, "interval_end"],
140             tm.loc[k, "source_right_endpoint_label"],
141         )
142         items.append(
143             _item(
144                 " 时间索引",
145                 f"k={k} 关键映射",
146                 got == triple,
147                 f" 实际 ={got}; 期望 ={triple}",
148             )
149         )
150
151     template_ok = bundle.time_map["template_slot"].tolist() == list(
152         range(config.intervals_per_day)
153     )
154     items.append(
155         _item(
156             " 时间索引",
157             "template_slot 仅按顺序映射",
158             template_ok,
159             " 要求 template_slot[k]=k; 模板显示文字不参与 k 推断。",
160         )
161     )
162
163     return items
164
165
166 def check_attachment1(
167     data: Attachment1Data,
168     config: ProjectConfig = DEFAULT_CONFIG,
169 ) -> list[CheckItem]:
170     items = []
171
172     items.append(
173         _item(
174             " 附件 1",
175             " 数据行数",
176             len(data.raw) == 144,
177             f" 实际 {len(data.raw)} 行, 期望 144 行。",
178         )
179     )
180
181     numeric_cols = [
182         "price_yuan_per_kwh",
183         "load_kw",
184         "pv_forecast_kw",
185     ]
186
187     for col in numeric_cols:
188         n_missing = int(data.aligned[col].isna().sum())
189         items.append(
190             _item(
191                 " 附件 1",
192                 f"{col}-缺失值",
193                 n_missing == 0,
194                 f" 缺失值数量 ={n_missing}",
195             )
196         )
197
198     source_labels_ok = (
199         data.aligned["source_right_endpoint_label"].iloc[0] == "0:10"
200         and data.aligned["source_right_endpoint_label"].iloc[-1] == "0:00+1"
201     )
202     items.append(
203         _item(
```

```
            " 附件 1",
            " 右端点标签首尾",
            source_labels_ok,
            (
                f" 首 ={data.aligned['source_right_endpoint_label'].iloc[0]}, "
                f" 尾 ={data.aligned['source_right_endpoint_label'].iloc[-1]}"
            ),
        )
    )

    return items


def check_attachment2(
    data: Attachment2Data,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> list[CheckItem]:
    items = []

    for name, frame in [
        (" 负荷", data.load_kw),
        (" 光伏", data.pv_kw),
    ]:
        items.extend(_check_date_index(frame.index, f" 附件 2-{name}", config))

        shape_ok = frame.shape == (365, 144)
        items.append(
            _item(
                " 附件 2",
                f"{name}-模型网格维度",
                shape_ok,
                f" 实际 ={frame.shape}, 期望 =(365, 144)",
            )
        )

        n_missing = int(frame.isna().sum().sum())
        items.append(
            _item(
                " 附件 2",
                f"{name}-缺失值",
                n_missing == 0,
                f" 缺失值数量 ={n_missing}",
            )
        )

    same_index = data.load_kw.index.equals(data.pv_kw.index)
    items.append(
        _item(
            " 附件 2",
            " 负荷与光伏日期完全一致",
            same_index,
            " 两个 sheet 必须共享同一 2025 日期索引。",
        )
    )

    return items


def check_attachment3(
    data: Attachment3Data,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> list[CheckItem]:
    items = []

    raw = data.raw
    rel = data.release_index

    items.append(
        _item(
```

```
            " 附件 3",
            " 原始数据行数",
            len(raw) == config.expected_attachment3_rows,
            (
                f" 实际 {len(raw)} 行, "
                f" 期望 {config.expected_attachment3_rows}=365×4。"
            ),
        )
    )

    expected_cols = 2 + config.forecast_horizons
    items.append(
        _item(
            " 附件 3",
            " 原始列数",
            raw.shape[1] == expected_cols,
            f" 实际 {raw.shape[1]} 列, 期望 {expected_cols} 列。",
        )
    )

    # 日期列允许非 0 点行存在空/共享字符串空标记, 因此单独报告。
    date_col = config.attachment3_date_col
    raw_date_missing = int(raw[date_col].isna().sum())
    items.append(
        _item(
            " 附件 3",
            " 日期列原始空值仅报告不修补",
            True,
            (
                f"pandas 直接识别的空值数量 ={raw_date_missing}; "
                " 另外部分读取链路可能将共享字符串空标记显示为'28'。"
                "raw 保持原样, release_index 负责解释。"
            ),
        )
    )

    forecast_cols = list(config.attachment3_forecast_columns)
    forecast_missing = int(raw[forecast_cols].isna().sum().sum())
    items.append(
        _item(
            " 附件 3",
            "24 小时预测字段缺失值",
            forecast_missing == 0,
            f" 预测字段缺失总数 ={forecast_missing}",
        )
    )

    # 每天四个发布时间
    expected_issue_times = {"0:00", "6:00", "12:00", "18:00"}
    grouped = rel.groupby("issue_date")["issue_time"].agg(list)

    day_count_ok = len(grouped) == 365
    items.append(
        _item(
            " 附件 3",
            " 自然日数量",
            day_count_ok,
            f" 实际 {len(grouped)} 日, 期望 365 日。",
        )
    )

    bad_days = []
    for d, times in grouped.items():
        if len(times) != 4 or set(times) != expected_issue_times:
            bad_days.append((d, times))

    items.append(
        _item(
            " 附件 3",
```

```
            " 每天 0/6/12/18 四次发布",
            len(bad_days) == 0,
            (
                " 全部通过。"
                if not bad_days
                else f" 异常日期前 5 个 ={bad_days[:5]}"
            ),
        )
    )

    items.extend(_check_date_index(grouped.index, " 附件 3-发布日期", config))

    return items


def check_attachment4(
    data: Attachment4Data,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> list[CheckItem]:
    items = []

    items.extend(_check_date_index(data.price.index, " 附件 4-价格", config))

    items.append(
        _item(
            " 附件 4",
            " 模型网格维度",
            data.price.shape == (365, 144),
            f" 实际 ={data.price.shape}, 期望 =(365, 144)",
        )
    )

    n_missing = int(data.price.isna().sum().sum())
    items.append(
        _item(
            " 附件 4",
            " 价格缺失值",
            n_missing == 0,
            f" 缺失值数量 ={n_missing}",
        )
    )

    return items


def build_check_report(
    bundle: DataBundle,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.DataFrame:
    """ 返回完整审计报告 DataFrame。"""
    items: List[CheckItem] = []

    items.extend(check_time_map(bundle, config))
    items.extend(check_attachment1(bundle.attachment1, config))
    items.extend(check_attachment2(bundle.attachment2, config))
    items.extend(check_attachment3(bundle.attachment3, config))
    items.extend(check_attachment4(bundle.attachment4, config))

    report = pd.DataFrame(
        [
            {
                "category": x.category,
                "item": x.item,
                "passed": x.passed,
                "detail": x.detail,
            }
            for x in items
        ]
    )
```

```
411
412     return report
413
414
415 def assert_all_passed(report: pd.DataFrame) -> None:
416     """
417     若存在失败项，抛出异常。
418     说明性项目（例如附件 3 日期列原始空值报告）本身标记 passed=True。
419     """
420     failed = report.loc[~report["passed"]]
421     if not failed.empty:
422         text = "\n".join(
423             f"[{row.category}] {row.item}: {row.detail}"
424             for row in failed.itertuples(index=False)
425         )
426         raise AssertionError(" 数据审计未通过：\n" + text)
427
428
429 def run_all_checks(
430     config: ProjectConfig = DEFAULT_CONFIG,
431     raise_on_error: bool = True,
432 ) -> tuple[DataBundle, pd.DataFrame]:
433     """
434     读取附件 1-4 并执行全部公共检查。
435
436     返回：
437         bundle: 后续模块可直接复用的数据对象
438         report: 审计报告
439
440     不提供 __main__ 入口，避免在本阶段写主程序。
441     """
442     bundle = load_all(config)
443     report = build_check_report(bundle, config)
444
445     if raise_on_error:
446         assert_all_passed(report)
447
448     return bundle, report
```

文件：code/forecast_parser.py

```
# -*- coding: utf-8 -*-
"""
forecast_parser.py
问题 3 附件 3 解析与“整点预测 -> 10 分钟物理区间电量”转换。

严格沿用前文：
- Attachment3Data.raw 保留原始宽表，不修改；
- Attachment3Data.release_index 提供每条原始记录的 issue_datetime；
- 物理时间仍由 time_index.py 统一管理；
- 不根据结果模板文字推断模型 k。

附件 3 规则
---------
[FP-01]
    target_datetime = issue_datetime + lead_hour

[FP-02]
    F0 = 当前决策时刻已经观测到的光伏功率

[FP-03]
    对相邻整点锚点线性插值：
        P(t) = (1-a)F_h + a F_{h+1}

[FP-04]
    每个 10 分钟物理区间用梯形积分：
        G_m = (P_m + P_{m+1})/2 * (1/6)  kWh

重要：
- F0 必须由调用者传入，解析器绝不读取未来实际光伏；
- 预报 1 小时 ~ 预报 24 小时均先转成绝对 target_datetime；
- 2025-12-31 18:00 + 24h -> 2026-01-01 18:00，不能截断。
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

import numpy as np
import pandas as pd

from config import DEFAULT_CONFIG, ProjectConfig
from data_loader import Attachment3Data


@dataclass
class ParsedForecastVersion:
    """ 一个附件 3 发布版本的完整 24 小时 10 分钟化结果。"""

    issue_datetime: pd.Timestamp
    issue_date: pd.Timestamp
    issue_hour: int

    # 25 个小时锚点：F0 + F1..F24
    hourly_anchors: pd.DataFrame

    # 145 个 10 分钟边界功率
    boundary_power_kw: pd.DataFrame

    # 144 个 10 分钟物理区间
    interval_forecast: pd.DataFrame

    @property
    def forecast_kwh(self) -> pd.Series:
        series = self.interval_forecast["forecast_kwh"].copy()
```

```
        series.index = range(len(series))
        series.name = "forecast_kwh"
        return series


def _normalize_issue_datetime(value) -> pd.Timestamp:
    ts = pd.Timestamp(value)
    if ts.second != 0 or ts.microsecond != 0:
        raise ValueError("issue_datetime 必须对齐到整点发布时刻。")
    if ts.minute != 0:
        raise ValueError(" 附件 3 发布时间必须为整点。")
    if ts.hour not in {0, 6, 12, 18}:
        raise ValueError(" 附件 3 发布时间只能为 0/6/12/18 点。")
    return ts


def _find_release_row(
    attachment3: Attachment3Data,
    issue_datetime: pd.Timestamp,
) -> int:
    """
    通过 release_index 中的绝对 issue_datetime 定位原始行。
    不按“每 4 行一天”硬编码。
    """
    rel = attachment3.release_index

    matches = rel.loc[
        pd.to_datetime(rel["issue_datetime"]) == issue_datetime
    ]

    if len(matches) != 1:
        raise KeyError(
            f" 附件 3 中 issue_datetime={issue_datetime} 应且只能匹配 1 行, "
            f" 实际匹配 {len(matches)} 行。"
        )

    return int(matches.iloc[0]["raw_row_position"])


def parse_forecast_version(
    attachment3: Attachment3Data,
    issue_datetime,
    *,
    current_observed_pv_kw: float,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> ParsedForecastVersion:
    """
    解析附件 3 某一发布版本并转换成未来 24 小时 144 个 10 分钟区间电量。

    参数
    ----
    attachment3:
        data_loader.load_attachment3() 结果。
    issue_datetime:
        0/6/12/18 点的绝对发布时间。
    current_observed_pv_kw:
        [FP-02] 发布时刻已经观测到的当前光伏功率 F0。
        必须由外部因果运行层传入, 本函数不读取附件 2 未来值。
    """
    issue = _normalize_issue_datetime(issue_datetime)

    if pd.isna(current_observed_pv_kw):
        raise ValueError("current_observed_pv_kw 不能缺失。")
    if float(current_observed_pv_kw) < -1e-12:
        raise ValueError(" 当前观测光伏功率不能为负。")

    raw_position = _find_release_row(
        attachment3,
        issue,
```

```
    )
    row = attachment3.raw.iloc[raw_position]

    anchor_rows = [
        {
            "lead_hour": 0,
            "target_datetime": issue,
            "forecast_kw": max(
                0.0,
                float(current_observed_pv_kw),
            ),
            "source": "current_observation_F0",
        }
    ]

    # [FP-01] 显式构造绝对目标时间
    for h in range(1, config.forecast_horizons + 1):
        column = f" 预报 {h} 小时"
        if column not in attachment3.raw.columns:
            raise ValueError(f" 附件 3 缺少字段 {column}。")

        value = pd.to_numeric(
            pd.Series([row[column]]),
            errors="raise",
        ).iloc[0]

        if pd.isna(value):
            raise ValueError(
                f"{issue} 发布版本的 {column} 存在缺失值。"
            )

        value = float(value)
        if value < -1e-12:
            raise ValueError(
                f"{issue} 发布版本的 {column} 出现负光伏预测。"
            )

        target_datetime = issue + pd.Timedelta(hours=h)

        anchor_rows.append(
            {
                "lead_hour": h,
                "target_datetime": target_datetime,
                "forecast_kw": max(0.0, value),
                "source": column,
            }
        )

    anchors = pd.DataFrame(anchor_rows)

    expected_targets = pd.date_range(
        issue,
        issue + pd.Timedelta(hours=24),
        freq="1h",
    )
    if not pd.DatetimeIndex(
        anchors["target_datetime"]
    ).equals(expected_targets):
        raise AssertionError(
            " 附件 3 target_datetime 映射错误。"
        )

    # ------------------------------------------------------------------
    # [FP-03] 25 个整点锚点 -> 145 个 10 分钟边界功率
    # ------------------------------------------------------------------
    boundary_times = pd.date_range(
        issue,
        issue + pd.Timedelta(hours=24),
        freq="10min",
```

```
    )

    anchor_ns = pd.DatetimeIndex(
        anchors["target_datetime"]
    ).asi8.astype(float)
    boundary_ns = boundary_times.asi8.astype(float)

    anchor_power = anchors["forecast_kw"].to_numpy(
        dtype=float
    )

    interpolated = np.interp(
        boundary_ns,
        anchor_ns,
        anchor_power,
    )

    boundary_power = pd.DataFrame(
        {
            "boundary_datetime": boundary_times,
            "forecast_kw": interpolated,
        }
    )

    # ------------------------------------------------------------------
    # [FP-04] 145 边界功率 -> 144 物理区间梯形积分
    # ------------------------------------------------------------------
    interval_rows = []
    dt_hours = config.dt_minutes / 60.0

    for m in range(config.intervals_per_day):
        start_dt = boundary_times[m]
        end_dt = boundary_times[m + 1]
        p_start = float(interpolated[m])
        p_end = float(interpolated[m + 1])
        p_avg = 0.5 * (p_start + p_end)
        energy = p_avg * dt_hours

        interval_rows.append(
            {
                "m": m,
                "interval_start_datetime": start_dt,
                "interval_end_datetime": end_dt,
                "forecast_kw_start": p_start,
                "forecast_kw_end": p_end,
                "forecast_kw_avg": p_avg,
                "forecast_kwh": energy,
            }
        )

    intervals = pd.DataFrame(interval_rows)

    if len(intervals) != config.intervals_per_day:
        raise AssertionError(
            " 附件 3 每个发布版本必须转换成 144 个 10 分钟物理区间。"
        )

    return ParsedForecastVersion(
        issue_datetime=issue,
        issue_date=issue.normalize(),
        issue_hour=int(issue.hour),
        hourly_anchors=anchors,
        boundary_power_kw=boundary_power,
        interval_forecast=intervals,
    )


def extract_current_day_model_kwh(
    parsed: ParsedForecastVersion,
```

```
    model_date,
    *,
    start_k: int,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Series:
    """
    从一个 24 小时发布版本中抽取“当前自然日尚未执行区间”的光伏预测。

    输出 index 严格为:
        start_k,...,143

    定位依据是绝对物理 interval_start_datetime,
    不是附件 3 列号或 Excel 时间字符串。
    """
    d = pd.Timestamp(model_date).normalize()

    if not (0 <= start_k < config.intervals_per_day):
        raise ValueError("start_k 必须位于 0~143。")

    target_start = d + pd.Timedelta(
        minutes=start_k * config.dt_minutes
    )
    target_end = d + pd.Timedelta(days=1)

    frame = parsed.interval_forecast

    selected = frame.loc[
        (frame["interval_start_datetime"] >= target_start)
        & (frame["interval_end_datetime"] <= target_end)
    ].copy()

    expected_len = config.intervals_per_day - start_k
    if len(selected) != expected_len:
        raise ValueError(
            f" 当前发布版本无法完整覆盖当天 k={start_k}~143。"
            f" 实际 {len(selected)} 个区间, 期望 {expected_len}。"
        )

    result = pd.Series(
        selected["forecast_kwh"].to_numpy(dtype=float),
        index=list(
            range(start_k, config.intervals_per_day)
        ),
        name="pv_forecast_kwh",
        dtype=float,
    )

    return result


def extract_preview_after_midnight(
    parsed: ParsedForecastVersion,
    model_date,
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.DataFrame:
    """
    提取当前发布版本中跨过当天 24:00 之后的预览部分。

    该部分只用于 MPC 评价当前日末 SOC 的未来价值,
    不能写成次日正式购电计划。
    """
    d = pd.Timestamp(model_date).normalize()
    midnight_next = d + pd.Timedelta(days=1)

    frame = parsed.interval_forecast.copy()

    preview = frame.loc[
        frame["interval_start_datetime"] >= midnight_next
```

```
    ].copy()

    preview.reset_index(drop=True, inplace=True)
    return preview
```

文件：code/forecast_model.py

```python
# -*- coding: utf-8 -*-
"""
forecast_model.py
问题 2 日前负荷/光伏预测与严格因果历史残差库。

只实现问题 2 需要的预测功能，不包含：
- 问题 3 附件 3 预测更新；
- 问题 4 动态电价预测；
- 任何优化模型。

核心信息规则
------------
对目标日期 d 在 0:00 作预测时，只允许使用日期 s<d 的附件 2 实际数据。
禁止读取 d 当天未来实际负荷/光伏参与：
1. 当天 0:00 预测；
2. 当天 0:00 场景生成；
3. 历史残差模型参数计算。

时间接口
--------
附件 2 已经由 data_loader.py 按唯一 time_map 对齐为：
    index = date
    columns = k=0,...,143

本文件只使用 columns 中的模型 k，不解析 Excel 时间字符串。

主预测方法
----------
负荷：
    优先使用上周同日同物理时段值 (lag-7)；
    历史不足 7 天时使用此前可用日期同物理时段均值。

光伏：
    使用最近 7 个已实现历史日同物理时段均值。

这与前面已经确定的简单、可解释、严格因果基线保持一致。
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

import numpy as np
import pandas as pd

from config import DEFAULT_CONFIG, ProjectConfig
from data_loader import Attachment2Data
from time_index import build_time_map, validate_time_map


@dataclass
class AvailableHistory:
    """ 目标日 0:00 时真正可用的历史信息。"""

    target_date: pd.Timestamp
    load_kw: pd.DataFrame
    pv_kw: pd.DataFrame

    @property
    def history_days(self) -> int:
        return len(self.load_kw)

    @property
    def last_available_date(self) -> Optional[pd.Timestamp]:
```

```
        if self.history_days == 0:
            return None
        return pd.Timestamp(self.load_kw.index.max()).normalize()


@dataclass
class DayAheadForecast:
    """ 问题 2 某日 0:00 生成的 144 区间日前预测。"""

    target_date: pd.Timestamp

    # 功率, kW
    load_kw: pd.Series
    pv_kw: pd.Series

    # 10 min 区间电量, kWh
    load_kwh: pd.Series
    pv_kwh: pd.Series

    history_days: int
    history_end_date: Optional[pd.Timestamp]

    load_method: str
    pv_method: str


@dataclass
class ResidualLibrary:
    """
    严格因果历史残差库。

    每一历史日 s 的残差都来自:
        actual(s) - forecast(s | 仅使用 t<s 的数据)

    因此不存在用全年拟合后残差回填历史的未来泄露。
    """

    target_date: pd.Timestamp

    load_residual_kw: pd.DataFrame
    pv_residual_kw: pd.DataFrame

    load_residual_kwh: pd.DataFrame
    pv_residual_kwh: pd.DataFrame

    @property
    def dates(self) -> pd.DatetimeIndex:
        return pd.DatetimeIndex(self.load_residual_kw.index)

    @property
    def n_days(self) -> int:
        return len(self.load_residual_kw)


def _normalize_date(value) -> pd.Timestamp:
    return pd.Timestamp(value).normalize()


def _validate_attachment2(
    attachment2: Attachment2Data,
    config: ProjectConfig,
) -> None:
    if not isinstance(attachment2, Attachment2Data):
        raise TypeError(
            "attachment2 必须是 data_loader.load_attachment2() 返回的"
            "Attachment2Data。"
        )

    expected_columns = list(range(config.intervals_per_day))
```

```
135
136     for name, frame in [
137         ("load_kw", attachment2.load_kw),
138         ("pv_kw", attachment2.pv_kw),
139     ]:
140         if list(frame.columns) != expected_columns:
141             raise ValueError(
142                 f"attachment2.{name} 列必须严格为模型 k=0~143。"
143             )
144
145         if not isinstance(frame.index, pd.DatetimeIndex):
146             raise TypeError(
147                 f"attachment2.{name}.index 必须为 DatetimeIndex。"
148             )
149
150         if frame.index.has_duplicates:
151             raise ValueError(f"attachment2.{name} 存在重复日期。")
152
153         if frame.isna().any().any():
154             raise ValueError(f"attachment2.{name} 存在缺失值。")
155
156
157 def _validate_series_k(
158     series: pd.Series,
159     config: ProjectConfig,
160     name: str,
161 ) -> pd.Series:
162     expected = list(range(config.intervals_per_day))
163     if not isinstance(series, pd.Series):
164         raise TypeError(f"{name} 必须为 pandas.Series。")
165     if list(series.index) != expected:
166         raise ValueError(f"{name}.index 必须严格为模型 k=0~143。")
167
168     result = pd.to_numeric(series, errors="raise").astype(float)
169     if result.isna().any():
170         raise ValueError(f"{name} 存在缺失值。")
171     return result
172
173
174 def build_available_history(
175     attachment2: Attachment2Data,
176     target_date,
177     *,
178     config: ProjectConfig = DEFAULT_CONFIG,
179 ) -> AvailableHistory:
180     """
181     构造目标日 d 在 0:00 真正可用的附件 2 历史信息。
182
183     强制:
184         history.index < target_date
185
186     不允许把 target_date 当天任何真实值放入历史集。
187     """
188     _validate_attachment2(attachment2, config)
189
190     d = _normalize_date(target_date)
191
192     load_history = attachment2.load_kw.loc[
193         attachment2.load_kw.index < d
194     ].copy()
195     pv_history = attachment2.pv_kw.loc[
196         attachment2.pv_kw.index < d
197     ].copy()
198
199     if not load_history.index.equals(pv_history.index):
200         raise ValueError(" 历史负荷与光伏日期索引不一致。")
201
202     if len(load_history) > 0:
203         if not (load_history.index < d).all():
```

```
            raise AssertionError(" 历史信息中混入目标日或未来日期。")

    return AvailableHistory(
        target_date=d,
        load_kw=load_history,
        pv_kw=pv_history,
    )


def forecast_load_day_ahead(
    history: AvailableHistory,
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Series:
    """
    生成当天 144 个物理区间的负荷功率预测 (kW)。

    主规则:
    1. 若 d-7 已经存在, 使用上周同日同 k:
           P^L_hat[d,k] = P^L[d-7,k]
    2. 若历史不足 7 天, 则使用所有可用历史日同 k 均值;
    3. 若完全没有历史 (仅可能出现在 1 月 1 日暖启动), 返回全 0 基线。
       实际运行仍由当前信息下的因果紧急购电/储能补救保证可行,
       且 1 月份不保存正式结果。
    """
    d = history.target_date
    expected_k = list(range(config.intervals_per_day))

    if history.history_days == 0:
        return pd.Series(
            np.zeros(config.intervals_per_day),
            index=expected_k,
            dtype=float,
            name="load_forecast_kw",
        )

    lag7_date = d - pd.Timedelta(days=7)

    if lag7_date in history.load_kw.index:
        result = history.load_kw.loc[lag7_date].astype(float).copy()
    else:
        result = history.load_kw.mean(axis=0).astype(float)

    result.index = expected_k
    result = result.clip(lower=0.0)
    result.name = "load_forecast_kw"
    return result


def forecast_pv_day_ahead(
    history: AvailableHistory,
    *,
    recent_days: int = 7,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Series:
    """
    生成当天 144 个物理区间的光伏功率预测 (kW)。

    使用最近 min(7, 可用历史天数) 个历史日同 k 平均。
    完全无历史时返回全 0 暖启动基线。
    """
    expected_k = list(range(config.intervals_per_day))

    if history.history_days == 0:
        return pd.Series(
            np.zeros(config.intervals_per_day),
            index=expected_k,
            dtype=float,
            name="pv_forecast_kw",
```

```
        )

    if recent_days <= 0:
        raise ValueError("recent_days 必须为正整数。")

    recent = history.pv_kw.tail(min(recent_days, history.history_days))
    result = recent.mean(axis=0).astype(float)
    result.index = expected_k
    result = result.clip(lower=0.0)
    result.name = "pv_forecast_kw"
    return result


def build_q2_day_ahead_forecast(
    attachment2: Attachment2Data,
    target_date,
    *,
    pv_recent_days: int = 7,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> DayAheadForecast:
    """
    问题 2 统一日前预测入口。

    对 target_date 的预测严格只使用 target_date 之前的数据。
    输出 144 个模型物理区间预测，不读取 Excel 时间字符串。
    """
    time_map = build_time_map(config)
    validate_time_map(time_map, config)

    history = build_available_history(
        attachment2,
        target_date,
        config=config,
    )

    load_kw = forecast_load_day_ahead(
        history,
        config=config,
    )
    pv_kw = forecast_pv_day_ahead(
        history,
        recent_days=pv_recent_days,
        config=config,
    )

    dt_hours = config.dt_minutes / 60.0
    load_kwh = load_kw * dt_hours
    pv_kwh = pv_kw * dt_hours
    load_kwh.name = "load_forecast_kwh"
    pv_kwh.name = "pv_forecast_kwh"

    if history.last_available_date is not None:
        if not history.last_available_date < history.target_date:
            raise AssertionError(" 日前预测使用了目标日或未来数据。")

    return DayAheadForecast(
        target_date=history.target_date,
        load_kw=load_kw,
        pv_kw=pv_kw,
        load_kwh=load_kwh,
        pv_kwh=pv_kwh,
        history_days=history.history_days,
        history_end_date=history.last_available_date,
        load_method="lag7_else_expanding_same_slot_mean",
        pv_method=f"recent_{pv_recent_days}_days_same_slot_mean",
    )


def build_causal_residual_library(
```

```
    attachment2: Attachment2Data,
    target_date,
    *,
    max_residual_days: Optional[int] = 60,
    pv_recent_days: int = 7,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> ResidualLibrary:
    """
    构造 target_date 在 0:00 可用的历史样本外残差库。

    对每个历史日期 s<target_date:
        1. 用 s 之前的数据生成 forecast(s);
        2. 再用 actual(s)-forecast(s) 形成残差。

    这样每一条残差轨迹都是严格因果的，不存在全年度参数回看。

    参数
    ----
    max_residual_days:
         只保留最近若干历史残差日以控制场景生成规模。
        None 表示保留全部因果残差。
    """
    _validate_attachment2(attachment2, config)
    d = _normalize_date(target_date)

    candidate_dates = pd.DatetimeIndex(
        attachment2.load_kw.index[
            attachment2.load_kw.index < d
        ]
    )

    # 至少需要一个更早历史日才能形成真正的样本外残差;
    # 因此历史第一天不进入残差库。
    if len(candidate_dates) <= 1:
        empty = pd.DataFrame(
            columns=list(range(config.intervals_per_day)),
            dtype=float,
        )
        empty.index = pd.DatetimeIndex([], name="date")
        return ResidualLibrary(
            target_date=d,
            load_residual_kw=empty.copy(),
            pv_residual_kw=empty.copy(),
            load_residual_kwh=empty.copy(),
            pv_residual_kwh=empty.copy(),
        )

    residual_dates = candidate_dates[1:]

    if max_residual_days is not None:
        if max_residual_days <= 0:
            raise ValueError("max_residual_days 必须为正整数或 None。")
        residual_dates = residual_dates[-max_residual_days:]

    load_rows = []
    pv_rows = []
    kept_dates = []

    for s in residual_dates:
        # 关键: forecast(s) 内部只读取 t<s。
        forecast_s = build_q2_day_ahead_forecast(
            attachment2,
            s,
            pv_recent_days=pv_recent_days,
            config=config,
        )

        if forecast_s.history_end_date is not None:
            if not forecast_s.history_end_date < s:
```

```
                raise AssertionError(
                    f" 构造 {s.date()} 残差时发生未来信息泄露。"
                )

        actual_load = attachment2.load_kw.loc[s].astype(float)
        actual_pv = attachment2.pv_kw.loc[s].astype(float)

        load_rows.append(
            (actual_load - forecast_s.load_kw).to_numpy(dtype=float)
        )
        pv_rows.append(
            (actual_pv - forecast_s.pv_kw).to_numpy(dtype=float)
        )
        kept_dates.append(s)

    columns = list(range(config.intervals_per_day))
    index = pd.DatetimeIndex(kept_dates, name="date")

    load_residual_kw = pd.DataFrame(
        load_rows,
        index=index,
        columns=columns,
        dtype=float,
    )
    pv_residual_kw = pd.DataFrame(
        pv_rows,
        index=index,
        columns=columns,
        dtype=float,
    )

    dt_hours = config.dt_minutes / 60.0

    load_residual_kwh = load_residual_kw * dt_hours
    pv_residual_kwh = pv_residual_kw * dt_hours

    if len(index) > 0 and not (index < d).all():
        raise AssertionError(" 残差库混入目标日或未来日期。")

    return ResidualLibrary(
        target_date=d,
        load_residual_kw=load_residual_kw,
        pv_residual_kw=pv_residual_kw,
        load_residual_kwh=load_residual_kwh,
        pv_residual_kwh=pv_residual_kwh,
    )



def validate_observed_prefix(
    observed_load_kwh: pd.Series,
    observed_pv_kwh: pd.Series,
    current_k: int,
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> tuple[pd.Series, pd.Series]:
    """
    校验实时因果补救在区间 k 开始时可读取的真实前缀。

    严格时序:
        当前要决定物理区间
            I_k = [tau_k, tau_{k+1})
        的储能动作时, 附件 2 中第 k 个区间的完整实际负荷/光伏
        尚未实现。

    因此在 tau_k 只能使用:
        index = 0,...,k-1

    特别地:
```

```
        k=0  -> 真实前缀必须为空;
        k=36 -> 6:00 决策时只允许 0..35;
        k=72 -> 12:00 决策时只允许 0..71;
        k=108-> 18:00 决策时只允许 0..107。

    明确禁止把 current_k 及其之后的真实值传入当前储能决策。
    当前区间真实值只能在动作决定以后用于事后能量平衡、
    紧急购电、弃光和计划富余结算。
    """
    if not (0 <= current_k < config.intervals_per_day):
        raise ValueError("current_k 必须位于 0~143。")

    expected_index = list(range(current_k))

    for name, series in [
        ("observed_load_kwh", observed_load_kwh),
        ("observed_pv_kwh", observed_pv_kwh),
    ]:
        if not isinstance(series, pd.Series):
            raise TypeError(f"{name} 必须为 pandas.Series。")

        if list(series.index) != expected_index:
            raise ValueError(
                f"{name} 在决定 k={current_k} 动作前只能包含已完成区间, "
                f"index 必须严格为 0~{current_k - 1 if current_k > 0 else '空'}; "
                " 禁止包含 current_k 实际值。"
            )

        if series.isna().any():
            raise ValueError(f"{name} 存在缺失值。")

    return (
        observed_load_kwh.astype(float).copy(),
        observed_pv_kwh.astype(float).copy(),
    )
```

文件：code/scenario_tree.py

```python
# -*- coding: utf-8 -*-
"""
scenario_tree.py
问题 2 严格非预见场景生成、场景压缩与多阶段场景树。

本模块不创建 Gurobi 变量，只负责随机信息结构。

核心原则
--------
1. 历史残差按“整日联合轨迹”抽样，保留负荷/光伏误差的日内相关性；
2. 完整样本轨迹只用于生成树，不能直接对应一套从 0:00 就知道未来的动作；
3. 通过逐阶段共同节点保证：
       同一已实现信息节点 -> 同一当前动作
4. stage k 场景树分支只使用已经完成的 0..k-1 历史，不使用当前 k 实现；
5. 条件滚动场景只接收过去已完成真实前缀，不接受 current_k 及未来真实值。
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, Iterable, List, Optional, Sequence, Tuple

import numpy as np
import pandas as pd

from config import DEFAULT_CONFIG, ProjectConfig
from forecast_model import (
    DayAheadForecast,
    ResidualLibrary,
    validate_observed_prefix,
)


@dataclass
class ScenarioTrajectorySet:
    """
    一组完整/剩余时域联合轨迹。

    load_kwh / pv_kwh:
        index = scenario_id
        columns = 连续模型 k，例如 0~143 或当前 k~143

    probability:
        index = scenario_id，和为 1

    source_dates:
        每条轨迹来自哪一个历史残差日；无历史时可为 NaT/None。
    """

    load_kwh: pd.DataFrame
    pv_kwh: pd.DataFrame
    probability: pd.Series
    source_dates: pd.Series

    def validate(self) -> None:
        if not self.load_kwh.index.equals(self.pv_kwh.index):
            raise ValueError("负荷场景和光伏场景 scenario_id 不一致。")

        if not self.load_kwh.columns.equals(self.pv_kwh.columns):
            raise ValueError("负荷场景和光伏场景 k 列不一致。")

        if not self.load_kwh.index.equals(self.probability.index):
            raise ValueError("场景概率 index 与 scenario_id 不一致。")

        if not self.load_kwh.index.equals(self.source_dates.index):
```

```
            raise ValueError("source_dates index 与 scenario_id 不一致。")

        columns = [int(k) for k in self.load_kwh.columns]
        if not columns:
            raise ValueError(" 场景轨迹不能没有物理区间。")

        expected = list(range(columns[0], columns[-1] + 1))
        if columns != expected:
            raise ValueError(" 场景轨迹 k 必须为连续物理区间。")

        if self.load_kwh.isna().any().any():
            raise ValueError(" 负荷场景存在缺失值。")
        if self.pv_kwh.isna().any().any():
            raise ValueError(" 光伏场景存在缺失值。")

        if (self.load_kwh < -1e-12).any().any():
            raise ValueError(" 负荷场景存在负值。")
        if (self.pv_kwh < -1e-12).any().any():
            raise ValueError(" 光伏场景存在负值。")

        prob = self.probability.astype(float)
        if (prob < -1e-15).any():
            raise ValueError(" 场景概率不能为负。")

        total = float(prob.sum())
        if total <= 0:
            raise ValueError(" 场景概率和必须为正。")

        if abs(total - 1.0) > 1e-10:
            self.probability = prob / total

    @property
    def scenario_ids(self) -> tuple[int, ...]:
        return tuple(int(i) for i in self.load_kwh.index)

    @property
    def k_values(self) -> tuple[int, ...]:
        return tuple(int(k) for k in self.load_kwh.columns)

    @property
    def start_k(self) -> int:
        return self.k_values[0]

    @property
    def end_k(self) -> int:
        return self.k_values[-1]

    @property
    def n_scenarios(self) -> int:
        return len(self.load_kwh)


@dataclass(frozen=True)
class ScenarioTreeNode:
    """ 一个信息节点，而不是一条完整叶场景的私有动作节点。"""

    node_id: int
    k: int
    parent_id: Optional[int]

    probability: float
    load_kwh: float
    pv_kwh: float

    member_scenarios: Tuple[int, ...]


class NonanticipativeScenarioTree:
    """
```

```
    多阶段非预见场景树。

    同一 node_id 代表同一已实现信息历史。
    后续优化模型必须对每个 node_id 只创建一套:
        q_ch, q_dis, q_emg, q_curt, q_sur, E
    从结构上保证非预见性。
    """

    def __init__(
        self,
        nodes: Dict[int, ScenarioTreeNode],
        *,
        start_k: int,
        end_k: int,
    ) -> None:
        self.nodes = dict(nodes)
        self.start_k = int(start_k)
        self.end_k = int(end_k)

        stage_map: Dict[int, List[int]] = {
            k: [] for k in range(self.start_k, self.end_k + 1)
        }
        children_map: Dict[int, List[int]] = {
            node_id: [] for node_id in self.nodes
        }

        for node_id, node in self.nodes.items():
            stage_map[node.k].append(node_id)
            if node.parent_id is not None:
                children_map[node.parent_id].append(node_id)

        self._stage_map = {
            k: tuple(sorted(v))
            for k, v in stage_map.items()
        }
        self._children_map = {
            node_id: tuple(sorted(v))
            for node_id, v in children_map.items()
        }

    def stage_node_ids(self, k: int) -> tuple[int, ...]:
        return self._stage_map[int(k)]

    def root_node_ids(self) -> tuple[int, ...]:
        return self.stage_node_ids(self.start_k)

    def children(self, node_id: int) -> tuple[int, ...]:
        return self._children_map[int(node_id)]

    def leaf_node_ids(self) -> tuple[int, ...]:
        return self.stage_node_ids(self.end_k)

    def path_to_leaf(self, leaf_id: int) -> tuple[int, ...]:
        """ 返回从当前树根阶段到该叶节点的唯一 node 路径。"""
        path = []
        current = int(leaf_id)

        while True:
            node = self.nodes[current]
            path.append(current)
            if node.parent_id is None:
                break
            current = int(node.parent_id)

        path.reverse()
        return tuple(path)

    def to_frame(self) -> pd.DataFrame:
        rows = []
```

```
        for node_id in sorted(self.nodes):
            node = self.nodes[node_id]
            rows.append(
                {
                    "node_id": node.node_id,
                    "k": node.k,
                    "parent_id": node.parent_id,
                    "probability": node.probability,
                    "load_kwh": node.load_kwh,
                    "pv_kwh": node.pv_kwh,
                    "member_scenarios": node.member_scenarios,
                    "member_count": len(node.member_scenarios),
                }
            )
        return pd.DataFrame(rows)

    def validate_nonanticipativity(self, tol: float = 1e-9) -> pd.DataFrame:
        """
        检查场景树结构是否真正满足 “共同历史共享节点”。

        这里不检查 Gurobi 动作数值，而是检查：
        - 每个节点只有一个父节点；
        - 父节点一定来自上一物理区间；
        - 子节点的成员场景是父节点成员场景子集；
        - 每个 stage 节点概率和为 1；
        - 叶节点概率和为 1；
        - 树结构只需要 node-indexed 动作，不需要 leaf-indexed 提前动作。
        """
        rows = []

        def add(item: str, passed: bool, detail: str) -> None:
            rows.append(
                {
                    "item": item,
                    "passed": bool(passed),
                    "detail": detail,
                }
            )

        # stage 概率
        bad_stage = []
        for k in range(self.start_k, self.end_k + 1):
            total = sum(
                self.nodes[n].probability
                for n in self.stage_node_ids(k)
            )
            if abs(total - 1.0) > tol:
                bad_stage.append((k, total))

        add(
            "stage_probability_sum",
            len(bad_stage) == 0,
            " 全部 stage 概率和为 1。"
            if not bad_stage
            else f" 异常 stage 前 5 个 ={bad_stage[:5]}",
        )

        # 父子阶段
        bad_parent = []
        bad_subset = []
        for node_id, node in self.nodes.items():
            if node.k == self.start_k:
                if node.parent_id is not None:
                    bad_parent.append((node_id, node.parent_id))
                continue

            if node.parent_id is None:
                bad_parent.append((node_id, None))
                continue
```

```
            parent = self.nodes[node.parent_id]
            if parent.k != node.k - 1:
                bad_parent.append(
                    (node_id, node.parent_id, parent.k, node.k)
                )

            if not set(node.member_scenarios).issubset(
                parent.member_scenarios
            ):
                bad_subset.append((node_id, node.parent_id))

        add(
            "parent_previous_stage",
            len(bad_parent) == 0,
            " 所有非根节点父节点均位于 k-1。"
            if not bad_parent
            else f" 异常 ={bad_parent[:5]}",
        )

        add(
            "child_history_subset",
            len(bad_subset) == 0,
            " 所有子节点场景成员均为父节点成员子集。"
            if not bad_subset
            else f" 异常 ={bad_subset[:5]}",
        )

        leaf_prob = sum(
            self.nodes[n].probability
            for n in self.leaf_node_ids()
        )
        add(
            "leaf_probability_sum",
            abs(leaf_prob - 1.0) <= tol,
            f" 叶节点概率和 ={leaf_prob:.12g}",
        )

        add(
            "node_indexed_policy_structure",
            True,
            (
                " 树按信息节点 node_id 组织。优化器对每个 node_id 只需一套动作, "
                " 不存在按完整叶场景在 0:00 提前创建独立全天动作的必要。"
            ),
        )

        return pd.DataFrame(rows)


def _standardized_feature_matrix(
    load: np.ndarray,
    pv: np.ndarray,
) -> np.ndarray:
    """ 负荷/光伏联合标准化特征。"""
    load_scale = max(float(np.std(load)), 1e-9)
    pv_scale = max(float(np.std(pv)), 1e-9)

    return np.hstack(
        [
            load / load_scale,
            pv / pv_scale,
        ]
    )


def _pairwise_squared_distance(x: np.ndarray) -> np.ndarray:
    diff = x[:, None, :] - x[None, :, :]
    return np.einsum("ijk,ijk->ij", diff, diff)
```

```
342
343
344 def _kmedoids_labels(
345     x: np.ndarray,
346     n_clusters: int,
347     *,
348     max_iter: int = 30,
349 ) -> tuple[np.ndarray, np.ndarray]:
350     """
351     轻量确定性 k-medoids，用于场景压缩/节点分裂。
352
353     不依赖 sklearn，便于比赛环境复现。
354     """
355     n = x.shape[0]
356     if n == 0:
357         raise ValueError(" 不能对空样本聚类。")
358
359     n_clusters = max(1, min(int(n_clusters), n))
360
361     if n_clusters == 1:
362         return np.zeros(n, dtype=int), np.array([0], dtype=int)
363
364     # 如果所有轨迹完全相同，不做无意义分裂。
365     if np.allclose(x, x[0], rtol=0.0, atol=1e-12):
366         return np.zeros(n, dtype=int), np.array([0], dtype=int)
367
368     dist = _pairwise_squared_distance(x)
369
370     # farthest-first 确定性初始化
371     mean = x.mean(axis=0)
372     first = int(np.argmin(((x - mean) ** 2).sum(axis=1)))
373     medoids = [first]
374
375     while len(medoids) < n_clusters:
376         nearest = dist[:, medoids].min(axis=1)
377         nearest[medoids] = -1.0
378         candidate = int(np.argmax(nearest))
379         if candidate in medoids:
380             break
381         medoids.append(candidate)
382
383     medoids = np.array(medoids, dtype=int)
384
385     for _ in range(max_iter):
386         labels = np.argmin(dist[:, medoids], axis=1)
387         new_medoids = medoids.copy()
388
389         for c in range(len(medoids)):
390             members = np.where(labels == c)[0]
391             if len(members) == 0:
392                 continue
393             within = dist[np.ix_(members, members)]
394             cost = within.sum(axis=1)
395             new_medoids[c] = members[int(np.argmin(cost))]
396
397         if np.array_equal(new_medoids, medoids):
398             break
399
400         medoids = new_medoids
401
402     labels = np.argmin(dist[:, medoids], axis=1)
403     return labels, medoids
404
405
406 def build_bootstrap_trajectory_set(
407     forecast: DayAheadForecast,
408     residual_library: ResidualLibrary,
409     *,
410     n_bootstrap: int = 40,
```

```
    random_state: int = 2025,
) -> ScenarioTrajectorySet:
    """
    0:00 历史联合残差 Bootstrap。

    从同一历史日期同时抽取:
        load residual trajectory
        pv residual trajectory

    以保留二者历史联合关系。

    若历史残差库为空（仅用于年初暖启动），退化为单一路径点预测。
    """
    if n_bootstrap <= 0:
        raise ValueError("n_bootstrap 必须为正整数。")

    columns = list(forecast.load_kwh.index)

    if residual_library.n_days == 0:
        load = pd.DataFrame(
            [forecast.load_kwh.to_numpy(dtype=float)],
            index=[0],
            columns=columns,
        )
        pv = pd.DataFrame(
            [forecast.pv_kwh.to_numpy(dtype=float)],
            index=[0],
            columns=columns,
        )
        probability = pd.Series([1.0], index=[0], dtype=float)
        source_dates = pd.Series([pd.NaT], index=[0], dtype=object)

        result = ScenarioTrajectorySet(
            load_kwh=load,
            pv_kwh=pv,
            probability=probability,
            source_dates=source_dates,
        )
        result.validate()
        return result

    rng = np.random.default_rng(random_state)
    available_dates = residual_library.dates

    sampled_dates = rng.choice(
        available_dates.to_numpy(),
        size=n_bootstrap,
        replace=True,
    )

    load_rows = []
    pv_rows = []

    for s in sampled_dates:
        s = pd.Timestamp(s).normalize()

        load_path = (
            forecast.load_kwh
            + residual_library.load_residual_kwh.loc[s]
        ).clip(lower=0.0)

        pv_path = (
            forecast.pv_kwh
            + residual_library.pv_residual_kwh.loc[s]
        ).clip(lower=0.0)

        load_rows.append(load_path.to_numpy(dtype=float))
        pv_rows.append(pv_path.to_numpy(dtype=float))

```

```
    ids = list(range(n_bootstrap))

    result = ScenarioTrajectorySet(
        load_kwh=pd.DataFrame(
            load_rows,
            index=ids,
            columns=columns,
            dtype=float,
        ),
        pv_kwh=pd.DataFrame(
            pv_rows,
            index=ids,
            columns=columns,
            dtype=float,
        ),
        probability=pd.Series(
            np.full(n_bootstrap, 1.0 / n_bootstrap),
            index=ids,
            dtype=float,
        ),
        source_dates=pd.Series(
            [pd.Timestamp(x).normalize() for x in sampled_dates],
            index=ids,
            dtype=object,
        ),
    )
    result.validate()
    return result



def build_conditional_trajectory_set(
    forecast: DayAheadForecast,
    residual_library: ResidualLibrary,
    *,
    observed_load_kwh: pd.Series,
    observed_pv_kwh: pd.Series,
    current_k: int,
    n_bootstrap: int = 16,
    conditioning_pool: int = 12,
    random_state: int = 2025,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> ScenarioTrajectorySet:
    """
    构造区间 k 开始时的严格因果条件未来场景。

    信息边界
    --------
    在 tau_k 决定 I_k=[tau_k,tau_{k+1}) 的储能动作时:
        已知真实区间: 0,...,k-1
        未知真实区间: k,...,143

    因而本函数只接受 0..k-1 真实前缀。
    current_k 本身及其未来全部由:
        当前点预测 + 历史残差后缀
    生成, 绝不使用当天真实 current_k。

    k=0 时没有当日真实前缀, 直接从全部可用历史残差日抽样。
    """
    observed_load, observed_pv = validate_observed_prefix(
        observed_load_kwh,
        observed_pv_kwh,
        current_k,
        config=config,
    )

    if n_bootstrap <= 0:
        raise ValueError("n_bootstrap 必须为正整数。")
    if conditioning_pool <= 0:
```

```
549             raise ValueError("conditioning_pool 必须为正整数。")
550
551         future_columns = list(
552             range(current_k, config.intervals_per_day)
553         )
554
555         # 年初无残差历史：退化为当前点预测路径。
556         # 注意：绝不使用 current_k 真实值覆盖预测。
557         if residual_library.n_days == 0:
558             load_path = forecast.load_kwh.loc[future_columns].copy()
559             pv_path = forecast.pv_kwh.loc[future_columns].copy()
560
561             result = ScenarioTrajectorySet(
562                 load_kwh=pd.DataFrame(
563                     [load_path.to_numpy(dtype=float)],
564                     index=[0],
565                     columns=future_columns,
566                 ),
567                 pv_kwh=pd.DataFrame(
568                     [pv_path.to_numpy(dtype=float)],
569                     index=[0],
570                     columns=future_columns,
571                 ),
572                 probability=pd.Series([1.0], index=[0], dtype=float),
573                 source_dates=pd.Series([pd.NaT], index=[0], dtype=object),
574             )
575             result.validate()
576             return result
577
578         # 已经完成的真实区间，仅 0..k-1。
579         prefix = list(range(current_k))
580
581         if current_k == 0:
582             # 当天尚无真实信息，所有历史残差日均可作为候选。
583             selected_dates = residual_library.dates
584         else:
585             observed_load_resid = (
586                 observed_load
587                 - forecast.load_kwh.loc[prefix]
588             )
589             observed_pv_resid = (
590                 observed_pv
591                 - forecast.pv_kwh.loc[prefix]
592             )
593
594             hist_load = residual_library.load_residual_kwh.loc[:, prefix]
595             hist_pv = residual_library.pv_residual_kwh.loc[:, prefix]
596
597             load_scale = max(
598                 float(hist_load.to_numpy(dtype=float).std()),
599                 1e-9,
600             )
601             pv_scale = max(
602                 float(hist_pv.to_numpy(dtype=float).std()),
603                 1e-9,
604             )
605
606             load_diff = (
607                 hist_load.to_numpy(dtype=float)
608                 - observed_load_resid.to_numpy(dtype=float)[None, :]
609             ) / load_scale
610
611             pv_diff = (
612                 hist_pv.to_numpy(dtype=float)
613                 - observed_pv_resid.to_numpy(dtype=float)[None, :]
614             ) / pv_scale
615
616             distance = np.sqrt(
617                 np.mean(load_diff**2, axis=1)
```

```
            + np.mean(pv_diff**2, axis=1)
        )

        order = np.argsort(distance)
        pool_size = min(conditioning_pool, len(order))
        selected_dates = residual_library.dates[order[:pool_size]]

    if len(selected_dates) == 0:
        raise ValueError(" 条件场景没有可用历史残差日。")

    rng = np.random.default_rng(random_state)
    sampled_dates = rng.choice(
        selected_dates.to_numpy(),
        size=n_bootstrap,
        replace=True,
    )

    load_rows = []
    pv_rows = []

    for s in sampled_dates:
        s = pd.Timestamp(s).normalize()

        # current_k 及以后全部来自预测 + 历史残差，绝不覆盖成当天真实值。
        load_path = (
            forecast.load_kwh.loc[future_columns]
            + residual_library.load_residual_kwh.loc[
                s, future_columns
            ]
        ).clip(lower=0.0)

        pv_path = (
            forecast.pv_kwh.loc[future_columns]
            + residual_library.pv_residual_kwh.loc[
                s, future_columns
            ]
        ).clip(lower=0.0)

        load_rows.append(load_path.to_numpy(dtype=float))
        pv_rows.append(pv_path.to_numpy(dtype=float))

    ids = list(range(n_bootstrap))

    result = ScenarioTrajectorySet(
        load_kwh=pd.DataFrame(
            load_rows,
            index=ids,
            columns=future_columns,
            dtype=float,
        ),
        pv_kwh=pd.DataFrame(
            pv_rows,
            index=ids,
            columns=future_columns,
            dtype=float,
        ),
        probability=pd.Series(
            np.full(n_bootstrap, 1.0 / n_bootstrap),
            index=ids,
            dtype=float,
        ),
        source_dates=pd.Series(
            [pd.Timestamp(x).normalize() for x in sampled_dates],
            index=ids,
            dtype=object,
        ),
    )
    result.validate()
    return result
```

```python


def reduce_trajectory_set(
    trajectories: ScenarioTrajectorySet,
    *,
    n_representatives: int = 10,
) -> ScenarioTrajectorySet:
    """
    通过联合负荷/光伏轨迹 k-medoids 压缩场景。

    每个代表场景概率 = 原场景簇概率和。
    代表轨迹取真实 medoid 轨迹，不人工平均出不存在的整日路径。
    """
    trajectories.validate()

    n = trajectories.n_scenarios
    if n_representatives <= 0:
        raise ValueError("n_representatives 必须为正整数。")

    if n <= n_representatives:
        result = ScenarioTrajectorySet(
            load_kwh=trajectories.load_kwh.copy(),
            pv_kwh=trajectories.pv_kwh.copy(),
            probability=(
                trajectories.probability
                / trajectories.probability.sum()
            ).copy(),
            source_dates=trajectories.source_dates.copy(),
        )
        result.validate()
        return result

    load = trajectories.load_kwh.to_numpy(dtype=float)
    pv = trajectories.pv_kwh.to_numpy(dtype=float)

    features = _standardized_feature_matrix(load, pv)
    labels, medoid_positions = _kmedoids_labels(
        features,
        n_representatives,
    )

    rows_load = []
    rows_pv = []
    probs = []
    sources = []

    old_ids = list(trajectories.load_kwh.index)

    for cluster_id, medoid_pos in enumerate(medoid_positions):
        members_pos = np.where(labels == cluster_id)[0]
        if len(members_pos) == 0:
            continue

        medoid_old_id = old_ids[int(medoid_pos)]

        rows_load.append(
            trajectories.load_kwh.loc[
                medoid_old_id
            ].to_numpy(dtype=float)
        )
        rows_pv.append(
            trajectories.pv_kwh.loc[
                medoid_old_id
            ].to_numpy(dtype=float)
        )

        member_ids = [old_ids[int(i)] for i in members_pos]
        probs.append(
            float(
```

```
                trajectories.probability.loc[member_ids].sum()
            )
        )

        source_values = tuple(
            trajectories.source_dates.loc[member_ids].tolist()
        )
        sources.append(source_values)

    new_ids = list(range(len(rows_load)))

    result = ScenarioTrajectorySet(
        load_kwh=pd.DataFrame(
            rows_load,
            index=new_ids,
            columns=trajectories.load_kwh.columns,
            dtype=float,
        ),
        pv_kwh=pd.DataFrame(
            rows_pv,
            index=new_ids,
            columns=trajectories.pv_kwh.columns,
            dtype=float,
        ),
        probability=pd.Series(
            probs,
            index=new_ids,
            dtype=float,
        ),
        source_dates=pd.Series(
            sources,
            index=new_ids,
            dtype=object,
        ),
    )

    result.probability = (
        result.probability / result.probability.sum()
    )
    result.validate()
    return result


def _split_members(
    trajectories: ScenarioTrajectorySet,
    members: Sequence[int],
    *,
    observed_columns: Sequence[int],
    max_branch: int,
) -> list[tuple[int, ...]]:
    """
    仅利用截至当前阶段已经“可观察”的列对父节点成员进行分裂。

    不允许用当前阶段之后的未来列决定当前节点身份。
    """
    members = tuple(int(x) for x in members)

    if len(members) <= 1 or max_branch <= 1:
        return [members]

    n_clusters = min(max_branch, len(members))

    load = trajectories.load_kwh.loc[
        list(members), list(observed_columns)
    ].to_numpy(dtype=float)
    pv = trajectories.pv_kwh.loc[
        list(members), list(observed_columns)
    ].to_numpy(dtype=float)

```

```
    features = _standardized_feature_matrix(load, pv)

    labels, medoids = _kmedoids_labels(
        features,
        n_clusters,
    )

    # _kmedoids_labels 可能因完全相同样本只返回 1 类。
    unique_labels = sorted(set(int(x) for x in labels))
    groups = []

    for c in unique_labels:
        group = tuple(
            members[i]
            for i in range(len(members))
            if int(labels[i]) == c
        )
        if group:
            groups.append(group)

    return groups



def build_nonanticipative_tree(
    trajectories: ScenarioTrajectorySet,
    *,
    branch_every: int = 6,
    max_branch: int = 3,
    force_single_root: bool = False,
) -> NonanticipativeScenarioTree:
    """
    将压缩联合轨迹转化为严格“决策先于当前区间实现”的非预见场景树。

    时序定义
    --------
    stage k 的节点动作发生在 tau_k。
    因此 stage k 最多只能依赖已经完成的:
        start_k,...,k-1
    不能使用第 k 个区间自身的负荷/PV 实现来区分当前动作。

    于是:
    - start_k 阶段永远只有一个根信息节点;
    - 若在 stage k 发生分支，聚类特征只使用
          observed_columns = start_k,...,k-1;
    - 从不使用 column k 或更未来的数据决定 stage k 节点身份。

    参数
    ----
    force_single_root:
        保留该参数以兼容前文接口。
        在严格时序下根节点无论如何都必须唯一。
        当其为 True 时，允许在 start_k+1 立即根据刚完成的 start_k 区间
        进行第一次分支；否则按 branch_every 控制首次分支。
    """
    trajectories.validate()

    if branch_every <= 0:
        raise ValueError("branch_every 必须为正整数。")
    if max_branch <= 0:
        raise ValueError("max_branch 必须为正整数。")

    start_k = trajectories.start_k
    end_k = trajectories.end_k
    all_members = tuple(trajectories.scenario_ids)

    nodes: Dict[int, ScenarioTreeNode] = {}
    next_node_id = 0

```

```
    def add_node(
        k: int,
        parent_id: Optional[int],
        members: tuple[int, ...],
    ) -> int:
        nonlocal next_node_id

        member_list = list(members)
        weights = trajectories.probability.loc[
            member_list
        ].to_numpy(dtype=float)

        probability = float(weights.sum())
        normalized = weights / probability

        # 当前 stage 尚未观测第 k 区间，因此节点中的 L/G 是该信息集下
        # 对 current interval 的条件均值，而不是当天真实值。
        load_values = trajectories.load_kwh.loc[
            member_list, k
        ].to_numpy(dtype=float)
        pv_values = trajectories.pv_kwh.loc[
            member_list, k
        ].to_numpy(dtype=float)

        load_kwh = float(np.dot(normalized, load_values))
        pv_kwh = float(np.dot(normalized, pv_values))

        node_id = next_node_id
        next_node_id += 1

        nodes[node_id] = ScenarioTreeNode(
            node_id=node_id,
            k=k,
            parent_id=parent_id,
            probability=probability,
            load_kwh=load_kwh,
            pv_kwh=pv_kwh,
            member_scenarios=tuple(sorted(members)),
        )

        return node_id

    # 严格非预见：当前 start_k 尚无当前区间实现，因此根节点唯一。
    current_groups = [all_members]
    group_to_node: Dict[tuple[int, ...], int] = {}

    root_id = add_node(start_k, None, all_members)
    group_to_node[tuple(sorted(all_members))] = root_id

    last_branch_k = start_k

    for k in range(start_k + 1, end_k + 1):
        if force_single_root and k == start_k + 1:
            should_branch = True
        else:
            should_branch = (k - last_branch_k) >= branch_every

        next_groups: list[tuple[int, ...]] = []

        if should_branch:
            # stage k 动作前只看已经完成到 k-1 的真实历史。
            observed_columns = list(range(start_k, k))

            for group in current_groups:
                split = _split_members(
                    trajectories,
                    group,
                    observed_columns=observed_columns,
                    max_branch=max_branch,
```

```
                )
                next_groups.extend(split)

            last_branch_k = k
        else:
            next_groups = list(current_groups)

        next_group_to_node: Dict[tuple[int, ...], int] = {}

        for group in next_groups:
            group_key = tuple(sorted(group))

            parent_group = None
            for candidate in current_groups:
                if set(group_key).issubset(candidate):
                    parent_group = tuple(sorted(candidate))
                    break

            if parent_group is None:
                raise AssertionError(" 无法为场景树子节点找到父信息节点。")

            parent_id = group_to_node[parent_group]
            node_id = add_node(k, parent_id, group_key)
            next_group_to_node[group_key] = node_id

        current_groups = next_groups
        group_to_node = next_group_to_node

    tree = NonanticipativeScenarioTree(
        nodes,
        start_k=start_k,
        end_k=end_k,
    )

    report = tree.validate_nonanticipativity()

    if len(tree.root_node_ids()) != 1:
        raise AssertionError(
            " 严格决策时序要求当前 stage 只能有一个根信息节点。"
        )

    if not report["passed"].all():
        failed = report.loc[~report["passed"]]
        raise AssertionError(
            " 生成的场景树未通过非预见性检查: \n"
            + failed.to_string(index=False)
        )

    return tree
```

文件：code/model_base.py

```
# -*- coding: utf-8 -*-
"""
model_base.py
C 题微网公共物理模型层 (Q1-Q4 复用)。

职责:
1. 只封装所有问题共同使用的物理变量与约束;
2. 严格使用 time_index.py 中已经确认的 k=0,...,143;
3. 能量平衡统一使用 kWh;
4. 默认采用连续 LP 变量;
5. 充放电互斥二进制变量仅作为可选开关，默认关闭;
6. 不建立任何具体问题的目标函数;
7. 不读取 Excel，不根据 Excel 表头或结果模板文字重新定位时间。

统一决策变量:
    q_buy[k]   外网正常购电量，kWh
    q_ch[k]    母线侧储能充电量，kWh
    q_dis[k]   母线侧储能放电量，kWh
    q_curt[k]  弃光量，kWh
    E[j]       储能内部电量，kWh，j=0,...,144
    z[k]       可选充放电互斥二进制变量; 默认不创建

具体问题自行增加:
    q_emg      紧急购电量 (Q2-Q4)
    q_sur      已计划但无法利用的外网电量富余 (Q2-Q4)
    初始/终端 SOC 约束
    跨日 SOC
    场景树非预见性
    MPC 冻结约束
    各问题目标函数

数学公式编号:
    [MB-01] 10 min 功率-电量换算
    [MB-02] 微网能量平衡
    [MB-03] 弃光约束
    [MB-04] 储能状态转移
    [MB-05] 储能电量上下界
    [MB-06] 单时段最大充放电量
    [MB-07] 可选充放电互斥
    [MB-08] 外网购电非负
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Mapping, Optional

import pandas as pd

try:
    import gurobipy as gp
    from gurobipy import GRB
except ImportError as exc:  # pragma: no cover
    raise ImportError(
        "model_base.py 依赖 gurobipy。请在已安装并配置 Gurobi 的环境中运行。"
    ) from exc

from config import DEFAULT_CONFIG, ProjectConfig
from time_index import build_time_map, validate_time_map


# ============================================================================
# 已确定的统一储能参数
# ============================================================================

```

```
ETA_C = 0.90
ETA_D = 0.90
E_MIN_KWH = 1200.0
E_MAX_KWH = 10800.0
P_BAT_MAX_KW = 5000.0


@dataclass
class BaseVariables:
    """
    公共决策变量容器。

    变量名严格沿用已经确定的符号体系:
        q_buy, q_ch, q_dis, q_curt, E, z
    """

    q_buy: gp.tupledict
    q_ch: gp.tupledict
    q_dis: gp.tupledict
    q_curt: gp.tupledict
    E: gp.tupledict
    z: Optional[gp.tupledict] = None


@dataclass
class BaseConstraintRefs:
    """ 公共约束引用，便于后续调试、IIS 分析和结果审计。"""

    energy_balance: Optional[gp.tupledict] = None
    curtailment_upper: Optional[gp.tupledict] = None
    storage_transition: Optional[gp.tupledict] = None
    charge_exclusivity: Optional[gp.tupledict] = None
    discharge_exclusivity: Optional[gp.tupledict] = None


def _validate_k_series(
    values: pd.Series,
    *,
    config: ProjectConfig,
    name: str,
    require_nonnegative: bool,
) -> pd.Series:
    """
    强制检查物理区间数据接口。

    只接受 pandas.Series，且 index 必须严格按顺序为:
        k = 0,1,...,143

    禁止通过 Excel 原始表头、paper_label 或 template_display_label 重新推断 k。
    """
    if not isinstance(values, pd.Series):
        raise TypeError(
            f"{name} 必须是 pandas.Series，且 index 严格为"
            f"k=0,...,{config.intervals_per_day - 1}。"
        )

    if len(values) != config.intervals_per_day:
        raise ValueError(
            f"{name} 长度必须为 {config.intervals_per_day}，实际为 {len(values)}。"
        )

    try:
        actual_index = [int(x) for x in values.index]
    except Exception as exc:
        raise ValueError(f"{name} 的 index 无法解释为模型 k。") from exc

    expected_index = list(range(config.intervals_per_day))
    if actual_index != expected_index:
        raise ValueError(
```

```
            f"{name} 的 index 必须严格为 0~{config.intervals_per_day - 1}, "
            " 不得根据 Excel 表头或结果模板显示文字重新定位。"
        )

    numeric = pd.to_numeric(values, errors="raise").astype(float).copy()
    numeric.index = expected_index

    if numeric.isna().any():
        bad = numeric[numeric.isna()].index.tolist()
        raise ValueError(f"{name} 存在缺失值, k={bad[:10]}。")

    if require_nonnegative and (numeric < 0).any():
        bad = numeric[numeric < 0]
        preview = {int(k): float(v) for k, v in bad.head(10).items()}
        raise ValueError(f"{name} 应非负, 发现负值: {preview}")

    return numeric


def kw_to_interval_kwh(
    power_kw: pd.Series,
    config: ProjectConfig = DEFAULT_CONFIG,
    *,
    name: str = "power_kw",
) -> pd.Series:
    """
    将按模型 k 索引的功率 (kW) 转换为单个 10 min 物理区间电量 (kWh)。

    [MB-01]
        q_k = P_k * Δt
        Δt = 10/60 = 1/6 h

    参数
    ----
    power_kw:
        pandas.Series, index 必须严格为 k=0,...,143, 单位 kW。

    返回
    ----
    pandas.Series
        相同 k 索引, 单位 kWh。
    """
    series = _validate_k_series(
        power_kw,
        config=config,
        name=name,
        require_nonnegative=False,
    )
    dt_hours = config.dt_minutes / 60.0
    result = series * dt_hours
    result.name = f"{name}_kwh"
    return result


class MicrogridBaseModel:
    """
    Q1-Q4 共同物理模型构建器。

    约束层只使用 time_map["k"] 作为优化时段索引。
    source_right_endpoint_label、paper_label、template_display_label
    仅用于解释和输出, 不参与变量定位。

    默认:
        enforce_charge_discharge_exclusivity=False

    因此默认公共模型全部为连续变量, 可用于 LP。
    """

    def __init__(
```

```
        self,
        model: gp.Model,
        *,
        config: ProjectConfig = DEFAULT_CONFIG,
        time_map: Optional[pd.DataFrame] = None,
        enforce_charge_discharge_exclusivity: bool = False,
        variable_name_prefix: str = "",
    ) -> None:
        self.model = model
        self.config = config
        self.enforce_charge_discharge_exclusivity = (
            enforce_charge_discharge_exclusivity
        )
        self.variable_name_prefix = variable_name_prefix

        # 唯一时间索引来源：time_index.py。
        if time_map is None:
            self.time_map = build_time_map(config)
        else:
            self.time_map = time_map.copy(deep=True)

        validate_time_map(self.time_map, config)

        # 优化模型只读取 time_map 中的 k，不读取任何 Excel 时间字符串。
        self.k = tuple(int(x) for x in self.time_map["k"].tolist())
        self.j = tuple(range(config.state_nodes_per_day))

        expected_k = tuple(range(config.intervals_per_day))
        expected_j = tuple(range(config.intervals_per_day + 1))

        if self.k != expected_k:
            raise ValueError(
                "model_base 只接受 time_index.py 定义的 k=0,...,143。"
            )

        if self.j != expected_j:
            raise ValueError(
                " 储能状态节点必须严格为 j=0,...,144。"
            )

        self.dt_hours = config.dt_minutes / 60.0

        # [MB-06] q_bat^max = P_bat^max * Δt
        self.q_bat_max_kwh = P_BAT_MAX_KW * self.dt_hours

        self.vars = self._create_common_variables()
        self.constraints = BaseConstraintRefs()

        # [MB-07] 仅显式开启时才创建并加入互斥二进制约束。
        if self.enforce_charge_discharge_exclusivity:
            self.add_charge_discharge_exclusivity()

    def _vname(self, base: str) -> str:
        """ 生成 Gurobi 变量名，不改变数学变量体系。"""
        if not self.variable_name_prefix:
            return base
        return f"{self.variable_name_prefix}{base}"

    def _cname(self, base: str, k: Optional[int] = None) -> str:
        """ 生成 Gurobi 约束名。"""
        prefix = self.variable_name_prefix
        if k is None:
            return f"{prefix}{base}"
        return f"{prefix}{base}[{k}]"

    def _create_common_variables(self) -> BaseVariables:
        """
        创建所有问题共有变量。

```

```
        [MB-05]
            1200 <= E_j <= 10800  kWh

        [MB-06]
            0 <= q_ch,k <= P_bat^max*Δt
            0 <= q_dis,k <= P_bat^max*Δt

        [MB-08]
            q_buy,k >= 0

        上述约束使用 Gurobi 变量上下界直接实现。
        """

        # [MB-08] 外网正常购电非负。
        q_buy = self.model.addVars(
            self.k,
            lb=0.0,
            vtype=GRB.CONTINUOUS,
            name=self._vname("q_buy"),
        )

        # [MB-06] 母线侧充电量，单位 kWh。
        q_ch = self.model.addVars(
            self.k,
            lb=0.0,
            ub=self.q_bat_max_kwh,
            vtype=GRB.CONTINUOUS,
            name=self._vname("q_ch"),
        )

        # [MB-06] 母线侧放电量，单位 kWh。
        q_dis = self.model.addVars(
            self.k,
            lb=0.0,
            ub=self.q_bat_max_kwh,
            vtype=GRB.CONTINUOUS,
            name=self._vname("q_dis"),
        )

        # [MB-03] q_curt 下界 0 在变量层设置；
        # 上界 q_curt<=G_k 在拿到 pv_kwh 后添加。
        q_curt = self.model.addVars(
            self.k,
            lb=0.0,
            vtype=GRB.CONTINUOUS,
            name=self._vname("q_curt"),
        )

        # [MB-05] 储能内部实际电量，单位 kWh。
        E = self.model.addVars(
            self.j,
            lb=E_MIN_KWH,
            ub=E_MAX_KWH,
            vtype=GRB.CONTINUOUS,
            name=self._vname("E"),
        )

        # [MB-07] 默认不创建二进制变量。
        z = None
        if self.enforce_charge_discharge_exclusivity:
            z = self.model.addVars(
                self.k,
                vtype=GRB.BINARY,
                name=self._vname("z"),
            )

        return BaseVariables(
            q_buy=q_buy,
            q_ch=q_ch,
```

```
            q_dis=q_dis,
            q_curt=q_curt,
            E=E,
            z=z,
        )

    def validate_energy_inputs(
        self,
        *,
        load_kwh: pd.Series,
        pv_kwh: pd.Series,
    ) -> tuple[pd.Series, pd.Series]:
        """
        检查能量平衡输入。

        load_kwh、pv_kwh 必须已经是每个 10 min 物理区间的 kWh,
        且索引严格为 time_index.py 定义的 k=0,...,143。
        """
        L = _validate_k_series(
            load_kwh,
            config=self.config,
            name="load_kwh",
            require_nonnegative=True,
        )
        G = _validate_k_series(
            pv_kwh,
            config=self.config,
            name="pv_kwh",
            require_nonnegative=True,
        )
        return L, G

    @staticmethod
    def _optional_var_at(
        mapping: Optional[Mapping[int, gp.Var]],
        k: int,
        name: str,
    ):
        """
        读取具体问题可选变量。

        Q1:
            q_emg=None, q_sur=None

        Q2-Q4:
            q_emg/q_sur 由对应求解器创建后传入。
        """
        if mapping is None:
            return 0.0

        if k not in mapping:
            raise KeyError(f"{name} 缺少模型索引 k={k}。")

        return mapping[k]

    def add_energy_balance(
        self,
        *,
        load_kwh: pd.Series,
        pv_kwh: pd.Series,
        q_emg: Optional[Mapping[int, gp.Var]] = None,
        q_sur: Optional[Mapping[int, gp.Var]] = None,
    ) -> gp.tupledict:
        """
        添加逐 10 min 微网能量平衡。

        [MB-02]
            q_buy,k + q_emg,k + G_k + q_dis,k
            =
```

```
            L_k + q_ch,k + q_curt,k + q_sur,k

        对 Q1:
            q_emg=None
            q_sur=None

        对 Q2-Q4:
            q_emg、q_sur 由具体问题求解器创建并传入。

        单位:
            q_buy/q_emg/G/q_dis/L/q_ch/q_curt/q_sur 全部为 kWh。

        注意:
            充放电效率不出现在母线能量平衡式，只出现在 [MB-04]。
        """
        L, G = self.validate_energy_inputs(
            load_kwh=load_kwh,
            pv_kwh=pv_kwh,
        )

        constr = gp.tupledict()

        for k in self.k:
            emg = self._optional_var_at(q_emg, k, "q_emg")
            sur = self._optional_var_at(q_sur, k, "q_sur")

            # [MB-02] 微网母线能量守恒。
            constr[k] = self.model.addConstr(
                self.vars.q_buy[k]
                + emg
                + float(G.loc[k])
                + self.vars.q_dis[k]
                ==
                float(L.loc[k])
                + self.vars.q_ch[k]
                + self.vars.q_curt[k]
                + sur,
                name=self._cname("energy_balance", k),
            )

        self.constraints.energy_balance = constr
        return constr

    def add_curtailment_constraints(
        self,
        *,
        pv_kwh: pd.Series,
    ) -> gp.tupledict:
        """
        添加弃光约束。

        [MB-03]
            0 <= q_curt,k <= G_k

        q_curt>=0 已在变量创建时设置;
        本函数添加数据相关的 q_curt<=G_k。
        """
        G = _validate_k_series(
            pv_kwh,
            config=self.config,
            name="pv_kwh",
            require_nonnegative=True,
        )

        constr = gp.tupledict()

        for k in self.k:
            # [MB-03]
            constr[k] = self.model.addConstr(
```

```
                self.vars.q_curt[k] <= float(G.loc[k]),
                name=self._cname("curtailment_upper", k),
            )

        self.constraints.curtailment_upper = constr
        return constr

    def add_storage_transition_constraints(self) -> gp.tupledict:
        """
        添加储能状态方程。

        [MB-04]
            E_{k+1}
            =
            E_k + eta_c*q_ch,k - q_dis,k/eta_d

        其中 q_ch、q_dis 已是 10 min 母线侧电量 (kWh),
        因此状态方程中不再乘 Δt。
        """
        constr = gp.tupledict()

        for k in self.k:
            # [MB-04]
            constr[k] = self.model.addConstr(
                self.vars.E[k + 1]
                ==
                self.vars.E[k]
                + ETA_C * self.vars.q_ch[k]
                - self.vars.q_dis[k] / ETA_D,
                name=self._cname("storage_transition", k),
            )

        self.constraints.storage_transition = constr
        return constr

    def add_charge_discharge_exclusivity(
        self,
    ) -> tuple[gp.tupledict, gp.tupledict]:
        """
        可选: 严格禁止同一 10 min 区间同时充电和放电。

        默认不开启。只有构造时显式设置:
            enforce_charge_discharge_exclusivity=True
        才创建 z[k] 并加入以下约束。

        [MB-07a]
            q_ch,k <= q_bat^max*z_k

        [MB-07b]
            q_dis,k <= q_bat^max*(1-z_k)

        [MB-07c]
            z_k in {0,1}

        开启后模型由连续 LP 扩展为 MILP。
        """
        if not self.enforce_charge_discharge_exclusivity:
            raise RuntimeError(
                " 当前模型未启用充放电互斥。"
                " 如确有必要, 请在构造时设置"
                "enforce_charge_discharge_exclusivity=True。"
            )

        if self.vars.z is None:
            raise RuntimeError(" 互斥开关已开启, 但 z 变量未创建。")

        # 防止重复添加。
        if (
            self.constraints.charge_exclusivity is not None
```

```
            and self.constraints.discharge_exclusivity is not None
        ):
            return (
                self.constraints.charge_exclusivity,
                self.constraints.discharge_exclusivity,
            )

        ch_constr = gp.tupledict()
        dis_constr = gp.tupledict()

        for k in self.k:
            # [MB-07a]
            ch_constr[k] = self.model.addConstr(
                self.vars.q_ch[k]
                <= self.q_bat_max_kwh * self.vars.z[k],
                name=self._cname("charge_exclusivity", k),
            )

            # [MB-07b]
            dis_constr[k] = self.model.addConstr(
                self.vars.q_dis[k]
                <= self.q_bat_max_kwh * (1 - self.vars.z[k]),
                name=self._cname("discharge_exclusivity", k),
            )

        self.constraints.charge_exclusivity = ch_constr
        self.constraints.discharge_exclusivity = dis_constr

        return ch_constr, dis_constr

    def add_common_physical_constraints(
        self,
        *,
        load_kwh: pd.Series,
        pv_kwh: pd.Series,
        q_emg: Optional[Mapping[int, gp.Var]] = None,
        q_sur: Optional[Mapping[int, gp.Var]] = None,
    ) -> BaseConstraintRefs:
        """
        一次性添加所有需要显式约束行的公共物理关系。

        添加:
            [MB-02] 能量平衡
            [MB-03] 弃光上界
            [MB-04] 储能状态方程

        已通过变量上下界自动实现:
            [MB-05] 储能电量上下界
            [MB-06] 充放电功率/单步电量限制
            [MB-08] 外网购电非负

        若互斥开关开启:
            [MB-07] 已在构造时自动加入，不重复添加。

        本函数明确不添加:
            - E[0] 初值
            - E[144] 终端约束
            - 跨日 SOC 约束
            - q_emg/q_sur 变量本身
            - 场景树非预见性
            - MPC 冻结约束
            - 任何具体问题目标函数
        """
        self.add_energy_balance(
            load_kwh=load_kwh,
            pv_kwh=pv_kwh,
            q_emg=q_emg,
            q_sur=q_sur,
        )
```

```
618         self.add_curtailment_constraints(
619             pv_kwh=pv_kwh,
620         )
621         self.add_storage_transition_constraints()
622         return self.constraints
623 
624     def get_physical_index(self) -> tuple[int, ...]:
625         """
626         返回唯一物理区间索引 k=0,...,143。
627 
628         后续所有问题应直接复用该索引，不允许通过：
629         - Excel 原始时间标签
630         - source_right_endpoint_label
631         - paper_label
632         - template_display_label
633         反向生成或偏移 k。
634         """
635         return self.k
636 
637     def get_state_index(self) -> tuple[int, ...]:
638         """ 返回储能状态索引 j=0,...,144。"""
639         return self.j
640 
641     def get_time_map(self) -> pd.DataFrame:
642         """
643         返回 time_map 副本，仅用于解释、结果标注和输出适配。
644 
645         优化变量的索引定位始终使用 self.k。
646         """
647         return self.time_map.copy(deep=True)
```

文件：code/question1_solver.py

```
# -*- coding: utf-8 -*-
"""
question1_solver.py
问题 1 确定性日内购电优化求解器。

本模块严格继承 model_base.py，不重新定义公共变量体系和时间索引。

输入
----
1. data_loader.Attachment1Data
   - attachment1.aligned["price_yuan_per_kwh"] : 元/kWh
   - attachment1.aligned["load_kw"]             : kW
   - attachment1.aligned["pv_forecast_kw"]      : kW
2. initial_energy_kwh
   - 问题 1 默认 6000 kWh
3. StorageParams
   - 与已经冻结的公共储能参数保持一致

输出
----
- 计划购电量 q_buy[k]，kWh
- 充电量 q_ch[k]，kWh
- 放电量 q_dis[k]，kWh
- 弃光量 q_curt[k]，kWh
- 储电量轨迹 E[j]，j=0,...,144，kWh
- 总购电量，kWh
- 总费用，元
- 论文指定时段提取结果
- 结果合法性检查报告

问题 1 新增约束
------------
[Q1-01] E_0 = E_init
[Q1-02] E_144 = E_0

问题 1 目标函数
-------------
[Q1-OBJ]
    min C_1 = sum_{k=0}^{143} p_k * q_buy[k]

时间提取规则
------------
论文指定：
    10:00-10:10 -> k=60
    12:00-12:10 -> k=72
    14:00-14:10 -> k=84
    16:00-16:10 -> k=96
    18:00-18:10 -> k=108
    20:00-20:10 -> k=120

禁止根据附件时间字符串或结果模板显示文字搜索这些时段。
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Optional, Sequence

import pandas as pd

try:
    import gurobipy as gp
    from gurobipy import GRB
except ImportError as exc:  # pragma: no cover
    raise ImportError(
```

```
        "question1_solver.py 依赖 gurobipy。"
        " 请在已安装并配置 Gurobi 的环境中运行。"
    ) from exc

from config import DEFAULT_CONFIG, ProjectConfig
from data_loader import Attachment1Data
from model_base import (
    ETA_C,
    ETA_D,
    E_MIN_KWH,
    E_MAX_KWH,
    P_BAT_MAX_KW,
    MicrogridBaseModel,
    kw_to_interval_kwh,
)
from time_index import build_time_map, validate_time_map


# ============================================================================
# 论文指定时段：只能通过模型 k 提取，不搜索任何 Excel/模板字符串
# ============================================================================

REPORT_INTERVAL_INDEX: tuple[int, ...] = (
    60,   # 10:00-10:10
    72,   # 12:00-12:10
    84,   # 14:00-14:10
    96,   # 16:00-16:10
    108,  # 18:00-18:10
    120,  # 20:00-20:10
)


@dataclass(frozen=True)
class StorageParams:
    """
    问题 1 储能参数输入。

    当前公共模型已经冻结为：
        eta_c = eta_d = 0.9
        E_min = 1200 kWh
        E_max = 10800 kWh
        P_bat_max = 5000 kW

    为防止 question1_solver 与 model_base 出现两套物理口径，
    本模块会强制检查传入参数与公共模型一致。
    """

    eta_c: float = ETA_C
    eta_d: float = ETA_D
    e_min_kwh: float = E_MIN_KWH
    e_max_kwh: float = E_MAX_KWH
    p_bat_max_kw: float = P_BAT_MAX_KW

    def validate_against_model_base(self, tol: float = 1e-12) -> None:
        expected = {
            "eta_c": ETA_C,
            "eta_d": ETA_D,
            "e_min_kwh": E_MIN_KWH,
            "e_max_kwh": E_MAX_KWH,
            "p_bat_max_kw": P_BAT_MAX_KW,
        }
        actual = {
            "eta_c": self.eta_c,
            "eta_d": self.eta_d,
            "e_min_kwh": self.e_min_kwh,
            "e_max_kwh": self.e_max_kwh,
            "p_bat_max_kw": self.p_bat_max_kw,
        }

```

```
        bad = {
            key: (actual[key], expected[key])
            for key in expected
            if abs(float(actual[key]) - float(expected[key])) > tol
        }

        if bad:
            raise ValueError(
                "question1_solver 的储能参数必须与 model_base.py 已冻结参数一致。"
                f" 发现差异: {bad}"
            )


@dataclass
class Question1Result:
    """ 问题 1 求解结果。"""

    status: int
    objective_value_yuan: float

    # k=0,...,143
    plan_purchase_kwh: pd.Series
    charge_kwh: pd.Series
    discharge_kwh: pd.Series
    curtailment_kwh: pd.Series

    # j=0,...,144
    energy_kwh: pd.Series

    total_purchase_kwh: float
    total_cost_yuan: float

    # 论文指定时段
    paper_intervals: pd.DataFrame

    # 结果合法性检查
    legality_report: pd.DataFrame


def _prepare_attachment1_for_q1(
    attachment1: Attachment1Data,
    *,
    config: ProjectConfig,
) -> tuple[pd.Series, pd.Series, pd.Series]:
    """
    从附件 1 处理结果中获取:
        price_yuan_per_kwh
        load_kwh
        pv_forecast_kwh

    只使用 attachment1.aligned 中的模型 k 顺序。
    不读取附件 1 原始时间字符串进行定位。
    """
    if not isinstance(attachment1, Attachment1Data):
        raise TypeError(
            "attachment1 必须是 data_loader.load_attachment1() 返回的"
            "Attachment1Data。"
        )

    aligned = attachment1.aligned

    required_columns = {
        "k",
        "price_yuan_per_kwh",
        "load_kw",
        "pv_forecast_kw",
    }
    missing = required_columns.difference(aligned.columns)
    if missing:
```

```
        raise ValueError(
            f" 附件 1 处理结果缺少 Q1 所需字段: {sorted(missing)}"
        )

    if len(aligned) != config.intervals_per_day:
        raise ValueError(
            f" 附件 1 处理结果必须为 {config.intervals_per_day} 行, "
            f" 实际为 {len(aligned)}。"
        )

    k_values = aligned["k"].astype(int).tolist()
    expected_k = list(range(config.intervals_per_day))
    if k_values != expected_k:
        raise ValueError(
            " 附件 1 处理结果的 k 必须严格为 0~143 且按顺序排列。"
            " 不得通过附件时间字符串重新定位。"
        )

    price = pd.to_numeric(
        aligned["price_yuan_per_kwh"],
        errors="raise",
    ).astype(float)
    price.index = expected_k
    price.name = "price_yuan_per_kwh"

    if price.isna().any():
        raise ValueError(" 附件 1 电价存在缺失值。")

    # 负荷/光伏原始单位 kW, 统一转换为每个 10 min 区间的 kWh。
    load_kw = pd.to_numeric(
        aligned["load_kw"],
        errors="raise",
    ).astype(float)
    load_kw.index = expected_k
    load_kw.name = "load_kw"

    pv_kw = pd.to_numeric(
        aligned["pv_forecast_kw"],
        errors="raise",
    ).astype(float)
    pv_kw.index = expected_k
    pv_kw.name = "pv_forecast_kw"

    if (load_kw < 0).any():
        raise ValueError(" 附件 1 负荷功率存在负值。")
    if (pv_kw < 0).any():
        raise ValueError(" 附件 1 光伏预测功率存在负值。")

    load_kwh = kw_to_interval_kwh(
        load_kw,
        config=config,
        name="load_kw",
    )
    pv_kwh = kw_to_interval_kwh(
        pv_kw,
        config=config,
        name="pv_forecast_kw",
    )

    load_kwh.name = "load_kwh"
    pv_kwh.name = "pv_forecast_kwh"

    return price, load_kwh, pv_kwh


def extract_paper_intervals(
    *,
    plan_purchase_kwh: pd.Series,
    charge_kwh: pd.Series,
```

```
    discharge_kwh: pd.Series,
    curtailment_kwh: pd.Series,
    energy_kwh: pd.Series,
    time_map: pd.DataFrame,
) -> pd.DataFrame:
    """
    固定按 REPORT_INTERVAL_INDEX 提取论文指定时段。

    重要:
    本函数不搜索:
    - 附件 1 中的'10:00'/'10:10'
    - source_right_endpoint_label
    - paper_label 字符串
    - template_display_label

    唯一定位方式是模型 k:
        60,72,84,96,108,120
    """
    validate_time_map(time_map)

    rows = []

    for k in REPORT_INTERVAL_INDEX:
        tm = time_map.loc[time_map["k"] == k]
        if len(tm) != 1:
            raise ValueError(f"time_map 中 k={k} 必须且只能出现一次。")

        tm_row = tm.iloc[0]

        rows.append(
            {
                "k": k,
                "paper_label": tm_row["paper_label"],
                "source_right_endpoint_label": (
                    tm_row["source_right_endpoint_label"]
                ),
                "plan_purchase_kwh": float(plan_purchase_kwh.loc[k]),
                "charge_kwh": float(charge_kwh.loc[k]),
                "discharge_kwh": float(discharge_kwh.loc[k]),
                "curtailment_kwh": float(curtailment_kwh.loc[k]),
                "energy_start_kwh": float(energy_kwh.loc[k]),
                "energy_end_kwh": float(energy_kwh.loc[k + 1]),
            }
        )

    result = pd.DataFrame(rows)

    expected = {
        60: "10:00-10:10",
        72: "12:00-12:10",
        84: "14:00-14:10",
        96: "16:00-16:10",
        108: "18:00-18:10",
        120: "20:00-20:10",
    }

    for row in result.itertuples(index=False):
        if row.paper_label != expected[row.k]:
            raise AssertionError(
                f"k={row.k} 物理区间异常: "
                f"{row.paper_label} != {expected[row.k]}"
            )

    return result


def check_question1_result_legality(
    *,
    result: Question1Result,
```

```
    attachment1: Attachment1Data,
    initial_energy_kwh: float,
    storage_params: StorageParams,
    config: ProjectConfig = DEFAULT_CONFIG,
    time_map: Optional[pd.DataFrame] = None,
    tolerance_kwh: float = 1e-5,
    tolerance_yuan: float = 1e-5,
    enforce_charge_discharge_exclusivity: bool = False,
) -> pd.DataFrame:
    """
    问题 1 结果合法性检查。

    检查内容:
    1. 索引和长度;
    2. 变量非负;
    3. [MB-02] 能量平衡;
    4. [MB-03] 弃光上界;
    5. [MB-04] SOC 状态方程;
    6. [MB-05] 储电量上下界;
    7. [MB-06] 充放电量上限;
    8. [MB-07] 可选互斥;
    9. [Q1-01] E_0=E_init;
    10. [Q1-02] E_144=E_0;
    11. 总购电量重算;
    12. 总费用重算;
    13. 论文指定 k 提取一致性。
    """
    storage_params.validate_against_model_base()

    if time_map is None:
        time_map = build_time_map(config)
    validate_time_map(time_map, config)

    price, load_kwh, pv_kwh = _prepare_attachment1_for_q1(
        attachment1,
        config=config,
    )

    q_buy = result.plan_purchase_kwh
    q_ch = result.charge_kwh
    q_dis = result.discharge_kwh
    q_curt = result.curtailment_kwh
    E = result.energy_kwh

    checks = []

    def add_check(
        item: str,
        passed: bool,
        detail: str,
        max_residual: Optional[float] = None,
    ) -> None:
        checks.append(
            {
                "item": item,
                "passed": bool(passed),
                "max_residual": max_residual,
                "detail": detail,
            }
        )

    expected_k = list(range(config.intervals_per_day))
    expected_j = list(range(config.state_nodes_per_day))

    # 1. 长度与索引
    for name, series, expected in [
        ("plan_purchase_kwh", q_buy, expected_k),
        ("charge_kwh", q_ch, expected_k),
        ("discharge_kwh", q_dis, expected_k),
```

```
        ("curtailment_kwh", q_curt, expected_k),
        ("energy_kwh", E, expected_j),
    ]:
        index_ok = list(series.index) == expected
        add_check(
            f"{name}-index",
            index_ok,
            f" 实际长度 ={len(series)}; 期望长度 ={len(expected)}。",
        )

    # 2. 非负性
    for name, series in [
        ("q_buy", q_buy),
        ("q_ch", q_ch),
        ("q_dis", q_dis),
        ("q_curt", q_curt),
    ]:
        min_value = float(series.min())
        add_check(
            f"{name}-nonnegative",
            min_value >= -tolerance_kwh,
            f" 最小值 ={min_value:.12g} kWh。",
            max_residual=max(0.0, -min_value),
        )

    # 3. [MB-02] q_buy + G + q_dis = L + q_ch + q_curt
    balance_residual = (
        q_buy
        + pv_kwh
        + q_dis
        - load_kwh
        - q_ch
        - q_curt
    )
    balance_max = float(balance_residual.abs().max())
    add_check(
        "[MB-02] energy_balance",
        balance_max <= tolerance_kwh,
        f" 最大绝对残差 ={balance_max:.12g} kWh。",
        max_residual=balance_max,
    )

    # 4. [MB-03] 0 <= q_curt <= G
    curt_upper_violation = float(
        (q_curt - pv_kwh).clip(lower=0).max()
    )
    add_check(
        "[MB-03] curtailment_upper",
        curt_upper_violation <= tolerance_kwh,
        f" 最大上界违反 ={curt_upper_violation:.12g} kWh。",
        max_residual=curt_upper_violation,
    )

    # 5. [MB-04] SOC 状态方程
    transition_residual = pd.Series(
        {
            k: (
                E.loc[k + 1]
                - E.loc[k]
                - storage_params.eta_c * q_ch.loc[k]
                + q_dis.loc[k] / storage_params.eta_d
            )
            for k in expected_k
        },
        dtype=float,
    )
    transition_max = float(transition_residual.abs().max())
    add_check(
        "[MB-04] storage_transition",
```

```
        transition_max <= tolerance_kwh,
        f" 最大绝对残差 ={transition_max:.12g} kWh。",
        max_residual=transition_max,
    )

    # 6. [MB-05] SOC 上下界
    lower_violation = float(
        (storage_params.e_min_kwh - E).clip(lower=0).max()
    )
    upper_violation = float(
        (E - storage_params.e_max_kwh).clip(lower=0).max()
    )
    soc_violation = max(lower_violation, upper_violation)
    add_check(
        "[MB-05] energy_bounds",
        soc_violation <= tolerance_kwh,
        (
            f" 下界最大违反 ={lower_violation:.12g} kWh; "
            f" 上界最大违反 ={upper_violation:.12g} kWh。"
        ),
        max_residual=soc_violation,
    )

    # 7. [MB-06] 充放电上限
    q_max = storage_params.p_bat_max_kw * config.dt_minutes / 60.0
    charge_violation = float(
        (q_ch - q_max).clip(lower=0).max()
    )
    discharge_violation = float(
        (q_dis - q_max).clip(lower=0).max()
    )
    power_violation = max(charge_violation, discharge_violation)
    add_check(
        "[MB-06] charge_discharge_limit",
        power_violation <= tolerance_kwh,
        (
            f"q_max={q_max:.12g} kWh; "
            f" 充电最大违反 ={charge_violation:.12g}; "
            f" 放电最大违反 ={discharge_violation:.12g}。"
        ),
        max_residual=power_violation,
    )

    # 8. [MB-07] 可选互斥
    if enforce_charge_discharge_exclusivity:
        simultaneous = pd.concat(
            [q_ch, q_dis],
            axis=1,
        ).min(axis=1)
        simultaneous_max = float(simultaneous.max())
        add_check(
            "[MB-07] charge_discharge_exclusivity",
            simultaneous_max <= tolerance_kwh,
            f" 最大同时充放电量 ={simultaneous_max:.12g} kWh。",
            max_residual=simultaneous_max,
        )
    else:
        add_check(
            "[MB-07] charge_discharge_exclusivity",
            True,
            " 连续 LP 主模型未强制互斥; 该项按模型设定通过。",
            max_residual=0.0,
        )

    # 9. [Q1-01] E_0 = initial
    init_residual = abs(float(E.loc[0]) - float(initial_energy_kwh))
    add_check(
        "[Q1-01] initial_energy",
        init_residual <= tolerance_kwh,
```

```
        f"|E_0-E_init|={init_residual:.12g} kWh。",
        max_residual=init_residual,
    )

    # 10. [Q1-02] E_144 = E_0
    terminal_residual = abs(
        float(E.loc[config.intervals_per_day]) - float(E.loc[0])
    )
    add_check(
        "[Q1-02] daily_energy_closure",
        terminal_residual <= tolerance_kwh,
        f"|E_144-E_0|={terminal_residual:.12g} kWh。",
        max_residual=terminal_residual,
    )

    # 11. 总购电量
    total_purchase_recalc = float(q_buy.sum())
    total_purchase_residual = abs(
        total_purchase_recalc - float(result.total_purchase_kwh)
    )
    add_check(
        "total_purchase_kwh",
        total_purchase_residual <= tolerance_kwh,
        (
            f" 结果 ={result.total_purchase_kwh:.12g}; "
            f" 重算 ={total_purchase_recalc:.12g} kWh。"
        ),
        max_residual=total_purchase_residual,
    )

    # 12. 总费用
    total_cost_recalc = float((price * q_buy).sum())
    total_cost_residual = abs(
        total_cost_recalc - float(result.total_cost_yuan)
    )
    add_check(
        "[Q1-OBJ] total_cost_yuan",
        total_cost_residual <= tolerance_yuan,
        (
            f" 结果 ={result.total_cost_yuan:.12g}; "
            f" 重算 ={total_cost_recalc:.12g} 元。"
        ),
        max_residual=total_cost_residual,
    )

    # 13. 论文指定 k 只能按常量索引提取
    expected_report_k = list(REPORT_INTERVAL_INDEX)
    actual_report_k = result.paper_intervals["k"].astype(int).tolist()
    add_check(
        "paper_report_fixed_k",
        actual_report_k == expected_report_k,
        (
            f" 实际 ={actual_report_k}; "
            f" 期望 ={expected_report_k}。"
        ),
    )

    report = pd.DataFrame(checks)
    return report


def assert_question1_result_legal(report: pd.DataFrame) -> None:
    """ 若问题 1 结果存在任何不合法项, 则直接抛出异常。"""
    failed = report.loc[~report["passed"]]

    if failed.empty:
        return

    lines = []
```

```
    for row in failed.itertuples(index=False):
        lines.append(
            f"{row.item}: {row.detail}"
        )

    raise AssertionError(
        " 问题 1 结果合法性检查未通过: \n"
        + "\n".join(lines)
    )


class Question1Solver(MicrogridBaseModel):
    """
    问题 1 求解器。

    继承:
        MicrogridBaseModel

    公共物理约束来自 model_base.py;
    本类只新增:
        [Q1-01] E_0 = E_init
        [Q1-02] E_144 = E_0
        [Q1-OBJ] 最小化正常外网购电费用
    """

    def __init__(
        self,
        *,
        attachment1: Attachment1Data,
        initial_energy_kwh: float = 6000.0,
        storage_params: StorageParams = StorageParams(),
        config: ProjectConfig = DEFAULT_CONFIG,
        model: Optional[gp.Model] = None,
        enforce_charge_discharge_exclusivity: bool = False,
        solver_output: bool = False,
    ) -> None:
        storage_params.validate_against_model_base()

        if not (
            storage_params.e_min_kwh
            <= initial_energy_kwh
            <= storage_params.e_max_kwh
        ):
            raise ValueError(
                "initial_energy_kwh 必须位于储能安全范围内: "
                f"[{storage_params.e_min_kwh}, "
                f"{storage_params.e_max_kwh}]。"
            )

        self.attachment1 = attachment1
        self.initial_energy_kwh = float(initial_energy_kwh)
        self.storage_params = storage_params

        self.price_yuan_per_kwh, self.load_kwh, self.pv_kwh = (
            _prepare_attachment1_for_q1(
                attachment1,
                config=config,
            )
        )

        if model is None:
            model = gp.Model("question1")

        model.Params.OutputFlag = 1 if solver_output else 0

        # 必须继承 model_base.py:
        # 公共变量、time_index、默认 LP/可选 MILP 接口由父类创建。
        super().__init__(
            model,
```

```
            config=config,
            time_map=build_time_map(config),
            enforce_charge_discharge_exclusivity=(
                enforce_charge_discharge_exclusivity
            ),
            variable_name_prefix="",
        )

        self._q1_constraints_added = False
        self._q1_objective_added = False

        self.initial_energy_constraint: Optional[gp.Constr] = None
        self.terminal_energy_constraint: Optional[gp.Constr] = None

    def add_question1_constraints(self) -> None:
        """
        添加问题 1 全部约束。

        先调用父类公共约束:
            [MB-02] 能量平衡
            [MB-03] 弃光
            [MB-04] SOC 状态方程
            [MB-05] SOC 上下界 (变量边界)
            [MB-06] 充放电限制 (变量边界)
            [MB-07] 可选互斥
            [MB-08] 外网购电非负 (变量边界)

        再新增:
            [Q1-01] E_0 = E_init
            [Q1-02] E_144 = E_0
        """
        if self._q1_constraints_added:
            return

        self.add_common_physical_constraints(
            load_kwh=self.load_kwh,
            pv_kwh=self.pv_kwh,
            q_emg=None,
            q_sur=None,
        )

        # [Q1-01] 日初储电量
        self.initial_energy_constraint = self.model.addConstr(
            self.vars.E[0] == self.initial_energy_kwh,
            name="q1_initial_energy",
        )

        # [Q1-02] 24:00 储电量与 0:00 完全相同
        self.terminal_energy_constraint = self.model.addConstr(
            self.vars.E[self.config.intervals_per_day]
            == self.vars.E[0],
            name="q1_terminal_energy_closure",
        )

        self._q1_constraints_added = True

    def add_question1_objective(self) -> None:
        """
        添加问题 1 唯一目标函数。

        [Q1-OBJ]
            min C_1 = sum_k p_k * q_buy[k]

        p_k 单位: 元/kWh
        q_buy[k] 单位: kWh
        因此目标函数单位: 元
        """
        if self._q1_objective_added:
            return
```

```
        objective = gp.quicksum(
            float(self.price_yuan_per_kwh.loc[k])
            * self.vars.q_buy[k]
            for k in self.k
        )

        self.model.setObjective(
            objective,
            GRB.MINIMIZE,
        )

        self._q1_objective_added = True

    @staticmethod
    def _extract_k_series(
        var_dict: gp.tupledict,
        *,
        index: Sequence[int],
        name: str,
    ) -> pd.Series:
        """ 从 Gurobi 变量字典提取固定索引 Series。"""
        return pd.Series(
            {
                int(i): float(var_dict[i].X)
                for i in index
            },
            dtype=float,
            name=name,
        )

    def solve(
        self,
        *,
        check_legality: bool = True,
        tolerance_kwh: float = 1e-5,
        tolerance_yuan: float = 1e-5,
    ) -> Question1Result:
        """
        求解问题 1 并返回结构化结果。

        不写 Excel，不处理问题 2 及以后逻辑。
        """
        self.add_question1_constraints()
        self.add_question1_objective()

        self.model.optimize()

        if self.model.Status != GRB.OPTIMAL:
            raise RuntimeError(
                " 问题 1 未得到最优解。"
                f"Gurobi Status={self.model.Status}"
            )

        q_buy = self._extract_k_series(
            self.vars.q_buy,
            index=self.k,
            name="plan_purchase_kwh",
        )
        q_ch = self._extract_k_series(
            self.vars.q_ch,
            index=self.k,
            name="charge_kwh",
        )
        q_dis = self._extract_k_series(
            self.vars.q_dis,
            index=self.k,
            name="discharge_kwh",
        )
```

```
        q_curt = self._extract_k_series(
            self.vars.q_curt,
            index=self.k,
            name="curtailment_kwh",
        )
        E = self._extract_k_series(
            self.vars.E,
            index=self.j,
            name="energy_kwh",
        )

        total_purchase_kwh = float(q_buy.sum())
        total_cost_yuan = float(
            (self.price_yuan_per_kwh * q_buy).sum()
        )

        paper_intervals = extract_paper_intervals(
            plan_purchase_kwh=q_buy,
            charge_kwh=q_ch,
            discharge_kwh=q_dis,
            curtailment_kwh=q_curt,
            energy_kwh=E,
            time_map=self.time_map,
        )

        # 先构造临时结果，合法性报告随后填充。
        result = Question1Result(
            status=int(self.model.Status),
            objective_value_yuan=float(self.model.ObjVal),
            plan_purchase_kwh=q_buy,
            charge_kwh=q_ch,
            discharge_kwh=q_dis,
            curtailment_kwh=q_curt,
            energy_kwh=E,
            total_purchase_kwh=total_purchase_kwh,
            total_cost_yuan=total_cost_yuan,
            paper_intervals=paper_intervals,
            legality_report=pd.DataFrame(),
        )

        legality_report = check_question1_result_legality(
            result=result,
            attachment1=self.attachment1,
            initial_energy_kwh=self.initial_energy_kwh,
            storage_params=self.storage_params,
            config=self.config,
            time_map=self.time_map,
            tolerance_kwh=tolerance_kwh,
            tolerance_yuan=tolerance_yuan,
            enforce_charge_discharge_exclusivity=(
                self.enforce_charge_discharge_exclusivity
            ),
        )

        result.legality_report = legality_report

        if check_legality:
            assert_question1_result_legal(legality_report)

        return result
```

文件：code/question2_solver.py

```
# -*- coding: utf-8 -*-
"""
question2_solver.py
问题 2：日前计划购电 + 多阶段非预见随机优化 + 10 分钟因果滚动补救。

严格继承前文已经冻结的：
- 变量命名：
    q_buy, q_ch, q_dis, q_emg, q_curt, q_sur, E
- 时间索引：
    time_index.py 中的 k=0,...,143
- 数据接口：
    Attachment1Data, Attachment2Data
- 单位：
    功率输入 kW；优化中的能量变量统一 kWh
- 储能参数：
    model_base.py 中 ETA_C/ETA_D/E_MIN/E_MAX/P_BAT_MAX

本文件不编写问题 3/4 逻辑。

信息边界
--------
1. 每天 0:00 生成 forecast(d) 时，只使用日期 s<d 的附件 2 历史；
2. 每天 0:00 残差库只包含 s<d 的严格样本外历史残差；
3. 当天未来真实负荷/光伏绝不进入 0:00 优化；
4. q_buy[d,k|0] 在 0:00 确定后全天固定；
5. 储能/弃光/计划富余/紧急购电按场景树 node_id 决策；
6. 不创建“每条完整叶场景一套从 0:00 就知道全天未来的动作”；
7. 实际运行采用条件滚动补救：
       在 tau_k 决定储能动作时只传入已完成真实前缀 0..k-1 和当前真实 SOC；
       current_k 及未来条件场景只来自预测与历史残差分布；
       current_k 真实负荷/PV 只在动作确定后用于事后紧急购电/弃光/富余；
       每次只执行 current_k 唯一根节点的储能第一步动作。
8. 实际 E_144 传递到下一天 E_0，不做每日 SOC 重置；
9. 默认从 2025-02-01 开始保留正式结果，1 月份用于暖启动和 SOC 推进。

数学编号
--------
[Q2-01] 0:00 计划购电费用
[Q2-02] 节点能量平衡
[Q2-03] 节点储能状态转移
[Q2-04] 节点 SOC 上下界
[Q2-05] 节点充放电上限
[Q2-06] 节点弃光上限
[Q2-07] 5 倍紧急购电期望费用
[Q2-08] 风险中性目标
[Q2-09] 非预见性：一节点一动作
[Q2-10] 跨日 SOC 连续
[Q2-11] CVaR 叶路径紧急购电损失（可选）
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Dict, Optional

import numpy as np
import pandas as pd

try:
    import gurobipy as gp
    from gurobipy import GRB
except ImportError as exc:  # pragma: no cover
    raise ImportError(
        "question2_solver.py 依赖 gurobipy。"
        " 请在已安装并配置 Gurobi 的环境中运行。"
```

```
    ) from exc

from config import DEFAULT_CONFIG, ProjectConfig
from data_loader import Attachment1Data, Attachment2Data
from forecast_model import (
    DayAheadForecast,
    ResidualLibrary,
    build_causal_residual_library,
    build_q2_day_ahead_forecast,
)
from model_base import (
    ETA_C,
    ETA_D,
    E_MIN_KWH,
    E_MAX_KWH,
    P_BAT_MAX_KW,
)
from scenario_tree import (
    NonanticipativeScenarioTree,
    ScenarioTrajectorySet,
    build_bootstrap_trajectory_set,
    build_conditional_trajectory_set,
    build_nonanticipative_tree,
    reduce_trajectory_set,
)
from time_index import build_time_map, validate_time_map


# ============================================================================
# 配置与结果结构
# ============================================================================


@dataclass(frozen=True)
class Question2Settings:
    """ 问题 2 随机优化和因果补救设置。"""

    # 因果残差库最多保留最近多少天
    max_residual_days: int = 60

    # 0:00 场景
    dayahead_bootstrap_scenarios: int = 40
    dayahead_reduced_scenarios: int = 10
    dayahead_branch_every: int = 6       # 6×10min = 1h
    dayahead_max_branch: int = 3

    # 实际运行的条件滚动补救场景
    rolling_bootstrap_scenarios: int = 16
    rolling_reduced_scenarios: int = 6
    rolling_branch_every: int = 12       # 12×10min = 2h
    rolling_max_branch: int = 2
    conditioning_pool: int = 12

    # 风险设置: 默认风险中性
    cvar_alpha: float = 0.95
    cvar_lambda: float = 0.0

    # 默认连续 LP; 如确有必要才开启 MILP 互斥
    enforce_charge_discharge_exclusivity: bool = False

    random_seed: int = 2025
    solver_output: bool = False

    def validate(self) -> None:
        integer_positive = {
            "max_residual_days": self.max_residual_days,
            "dayahead_bootstrap_scenarios": self.dayahead_bootstrap_scenarios,
            "dayahead_reduced_scenarios": self.dayahead_reduced_scenarios,
            "dayahead_branch_every": self.dayahead_branch_every,
```

```
            "dayahead_max_branch": self.dayahead_max_branch,
            "rolling_bootstrap_scenarios": self.rolling_bootstrap_scenarios,
            "rolling_reduced_scenarios": self.rolling_reduced_scenarios,
            "rolling_branch_every": self.rolling_branch_every,
            "rolling_max_branch": self.rolling_max_branch,
            "conditioning_pool": self.conditioning_pool,
        }

        for name, value in integer_positive.items():
            if int(value) <= 0:
                raise ValueError(f"{name} 必须为正整数。")

        if not (0.0 < self.cvar_alpha < 1.0):
            raise ValueError("cvar_alpha 必须位于 (0,1)。")

        if not (0.0 <= self.cvar_lambda <= 1.0):
            raise ValueError("cvar_lambda 必须位于 [0,1]。")


@dataclass
class TreeOptimizationSolution:
    """ 一个场景树优化问题的内部求解结果。"""

    q_buy_kwh: pd.Series
    node_policy: pd.DataFrame

    objective_value: float
    expected_emergency_cost_yuan: float
    plan_cost_yuan: float

    leaf_emergency_loss: pd.DataFrame
    cvar_value_yuan: Optional[float]


@dataclass
class Question2DayResult:
    """ 问题 2 单日正式/暖启动运行结果。"""

    date: pd.Timestamp
    initial_energy_kwh: float
    end_energy_kwh: float

    # 0:00 唯一锁定全天计划购电
    plan_purchase_kwh: pd.Series

    # 实际因果运行轨迹
    actual_charge_kwh: pd.Series
    actual_discharge_kwh: pd.Series
    actual_emergency_kwh: pd.Series
    actual_curtailment_kwh: pd.Series
    actual_surplus_kwh: pd.Series

    # j=0,...,144
    energy_kwh: pd.Series

    # 费用分解
    plan_cost_yuan: float
    emergency_cost_yuan: float
    total_cost_yuan: float
    total_plan_purchase_kwh: float
    total_emergency_kwh: float

    # 0:00 预测
    load_forecast_kwh: pd.Series
    pv_forecast_kwh: pd.Series

    # 0:00 场景树、节点策略及非预见性检查
    day_ahead_tree_nodes: pd.DataFrame
    day_ahead_node_policy: pd.DataFrame
```

```
    day_ahead_nonanticipativity_report: pd.DataFrame

    # 每个 10 分钟实际执行时的信息边界检查
    runtime_causality_report: pd.DataFrame

    # 最终物理/费用合法性
    legality_report: pd.DataFrame


@dataclass
class Question2PeriodResult:
    """
    多日滚动结果。

    formal_results:
        默认只保存 2025-02-01 及以后单日结果。

    cross_day_report:
        检查 [Q2-10] E[d+1,0]=E[d,144]。
    """

    formal_results: Dict[pd.Timestamp, Question2DayResult]
    cross_day_report: pd.DataFrame
    warmup_end_energy_kwh: Optional[float]
    final_energy_kwh: float


# ============================================================================
# 公共准备函数
# ============================================================================


def _normalize_date(value) -> pd.Timestamp:
    return pd.Timestamp(value).normalize()


def _prepare_fixed_price(
    attachment1: Attachment1Data,
    config: ProjectConfig,
) -> pd.Series:
    """
    问题 2 沿用附件 1 固定日内电价。

    只按 attachment1.aligned 中的模型 k=0~143 读取，
    不搜索任何 Excel 时间字符串。
    """
    aligned = attachment1.aligned

    required = {"k", "price_yuan_per_kwh"}
    missing = required.difference(aligned.columns)
    if missing:
        raise ValueError(
            f" 附件 1 处理结果缺少字段：{sorted(missing)}"
        )

    expected_k = list(range(config.intervals_per_day))

    if aligned["k"].astype(int).tolist() != expected_k:
        raise ValueError(
            " 附件 1.aligned 的 k 必须严格为 0~143。"
        )

    price = pd.to_numeric(
        aligned["price_yuan_per_kwh"],
        errors="raise",
    ).astype(float)
    price.index = expected_k
    price.name = "price_yuan_per_kwh"

```

```
    if price.isna().any():
        raise ValueError(" 固定电价存在缺失值。")

    if (price < 0).any():
        raise ValueError(" 当前问题 2 实现要求电价非负。")

    return price


def _validate_attachment2_model_grid(
    attachment2: Attachment2Data,
    config: ProjectConfig,
) -> None:
    expected_k = list(range(config.intervals_per_day))

    for name, frame in [
        ("load_kw", attachment2.load_kw),
        ("pv_kw", attachment2.pv_kw),
    ]:
        if list(frame.columns) != expected_k:
            raise ValueError(
                f"attachment2.{name} 必须严格使用模型 k=0~143。"
            )

        if frame.isna().any().any():
            raise ValueError(
                f"attachment2.{name} 存在缺失值。"
            )

    if not attachment2.load_kw.index.equals(
        attachment2.pv_kw.index
    ):
        raise ValueError(" 附件 2 负荷/光伏日期不一致。")


def _seed_for(
    base_seed: int,
    date: pd.Timestamp,
    k: int = 0,
) -> int:
    """
    生成可复现实验随机种子。
    不读取任何未来数据。
    """
    day_number = int(date.dayofyear)
    return int(base_seed + day_number * 1000 + int(k))


def settle_realized_interval_after_storage(
    *,
    q_buy_kwh: float,
    q_ch_kwh: float,
    q_dis_kwh: float,
    actual_load_kwh: float,
    actual_pv_kwh: float,
    tolerance_kwh: float = 1e-9,
) -> tuple[float, float, float, float]:
    """
    在第 k 个 10 分钟储能动作已经确定之后，根据该区间最终实现的
    真实负荷/光伏计算事后补救量。

    时序：
        tau_k：
            q_buy[k] 已由 0:00 锁定；
            q_ch[k], q_dis[k] 只基于 0..k-1 历史和 E_k 决定。
        [tau_k, tau_{k+1}) 结束：
            actual_load[k], actual_pv[k] 才完整实现；
            随后计算 q_emg[k], q_curt[k], q_sur[k] 完成实际能量平衡。

```

```
    实际能量平衡:
        q_buy + q_emg + G_actual + q_dis
        =
        L_actual + q_ch + q_curt + q_sur

    分配规则:
    1. 若出现缺口，全部由紧急购电 q_emg 补齐;
    2. 若出现富余，优先将可再生光伏计为弃光，剩余部分计入 q_sur;
    3. q_curt 永远不超过 actual_pv;
    4. 不允许负购电或售电。

    返回:
        q_emg, q_curt, q_sur, balance_residual
    """
    values = {
        "q_buy_kwh": q_buy_kwh,
        "q_ch_kwh": q_ch_kwh,
        "q_dis_kwh": q_dis_kwh,
        "actual_load_kwh": actual_load_kwh,
        "actual_pv_kwh": actual_pv_kwh,
    }

    for name, value in values.items():
        if float(value) < -tolerance_kwh:
            raise ValueError(f"{name} 不能为负: {value}")

    q_buy = max(0.0, float(q_buy_kwh))
    q_ch = max(0.0, float(q_ch_kwh))
    q_dis = max(0.0, float(q_dis_kwh))
    load = max(0.0, float(actual_load_kwh))
    pv = max(0.0, float(actual_pv_kwh))

    net_before_expost = (
        q_buy + pv + q_dis - load - q_ch
    )

    if net_before_expost < 0.0:
        q_emg = -net_before_expost
        q_curt = 0.0
        q_sur = 0.0
    else:
        q_emg = 0.0
        q_curt = min(pv, net_before_expost)
        q_sur = net_before_expost - q_curt

    residual = (
        q_buy
        + q_emg
        + pv
        + q_dis
        - load
        - q_ch
        - q_curt
        - q_sur
    )

    if abs(residual) > max(tolerance_kwh, 1e-8):
        raise AssertionError(
            " 事后补救未能闭合实际能量平衡: "
            f"residual={residual}"
        )

    return (
        float(q_emg),
        float(q_curt),
        float(q_sur),
        float(residual),
    )

```

```
411
412 # ================================================================================
413 # 场景树 Gurobi 模型
414 # ================================================================================
415
416
417 def _solve_tree_optimization(
418     *,
419     tree: NonanticipativeScenarioTree,
420     price_yuan_per_kwh: pd.Series,
421     initial_energy_kwh: float,
422     config: ProjectConfig,
423     settings: Question2Settings,
424     q_buy_fixed_kwh: Optional[pd.Series] = None,
425     model_name: str = "q2_tree",
426 ) -> TreeOptimizationSolution:
427     """
428     对一个非预见场景树求解 Q2 运行模型。
429
430     两种模式
431     --------
432     1. q_buy_fixed_kwh is None:
433        0:00 日前优化。创建唯一的 q_buy[k], 对所有未来场景共享。
434
435     2. q_buy_fixed_kwh is not None:
436        实时条件滚动补救。q_buy 全天已经在 0:00 锁定,
437        本模型不能改变它, 只优化节点补救动作。
438
439     节点动作
440     --------
441     q_ch[node_id]
442     q_dis[node_id]
443     q_emg[node_id]
444     q_curt[node_id]
445     q_sur[node_id]
446     E[node_id]
447
448     因此没有 leaf-indexed 全天储能动作。
449     """
450     settings.validate()
451
452     if not (
453         E_MIN_KWH <= initial_energy_kwh <= E_MAX_KWH
454     ):
455         raise ValueError(
456             f"initial_energy_kwh={initial_energy_kwh} 超出"
457             f"[{E_MIN_KWH},{E_MAX_KWH}]。"
458         )
459
460     expected_k = list(range(config.intervals_per_day))
461     if list(price_yuan_per_kwh.index) != expected_k:
462         raise ValueError("price_yuan_per_kwh.index 必须严格为 0~143。")
463
464     if q_buy_fixed_kwh is not None:
465         if list(q_buy_fixed_kwh.index) != expected_k:
466             raise ValueError(
467                 "q_buy_fixed_kwh.index 必须严格为 0~143。"
468             )
469         if (q_buy_fixed_kwh < -1e-9).any():
470             raise ValueError(" 固定的 0:00 计划购电量不能为负。")
471
472     # 先验证随机信息结构。
473     tree_report = tree.validate_nonanticipativity()
474     if not tree_report["passed"].all():
475         raise AssertionError(
476             " 场景树未通过非预见性检查, 禁止进入优化。"
477         )
478
479     model = gp.Model(model_name)
```

```
    model.Params.OutputFlag = 1 if settings.solver_output else 0

    node_ids = tuple(sorted(tree.nodes))
    k_all = tuple(range(config.intervals_per_day))
    q_max = P_BAT_MAX_KW * config.dt_minutes / 60.0

    # ---------------------------------------------------------------------
    # 0:00 唯一计划购电变量
    # ---------------------------------------------------------------------
    if q_buy_fixed_kwh is None:
        # [Q2-01] 一套 q_buy[k] 对所有场景共同生效。
        q_buy = model.addVars(
            k_all,
            lb=0.0,
            vtype=GRB.CONTINUOUS,
            name="q_buy",
        )
    else:
        q_buy = None

    # ---------------------------------------------------------------------
    # 节点补救变量: 每个信息节点仅一套动作 [Q2-09]
    # ---------------------------------------------------------------------
    q_ch = model.addVars(
        node_ids,
        lb=0.0,
        ub=q_max,
        vtype=GRB.CONTINUOUS,
        name="q_ch",
    )
    q_dis = model.addVars(
        node_ids,
        lb=0.0,
        ub=q_max,
        vtype=GRB.CONTINUOUS,
        name="q_dis",
    )
    q_emg = model.addVars(
        node_ids,
        lb=0.0,
        vtype=GRB.CONTINUOUS,
        name="q_emg",
    )
    q_curt = model.addVars(
        node_ids,
        lb=0.0,
        vtype=GRB.CONTINUOUS,
        name="q_curt",
    )
    q_sur = model.addVars(
        node_ids,
        lb=0.0,
        vtype=GRB.CONTINUOUS,
        name="q_sur",
    )

    # [Q2-04] 节点动作执行前的储能内部电量。
    E = model.addVars(
        node_ids,
        lb=E_MIN_KWH,
        ub=E_MAX_KWH,
        vtype=GRB.CONTINUOUS,
        name="E",
    )

    z = None
    if settings.enforce_charge_discharge_exclusivity:
        z = model.addVars(
            node_ids,
```

```
            vtype=GRB.BINARY,
            name="z",
        )

    # ------------------------------------------------------------------
    # 根节点当前真实/初始 SOC
    # ------------------------------------------------------------------
    for root_id in tree.root_node_ids():
        model.addConstr(
            E[root_id] == float(initial_energy_kwh),
            name=f"root_energy[{root_id}]",
        )

    # ------------------------------------------------------------------
    # [Q2-02] 节点能量平衡
    # ------------------------------------------------------------------
    for node_id in node_ids:
        node = tree.nodes[node_id]
        k = node.k

        if q_buy is None:
            buy_expr = float(q_buy_fixed_kwh.loc[k])
        else:
            buy_expr = q_buy[k]

        model.addConstr(
            buy_expr
            + q_emg[node_id]
            + float(node.pv_kwh)
            + q_dis[node_id]
            ==
            float(node.load_kwh)
            + q_ch[node_id]
            + q_curt[node_id]
            + q_sur[node_id],
            name=f"energy_balance[{node_id}]",
        )

        # [Q2-06] 0 <= q_curt <= 当前节点光伏
        model.addConstr(
            q_curt[node_id] <= float(node.pv_kwh),
            name=f"curtailment_upper[{node_id}]",
        )

        if z is not None:
            # 可选 MILP 互斥；默认不启用。
            model.addConstr(
                q_ch[node_id] <= q_max * z[node_id],
                name=f"charge_exclusivity[{node_id}]",
            )
            model.addConstr(
                q_dis[node_id] <= q_max * (1 - z[node_id]),
                name=f"discharge_exclusivity[{node_id}]",
            )

    # ------------------------------------------------------------------
    # [Q2-03] 父子节点 SOC 状态转移
    # ------------------------------------------------------------------
    for node_id in node_ids:
        node = tree.nodes[node_id]

        if node.parent_id is None:
            continue

        parent_id = int(node.parent_id)

        model.addConstr(
            E[node_id]
            ==
```

```
            E[parent_id]
            + ETA_C * q_ch[parent_id]
            - q_dis[parent_id] / ETA_D,
            name=f"storage_transition[{parent_id}->{node_id}]",
        )

    # 最后一个 10 min 动作执行后也必须保持 SOC 安全。
    for leaf_id in tree.leaf_node_ids():
        terminal_expr = (
            E[leaf_id]
            + ETA_C * q_ch[leaf_id]
            - q_dis[leaf_id] / ETA_D
        )

        model.addConstr(
            terminal_expr >= E_MIN_KWH,
            name=f"terminal_energy_lower[{leaf_id}]",
        )
        model.addConstr(
            terminal_expr <= E_MAX_KWH,
            name=f"terminal_energy_upper[{leaf_id}]",
        )

    # ------------------------------------------------------------------
    # [Q2-01] 计划购电费用
    # ------------------------------------------------------------------
    if q_buy is None:
        plan_cost_expr = gp.LinExpr(0.0)
    else:
        plan_cost_expr = gp.quicksum(
            float(price_yuan_per_kwh.loc[k]) * q_buy[k]
            for k in k_all
        )

    # ------------------------------------------------------------------
    # [Q2-07] 5 倍紧急购电期望费用
    # E(C_emg)=5 sum_n rho_n p_k q_emg[n]
    # ------------------------------------------------------------------
    expected_emergency_expr = gp.quicksum(
        5.0
        * float(tree.nodes[node_id].probability)
        * float(price_yuan_per_kwh.loc[tree.nodes[node_id].k])
        * q_emg[node_id]
        for node_id in node_ids
    )

    # ------------------------------------------------------------------
    # [Q2-11] 可选 CVaR：严格基于叶路径紧急购电损失
    # ------------------------------------------------------------------
    cvar_expr = None
    zeta = None
    u = None

    if settings.cvar_lambda > 0.0:
        leaf_ids = tree.leaf_node_ids()

        zeta = model.addVar(
            lb=0.0,
            vtype=GRB.CONTINUOUS,
            name="cvar_zeta",
        )
        u = model.addVars(
            leaf_ids,
            lb=0.0,
            vtype=GRB.CONTINUOUS,
            name="cvar_u",
        )

        for leaf_id in leaf_ids:
```

```
            path = tree.path_to_leaf(leaf_id)

            leaf_loss_expr = gp.quicksum(
                5.0
                * float(
                    price_yuan_per_kwh.loc[
                        tree.nodes[node_id].k
                    ]
                )
                * q_emg[node_id]
                for node_id in path
            )

            model.addConstr(
                u[leaf_id] >= leaf_loss_expr - zeta,
                name=f"cvar_excess[{leaf_id}]",
            )

        cvar_expr = (
            zeta
            + 1.0
            / (1.0 - settings.cvar_alpha)
            * gp.quicksum(
                float(tree.nodes[leaf_id].probability)
                * u[leaf_id]
                for leaf_id in leaf_ids
            )
        )

        # [Q2-08] + [Q2-11]
        objective = (
            plan_cost_expr
            + (1.0 - settings.cvar_lambda)
            * expected_emergency_expr
            + settings.cvar_lambda * cvar_expr
        )
    else:
        # [Q2-08] 风险中性主模型
        objective = (
            plan_cost_expr
            + expected_emergency_expr
        )

    model.setObjective(objective, GRB.MINIMIZE)
    model.optimize()

    if model.Status != GRB.OPTIMAL:
        raise RuntimeError(
            f"{model_name} 未获得最优解, Gurobi Status={model.Status}"
        )

    # ------------------------------------------------------------------
    # 提取 q_buy
    # ------------------------------------------------------------------
    if q_buy is None:
        q_buy_result = q_buy_fixed_kwh.astype(float).copy()
    else:
        q_buy_result = pd.Series(
            {
                k: float(q_buy[k].X)
                for k in k_all
            },
            dtype=float,
            name="plan_purchase_kwh",
        )

    # ------------------------------------------------------------------
    # 提取 node policy: 一 node 一动作
    # ------------------------------------------------------------------
```

```
    policy_rows = []

    for node_id in node_ids:
        node = tree.nodes[node_id]

        policy_rows.append(
            {
                "node_id": node_id,
                "k": node.k,
                "parent_id": node.parent_id,
                "probability": node.probability,
                "member_scenarios": node.member_scenarios,
                "load_kwh": node.load_kwh,
                "pv_kwh": node.pv_kwh,
                "q_ch_kwh": float(q_ch[node_id].X),
                "q_dis_kwh": float(q_dis[node_id].X),
                "q_emg_kwh": float(q_emg[node_id].X),
                "q_curt_kwh": float(q_curt[node_id].X),
                "q_sur_kwh": float(q_sur[node_id].X),
                "E_pre_kwh": float(E[node_id].X),
            }
        )

    node_policy = pd.DataFrame(policy_rows)

    expected_emergency_cost = float(
        sum(
            5.0
            * tree.nodes[node_id].probability
            * float(
                price_yuan_per_kwh.loc[
                    tree.nodes[node_id].k
                ]
            )
            * float(q_emg[node_id].X)
            for node_id in node_ids
        )
    )

    if q_buy is None:
        plan_cost_value = 0.0
    else:
        plan_cost_value = float(
            sum(
                float(price_yuan_per_kwh.loc[k])
                * float(q_buy[k].X)
                for k in k_all
            )
        )

    # 叶路径紧急购电损失，用于 CVaR 审计。
    leaf_rows = []
    for leaf_id in tree.leaf_node_ids():
        path = tree.path_to_leaf(leaf_id)
        loss = float(
            sum(
                5.0
                * float(
                    price_yuan_per_kwh.loc[
                        tree.nodes[node_id].k
                    ]
                )
                * float(q_emg[node_id].X)
                for node_id in path
            )
        )

        leaf_rows.append(
            {
```

```
                "leaf_id": leaf_id,
                "probability": tree.nodes[leaf_id].probability,
                "emergency_loss_yuan": loss,
                "path_node_ids": path,
            }
        )

    leaf_emergency_loss = pd.DataFrame(leaf_rows)

    cvar_value = None
    if cvar_expr is not None:
        cvar_value = float(cvar_expr.getValue())

    return TreeOptimizationSolution(
        q_buy_kwh=q_buy_result,
        node_policy=node_policy,
        objective_value=float(model.ObjVal),
        expected_emergency_cost_yuan=expected_emergency_cost,
        plan_cost_yuan=plan_cost_value,
        leaf_emergency_loss=leaf_emergency_loss,
        cvar_value_yuan=cvar_value,
    )


def _build_solver_nonanticipativity_report(
    tree: NonanticipativeScenarioTree,
    solution: TreeOptimizationSolution,
) -> pd.DataFrame:
    """
    场景树结构 + 优化动作索引联合检查。

    重点确认:
    - 每个 node_id 只有一行动作;
    - 动作表没有 leaf_id 索引;
    - q_buy 只有一条 144 时段计划, 不按场景重复。
    """
    base = tree.validate_nonanticipativity().copy()

    extra_rows = []

    policy = solution.node_policy
    node_unique = (
        len(policy) == len(tree.nodes)
        and policy["node_id"].is_unique
        and set(policy["node_id"]) == set(tree.nodes)
    )

    extra_rows.append(
        {
            "item": "one_action_row_per_information_node",
            "passed": bool(node_unique),
            "detail": (
                f"node 数量 ={len(tree.nodes)}; "
                f"policy 行数 ={len(policy)}; "
                f"node_id 唯一 ={policy['node_id'].is_unique}"
            ),
        }
    )

    extra_rows.append(
        {
            "item": "no_leaf_indexed_advance_action",
            "passed": "leaf_id" not in policy.columns,
            "detail": (
                " 储能/紧急购电等动作只按 node_id 索引, "
                " 没有按完整叶场景提前创建全天动作。"
            ),
        }
    )
```

```
894
895     q_buy_ok = (
896         list(solution.q_buy_kwh.index) == list(range(144))
897     )
898     extra_rows.append(
899         {
900             "item": "single_day_ahead_q_buy_plan",
901             "passed": bool(q_buy_ok),
902             "detail": (
903                 "q_buy 仅有 k=0~143 一套计划，对全部未来场景共享。"
904             ),
905         }
906     )
907
908     return pd.concat(
909         [base, pd.DataFrame(extra_rows)],
910         ignore_index=True,
911     )
912
913
914 # ============================================================================
915 # 结果合法性检查
916 # ============================================================================
917
918
919 def check_question2_day_result_legality(
920     *,
921     result: Question2DayResult,
922     attachment2: Attachment2Data,
923     price_yuan_per_kwh: pd.Series,
924     config: ProjectConfig = DEFAULT_CONFIG,
925     tolerance_kwh: float = 1e-5,
926     tolerance_yuan: float = 1e-5,
927     enforce_charge_discharge_exclusivity: bool = False,
928 ) -> pd.DataFrame:
929     """
930     单日运行结束后的事后合法性检查。
931
932     注意：
933     本函数可以在一天结束后读取完整当天实际轨迹，
934     因为它只做事后审计，不参与此前任何决策。
935     """
936     d = result.date
937     k_all = list(range(config.intervals_per_day))
938     j_all = list(range(config.state_nodes_per_day))
939
940     dt_hours = config.dt_minutes / 60.0
941
942     actual_load_kwh = (
943         attachment2.load_kw.loc[d].astype(float)
944         * dt_hours
945     )
946     actual_pv_kwh = (
947         attachment2.pv_kw.loc[d].astype(float)
948         * dt_hours
949     )
950     actual_load_kwh.index = k_all
951     actual_pv_kwh.index = k_all
952
953     q_buy = result.plan_purchase_kwh
954     q_ch = result.actual_charge_kwh
955     q_dis = result.actual_discharge_kwh
956     q_emg = result.actual_emergency_kwh
957     q_curt = result.actual_curtailment_kwh
958     q_sur = result.actual_surplus_kwh
959     E = result.energy_kwh
960
961     rows = []
962
```

```
    def add(
        item: str,
        passed: bool,
        detail: str,
        residual: Optional[float] = None,
    ) -> None:
        rows.append(
            {
                "item": item,
                "passed": bool(passed),
                "max_residual": residual,
                "detail": detail,
            }
        )

    for name, series, expected in [
        ("q_buy", q_buy, k_all),
        ("q_ch", q_ch, k_all),
        ("q_dis", q_dis, k_all),
        ("q_emg", q_emg, k_all),
        ("q_curt", q_curt, k_all),
        ("q_sur", q_sur, k_all),
        ("E", E, j_all),
    ]:
        ok = list(series.index) == expected
        add(
            f"{name}_index",
            ok,
            f" 长度 ={len(series)}; 期望 ={len(expected)}。",
        )

    # 非负
    for name, series in [
        ("q_buy", q_buy),
        ("q_ch", q_ch),
        ("q_dis", q_dis),
        ("q_emg", q_emg),
        ("q_curt", q_curt),
        ("q_sur", q_sur),
    ]:
        min_value = float(series.min())
        add(
            f"{name}_nonnegative",
            min_value >= -tolerance_kwh,
            f" 最小值 ={min_value:.12g}",
            max(0.0, -min_value),
        )

    # [Q2-02] 实际轨迹能量平衡
    balance = (
        q_buy
        + q_emg
        + actual_pv_kwh
        + q_dis
        - actual_load_kwh
        - q_ch
        - q_curt
        - q_sur
    )
    balance_max = float(balance.abs().max())
    add(
        "[Q2-02] actual_energy_balance",
        balance_max <= tolerance_kwh,
        f" 最大绝对残差 ={balance_max:.12g} kWh。",
        balance_max,
    )

    # [Q2-03] 实际 SOC 转移
    transition = pd.Series(
```

```
        {
            k: (
                E.loc[k + 1]
                - E.loc[k]
                - ETA_C * q_ch.loc[k]
                + q_dis.loc[k] / ETA_D
            )
            for k in k_all
        },
        dtype=float,
    )
    transition_max = float(transition.abs().max())
    add(
        "[Q2-03] actual_storage_transition",
        transition_max <= tolerance_kwh,
        f" 最大绝对残差 ={transition_max:.12g} kWh。",
        transition_max,
    )

    # [Q2-04] SOC 边界
    lower = float((E_MIN_KWH - E).clip(lower=0).max())
    upper = float((E - E_MAX_KWH).clip(lower=0).max())
    soc_violation = max(lower, upper)
    add(
        "[Q2-04] actual_energy_bounds",
        soc_violation <= tolerance_kwh,
        f" 下界违反 ={lower:.12g}; 上界违反 ={upper:.12g}。",
        soc_violation,
    )

    # [Q2-05] 充放电限制
    q_max = P_BAT_MAX_KW * dt_hours
    ch_v = float((q_ch - q_max).clip(lower=0).max())
    dis_v = float((q_dis - q_max).clip(lower=0).max())
    limit_v = max(ch_v, dis_v)
    add(
        "[Q2-05] charge_discharge_limit",
        limit_v <= tolerance_kwh,
        f"q_max={q_max:.12g}; ch_v={ch_v:.12g}; dis_v={dis_v:.12g}",
        limit_v,
    )

    # [Q2-06] 弃光
    curt_v = float(
        (q_curt - actual_pv_kwh).clip(lower=0).max()
    )
    add(
        "[Q2-06] curtailment_upper",
        curt_v <= tolerance_kwh,
        f" 最大违反 ={curt_v:.12g} kWh。",
        curt_v,
    )

    if enforce_charge_discharge_exclusivity:
        simultaneous = pd.concat(
            [q_ch, q_dis],
            axis=1,
        ).min(axis=1)
        sim_max = float(simultaneous.max())
        add(
            "charge_discharge_exclusivity",
            sim_max <= tolerance_kwh,
            f" 最大同时充放电 ={sim_max:.12g} kWh。",
            sim_max,
        )
    else:
        add(
            "charge_discharge_exclusivity",
            True,
```

```
            " 连续 LP 主模型默认未强制二进制互斥。",
            0.0,
        )

    # 初始/末端
    initial_resid = abs(
        float(E.loc[0]) - float(result.initial_energy_kwh)
    )
    end_resid = abs(
        float(E.loc[144]) - float(result.end_energy_kwh)
    )
    add(
        "initial_energy_consistency",
        initial_resid <= tolerance_kwh,
        f" 残差 ={initial_resid:.12g} kWh。",
        initial_resid,
    )
    add(
        "end_energy_consistency",
        end_resid <= tolerance_kwh,
        f" 残差 ={end_resid:.12g} kWh。",
        end_resid,
    )

    # 费用
    plan_cost = float((price_yuan_per_kwh * q_buy).sum())
    emergency_cost = float(
        (
            5.0
            * price_yuan_per_kwh
            * q_emg
        ).sum()
    )
    total_cost = plan_cost + emergency_cost

    add(
        "[Q2-01] plan_cost",
        abs(plan_cost - result.plan_cost_yuan)
        <= tolerance_yuan,
        (
            f" 结果 ={result.plan_cost_yuan:.12g}; "
            f" 重算 ={plan_cost:.12g}"
        ),
        abs(plan_cost - result.plan_cost_yuan),
    )
    add(
        "[Q2-07] emergency_cost",
        abs(emergency_cost - result.emergency_cost_yuan)
        <= tolerance_yuan,
        (
            f" 结果 ={result.emergency_cost_yuan:.12g}; "
            f" 重算 ={emergency_cost:.12g}"
        ),
        abs(emergency_cost - result.emergency_cost_yuan),
    )
    add(
        "[Q2-08] total_cost",
        abs(total_cost - result.total_cost_yuan)
        <= tolerance_yuan,
        (
            f" 结果 ={result.total_cost_yuan:.12g}; "
            f" 重算 ={total_cost:.12g}"
        ),
        abs(total_cost - result.total_cost_yuan),
    )

    # 非预见性
    day_tree_ok = bool(
        result.day_ahead_nonanticipativity_report["passed"].all()
```

```
    )
    add(
        "[Q2-09] day_ahead_nonanticipativity",
        day_tree_ok,
        "0:00 场景树及 node-indexed 动作检查结果。",
    )

    runtime = result.runtime_causality_report
    runtime_ok = bool(
        runtime["future_actual_used"].eq(False).all()
        and (
            runtime["max_observed_k"].astype(int)
            == runtime["k"].astype(int) - 1
        ).all()
        and (
            runtime["observed_points"].astype(int)
            == runtime["k"].astype(int)
        ).all()
        and runtime[
            "current_actual_used_for_storage_decision"
        ].eq(False).all()
        and runtime[
            "current_actual_used_only_for_expost_balance"
        ].eq(True).all()
        and runtime["single_current_root"].eq(True).all()
        and runtime["conditional_tree_passed"].eq(True).all()
        and (
            runtime["expost_balance_residual_kwh"]
            .abs()
            .max()
            <= tolerance_kwh
        )
    )
    add(
        "runtime_causal_recourse",
        runtime_ok,
        (
            " 在 tau_k 储能决策前只允许 0..k-1 真实前缀; "
            "current_k 真实值只在动作确定后用于紧急购电/弃光/富余结算; "
            " 当前条件树唯一根且事后能量平衡闭合。"
        ),
    )

    return pd.DataFrame(rows)


def assert_question2_day_result_legal(
    report: pd.DataFrame,
) -> None:
    failed = report.loc[~report["passed"]]
    if failed.empty:
        return

    raise AssertionError(
        " 问题 2 单日结果合法性检查失败: \n"
        + failed.to_string(index=False)
    )


# ============================================================================
# Question2Solver
# ============================================================================


class Question2Solver:
    """
    问题 2 全年/单日求解器。

    注意:
```

```
    model_base.py 的单路径公共变量是按 k 定义；
    问题 2 实时补救必须升级为场景树 node_id 索引。
    因此本求解器复用 model_base 已冻结的参数、单位和变量命名，
    但节点动作必须按 node_id 重新创建，不能错误复用单路径 q_ch[k]
    来表示多个不确定节点。
    """

    def __init__(
        self,
        *,
        attachment1: Attachment1Data,
        attachment2: Attachment2Data,
        settings: Question2Settings = Question2Settings(),
        config: ProjectConfig = DEFAULT_CONFIG,
    ) -> None:
        settings.validate()
        _validate_attachment2_model_grid(
            attachment2,
            config,
        )

        self.attachment1 = attachment1
        self.attachment2 = attachment2
        self.settings = settings
        self.config = config

        self.time_map = build_time_map(config)
        validate_time_map(self.time_map, config)

        self.price_yuan_per_kwh = _prepare_fixed_price(
            attachment1,
            config,
        )

        self.dt_hours = config.dt_minutes / 60.0

    def _check_date_exists(
        self,
        date,
    ) -> pd.Timestamp:
        d = _normalize_date(date)

        if d not in self.attachment2.load_kw.index:
            raise KeyError(f" 附件 2 中不存在日期 {d.date()}。")
        if d not in self.attachment2.pv_kw.index:
            raise KeyError(f" 附件 2 光伏中不存在日期 {d.date()}。")

        return d

    def _build_day_ahead_tree(
        self,
        date: pd.Timestamp,
    ) -> tuple[
        DayAheadForecast,
        ResidualLibrary,
        ScenarioTrajectorySet,
        NonanticipativeScenarioTree,
    ]:
        """
        0:00 只使用 date 之前历史构造预测、残差和场景树。
        """
        forecast = build_q2_day_ahead_forecast(
            self.attachment2,
            date,
            config=self.config,
        )

        residual_library = build_causal_residual_library(
            self.attachment2,
```

```
            date,
            max_residual_days=self.settings.max_residual_days,
            config=self.config,
        )

        # 强制信息边界审计
        if forecast.history_end_date is not None:
            if not forecast.history_end_date < date:
                raise AssertionError("0:00 预测读取了当天/未来真实数据。")

        if residual_library.n_days > 0:
            if not (residual_library.dates < date).all():
                raise AssertionError("0:00 残差库混入当天/未来数据。")

        bootstrap = build_bootstrap_trajectory_set(
            forecast,
            residual_library,
            n_bootstrap=self.settings.dayahead_bootstrap_scenarios,
            random_state=_seed_for(
                self.settings.random_seed,
                date,
                0,
            ),
        )

        reduced = reduce_trajectory_set(
            bootstrap,
            n_representatives=(
                self.settings.dayahead_reduced_scenarios
            ),
        )

        tree = build_nonanticipative_tree(
            reduced,
            branch_every=self.settings.dayahead_branch_every,
            max_branch=self.settings.dayahead_max_branch,
            force_single_root=False,
        )

        return forecast, residual_library, reduced, tree


    def _read_current_actual_kwh(
        self,
        date: pd.Timestamp,
        k: int,
    ) -> tuple[float, float]:
        """
        读取刚刚完成的第 k 个物理区间真实负荷/光伏。

        重要:
        本函数虽然仍保留原接口名以兼容前文代码, 但只能在:
            第 k 个区间储能动作 q_ch[k]/q_dis[k] 已经决定之后
        调用。

        它返回的是:
            I_k=[tau_k,tau_{k+1})
        整个区间最终实现的 10 分钟电量。

        禁止在 tau_k 决定当前储能动作之前调用本函数。
        """
        if not (0 <= k < self.config.intervals_per_day):
            raise ValueError("k 必须位于 0~143。")

        load_kw = float(
            self.attachment2.load_kw.at[date, k]
        )
        pv_kw = float(
            self.attachment2.pv_kw.at[date, k]
```

```
        )

        return (
            load_kw * self.dt_hours,
            pv_kw * self.dt_hours,
        )


    def _run_causal_actual_day(
        self,
        *,
        date: pd.Timestamp,
        forecast: DayAheadForecast,
        residual_library: ResidualLibrary,
        plan_purchase_kwh: pd.Series,
        initial_energy_kwh: float,
    ) -> tuple[
        pd.Series,
        pd.Series,
        pd.Series,
        pd.Series,
        pd.Series,
        pd.Series,
        pd.DataFrame,
    ]:
        """
        严格因果的 10 分钟滚动补救。

        对每个物理区间 k，执行顺序严格为：

        A. tau_k 决策前：
           - 已知真实历史只有 0..k-1；
           - 已知当前真实 SOC E_k；
           - q_buy[k] 仍是 0:00 锁定计划；
           - 基于历史分布构造 k..143 条件场景；
           - 只决定当前 q_ch[k]/q_dis[k]。

        B. 区间 I_k 结束后：
           - 第 k 区间真实负荷/光伏才完整实现；
           - 再计算实际 q_emg[k]/q_curt[k]/q_sur[k] 闭合能量平衡；
           - 更新 E_{k+1}；
           - 把实际 k 加入历史，供下一时段 k+1 使用。

        因此 current_k 真实值绝不会用于 current_k 储能动作。
        """
        k_all = list(range(self.config.intervals_per_day))

        # 在决定 k 动作前，这两个 Series 只能包含 0..k-1。
        observed_load = pd.Series(dtype=float)
        observed_pv = pd.Series(dtype=float)

        q_ch_actual = pd.Series(
            index=k_all,
            dtype=float,
            name="actual_charge_kwh",
        )
        q_dis_actual = pd.Series(
            index=k_all,
            dtype=float,
            name="actual_discharge_kwh",
        )
        q_emg_actual = pd.Series(
            index=k_all,
            dtype=float,
            name="actual_emergency_kwh",
        )
        q_curt_actual = pd.Series(
            index=k_all,
            dtype=float,
```

```
            name="actual_curtailment_kwh",
        )
        q_sur_actual = pd.Series(
            index=k_all,
            dtype=float,
            name="actual_surplus_kwh",
        )

        E_actual = pd.Series(
            index=list(range(self.config.state_nodes_per_day)),
            dtype=float,
            name="energy_kwh",
        )
        E_actual.loc[0] = float(initial_energy_kwh)

        runtime_rows = []
        current_energy = float(initial_energy_kwh)

        for k in k_all:
            # ------------------------------------------------------------
            # 1. tau_k: 严格检查真实信息只有 0..k-1
            # ------------------------------------------------------------
            expected_prefix = list(range(k))
            if list(observed_load.index) != expected_prefix:
                raise AssertionError(
                    f"k={k} 决策前真实负荷前缀必须严格为 0..{k-1}。"
                )
            if list(observed_pv.index) != expected_prefix:
                raise AssertionError(
                    f"k={k} 决策前真实光伏前缀必须严格为 0..{k-1}。"
                )

            conditional_bootstrap = (
                build_conditional_trajectory_set(
                    forecast,
                    residual_library,
                    observed_load_kwh=observed_load.copy(),
                    observed_pv_kwh=observed_pv.copy(),
                    current_k=k,
                    n_bootstrap=(
                        self.settings.rolling_bootstrap_scenarios
                    ),
                    conditioning_pool=(
                        self.settings.conditioning_pool
                    ),
                    random_state=_seed_for(
                        self.settings.random_seed,
                        date,
                        k,
                    ),
                    config=self.config,
                )
            )

            conditional_reduced = reduce_trajectory_set(
                conditional_bootstrap,
                n_representatives=(
                    self.settings.rolling_reduced_scenarios
                ),
            )

            conditional_tree = build_nonanticipative_tree(
                conditional_reduced,
                branch_every=(
                    self.settings.rolling_branch_every
                ),
                max_branch=self.settings.rolling_max_branch,
                force_single_root=True,
            )
```

```
            tree_check = (
                conditional_tree.validate_nonanticipativity()
            )
            tree_passed = bool(tree_check["passed"].all())

            root_ids = conditional_tree.root_node_ids()
            single_root = len(root_ids) == 1

            if not single_root:
                raise AssertionError(
                    f"current_k={k} 条件补救树必须只有一个当前信息根节点。"
                )

            # ------------------------------------------------------------
            # 2. 在不知道 current_k 真实值的条件下决定储能动作
            # ------------------------------------------------------------
            recourse_solution = _solve_tree_optimization(
                tree=conditional_tree,
                price_yuan_per_kwh=self.price_yuan_per_kwh,
                initial_energy_kwh=current_energy,
                config=self.config,
                settings=self.settings,
                q_buy_fixed_kwh=plan_purchase_kwh,
                model_name=f"q2_recourse_{date.date()}_k{k}",
            )

            root_id = int(root_ids[0])
            root_row = recourse_solution.node_policy.loc[
                recourse_solution.node_policy["node_id"]
                == root_id
            ]

            if len(root_row) != 1:
                raise AssertionError(
                    " 当前根节点必须且只能有一行预决策动作。"
                )

            root_row = root_row.iloc[0]

            # 只有储能动作在 tau_k 真正执行。
            q_ch = float(root_row["q_ch_kwh"])
            q_dis = float(root_row["q_dis_kwh"])

            # ------------------------------------------------------------
            # 3. 储能动作确定以后，第 k 区间真实负荷/PV 才实现
            # ------------------------------------------------------------
            load_now, pv_now = self._read_current_actual_kwh(
                date,
                k,
            )

            # ------------------------------------------------------------
            # 4. 事后紧急购电/弃光/计划富余闭合真实能量平衡
            # ------------------------------------------------------------
            (
                q_emg,
                q_curt,
                q_sur,
                balance_residual,
            ) = settle_realized_interval_after_storage(
                q_buy_kwh=float(plan_purchase_kwh.loc[k]),
                q_ch_kwh=q_ch,
                q_dis_kwh=q_dis,
                actual_load_kwh=load_now,
                actual_pv_kwh=pv_now,
            )

            q_ch_actual.loc[k] = q_ch
```

```
            q_dis_actual.loc[k] = q_dis
            q_emg_actual.loc[k] = q_emg
            q_curt_actual.loc[k] = q_curt
            q_sur_actual.loc[k] = q_sur

            next_energy = (
                current_energy
                + ETA_C * q_ch
                - q_dis / ETA_D
            )

            if (
                next_energy < E_MIN_KWH - 1e-6
                or next_energy > E_MAX_KWH + 1e-6
            ):
                raise AssertionError(
                    f"k={k} 执行后 SOC 越界: {next_energy}"
                )

            E_actual.loc[k] = current_energy
            E_actual.loc[k + 1] = next_energy
            current_energy = next_energy

            # ------------------------------------------------------------
            # 5. current_k 结束后才写入真实历史，供 k+1 使用
            # ------------------------------------------------------------
            observed_load.loc[k] = load_now
            observed_pv.loc[k] = pv_now

            runtime_rows.append(
                {
                    "k": k,
                    "max_observed_k": k - 1,
                    "observed_points": k,
                    "future_actual_used": False,
                    "current_actual_used_for_storage_decision": False,
                    "current_actual_used_only_for_expost_balance": True,
                    "q_buy_plan_modified": False,
                    "single_current_root": single_root,
                    "conditional_tree_passed": tree_passed,
                    "current_root_id": root_id,
                    "current_root_expected_load_kwh": float(
                        root_row["load_kwh"]
                    ),
                    "current_root_expected_pv_kwh": float(
                        root_row["pv_kwh"]
                    ),
                    "realized_load_kwh": load_now,
                    "realized_pv_kwh": pv_now,
                    "predicted_root_q_emg_kwh_not_executed": float(
                        root_row["q_emg_kwh"]
                    ),
                    "expost_balance_residual_kwh": (
                        balance_residual
                    ),
                }
            )

        return (
            q_ch_actual,
            q_dis_actual,
            q_emg_actual,
            q_curt_actual,
            q_sur_actual,
            E_actual,
            pd.DataFrame(runtime_rows),
        )

    def solve_day(
```

```
        self,
        date,
        *,
        initial_energy_kwh: float,
        check_legality: bool = True,
    ) -> Question2DayResult:
        """
        求解并真实因果执行一天。

        0:00 阶段:
            预测 -> 历史残差 -> 非预见场景树 -> 唯一 q_buy 计划

        实时阶段:
            current actual + current SOC
            -> 条件历史场景
            -> 固定 q_buy 的剩余期补救模型
            -> 只执行当前第一步
        """
        d = self._check_date_exists(date)

        if not (
            E_MIN_KWH
            <= float(initial_energy_kwh)
            <= E_MAX_KWH
        ):
            raise ValueError("initial_energy_kwh 超出储能安全范围。")

        (
            forecast,
            residual_library,
            reduced,
            day_tree,
        ) = self._build_day_ahead_tree(d)

        # 0:00 场景树模型: 只锁定一套 q_buy。
        day_solution = _solve_tree_optimization(
            tree=day_tree,
            price_yuan_per_kwh=self.price_yuan_per_kwh,
            initial_energy_kwh=float(initial_energy_kwh),
            config=self.config,
            settings=self.settings,
            q_buy_fixed_kwh=None,
            model_name=f"q2_dayahead_{d.date()}",
        )

        nonanticipativity_report = (
            _build_solver_nonanticipativity_report(
                day_tree,
                day_solution,
            )
        )

        if not nonanticipativity_report["passed"].all():
            raise AssertionError(
                "0:00 模型未通过非预见性检查。"
            )

        plan_purchase = (
            day_solution.q_buy_kwh.astype(float).copy()
        )
        plan_purchase.name = "plan_purchase_kwh"

        (
            q_ch_actual,
            q_dis_actual,
            q_emg_actual,
            q_curt_actual,
            q_sur_actual,
            E_actual,
```

```
            runtime_report,
        ) = self._run_causal_actual_day(
            date=d,
            forecast=forecast,
            residual_library=residual_library,
            plan_purchase_kwh=plan_purchase,
            initial_energy_kwh=float(initial_energy_kwh),
        )

        plan_cost = float(
            (
                self.price_yuan_per_kwh
                * plan_purchase
            ).sum()
        )

        emergency_cost = float(
            (
                5.0
                * self.price_yuan_per_kwh
                * q_emg_actual
            ).sum()
        )

        total_cost = plan_cost + emergency_cost

        result = Question2DayResult(
            date=d,
            initial_energy_kwh=float(initial_energy_kwh),
            end_energy_kwh=float(E_actual.loc[144]),
            plan_purchase_kwh=plan_purchase,
            actual_charge_kwh=q_ch_actual,
            actual_discharge_kwh=q_dis_actual,
            actual_emergency_kwh=q_emg_actual,
            actual_curtailment_kwh=q_curt_actual,
            actual_surplus_kwh=q_sur_actual,
            energy_kwh=E_actual,
            plan_cost_yuan=plan_cost,
            emergency_cost_yuan=emergency_cost,
            total_cost_yuan=total_cost,
            total_plan_purchase_kwh=float(
                plan_purchase.sum()
            ),
            total_emergency_kwh=float(
                q_emg_actual.sum()
            ),
            load_forecast_kwh=forecast.load_kwh.copy(),
            pv_forecast_kwh=forecast.pv_kwh.copy(),
            day_ahead_tree_nodes=day_tree.to_frame(),
            day_ahead_node_policy=(
                day_solution.node_policy.copy()
            ),
            day_ahead_nonanticipativity_report=(
                nonanticipativity_report
            ),
            runtime_causality_report=runtime_report,
            legality_report=pd.DataFrame(),
        )

        legality = check_question2_day_result_legality(
            result=result,
            attachment2=self.attachment2,
            price_yuan_per_kwh=self.price_yuan_per_kwh,
            config=self.config,
            enforce_charge_discharge_exclusivity=(
                self.settings
                .enforce_charge_discharge_exclusivity
            ),
        )
```

```
        result.legality_report = legality

        if check_legality:
            assert_question2_day_result_legal(legality)

        return result

    def solve_period(
        self,
        *,
        start_date: str = "2025-01-01",
        end_date: str = "2025-12-31",
        initial_energy_kwh: float = 6000.0,
        save_from_date: str = "2025-02-01",
        check_legality: bool = True,
    ) -> Question2PeriodResult:
        """
        多日顺序滚动。

        [Q2-10]
            E^{act}_{d+1,0} = E^{act}_{d,144}

        默认:
        - 2025-01-01 从 6000 kWh 开始;
        - 1 月份正常模拟但不保存正式结果;
        - 2025-02-01 起保存 formal_results;
        - 绝不在 2 月 1 日重新把 SOC 设为 6000。
        """
        start = _normalize_date(start_date)
        end = _normalize_date(end_date)
        save_from = _normalize_date(save_from_date)

        if end < start:
            raise ValueError("end_date 不能早于 start_date。")

        dates = pd.date_range(
            start,
            end,
            freq="D",
        )

        formal_results: Dict[
            pd.Timestamp,
            Question2DayResult
        ] = {}

        cross_rows = []

        current_energy = float(initial_energy_kwh)
        warmup_end_energy = None

        previous_date = None
        previous_end_energy = None

        for d in dates:
            if d not in self.attachment2.load_kw.index:
                raise KeyError(
                    f" 附件 2 缺少日期 {d.date()}, 无法连续模拟。"
                )

            day_initial = current_energy

            # [Q2-10] 与前一天末 SOC 完全连续。
            if previous_date is not None:
                residual = abs(
                    day_initial - float(previous_end_energy)
                )
                cross_rows.append(
```

```
                    {
                        "previous_date": previous_date,
                        "current_date": d,
                        "previous_end_energy_kwh": (
                            previous_end_energy
                        ),
                        "current_initial_energy_kwh": (
                            day_initial
                        ),
                        "residual_kwh": residual,
                        "passed": residual <= 1e-8,
                    }
                )

            day_result = self.solve_day(
                d,
                initial_energy_kwh=day_initial,
                check_legality=check_legality,
            )

            current_energy = float(
                day_result.end_energy_kwh
            )

            print(
                f"[Q2] {d.date()} 完成: 总成本 {day_result.total_cost_yuan:.2f} 元, "
                f" 末 SOC {current_energy:.2f} kWh",
                flush=True,
            )

            previous_date = d
            previous_end_energy = current_energy

            if d < save_from:
                # 暖启动结果不进入正式输出。
                warmup_end_energy = current_energy
            else:
                formal_results[d] = day_result

        cross_report = pd.DataFrame(cross_rows)

        if (
            not cross_report.empty
            and not cross_report["passed"].all()
        ):
            raise AssertionError(
                " 跨日 SOC 连续性检查失败。"
            )

        return Question2PeriodResult(
            formal_results=formal_results,
            cross_day_report=cross_report,
            warmup_end_energy_kwh=warmup_end_energy,
            final_energy_kwh=float(current_energy),
        )
```

文件：code/question3_solver.py

```
# -*- coding: utf-8 -*-
"""
question3_solver.py
问题 3：0/6/12/18 滚动购电调整 + 附件 3 光伏预测 + 因果实时补救。

严格继承前文工程：
- config.py / time_index.py / data_loader.py
- forecast_model.py
- scenario_tree.py
- question2_solver.py 的条件场景实时补救求解器
- forecast_parser.py
- rolling_scheduler.py

不重新定义前文时间映射、变量单位或问题 2 变量命名。

问题 3 核心规则
-------------
1. 0:00：
       k>=0 可优化，形成唯一原始计划 G0。
2. 6:00：
       冻结 k<36，只允许更新 k>=36。
3. 12:00：
       冻结 k<72，只允许更新 k>=72。
4. 18:00：
       冻结 k<108，只允许更新 k>=108。
5. 每次更新使用真实当前 SOC E_act[r]。
6. 正常购电更新只用：
       当前/过去真实状态 + 当前已发布预测 + 历史误差分布。
   禁止读取未来真实负荷/光伏。
7. 每 10 分钟在 tau_k 先基于 0..k-1 真实历史和 E_k 决定储能动作；
   current_k 真实负荷/PV 在区间结束后才用于紧急购电、弃光和计划富余；
   不允许 current_k 实际值参与 current_k 储能动作。
8. 18:00 发布版本可以把次日 0:00~18:00 作为 24 小时预览窗口，
   只用于评价当前 SOC 的未来价值；次日预览购电变量全部丢弃，
   绝不提前写成次日正式计划。
9. 结算严格调用 rolling_scheduler.settle_question3_cost()：
       C_reg = sum c[GF + 0.5(dG+ + dG-)]
       C_adj = sum c[1.5dG+ - 0.5dG-]
       C_total = C_reg + C_emg
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Callable, Dict, Optional, Sequence

import numpy as np
import pandas as pd

try:
    import gurobipy as gp
    from gurobipy import GRB
except ImportError as exc:  # pragma: no cover
    raise ImportError(
        "question3_solver.py 依赖 gurobipy。"
        " 请在已安装并配置 Gurobi 的环境中运行。"
    ) from exc

from config import DEFAULT_CONFIG, ProjectConfig
from data_loader import (
    Attachment1Data,
    Attachment2Data,
    Attachment3Data,
)
from forecast_model import (
```

```
    AvailableHistory,
    DayAheadForecast,
    ResidualLibrary,
    build_available_history,
    build_causal_residual_library,
    build_q2_day_ahead_forecast,
)
from forecast_parser import (
    ParsedForecastVersion,
    extract_current_day_model_kwh,
    parse_forecast_version,
)
from model_base import (
    ETA_C,
    ETA_D,
    E_MIN_KWH,
    E_MAX_KWH,
    P_BAT_MAX_KW,
)
from question2_solver import (
    Question2Settings,
    _solve_tree_optimization,
    settle_realized_interval_after_storage,
)
from rolling_scheduler import (
    ABLATION_UPDATE_COMBINATIONS,
    SETTLEMENT_SELF_TEST_REPORT,
    UPDATE_K,
    RollingPlanBook,
    SettlementResult,
    settle_question3_cost,
    validate_update_k,
)
from scenario_tree import (
    build_conditional_trajectory_set,
    build_nonanticipative_tree,
    reduce_trajectory_set,
)
from time_index import build_time_map, validate_time_map


# ============================================================================
# 配置 / 结果
# ============================================================================


@dataclass(frozen=True)
class Question3Settings:
    """ 问题 3 滚动优化设置。"""

    # 默认使用全部 4 次更新
    update_combination: tuple[int, ...] = (
        0,
        36,
        72,
        108,
    )

    # 每次更新使用 24h 滚动窗口
    rolling_horizon_intervals: int = 144

    # 实时因果补救沿用 Q2 条件场景
    q2_runtime_settings: Question2Settings = field(
        default_factory=Question2Settings
    )

    solver_output: bool = False
    enforce_charge_discharge_exclusivity: bool = False

```

```
135     def validate(
136         self,
137         config: ProjectConfig = DEFAULT_CONFIG,
138     ) -> None:
139         if 0 not in self.update_combination:
140             raise ValueError(" 问题 3 更新组合必须包含 0:00。")
141 
142         if tuple(sorted(set(self.update_combination))) != (
143             self.update_combination
144         ):
145             raise ValueError(
146                 "update_combination 必须升序且不得重复。"
147             )
148 
149         for r in self.update_combination:
150             validate_update_k(r, config=config)
151 
152         if self.rolling_horizon_intervals != 144:
153             raise ValueError(
154                 " 附件 3 每次发布覆盖 24h, 本实现固定 144 个 10 分钟区间。"
155             )
156 
157         self.q2_runtime_settings.validate()
158 
159 
160 @dataclass
161 class MPCUpdateRecord:
162     """ 一次 0/6/12/18 更新记录。"""
163 
164     r: int
165     issue_datetime: pd.Timestamp
166     current_energy_kwh: float
167 
168     current_observed_pv_kw: float
169 
170     formal_future_plan_kwh: pd.Series
171 
172     # 只用于 SOC 未来价值, 不进入正式次日计划。
173     preview_purchase_kwh: pd.DataFrame
174 
175     objective_value_yuan: float
176 
177     parser_audit: pd.DataFrame
178 
179 
180 @dataclass
181 class Question3DayResult:
182     """ 问题 3 单日结果。"""
183 
184     date: pd.Timestamp
185     update_combination: tuple[int, ...]
186 
187     initial_energy_kwh: float
188     end_energy_kwh: float
189 
190     # G0 / GF
191     original_plan_kwh: pd.Series
192     final_effective_plan_kwh: pd.Series
193 
194     # 每次更新后的计划版本
195     plan_versions: Dict[int, pd.Series]
196     update_records: Dict[int, MPCUpdateRecord]
197 
198     # 实际因果运行
199     actual_charge_kwh: pd.Series
200     actual_discharge_kwh: pd.Series
201     actual_emergency_kwh: pd.Series
202     actual_curtailment_kwh: pd.Series
203     actual_surplus_kwh: pd.Series
```

```
    energy_kwh: pd.Series

    # 结算
    dG_plus_kwh: pd.Series
    dG_minus_kwh: pd.Series
    C_plan_yuan: float
    C_reg_yuan: float
    C_adj_yuan: float
    C_emg_yuan: float
    C_total_yuan: float

    # 因果/冻结/结算审计
    runtime_causality_report: pd.DataFrame
    update_freeze_report: pd.DataFrame
    settlement_test_report: pd.DataFrame
    legality_report: pd.DataFrame


@dataclass
class Question3PeriodResult:
    formal_results: Dict[pd.Timestamp, Question3DayResult]
    cross_day_report: pd.DataFrame
    warmup_end_energy_kwh: Optional[float]
    final_energy_kwh: float


# ============================================================================
# 数据准备
# ============================================================================


def _normalize_date(value) -> pd.Timestamp:
    return pd.Timestamp(value).normalize()


def _issue_datetime_from_update_k(
    date: pd.Timestamp,
    r: int,
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Timestamp:
    """
    将滚动更新索引 r 映射为附件 3 的真实发布时间。

    物理映射固定为:
        r=0   -> 0:00
        r=36  -> 6:00
        r=72  -> 12:00
        r=108 -> 18:00

    这里显式使用 r * dt_minutes，禁止再额外加一个 10 分钟步长。
    """
    validate_update_k(r, config=config)
    issue = pd.Timestamp(date).normalize() + pd.Timedelta(
        minutes=int(r) * int(config.dt_minutes)
    )

    expected_hour = int(r * config.dt_minutes // 60)
    if issue.minute != 0 or issue.hour != expected_hour:
        raise AssertionError(
            f" 附件 3 发布时间映射错误: r={r}, issue={issue}, "
            f"expected_hour={expected_hour}:00"
        )
    if issue.hour not in set(config.issue_hours):
        raise AssertionError(
            f" 附件 3 发布时间不在允许集合 {config.issue_hours} 中: {issue}"
        )
    return issue

```

```
273
274 def _prepare_fixed_price(
275     attachment1: Attachment1Data,
276     config: ProjectConfig,
277 ) -> pd.Series:
278     """ 严格按 attachment1.aligned 中的 k 读取固定日内电价。"""
279     aligned = attachment1.aligned
280     expected = list(range(config.intervals_per_day))
281
282     if aligned["k"].astype(int).tolist() != expected:
283         raise ValueError(
284             "attachment1.aligned 必须严格使用 k=0~143。"
285         )
286
287     price = pd.to_numeric(
288         aligned["price_yuan_per_kwh"],
289         errors="raise",
290     ).astype(float)
291     price.index = expected
292     price.name = "price_yuan_per_kwh"
293
294     if price.isna().any():
295         raise ValueError(" 电价存在缺失值。")
296
297     return price
298
299
300 def _causal_preview_load_for_next_day(
301     history: AvailableHistory,
302     *,
303     current_date: pd.Timestamp,
304     config: ProjectConfig,
305 ) -> pd.Series:
306     """
307     为 24h MPC 跨日预览构造次日负荷预测。
308
309     重要:
310     只能使用 current_date 0:00 以前的 AvailableHistory。
311     绝不调用 target=next_day 的普通预测函数, 否则可能读入 current_date 实际值。
312
313     规则:
314     - next_day 对应的上周同星期日为 current_date-6;
315     - 若该日已存在于历史, 则使用其同 k 负荷;
316     - 否则使用已有历史同 k 均值;
317     - 无历史则返回 0 暖启动。
318     """
319     expected = list(range(config.intervals_per_day))
320
321     if history.history_days == 0:
322         return pd.Series(
323             np.zeros(config.intervals_per_day),
324             index=expected,
325             dtype=float,
326             name="next_day_preview_load_kwh",
327         )
328
329     next_day = current_date + pd.Timedelta(days=1)
330     lag7_for_next = next_day - pd.Timedelta(days=7)
331
332     if lag7_for_next in history.load_kw.index:
333         load_kw = history.load_kw.loc[
334             lag7_for_next
335         ].astype(float)
336     else:
337         load_kw = history.load_kw.mean(axis=0).astype(float)
338
339     load_kw.index = expected
340
341     dt_hours = config.dt_minutes / 60.0
```

```
    result = load_kw.clip(lower=0.0) * dt_hours
    result.name = "next_day_preview_load_kwh"
    return result


def _current_observed_pv_kw(
    attachment2: Attachment2Data,
    date: pd.Timestamp,
    r: int,
    *,
    config: ProjectConfig,
) -> float:
    """
    生成附件 3 线性插值所需 F0。

    r>0:
        使用刚刚完成的最后一个 10min 实际区间 k=r-1 的平均功率,
        作为发布边界时刻当前观测功率的可复现代理。
        例如 6:00 使用 k=35, 即附件 2 右端点 6:00。

    r=0:
        使用前一自然日 k=143 (右端点 0:00+1) 的已实现功率;
        若不存在前一日 (2025-01-01), 使用 0。

    从不读取当前未来区间 k=r 的实际光伏。
    """
    validate_update_k(r, config=config)

    if r > 0:
        return max(
            0.0,
            float(attachment2.pv_kw.at[date, r - 1]),
        )

    previous_date = date - pd.Timedelta(days=1)
    if previous_date in attachment2.pv_kw.index:
        return max(
            0.0,
            float(attachment2.pv_kw.at[previous_date, 143]),
        )

    return 0.0


def _build_effective_q3_forecast(
    base_forecast: DayAheadForecast,
    effective_pv_kwh: pd.Series,
) -> DayAheadForecast:
    """
    仅替换 Q3 当前已发布版本对未来的 PV 预测,
    负荷日前预测和历史信息边界保持 Q2 结构。
    """
    expected = list(base_forecast.load_kwh.index)

    if list(effective_pv_kwh.index) != expected:
        raise ValueError(
            "effective_pv_kwh 必须覆盖当天 k=0~143。"
        )

    dt_hours = 1.0 / 6.0
    pv_kw = effective_pv_kwh / dt_hours
    pv_kw.name = "pv_forecast_kw"

    return DayAheadForecast(
        target_date=base_forecast.target_date,
        load_kw=base_forecast.load_kw.copy(),
        pv_kw=pv_kw,
        load_kwh=base_forecast.load_kwh.copy(),
        pv_kwh=effective_pv_kwh.copy(),
```

```
        history_days=base_forecast.history_days,
        history_end_date=base_forecast.history_end_date,
        load_method=base_forecast.load_method,
        pv_method="attachment3_latest_released_version",
    )


# ============================================================================
# 24 小时滚动计划优化
# ============================================================================


def _solve_deterministic_mpc_update(
    *,
    date: pd.Timestamp,
    r: int,
    current_energy_kwh: float,
    original_plan_kwh: Optional[pd.Series],
    load_today_kwh: pd.Series,
    load_next_day_preview_kwh: pd.Series,
    parsed_pv: ParsedForecastVersion,
    price_provider: Callable[
        [pd.Timestamp, int, pd.Timestamp], float
    ],
    config: ProjectConfig,
    enforce_charge_discharge_exclusivity: bool,
    solver_output: bool,
) -> tuple[pd.Series, pd.DataFrame, float]:
    """
    当前更新时刻的 24h 确定性 MPC。

    current-day 区间:
        q_buy 为正式购电计划变量。

    next-day 预览区间:
        q_buy_preview 只用于评价当前 SOC 未来价值,
        求解后全部丢弃, 不写入次日正式计划。

    price_provider(abs_date, model_k, decision_datetime):
        返回当前决策时刻允许使用的预测/已知价格。
        Q3 固定电价时忽略 decision_datetime;
        Q4-3 可传入严格因果动态价格预测。
    """
    validate_update_k(r, config=config)

    issue_datetime = _issue_datetime_from_update_k(
        date,
        r,
        config=config,
    )

    frame = parsed_pv.interval_forecast
    if len(frame) != 144:
        raise ValueError(" 附件 3 发布版本必须恰好 144 个 10 分钟区间。")

    if not (
        E_MIN_KWH
        <= current_energy_kwh
        <= E_MAX_KWH
    ):
        raise ValueError(" 当前真实 SOC 超出安全区间。")

    horizon = list(range(144))
    dt_hours = config.dt_minutes / 60.0
    q_max = P_BAT_MAX_KW * dt_hours

    model = gp.Model(
        f"q3_mpc_{date.date()}_r{r}"
    )
```

```
    model.Params.OutputFlag = 1 if solver_output else 0

    q_buy_h = model.addVars(
        horizon,
        lb=0.0,
        vtype=GRB.CONTINUOUS,
        name="q_buy",
    )
    q_ch = model.addVars(
        horizon,
        lb=0.0,
        ub=q_max,
        vtype=GRB.CONTINUOUS,
        name="q_ch",
    )
    q_dis = model.addVars(
        horizon,
        lb=0.0,
        ub=q_max,
        vtype=GRB.CONTINUOUS,
        name="q_dis",
    )
    q_emg = model.addVars(
        horizon,
        lb=0.0,
        vtype=GRB.CONTINUOUS,
        name="q_emg",
    )
    q_curt = model.addVars(
        horizon,
        lb=0.0,
        vtype=GRB.CONTINUOUS,
        name="q_curt",
    )
    q_sur = model.addVars(
        horizon,
        lb=0.0,
        vtype=GRB.CONTINUOUS,
        name="q_sur",
    )
    E = model.addVars(
        range(145),
        lb=E_MIN_KWH,
        ub=E_MAX_KWH,
        vtype=GRB.CONTINUOUS,
        name="E",
    )

    z = None
    if enforce_charge_discharge_exclusivity:
        z = model.addVars(
            horizon,
            vtype=GRB.BINARY,
            name="z",
        )

    model.addConstr(
        E[0] == float(current_energy_kwh),
        name="current_real_energy",
    )

    # 记录每个 h 对应绝对物理时间/自然日 k
    mapping_rows = []
    current_day_h = []
    preview_h = []

    objective_terms = []

    dplus = {}
```

```
    dminus = {}

    for h in horizon:
        interval_start = pd.Timestamp(
            frame.iloc[h][
                "interval_start_datetime"
            ]
        )
        interval_end = pd.Timestamp(
            frame.iloc[h][
                "interval_end_datetime"
            ]
        )

        abs_date = interval_start.normalize()
        minute_of_day = (
            interval_start.hour * 60
            + interval_start.minute
        )
        model_k = minute_of_day // config.dt_minutes

        if not (0 <= model_k < 144):
            raise AssertionError(" 预览区间 model_k 异常。")

        if abs_date == date:
            if model_k < r:
                raise AssertionError(
                    "24h MPC 窗口不应包含已执行当前日过去区间。"
                )
            load_kwh = float(load_today_kwh.loc[model_k])
            is_formal = True
            current_day_h.append((h, model_k))
        else:
            # 当前附件 3 版本最多跨到次日。
            if abs_date != date + pd.Timedelta(days=1):
                raise AssertionError(
                    " 当前 24h 附件 3 窗口出现超过次日的区间。"
                )

            load_kwh = float(
                load_next_day_preview_kwh.loc[model_k]
            )
            is_formal = False
            preview_h.append((h, model_k))

        pv_kwh = float(
            frame.iloc[h]["forecast_kwh"]
        )

        price = float(
            price_provider(
                abs_date,
                int(model_k),
                issue_datetime,
            )
        )

        if price < 0:
            raise ValueError(" 当前实现要求预测/已知电价非负。")

        # 物理能量平衡
        model.addConstr(
            q_buy_h[h]
            + q_emg[h]
            + pv_kwh
            + q_dis[h]
            ==
            load_kwh
            + q_ch[h]
```

```
            + q_curt[h]
            + q_sur[h],
            name=f"energy_balance[{h}]",
        )

        model.addConstr(
            q_curt[h] <= pv_kwh,
            name=f"curtailment_upper[{h}]",
        )

        model.addConstr(
            E[h + 1]
            ==
            E[h]
            + ETA_C * q_ch[h]
            - q_dis[h] / ETA_D,
            name=f"storage_transition[{h}]",
        )

        if z is not None:
            model.addConstr(
                q_ch[h] <= q_max * z[h],
                name=f"charge_exclusive[{h}]",
            )
            model.addConstr(
                q_dis[h] <= q_max * (1 - z[h]),
                name=f"discharge_exclusive[{h}]",
            )

        if is_formal and r > 0:
            if original_plan_kwh is None:
                raise ValueError(
                    "r>0 调整时必须提供 0:00 原始计划 G0。"
                )

            # dG_plus / dG_minus
            dplus[h] = model.addVar(
                lb=0.0,
                vtype=GRB.CONTINUOUS,
                name=f"dG_plus[{model_k}]",
            )
            dminus[h] = model.addVar(
                lb=0.0,
                vtype=GRB.CONTINUOUS,
                name=f"dG_minus[{model_k}]",
            )

            model.addConstr(
                q_buy_h[h]
                - float(original_plan_kwh.loc[model_k])
                ==
                dplus[h] - dminus[h],
                name=f"adjustment_definition[{model_k}]",
            )

            # 正常购电及调整直接结算形式:
            # c * [GF + 0.5(dG+ + dG-)]
            objective_terms.append(
                price
                * (
                    q_buy_h[h]
                    + 0.5 * (dplus[h] + dminus[h])
                )
            )
        else:
            # r=0 当前日正式计划, 或跨日 preview:
            # 只按正常购电成本评价。
            objective_terms.append(
                price * q_buy_h[h]
```

```
            )

        # forecast shortage emergency
        objective_terms.append(
            5.0 * price * q_emg[h]
        )

        mapping_rows.append(
            {
                "h": h,
                "interval_start_datetime": interval_start,
                "interval_end_datetime": interval_end,
                "date": abs_date,
                "model_k": int(model_k),
                "is_formal_current_day": is_formal,
                "price_used_yuan_per_kwh": price,
            }
        )

    model.setObjective(
        gp.quicksum(objective_terms),
        GRB.MINIMIZE,
    )
    model.optimize()

    if model.Status != GRB.OPTIMAL:
        raise RuntimeError(
            f" 问题 3 r={r} MPC 未获得最优解, "
            f"Gurobi Status={model.Status}"
        )

    formal = pd.Series(
        {
            model_k: float(q_buy_h[h].X)
            for h, model_k in current_day_h
        },
        dtype=float,
        name="formal_future_plan_kwh",
    )
    formal = formal.sort_index()

    expected_formal_index = list(range(r, 144))
    if list(formal.index) != expected_formal_index:
        raise AssertionError(
            f"r={r} 正式计划必须只覆盖 k={r}~143。"
        )

    preview_rows = []
    for h, model_k in preview_h:
        map_row = mapping_rows[h]
        preview_rows.append(
            {
                "h": h,
                "interval_start_datetime": (
                    map_row["interval_start_datetime"]
                ),
                "interval_end_datetime": (
                    map_row["interval_end_datetime"]
                ),
                "model_k": model_k,
                "preview_purchase_kwh": float(
                    q_buy_h[h].X
                ),
                "preview_energy_start_kwh": float(
                    E[h].X
                ),
                "preview_energy_end_kwh": float(
                    E[h + 1].X
                ),
```

```
            }
        )

    preview = pd.DataFrame(preview_rows)

    return formal, preview, float(model.ObjVal)


# ============================================================================
# 合法性检查
# ============================================================================


def check_question3_day_result_legality(
    *,
    result: Question3DayResult,
    attachment2: Attachment2Data,
    price_yuan_per_kwh: pd.Series,
    config: ProjectConfig = DEFAULT_CONFIG,
    tolerance_kwh: float = 1e-5,
    tolerance_yuan: float = 1e-5,
) -> pd.DataFrame:
    """ 问题 3 事后合法性审计；只在一天执行结束后读取完整真实轨迹。"""
    d = result.date
    k_all = list(range(144))
    dt_hours = config.dt_minutes / 60.0

    actual_load = (
        attachment2.load_kw.loc[d].astype(float)
        * dt_hours
    )
    actual_pv = (
        attachment2.pv_kw.loc[d].astype(float)
        * dt_hours
    )
    actual_load.index = k_all
    actual_pv.index = k_all

    rows = []

    def add(
        item: str,
        passed: bool,
        detail: str,
        residual: Optional[float] = None,
    ):
        rows.append(
            {
                "item": item,
                "passed": bool(passed),
                "max_residual": residual,
                "detail": detail,
            }
        )

    q_buy = result.final_effective_plan_kwh
    q_ch = result.actual_charge_kwh
    q_dis = result.actual_discharge_kwh
    q_emg = result.actual_emergency_kwh
    q_curt = result.actual_curtailment_kwh
    q_sur = result.actual_surplus_kwh
    E = result.energy_kwh

    balance = (
        q_buy + q_emg + actual_pv + q_dis
        - actual_load - q_ch - q_curt - q_sur
    )
    balance_max = float(balance.abs().max())
    add(
```

```
        "actual_energy_balance",
        balance_max <= tolerance_kwh,
        f" 最大绝对残差 ={balance_max:.12g} kWh。",
        balance_max,
    )

    transition = pd.Series(
        {
            k: (
                E.loc[k + 1]
                - E.loc[k]
                - ETA_C * q_ch.loc[k]
                + q_dis.loc[k] / ETA_D
            )
            for k in k_all
        },
        dtype=float,
    )
    transition_max = float(transition.abs().max())
    add(
        "actual_storage_transition",
        transition_max <= tolerance_kwh,
        f" 最大绝对残差 ={transition_max:.12g} kWh。",
        transition_max,
    )

    lower = float((E_MIN_KWH - E).clip(lower=0).max())
    upper = float((E - E_MAX_KWH).clip(lower=0).max())
    add(
        "energy_bounds",
        max(lower, upper) <= tolerance_kwh,
        f"lower={lower:.12g}; upper={upper:.12g}",
        max(lower, upper),
    )

    # 冻结检查
    freeze_ok = bool(
        result.update_freeze_report["passed"].all()
    )
    add(
        "past_frozen_at_all_updates",
        freeze_ok,
        " 每次 r 只允许修改 k>=r。",
    )

    runtime = result.runtime_causality_report
    causal_ok = bool(
        runtime["future_actual_used"].eq(False).all()
        and runtime["max_observed_k"].astype(int)
        .eq(runtime["k"].astype(int) - 1)
        .all()
        and runtime["observed_points"].astype(int)
        .eq(runtime["k"].astype(int))
        .all()
        and runtime[
            "current_actual_used_for_storage_decision"
        ].eq(False).all()
        and runtime[
            "current_actual_used_only_for_expost_balance"
        ].eq(True).all()
        and runtime["single_current_root"].eq(True).all()
        and runtime["conditional_tree_passed"].eq(True).all()
        and (
            runtime["expost_balance_residual_kwh"]
            .abs()
            .max()
            <= tolerance_kwh
        )
    )
```

```
    add(
        "runtime_no_future_actual",
        causal_ok,
        (
            "tau_k 储能决策只读取 0..k-1 真实前缀; "
            "current_k 实际值仅在储能动作确定后用于事后平衡。"
        ),
    )

    # 费用重算
    settlement = settle_question3_cost(
        G0_kwh=result.original_plan_kwh,
        GF_kwh=result.final_effective_plan_kwh,
        price_yuan_per_kwh=price_yuan_per_kwh,
        G_emg_kwh=result.actual_emergency_kwh,
        config=config,
    )

    costs = [
        (
            "C_plan",
            result.C_plan_yuan,
            settlement.C_plan_yuan,
        ),
        (
            "C_reg",
            result.C_reg_yuan,
            settlement.C_reg_yuan,
        ),
        (
            "C_adj",
            result.C_adj_yuan,
            settlement.C_adj_yuan,
        ),
        (
            "C_emg",
            result.C_emg_yuan,
            settlement.C_emg_yuan,
        ),
        (
            "C_total",
            result.C_total_yuan,
            settlement.C_total_yuan,
        ),
    ]

    for name, stored, recalculated in costs:
        residual = abs(float(stored) - float(recalculated))
        add(
            name,
            residual <= tolerance_yuan,
            (
                f"stored={stored:.12g}; "
                f"recalculated={recalculated:.12g}"
            ),
            residual,
        )

    add(
        "settlement_identity",
        settlement.identity_residual_yuan
        <= tolerance_yuan,
        (
            "C_reg 必须等于 C_plan+C_adj; "
            f" 残差 ={settlement.identity_residual_yuan:.12g}"
        ),
        settlement.identity_residual_yuan,
    )

```

```
    return pd.DataFrame(rows)


# ============================================================================
# Question3Solver
# ============================================================================


class Question3Solver:
    """
    问题 3 求解器。

    price_provider 为可选扩展点:
    - 默认 Q3 使用附件 1 固定日内价格;
    - Q4-3 可传入严格因果动态价格预测 provider,
      从而复用整个 Question3Solver 而不复制代码。
    """

    def __init__(
        self,
        *,
        attachment1: Attachment1Data,
        attachment2: Attachment2Data,
        attachment3: Attachment3Data,
        settings: Question3Settings = Question3Settings(),
        config: ProjectConfig = DEFAULT_CONFIG,
        price_provider: Optional[
            Callable[
                [pd.Timestamp, int, pd.Timestamp],
                float,
            ]
        ] = None,
    ) -> None:
        settings.validate(config)

        self.attachment1 = attachment1
        self.attachment2 = attachment2
        self.attachment3 = attachment3
        self.settings = settings
        self.config = config

        self.time_map = build_time_map(config)
        validate_time_map(self.time_map, config)

        self.fixed_price = _prepare_fixed_price(
            attachment1,
            config,
        )

        if price_provider is None:
            self.price_provider = self._fixed_price_provider
        else:
            self.price_provider = price_provider

        self.dt_hours = config.dt_minutes / 60.0

    def _fixed_price_provider(
        self,
        abs_date: pd.Timestamp,
        model_k: int,
        decision_datetime: pd.Timestamp,
    ) -> float:
        del abs_date, decision_datetime
        return float(self.fixed_price.loc[model_k])

    def _price_series_for_decision(
        self,
        date: pd.Timestamp,
        decision_k: int,
```

```
    ) -> pd.Series:
        """
        为 Q2 条件实时补救构造当前决策时刻可使用的 144 价格向量。

        Q3 固定价格下等于附件 1 价格;
        Q4-3 通过 price_provider 得到严格因果预测价格。
        """
        decision_datetime = (
            date
            + pd.Timedelta(
                minutes=decision_k * self.config.dt_minutes
            )
        )

        values = {}
        for k in range(144):
            # 当前日每个 model_k 的定价预测接口;
            # provider 必须自行保证动态价格因果边界。
            values[k] = float(
                self.price_provider(
                    date,
                    k,
                    decision_datetime,
                )
            )

        result = pd.Series(
            values,
            dtype=float,
            name="price_yuan_per_kwh",
        )

        if (result < 0).any():
            raise ValueError(" 价格预测不能为负。")

        return result

    def _check_date_exists(
        self,
        date,
    ) -> pd.Timestamp:
        d = _normalize_date(date)
        if d not in self.attachment2.load_kw.index:
            raise KeyError(f" 附件 2 缺少日期 {d.date()}。")
        return d

    def _build_parser_version(
        self,
        date: pd.Timestamp,
        r: int,
    ) -> tuple[ParsedForecastVersion, float]:
        issue = _issue_datetime_from_update_k(
            date,
            r,
            config=self.config,
        )

        F0 = _current_observed_pv_kw(
            self.attachment2,
            date,
            r,
            config=self.config,
        )

        parsed = parse_forecast_version(
            self.attachment3,
            issue,
            current_observed_pv_kw=F0,
            config=self.config,
```

```
        )

        return parsed, F0


    def _runtime_recourse_step(
        self,
        *,
        date: pd.Timestamp,
        k: int,
        current_energy_kwh: float,
        observed_load_kwh: pd.Series,
        observed_pv_kwh: pd.Series,
        effective_forecast: DayAheadForecast,
        residual_library: ResidualLibrary,
        fixed_plan_kwh: pd.Series,
    ) -> tuple[
        float,
        float,
        float,
        float,
        float,
        pd.DataFrame,
    ]:
        """
        复用问题 2 严格因果条件场景树和 tree optimizer。

        在 tau_k 调用本函数时:
        - observed_load_kwh / observed_pv_kwh 只能包含 0..k-1;
        - current_k 真实负荷/PV 尚未知;
        - 当前真实 SOC E_k 已知;
        - 正常购电计划固定为最近一次滚动更新后的生效计划。

        本函数只真正执行根节点的 q_ch/q_dis。
        根节点预测出的 q_emg/q_curt/q_sur 只作为优化内部预估,
        不能当作真实执行量; 实际值必须在 current_k 真实负荷/PV 实现后
        由 settle_realized_interval_after_storage() 计算。
        """
        runtime_settings = (
            self.settings.q2_runtime_settings
        )

        expected_prefix = list(range(k))
        if list(observed_load_kwh.index) != expected_prefix:
            raise ValueError(
                f"k={k} 储能决策前真实负荷前缀必须严格为 0..{k-1}。"
            )
        if list(observed_pv_kwh.index) != expected_prefix:
            raise ValueError(
                f"k={k} 储能决策前真实光伏前缀必须严格为 0..{k-1}。"
            )

        bootstrap = build_conditional_trajectory_set(
            effective_forecast,
            residual_library,
            observed_load_kwh=observed_load_kwh.copy(),
            observed_pv_kwh=observed_pv_kwh.copy(),
            current_k=k,
            n_bootstrap=(
                runtime_settings
                .rolling_bootstrap_scenarios
            ),
            conditioning_pool=(
                runtime_settings.conditioning_pool
            ),
            random_state=(
                runtime_settings.random_seed
                + int(date.dayofyear) * 1000
                + k
```

```
            ),
            config=self.config,
        )

        reduced = reduce_trajectory_set(
            bootstrap,
            n_representatives=(
                runtime_settings
                .rolling_reduced_scenarios
            ),
        )

        tree = build_nonanticipative_tree(
            reduced,
            branch_every=(
                runtime_settings.rolling_branch_every
            ),
            max_branch=(
                runtime_settings.rolling_max_branch
            ),
            force_single_root=True,
        )

        check = tree.validate_nonanticipativity()
        if not check["passed"].all():
            raise AssertionError(
                " 问题 3 实时条件树未通过非预见性检查。"
            )

        root_ids = tree.root_node_ids()
        if len(root_ids) != 1:
            raise AssertionError(
                " 问题 3 实时补救当前 stage 必须唯一根节点。"
            )

        price_for_runtime = (
            self._price_series_for_decision(
                date,
                k,
            )
        )

        solution = _solve_tree_optimization(
            tree=tree,
            price_yuan_per_kwh=price_for_runtime,
            initial_energy_kwh=current_energy_kwh,
            config=self.config,
            settings=runtime_settings,
            q_buy_fixed_kwh=fixed_plan_kwh,
            model_name=f"q3_recourse_{date.date()}_k{k}",
        )

        root_id = int(root_ids[0])
        row = solution.node_policy.loc[
            solution.node_policy["node_id"] == root_id
        ]

        if len(row) != 1:
            raise AssertionError(" 当前根节点动作不唯一。")

        row = row.iloc[0]

        audit = pd.DataFrame(
            [
                {
                    "k": k,
                    "max_observed_k": k - 1,
                    "observed_points": k,
                    "future_actual_used": False,
```

```
                    "current_actual_used_for_storage_decision": False,
                    "current_actual_used_only_for_expost_balance": True,
                    "single_current_root": True,
                    "conditional_tree_passed": True,
                    "current_root_id": root_id,
                    "current_root_expected_load_kwh": float(
                        row["load_kwh"]
                    ),
                    "current_root_expected_pv_kwh": float(
                        row["pv_kwh"]
                    ),
                    "predicted_root_q_emg_kwh_not_executed": float(
                        row["q_emg_kwh"]
                    ),
                }
            ]
        )

        return (
            float(row["q_ch_kwh"]),
            float(row["q_dis_kwh"]),
            float(row["q_emg_kwh"]),
            float(row["q_curt_kwh"]),
            float(row["q_sur_kwh"]),
            audit,
        )

    def solve_day(
        self,
        date,
        *,
        initial_energy_kwh: float,
        update_combination: Optional[
            tuple[int, ...]
        ] = None,
        check_legality: bool = True,
    ) -> Question3DayResult:
        """
        求解并真实因果执行一天。
        """
        d = self._check_date_exists(date)

        updates = (
            self.settings.update_combination
            if update_combination is None
            else update_combination
        )

        local_settings = Question3Settings(
            update_combination=updates,
            rolling_horizon_intervals=(
                self.settings.rolling_horizon_intervals
            ),
            q2_runtime_settings=(
                self.settings.q2_runtime_settings
            ),
            solver_output=self.settings.solver_output,
            enforce_charge_discharge_exclusivity=(
                self.settings
                .enforce_charge_discharge_exclusivity
            ),
        )
        local_settings.validate(self.config)

        if not (
            E_MIN_KWH
            <= initial_energy_kwh
            <= E_MAX_KWH
        ):
```

```
            raise ValueError("initial_energy_kwh 越界。")

        # ------------------------------------------------------------
        # 0:00 因果负荷预测 / 历史残差
        # ------------------------------------------------------------
        base_forecast = build_q2_day_ahead_forecast(
            self.attachment2,
            d,
            config=self.config,
        )

        residual_library = build_causal_residual_library(
            self.attachment2,
            d,
            max_residual_days=(
                self.settings.q2_runtime_settings
                .max_residual_days
            ),
            config=self.config,
        )

        history = build_available_history(
            self.attachment2,
            d,
            config=self.config,
        )

        next_day_load_preview = (
            _causal_preview_load_for_next_day(
                history,
                current_date=d,
                config=self.config,
            )
        )

        # 当前 Q3 生效的 PV 预测基线。
        effective_pv_forecast = pd.Series(
            np.zeros(144),
            index=list(range(144)),
            dtype=float,
            name="effective_pv_forecast_kwh",
        )

        observed_load = pd.Series(dtype=float)
        observed_pv = pd.Series(dtype=float)

        q_ch_actual = pd.Series(
            index=range(144),
            dtype=float,
            name="actual_charge_kwh",
        )
        q_dis_actual = pd.Series(
            index=range(144),
            dtype=float,
            name="actual_discharge_kwh",
        )
        q_emg_actual = pd.Series(
            index=range(144),
            dtype=float,
            name="actual_emergency_kwh",
        )
        q_curt_actual = pd.Series(
            index=range(144),
            dtype=float,
            name="actual_curtailment_kwh",
        )
        q_sur_actual = pd.Series(
            index=range(144),
            dtype=float,
```

```
            name="actual_surplus_kwh",
        )

        E_actual = pd.Series(
            index=range(145),
            dtype=float,
            name="energy_kwh",
        )
        E_actual.loc[0] = float(initial_energy_kwh)

        plan_book: Optional[RollingPlanBook] = None
        update_records: Dict[int, MPCUpdateRecord] = {}
        runtime_rows = []
        freeze_rows = []

        current_energy = float(initial_energy_kwh)

        for k in range(144):
            # ------------------------------------------------------------
            # 0/6/12/18：只在组合中启用的时刻重算未来计划
            # 更新发生在当前 k 实际区间开始之前。
            # ------------------------------------------------------------
            if k in updates:
                parsed, F0 = self._build_parser_version(
                    d,
                    k,
                )

                future_pv = extract_current_day_model_kwh(
                    parsed,
                    d,
                    start_k=k,
                    config=self.config,
                )

                # 只覆盖未来 k>=r；过去预测值保留用于实时残差条件。
                effective_pv_forecast.loc[k:] = (
                    future_pv.to_numpy(dtype=float)
                )

                # 计划优化价格 provider 可由 Q4 注入。
                formal_future, preview, obj = (
                    _solve_deterministic_mpc_update(
                        date=d,
                        r=k,
                        current_energy_kwh=current_energy,
                        original_plan_kwh=(
                            None
                            if plan_book is None
                            else plan_book.original_plan_kwh
                        ),
                        load_today_kwh=(
                            base_forecast.load_kwh
                        ),
                        load_next_day_preview_kwh=(
                            next_day_load_preview
                        ),
                        parsed_pv=parsed,
                        price_provider=self.price_provider,
                        config=self.config,
                        enforce_charge_discharge_exclusivity=(
                            local_settings
                            .enforce_charge_discharge_exclusivity
                        ),
                        solver_output=(
                            local_settings.solver_output
                        ),
                    )
                )
```

```
                if k == 0:
                    if list(formal_future.index) != list(
                        range(144)
                    ):
                        raise AssertionError(
                            "0:00 必须形成完整 144 点原始计划。"
                        )

                    plan_book = (
                        RollingPlanBook.from_initial_plan(
                            formal_future,
                            config=self.config,
                        )
                    )
                else:
                    if plan_book is None:
                        raise AssertionError(
                            " 非 0 点更新前必须已有 0:00 原始计划。"
                        )

                    frozen_before = (
                        plan_book.effective_plan_kwh.loc[
                            : k - 1
                        ].copy()
                    )

                    plan_book.update_future(
                        k,
                        formal_future,
                        config=self.config,
                    )

                    frozen_after = (
                        plan_book.effective_plan_kwh.loc[
                            : k - 1
                        ].copy()
                    )

                    passed = bool(
                        np.array_equal(
                            frozen_before.to_numpy(),
                            frozen_after.to_numpy(),
                        )
                    )

                    freeze_rows.append(
                        {
                            "r": k,
                            "frozen_range": f"k<{k}",
                            "passed": passed,
                        }
                    )

                    if not passed:
                        raise AssertionError(
                            f"r={k} 更新修改了过去计划。"
                        )

                parser_audit = parsed.hourly_anchors[
                    [
                        "lead_hour",
                        "target_datetime",
                        "forecast_kw",
                        "source",
                    ]
                ].copy()

                update_records[k] = MPCUpdateRecord(
```

```
                r=k,
                issue_datetime=parsed.issue_datetime,
                current_energy_kwh=current_energy,
                current_observed_pv_kw=F0,
                formal_future_plan_kwh=(
                    formal_future.copy()
                ),
                preview_purchase_kwh=preview.copy(),
                objective_value_yuan=obj,
                parser_audit=parser_audit,
            )

        if plan_book is None:
            raise AssertionError(
                " 问题 3 必须在 k=0 建立原始计划。"
            )

        # ------------------------------------------------------------
        # tau_k: 当前储能动作决策前, 只允许真实历史 0..k-1
        # ------------------------------------------------------------
        if list(observed_load.index) != list(range(k)):
            raise AssertionError(
                f"k={k} 决策前真实负荷前缀必须严格为 0..{k-1}。"
            )
        if list(observed_pv.index) != list(range(k)):
            raise AssertionError(
                f"k={k} 决策前真实光伏前缀必须严格为 0..{k-1}。"
            )

        effective_forecast = (
            _build_effective_q3_forecast(
                base_forecast,
                effective_pv_forecast,
            )
        )

        # ------------------------------------------------------------
        # 1. 不知道 current_k 实际值时先决定 q_ch/q_dis
        # ------------------------------------------------------------
        (
            q_ch,
            q_dis,
            predicted_q_emg,
            predicted_q_curt,
            predicted_q_sur,
            step_audit,
        ) = self._runtime_recourse_step(
            date=d,
            k=k,
            current_energy_kwh=current_energy,
            observed_load_kwh=observed_load,
            observed_pv_kwh=observed_pv,
            effective_forecast=effective_forecast,
            residual_library=residual_library,
            fixed_plan_kwh=(
                plan_book.effective_plan_kwh
            ),
        )

        # ------------------------------------------------------------
        # 2. 储能动作锁定后, 第 k 区间真实负荷/PV 才完整实现
        # ------------------------------------------------------------
        load_now = float(
            self.attachment2.load_kw.at[d, k]
        ) * self.dt_hours

        pv_now = float(
            self.attachment2.pv_kw.at[d, k]
        ) * self.dt_hours
```

```
            # ------------------------------------------------------------
            # 3. current_k 实际实现后再计算紧急购电/弃光/计划富余
            # ------------------------------------------------------------
            (
                q_emg,
                q_curt,
                q_sur,
                balance_residual,
            ) = settle_realized_interval_after_storage(
                q_buy_kwh=float(
                    plan_book.effective_plan_kwh.loc[k]
                ),
                q_ch_kwh=q_ch,
                q_dis_kwh=q_dis,
                actual_load_kwh=load_now,
                actual_pv_kwh=pv_now,
            )

            q_ch_actual.loc[k] = q_ch
            q_dis_actual.loc[k] = q_dis
            q_emg_actual.loc[k] = q_emg
            q_curt_actual.loc[k] = q_curt
            q_sur_actual.loc[k] = q_sur

            next_energy = (
                current_energy
                + ETA_C * q_ch
                - q_dis / ETA_D
            )

            if (
                next_energy < E_MIN_KWH - 1e-6
                or next_energy > E_MAX_KWH + 1e-6
            ):
                raise AssertionError(
                    f"k={k} 实际执行后 SOC 越界。"
                )

            E_actual.loc[k] = current_energy
            E_actual.loc[k + 1] = next_energy
            current_energy = next_energy

            # ------------------------------------------------------------
            # 4. 区间结束后才把 actual k 加入下一时段可用历史
            # ------------------------------------------------------------
            observed_load.loc[k] = load_now
            observed_pv.loc[k] = pv_now

            audit_row = step_audit.iloc[0].to_dict()
            audit_row.update(
                {
                    "realized_load_kwh": load_now,
                    "realized_pv_kwh": pv_now,
                    "predicted_q_emg_kwh_not_executed": (
                        predicted_q_emg
                    ),
                    "predicted_q_curt_kwh_not_executed": (
                        predicted_q_curt
                    ),
                    "predicted_q_sur_kwh_not_executed": (
                        predicted_q_sur
                    ),
                    "expost_balance_residual_kwh": (
                        balance_residual
                    ),
                }
            )
            runtime_rows.append(audit_row)
```

```
        if plan_book is None:
            raise AssertionError(" 全天计划簿未建立。")

        GF = plan_book.final_effective_plan()

        # Q3 固定价格结算; Q4-3 会在外层用附件 4 实际价格重新结算。
        # 当前 Q3 必须使用附件 1 固定价格。
        settlement = settle_question3_cost(
            G0_kwh=plan_book.original_plan_kwh,
            GF_kwh=GF,
            price_yuan_per_kwh=self.fixed_price,
            G_emg_kwh=q_emg_actual,
            config=self.config,
        )

        if not freeze_rows:
            # 仅 0:00 组合没有调整, 冻结检查自然通过。
            freeze_rows.append(
                {
                    "r": 0,
                    "frozen_range": "none",
                    "passed": True,
                }
            )

        result = Question3DayResult(
            date=d,
            update_combination=updates,
            initial_energy_kwh=float(initial_energy_kwh),
            end_energy_kwh=float(E_actual.loc[144]),
            original_plan_kwh=(
                plan_book.original_plan_kwh.copy()
            ),
            final_effective_plan_kwh=GF.copy(),
            plan_versions={
                r: s.copy()
                for r, s in plan_book.versions.items()
            },
            update_records=update_records,
            actual_charge_kwh=q_ch_actual,
            actual_discharge_kwh=q_dis_actual,
            actual_emergency_kwh=q_emg_actual,
            actual_curtailment_kwh=q_curt_actual,
            actual_surplus_kwh=q_sur_actual,
            energy_kwh=E_actual,
            dG_plus_kwh=settlement.dG_plus_kwh,
            dG_minus_kwh=settlement.dG_minus_kwh,
            C_plan_yuan=settlement.C_plan_yuan,
            C_reg_yuan=settlement.C_reg_yuan,
            C_adj_yuan=settlement.C_adj_yuan,
            C_emg_yuan=settlement.C_emg_yuan,
            C_total_yuan=settlement.C_total_yuan,
            runtime_causality_report=pd.DataFrame(
                runtime_rows
            ),
            update_freeze_report=pd.DataFrame(
                freeze_rows
            ),
            settlement_test_report=(
                SETTLEMENT_SELF_TEST_REPORT.copy()
            ),
            legality_report=pd.DataFrame(),
        )

        legality = check_question3_day_result_legality(
            result=result,
            attachment2=self.attachment2,
            price_yuan_per_kwh=self.fixed_price,
```

```
            config=self.config,
        )
        result.legality_report = legality

        if check_legality:
            failed = legality.loc[~legality["passed"]]
            if not failed.empty:
                raise AssertionError(
                    " 问题 3 结果合法性检查失败: \n"
                    + failed.to_string(index=False)
                )

        return result

    def run_ablation_day(
        self,
        date,
        *,
        initial_energy_kwh: float,
        check_legality: bool = True,
    ) -> Dict[str, Question3DayResult]:
        """
        四种更新时间组合消融。

        每种组合从同一个 initial_energy_kwh 开始,
        便于单日公平比较。
        """
        results = {}

        for name, updates in (
            ABLATION_UPDATE_COMBINATIONS.items()
        ):
            results[name] = self.solve_day(
                date,
                initial_energy_kwh=initial_energy_kwh,
                update_combination=updates,
                check_legality=check_legality,
            )

        return results

    def solve_period(
        self,
        *,
        start_date: str = "2025-01-01",
        end_date: str = "2025-12-31",
        initial_energy_kwh: float = 6000.0,
        save_from_date: str = "2025-02-01",
        update_combination: Optional[
            tuple[int, ...]
        ] = None,
        check_legality: bool = True,
    ) -> Question3PeriodResult:
        """
        跨日 SOC 连续滚动。

        次日正式购电计划始终在次日 0:00 重新生成;
        前一日 18:00 得到的 next-day preview 从不写入正式计划。
        """
        start = _normalize_date(start_date)
        end = _normalize_date(end_date)
        save_from = _normalize_date(save_from_date)

        if end < start:
            raise ValueError("end_date 不能早于 start_date。")

        formal_results = {}
        cross_rows = []

```

```
        current_energy = float(initial_energy_kwh)
        warmup_end_energy = None

        previous_date = None
        previous_end = None

        for d in pd.date_range(start, end, freq="D"):
            if previous_date is not None:
                residual = abs(
                    current_energy - float(previous_end)
                )
                cross_rows.append(
                    {
                        "previous_date": previous_date,
                        "current_date": d,
                        "previous_end_energy_kwh": previous_end,
                        "current_initial_energy_kwh": (
                            current_energy
                        ),
                        "residual_kwh": residual,
                        "passed": residual <= 1e-8,
                    }
                )

            day = self.solve_day(
                d,
                initial_energy_kwh=current_energy,
                update_combination=update_combination,
                check_legality=check_legality,
            )

            current_energy = float(day.end_energy_kwh)
            previous_end = current_energy
            previous_date = d

            print(
                f"[Q3] {d.date()} 完成: 总成本 {day.C_total_yuan:.2f} 元, "
                f" 末 SOC {current_energy:.2f} kWh",
                flush=True,
            )

            if d < save_from:
                warmup_end_energy = current_energy
            else:
                formal_results[d] = day

        cross_report = pd.DataFrame(cross_rows)

        if (
            not cross_report.empty
            and not cross_report["passed"].all()
        ):
            raise AssertionError(" 问题 3 跨日 SOC 连续性失败。")

        return Question3PeriodResult(
            formal_results=formal_results,
            cross_day_report=cross_report,
            warmup_end_energy_kwh=warmup_end_energy,
            final_energy_kwh=current_energy,
        )
```

文件：code/question4_solver.py

```
# -*- coding: utf-8 -*-
"""
question4_solver.py
问题 4：动态实时电价下复用问题 2 / 问题 3。

本文件不复制问题 2、问题 3 整套优化代码。

复用关系
--------
Q4-2:
    CausalPriceModule
    + Question2Solver
    -> result4-2

Q4-3:
    CausalPriceModule.price_provider
    + Question3Solver
    -> result4-3

附件 4 时间映射
-------------
只使用 data_loader.Attachment4Data.price：
    index = date
    columns = model k=0,...,143

该 DataFrame 已经由 data_loader.py + time_index.py 按“右端点标签”对齐：
    k=0   -> [0:00,0:10)   <- source 0:10
    k=36  -> [6:00,6:10)   <- source 6:10
    k=60  -> [10:00,10:10) <- source 10:10
    k=143 -> [23:50,24:00) <- source 0:00+1

本文件绝不直接读取附件 4 原始表头来推断 k。

价格主策略
----------
未来实际价格未知，采用严格因果预测：
1. 基础：最近最多 4 个“目标日期同星期属性”的历史日同 k 均值；
2. 决策日内：只用 decision_datetime 之前已经实现的当前日价格偏差
   对未来价格整体修正；
3. 当前时刻以后的附件 4 实际价格只能用于后续实现和最终结算。

完美预知价格 Oracle
------------------
可选，仅用于基准比较，默认不启用。
Oracle 允许把附件 4 未来实际价直接作为优化价格，但必须明确标记，
不得作为主策略结果。

Q4 费用分解
----------
Q4-2:
    C_plan = sum p_act * G0
    C_reg  = C_plan
    C_adj  = 0
    C_emg  = 5 sum p_act * G_emg
    C_total= C_reg + C_emg

Q4-3:
    严格沿用 rolling_scheduler 问题 3 最终结算：
    C_reg=sum p_act[GF+0.5(dG_plus+dG_minus)]
    C_plan=sum p_act G0
    C_adj=sum p_act[1.5dG_plus-0.5dG_minus]
    C_emg=5 sum p_act G_emg
    C_total=C_reg+C_emg
"""

```

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, Optional, Tuple

import numpy as np
import pandas as pd

from config import DEFAULT_CONFIG, ProjectConfig
from data_loader import (
    Attachment1Data,
    Attachment2Data,
    Attachment3Data,
    Attachment4Data,
)
from question2_solver import (
    Question2DayResult,
    Question2Settings,
    Question2Solver,
)
from question3_solver import (
    Question3DayResult,
    Question3Settings,
    Question3Solver,
)
from rolling_scheduler import (
    SettlementResult,
    settle_question3_cost,
)
from time_index import build_time_map, validate_time_map


# ============================================================================
# 因果动态价格模块
# ============================================================================


@dataclass
class PriceForecastResult:
    target_date: pd.Timestamp
    decision_datetime: pd.Timestamp
    forecast_yuan_per_kwh: pd.Series
    history_dates: tuple[pd.Timestamp, ...]
    observed_current_day_intervals: int
    observed_actual_k_max: Optional[int]
    future_actual_used: bool


class CausalPriceModule:
    """
    问题 4 严格因果价格预测模块。

    任何预测只允许使用:
    - decision day 之前完整历史日;
    - decision_datetime 之前当前日已经实现的 k;
    - 已知 fallback 基准 (附件 1 固定日内电价)。

    禁止使用:
    - 当前日 decision 时刻以后的附件 4 真实价格;
    - target future day 自身真实价格。
    """

    def __init__(
        self,
        *,
        attachment4: Attachment4Data,
        fallback_price_yuan_per_kwh: pd.Series,
        config: ProjectConfig = DEFAULT_CONFIG,
        similar_days: int = 4,
```

```
    ) -> None:
        self.attachment4 = attachment4
        self.config = config
        self.similar_days = int(similar_days)

        if self.similar_days <= 0:
            raise ValueError("similar_days 必须为正整数。")

        expected_k = list(range(config.intervals_per_day))

        if list(attachment4.price.columns) != expected_k:
            raise ValueError(
                "Attachment4Data.price 必须严格为模型 k=0~143。"
            )

        if attachment4.price.isna().any().any():
            raise ValueError(" 附件 4 模型价格网格存在缺失值。")

        if list(fallback_price_yuan_per_kwh.index) != expected_k:
            raise ValueError(
                "fallback_price_yuan_per_kwh.index 必须为 0~143。"
            )

        self.fallback_price = pd.to_numeric(
            fallback_price_yuan_per_kwh,
            errors="raise",
        ).astype(float)

        self.fallback_price.index = expected_k

        self.time_map = build_time_map(config)
        validate_time_map(self.time_map, config)

        self._cache: Dict[
            tuple[pd.Timestamp, pd.Timestamp],
            PriceForecastResult,
        ] = {}

    @staticmethod
    def _normalize_date(value) -> pd.Timestamp:
        return pd.Timestamp(value).normalize()

    def _base_forecast(
        self,
        target_date: pd.Timestamp,
        decision_datetime: pd.Timestamp,
    ) -> tuple[pd.Series, tuple[pd.Timestamp, ...]]:
        """
        基础相似日预测。

        完整历史日必须严格早于 decision_datetime 所在自然日。
        即使预测的是次日，也不能把“当前日未来尚未实现部分”当完整历史日。
        """
        decision_day = decision_datetime.normalize()

        history = self.attachment4.price.loc[
            self.attachment4.price.index < decision_day
        ]

        if history.empty:
            return self.fallback_price.copy(), tuple()

        same_weekday = history.loc[
            history.index.weekday == target_date.weekday()
        ]

        if same_weekday.empty:
            selected = history.tail(self.similar_days)
        else:
```

```
            selected = same_weekday.tail(self.similar_days)

        base = selected.mean(axis=0).astype(float)
        base.index = list(range(self.config.intervals_per_day))

        return (
            base,
            tuple(
                pd.Timestamp(x).normalize()
                for x in selected.index
            ),
        )

    def forecast(
        self,
        target_date,
        decision_datetime,
    ) -> PriceForecastResult:
        """
        返回 target_date 的 144 点价格预测。

        decision_datetime 可为任意 10 分钟决策时刻。
        当前日真实偏差只使用 k < current_k 的已实现区间。

        例如:
        6:00 决策 -> current_k=36 -> 最多读取实际 k=0..35,
        对应附件 4 最后可用右端点标签 6:00。
        绝不读取 source 6:10 (k=36)。
        """
        target = self._normalize_date(target_date)
        decision = pd.Timestamp(decision_datetime)

        cache_key = (target, decision)
        if cache_key in self._cache:
            cached = self._cache[cache_key]
            return PriceForecastResult(
                target_date=cached.target_date,
                decision_datetime=cached.decision_datetime,
                forecast_yuan_per_kwh=(
                    cached.forecast_yuan_per_kwh.copy()
                ),
                history_dates=cached.history_dates,
                observed_current_day_intervals=(
                    cached.observed_current_day_intervals
                ),
                observed_actual_k_max=(
                    cached.observed_actual_k_max
                ),
                future_actual_used=False,
            )

        base, history_dates = self._base_forecast(
            target,
            decision,
        )

        decision_day = decision.normalize()
        minute = decision.hour * 60 + decision.minute

        if minute % self.config.dt_minutes != 0:
            raise ValueError(
                "decision_datetime 必须落在 10 分钟边界。"
            )

        current_k = minute // self.config.dt_minutes

        # 只允许读取当前日已经完成的 k<current_k。
        observed_count = 0
        observed_k_max = None
```

```
        bias = 0.0

        if (
            decision_day in self.attachment4.price.index
            and current_k > 0
        ):
            observed_index = list(range(current_k))

            actual_prefix = (
                self.attachment4.price.loc[
                    decision_day,
                    observed_index,
                ].astype(float)
            )

            # 当前日基准也必须是仅由更早完整历史日生成。
            current_day_base, _ = self._base_forecast(
                decision_day,
                decision,
            )

            residual_prefix = (
                actual_prefix
                - current_day_base.loc[observed_index]
            )

            bias = float(residual_prefix.mean())
            observed_count = current_k
            observed_k_max = current_k - 1

        forecast = (base + bias).clip(lower=0.0)
        forecast.name = "price_forecast_yuan_per_kwh"

        result = PriceForecastResult(
            target_date=target,
            decision_datetime=decision,
            forecast_yuan_per_kwh=forecast,
            history_dates=history_dates,
            observed_current_day_intervals=observed_count,
            observed_actual_k_max=observed_k_max,
            future_actual_used=False,
        )

        self._cache[cache_key] = result
        return PriceForecastResult(
            target_date=result.target_date,
            decision_datetime=result.decision_datetime,
            forecast_yuan_per_kwh=result.forecast_yuan_per_kwh.copy(),
            history_dates=result.history_dates,
            observed_current_day_intervals=(
                result.observed_current_day_intervals
            ),
            observed_actual_k_max=result.observed_actual_k_max,
            future_actual_used=False,
        )

    def price_provider(
        self,
        abs_date: pd.Timestamp,
        model_k: int,
        decision_datetime: pd.Timestamp,
    ) -> float:
        """
        给 Question3Solver 注入的严格因果价格接口。
        """
        forecast = self.forecast(
            abs_date,
            decision_datetime,
        )
```

```
342         return float(
343             forecast.forecast_yuan_per_kwh.loc[
344                 int(model_k)
345             ]
346         )
347 
348     def actual_price(
349         self,
350         date,
351     ) -> pd.Series:
352         """
353         事后真实价格。
354 
355         只从 Attachment4Data.price 按模型 k 读取,
356         不解析附件 4 原始时间字符串。
357         """
358         d = self._normalize_date(date)
359 
360         if d not in self.attachment4.price.index:
361             raise KeyError(f" 附件 4 缺少日期 {d.date()}。")
362 
363         result = (
364             self.attachment4.price.loc[d]
365             .astype(float)
366             .copy()
367         )
368         result.index = list(range(self.config.intervals_per_day))
369         result.name = "actual_price_yuan_per_kwh"
370         return result
371 
372 
373 class OraclePriceModule(CausalPriceModule):
374     """
375     完美预知价格 Oracle, 仅用于基准。
376 
377     本类故意允许使用 target_date 完整真实价格。
378     绝不能作为 main_strategy=True 的正式策略。
379     """
380 
381     def forecast(
382         self,
383         target_date,
384         decision_datetime,
385     ) -> PriceForecastResult:
386         target = self._normalize_date(target_date)
387         decision = pd.Timestamp(decision_datetime)
388 
389         actual = self.actual_price(target)
390         return PriceForecastResult(
391             target_date=target,
392             decision_datetime=decision,
393             forecast_yuan_per_kwh=actual,
394             history_dates=tuple(),
395             observed_current_day_intervals=0,
396             observed_actual_k_max=None,
397             future_actual_used=True,
398         )
399 
400 
401 # ============================================================================
402 # Q4 结果
403 # ============================================================================
404 
405 
406 @dataclass
407 class Question42DayResult:
408     date: pd.Timestamp
409 
410     base_result: Question2DayResult
```

```
411
412     day_ahead_price_forecast: pd.Series
413     actual_price_yuan_per_kwh: pd.Series
414     price_forecast_audit: PriceForecastResult
415
416     C_plan_yuan: float
417     C_reg_yuan: float
418     C_adj_yuan: float
419     C_emg_yuan: float
420     C_total_yuan: float
421
422     is_oracle: bool
423
424
425 @dataclass
426 class Question43DayResult:
427     date: pd.Timestamp
428
429     base_result: Question3DayResult
430
431     day_ahead_price_forecast: pd.Series
432     actual_price_yuan_per_kwh: pd.Series
433     price_forecast_audit: PriceForecastResult
434
435     C_plan_yuan: float
436     C_reg_yuan: float
437     C_adj_yuan: float
438     C_emg_yuan: float
439     C_total_yuan: float
440
441     settlement_identity_residual_yuan: float
442     is_oracle: bool
443
444
445 @dataclass
446 class Question4BothResult:
447     """ 同时返回 result4-2 和 result4-3。"""
448
449     result4_2: Question42DayResult
450     result4_3: Question43DayResult
451
452
453 @dataclass
454 class Question4PeriodResult:
455     formal_results: Dict[pd.Timestamp, object]
456     cross_day_report: pd.DataFrame
457     warmup_end_energy_kwh: Optional[float]
458     final_energy_kwh: float
459
460
461 # ============================================================================
462 # Question4Solver: 薄适配层
463 # ============================================================================
464
465
466 class Question4Solver:
467     """
468     问题 4 薄封装。
469
470     不复制 Q2/Q3 模型:
471     - Q4-2 直接实例化 Question2Solver 并注入 0:00 因果价格预测;
472     - Q4-3 直接实例化 Question3Solver 并注入 price_provider。
473     """
474
475     def __init__(
476         self,
477         *,
478         attachment1: Attachment1Data,
479         attachment2: Attachment2Data,
```

```
        attachment3: Attachment3Data,
        attachment4: Attachment4Data,
        q2_settings: Question2Settings = Question2Settings(),
        q3_settings: Question3Settings = Question3Settings(),
        config: ProjectConfig = DEFAULT_CONFIG,
    ) -> None:
        self.attachment1 = attachment1
        self.attachment2 = attachment2
        self.attachment3 = attachment3
        self.attachment4 = attachment4
        self.q2_settings = q2_settings
        self.q3_settings = q3_settings
        self.config = config

        self.time_map = build_time_map(config)
        validate_time_map(self.time_map, config)

        expected_k = list(range(config.intervals_per_day))
        if list(attachment4.price.columns) != expected_k:
            raise ValueError(
                " 附件 4 必须通过 data_loader/time_index 按右端点映射后再进入 Q4。"
            )

        fallback_price = (
            attachment1.aligned["price_yuan_per_kwh"]
            .astype(float)
            .copy()
        )
        fallback_price.index = expected_k

        self.price_module = CausalPriceModule(
            attachment4=attachment4,
            fallback_price_yuan_per_kwh=fallback_price,
            config=config,
        )

        self.oracle_price_module = OraclePriceModule(
            attachment4=attachment4,
            fallback_price_yuan_per_kwh=fallback_price,
            config=config,
        )

    @staticmethod
    def _normalize_date(value) -> pd.Timestamp:
        return pd.Timestamp(value).normalize()

    def solve_q4_2_day(
        self,
        date,
        *,
        initial_energy_kwh: float,
        use_oracle: bool = False,
        check_legality: bool = True,
    ) -> Question42DayResult:
        """
        Q4-2 = 动态电价下复用 Question2Solver。

        主策略:
            use_oracle=False

        Oracle:
            use_oracle=True, 仅用于完美价格信息基准。
        """
        d = self._normalize_date(date)

        price_module = (
            self.oracle_price_module
            if use_oracle
            else self.price_module
```

```
        )

        decision_datetime = d
        price_forecast = price_module.forecast(
            d,
            decision_datetime,
        )

        if (
            not use_oracle
            and price_forecast.future_actual_used
        ):
            raise AssertionError(
                "Q4-2 主策略价格预测发生未来信息泄露。"
            )

        # ------------------------------------------------------------
        # 直接复用 Question2Solver，不复制问题 2 优化代码。
        # ------------------------------------------------------------
        solver = Question2Solver(
            attachment1=self.attachment1,
            attachment2=self.attachment2,
            settings=self.q2_settings,
            config=self.config,
        )

        # Q2 内部所有价格位置统一使用这一 144 点预测。
        solver.price_yuan_per_kwh = (
            price_forecast
            .forecast_yuan_per_kwh
            .copy()
        )

        base = solver.solve_day(
            d,
            initial_energy_kwh=initial_energy_kwh,
            check_legality=check_legality,
        )

        # ------------------------------------------------------------
        # 事后按附件 4 真实价格重新结算
        # ------------------------------------------------------------
        actual_price = self.price_module.actual_price(d)

        G0 = base.plan_purchase_kwh
        G_emg = base.actual_emergency_kwh

        C_plan = float((actual_price * G0).sum())

        # Q4-2 没有日内正常购电调整：
        C_reg = C_plan
        C_adj = 0.0

        C_emg = float(
            (5.0 * actual_price * G_emg).sum()
        )

        C_total = C_reg + C_emg

        return Question42DayResult(
            date=d,
            base_result=base,
            day_ahead_price_forecast=(
                price_forecast
                .forecast_yuan_per_kwh.copy()
            ),
            actual_price_yuan_per_kwh=actual_price,
            price_forecast_audit=price_forecast,
            C_plan_yuan=C_plan,
```

```
            C_reg_yuan=C_reg,
            C_adj_yuan=C_adj,
            C_emg_yuan=C_emg,
            C_total_yuan=C_total,
            is_oracle=bool(use_oracle),
        )

    def solve_q4_3_day(
        self,
        date,
        *,
        initial_energy_kwh: float,
        use_oracle: bool = False,
        update_combination: Optional[
            tuple[int, ...]
        ] = None,
        check_legality: bool = True,
    ) -> Question43DayResult:
        """
        Q4-3 = 动态电价下复用 Question3Solver。

        Question3Solver 的所有滚动购电调整和 10 分钟实时补救代码不复制,
        这里只注入动态 price_provider。
        """
        d = self._normalize_date(date)

        price_module = (
            self.oracle_price_module
            if use_oracle
            else self.price_module
        )

        day_ahead_audit = price_module.forecast(
            d,
            d,
        )

        if (
            not use_oracle
            and day_ahead_audit.future_actual_used
        ):
            raise AssertionError(
                "Q4-3 主策略 0:00 价格预测发生未来泄露。"
            )

        solver = Question3Solver(
            attachment1=self.attachment1,
            attachment2=self.attachment2,
            attachment3=self.attachment3,
            settings=self.q3_settings,
            config=self.config,
            price_provider=price_module.price_provider,
        )

        base = solver.solve_day(
            d,
            initial_energy_kwh=initial_energy_kwh,
            update_combination=update_combination,
            check_legality=check_legality,
        )

        # ------------------------------------------------------------
        # 最终 Q4-3 只按附件 4 真实实现价格结算。
        # ------------------------------------------------------------
        actual_price = self.price_module.actual_price(d)

        settlement = settle_question3_cost(
            G0_kwh=base.original_plan_kwh,
            GF_kwh=base.final_effective_plan_kwh,
```

```
            price_yuan_per_kwh=actual_price,
            G_emg_kwh=base.actual_emergency_kwh,
            config=self.config,
        )

        if settlement.identity_residual_yuan > 1e-8:
            raise AssertionError(
                "Q4-3 真实动态价格下 C_reg != C_plan + C_adj。"
            )

        return Question43DayResult(
            date=d,
            base_result=base,
            day_ahead_price_forecast=(
                day_ahead_audit
                .forecast_yuan_per_kwh.copy()
            ),
            actual_price_yuan_per_kwh=actual_price,
            price_forecast_audit=day_ahead_audit,
            C_plan_yuan=settlement.C_plan_yuan,
            C_reg_yuan=settlement.C_reg_yuan,
            C_adj_yuan=settlement.C_adj_yuan,
            C_emg_yuan=settlement.C_emg_yuan,
            C_total_yuan=settlement.C_total_yuan,
            settlement_identity_residual_yuan=(
                settlement.identity_residual_yuan
            ),
            is_oracle=bool(use_oracle),
        )

    def solve_both_day(
        self,
        date,
        *,
        initial_energy_q4_2_kwh: float,
        initial_energy_q4_3_kwh: float,
        check_legality: bool = True,
    ) -> Question4BothResult:
        """
        同一天同时得到 result4-2 / result4-3 主策略。
        """
        result4_2 = self.solve_q4_2_day(
            date,
            initial_energy_kwh=initial_energy_q4_2_kwh,
            use_oracle=False,
            check_legality=check_legality,
        )

        result4_3 = self.solve_q4_3_day(
            date,
            initial_energy_kwh=initial_energy_q4_3_kwh,
            use_oracle=False,
            check_legality=check_legality,
        )

        return Question4BothResult(
            result4_2=result4_2,
            result4_3=result4_3,
        )

    def solve_q4_2_period(
        self,
        *,
        start_date: str = "2025-01-01",
        end_date: str = "2025-12-31",
        initial_energy_kwh: float = 6000.0,
        save_from_date: str = "2025-02-01",
        use_oracle: bool = False,
        check_legality: bool = True,
```

```
    ) -> Question4PeriodResult:
        """
        Q4-2 跨日连续运行。
        """
        return self._solve_period(
            mode="q4-2",
            start_date=start_date,
            end_date=end_date,
            initial_energy_kwh=initial_energy_kwh,
            save_from_date=save_from_date,
            use_oracle=use_oracle,
            check_legality=check_legality,
        )

    def solve_q4_3_period(
        self,
        *,
        start_date: str = "2025-01-01",
        end_date: str = "2025-12-31",
        initial_energy_kwh: float = 6000.0,
        save_from_date: str = "2025-02-01",
        use_oracle: bool = False,
        update_combination: Optional[
            tuple[int, ...]
        ] = None,
        check_legality: bool = True,
    ) -> Question4PeriodResult:
        """
        Q4-3 跨日连续运行。
        """
        return self._solve_period(
            mode="q4-3",
            start_date=start_date,
            end_date=end_date,
            initial_energy_kwh=initial_energy_kwh,
            save_from_date=save_from_date,
            use_oracle=use_oracle,
            update_combination=update_combination,
            check_legality=check_legality,
        )

    def _solve_period(
        self,
        *,
        mode: str,
        start_date: str,
        end_date: str,
        initial_energy_kwh: float,
        save_from_date: str,
        use_oracle: bool,
        check_legality: bool,
        update_combination: Optional[
            tuple[int, ...]
        ] = None,
    ) -> Question4PeriodResult:
        start = self._normalize_date(start_date)
        end = self._normalize_date(end_date)
        save_from = self._normalize_date(save_from_date)

        if mode not in {"q4-2", "q4-3"}:
            raise ValueError("mode 只能为 q4-2 或 q4-3。")

        formal_results = {}
        cross_rows = []

        current_energy = float(initial_energy_kwh)
        warmup_end = None
        previous_date = None
        previous_end = None
```

```
825
826         for d in pd.date_range(start, end, freq="D"):
827             if previous_date is not None:
828                 residual = abs(
829                     current_energy - float(previous_end)
830                 )
831                 cross_rows.append(
832                     {
833                         "previous_date": previous_date,
834                         "current_date": d,
835                         "previous_end_energy_kwh": previous_end,
836                         "current_initial_energy_kwh": current_energy,
837                         "residual_kwh": residual,
838                         "passed": residual <= 1e-8,
839                     }
840                 )
841
842             if mode == "q4-2":
843                 day = self.solve_q4_2_day(
844                     d,
845                     initial_energy_kwh=current_energy,
846                     use_oracle=use_oracle,
847                     check_legality=check_legality,
848                 )
849             else:
850                 day = self.solve_q4_3_day(
851                     d,
852                     initial_energy_kwh=current_energy,
853                     use_oracle=use_oracle,
854                     update_combination=update_combination,
855                     check_legality=check_legality,
856                 )
857
858             current_energy = float(
859                 day.base_result.end_energy_kwh
860             )
861             previous_end = current_energy
862             previous_date = d
863
864             print(
865                 f"[Q4] {mode} {d.date()} 完成: 总成本 "
866                 f"{day.base_result.C_total_yuan:.2f} 元, "
867                 f" 末 SOC {current_energy:.2f} kWh",
868                 flush=True,
869             )
870
871             if d < save_from:
872                 warmup_end = current_energy
873             else:
874                 formal_results[d] = day
875
876         cross_report = pd.DataFrame(cross_rows)
877
878         if (
879             not cross_report.empty
880             and not cross_report["passed"].all()
881         ):
882             raise AssertionError("Q4 跨日 SOC 连续性失败。")
883
884         return Question4PeriodResult(
885             formal_results=formal_results,
886             cross_day_report=cross_report,
887             warmup_end_energy_kwh=warmup_end,
888             final_energy_kwh=current_energy,
889         )
890
891
892 # ============================================================================
893 # 论文可视化代码
```

```
# ============================================================================
# 说明:
# - 使用 matplotlib, 不使用 seaborn;
# - 每个函数只创建一张独立图, 不使用 subplots;
# - 不手工指定颜色, 交给 matplotlib 默认循环;
# - 图中标题/坐标/图例默认使用英文, 便于避免环境中文字体问题。
# ============================================================================


def plot_price_forecast_vs_actual(
    result: Question42DayResult | Question43DayResult,
    *,
    output_path: Optional[str] = None,
):
    """
    论文图: 0:00 动态电价预测 vs 实际实时电价。
    """
    import matplotlib.pyplot as plt

    x = np.arange(144) / 6.0

    fig, ax = plt.subplots(figsize=(9, 4.8))
    ax.plot(
        x,
        result.day_ahead_price_forecast.to_numpy(dtype=float),
        label="Day-ahead price forecast",
    )
    ax.plot(
        x,
        result.actual_price_yuan_per_kwh.to_numpy(dtype=float),
        label="Actual real-time price",
    )

    ax.set_xlabel("Time of day (hour)")
    ax.set_ylabel("Electricity price (CNY/kWh)")
    ax.set_title(
        f"Dynamic Price Forecast vs Actual - "
        f"{result.date.date()}"
    )
    ax.legend()
    ax.grid(True, alpha=0.25)

    fig.tight_layout()

    if output_path is not None:
        fig.savefig(
            output_path,
            dpi=300,
            bbox_inches="tight",
        )

    return fig, ax


def plot_q4_cost_breakdown(
    result4_2: Question42DayResult,
    result4_3: Question43DayResult,
    *,
    output_path: Optional[str] = None,
):
    """
    论文图: Q4-2 与 Q4-3 费用分解。
    """
    import matplotlib.pyplot as plt

    labels = [
        "Plan cost",
        "Adjustment net",
        "Emergency cost",
```

```
        "Total cost",
    ]

    values_42 = [
        result4_2.C_plan_yuan,
        result4_2.C_adj_yuan,
        result4_2.C_emg_yuan,
        result4_2.C_total_yuan,
    ]

    values_43 = [
        result4_3.C_plan_yuan,
        result4_3.C_adj_yuan,
        result4_3.C_emg_yuan,
        result4_3.C_total_yuan,
    ]

    x = np.arange(len(labels))
    width = 0.36

    fig, ax = plt.subplots(figsize=(9, 4.8))
    ax.bar(
        x - width / 2,
        values_42,
        width,
        label="Q4-2",
    )
    ax.bar(
        x + width / 2,
        values_43,
        width,
        label="Q4-3",
    )

    ax.set_xticks(x)
    ax.set_xticklabels(labels)
    ax.set_ylabel("Cost (CNY)")
    ax.set_title("Cost Breakdown under Dynamic Electricity Price")
    ax.legend()
    ax.grid(True, axis="y", alpha=0.25)

    fig.tight_layout()

    if output_path is not None:
        fig.savefig(
            output_path,
            dpi=300,
            bbox_inches="tight",
        )

    return fig, ax


def plot_soc_profile(
    result: Question42DayResult | Question43DayResult,
    *,
    output_path: Optional[str] = None,
):
    """
    论文图：145 节点 SOC/储能电量轨迹。
    """
    import matplotlib.pyplot as plt

    E = result.base_result.energy_kwh
    x = np.arange(145) / 6.0

    fig, ax = plt.subplots(figsize=(9, 4.8))
    ax.plot(
        x,
```

```
        E.to_numpy(dtype=float),
        label="Battery energy",
    )

    ax.set_xlabel("Time of day (hour)")
    ax.set_ylabel("Battery energy (kWh)")
    ax.set_title(
        f"Battery Energy Trajectory - "
        f"{result.date.date()}"
    )
    ax.legend()
    ax.grid(True, alpha=0.25)

    fig.tight_layout()

    if output_path is not None:
        fig.savefig(
            output_path,
            dpi=300,
            bbox_inches="tight",
        )

    return fig, ax


def plot_purchase_and_emergency(
    result: Question42DayResult | Question43DayResult,
    *,
    output_path: Optional[str] = None,
):
    """
    论文图：正常购电与紧急购电 10 分钟电量。
    """
    import matplotlib.pyplot as plt

    x = np.arange(144) / 6.0

    if isinstance(result, Question43DayResult):
        normal = (
            result.base_result.final_effective_plan_kwh
        )
    else:
        normal = result.base_result.plan_purchase_kwh

    emergency = result.base_result.actual_emergency_kwh

    fig, ax = plt.subplots(figsize=(9, 4.8))
    ax.plot(
        x,
        normal.to_numpy(dtype=float),
        label="Normal grid purchase",
    )
    ax.plot(
        x,
        emergency.to_numpy(dtype=float),
        label="Emergency purchase",
    )

    ax.set_xlabel("Time of day (hour)")
    ax.set_ylabel("Energy per 10-min interval (kWh)")
    ax.set_title(
        f"Grid Purchase Profile - "
        f"{result.date.date()}"
    )
    ax.legend()
    ax.grid(True, alpha=0.25)

    fig.tight_layout()

```

```
    if output_path is not None:
        fig.savefig(
            output_path,
            dpi=300,
            bbox_inches="tight",
        )

    return fig, ax
```

文件：code/rolling_scheduler.py

```
# -*- coding: utf-8 -*-
"""
rolling_scheduler.py
问题 3 滚动购电计划冻结、版本管理、结算与消融组合。

时间更新点严格固定:
    0:00  -> r=0   -> k>=0 可优化
    6:00  -> r=36  -> 冻结 k<36, 优化 k>=36
    12:00 -> r=72  -> 冻结 k<72, 优化 k>=72
    18:00 -> r=108 -> 冻结 k<108, 优化 k>=108

不允许根据 Excel/模板时间字符串重新定位 r。

结算主口径
----------
每个交割时段只结算一次:
    G0 = 0:00 原始计划
    GF = 执行前最终生效购电量

[RS-01]
    dG_plus  = max(GF-G0, 0)
    dG_minus = max(G0-GF, 0)

[RS-02]
    C_reg = sum c * [GF + 0.5(dG_plus+dG_minus)]

[RS-03]
    C_plan = sum c * G0

[RS-04]
    C_adj = sum c * [1.5*dG_plus - 0.5*dG_minus]

[RS-05]
    C_reg = C_plan + C_adj

[RS-06]
    C_emg = 5 * sum c * G_emg

[RS-07]
    C_total = C_reg + C_emg

严禁:
    在完整 C_plan 之外再额外写 +0.5*dG_minus。
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Dict, Iterable, Sequence

import numpy as np
import pandas as pd

from config import DEFAULT_CONFIG, ProjectConfig


UPDATE_K: tuple[int, ...] = (0, 36, 72, 108)

# 四种递进式预测更新时间组合，用于消融实验。
ABLATION_UPDATE_COMBINATIONS: Dict[str, tuple[int, ...]] = {
    "00_only": (0,),
    "00_06": (0, 36),
    "00_06_12": (0, 36, 72),
    "00_06_12_18": (0, 36, 72, 108),
}
```

```
66
67
68  def validate_update_k(
69      r: int,
70      *,
71      config: ProjectConfig = DEFAULT_CONFIG,
72  ) -> None:
73      if r not in UPDATE_K:
74          raise ValueError(
75              f" 问题 3 更新索引只能为 {UPDATE_K}, 实际 r={r}。"
76          )
77
78      if not (0 <= r < config.intervals_per_day):
79          raise ValueError(" 更新索引超出 0~143。")
80
81
82  @dataclass
83  class RollingPlanBook:
84      """
85      问题 3 正常购电计划版本簿。
86
87      original_plan_kwh:
88          0:00 G0, 全天 144 点, 结算唯一基准。
89
90      effective_plan_kwh:
91          当前最新生效计划。
92
93      versions:
94          每次更新时间之后的完整 144 点快照。
95          过去时段必须保持冻结。
96      """
97
98      original_plan_kwh: pd.Series
99      effective_plan_kwh: pd.Series
100     versions: Dict[int, pd.Series] = field(default_factory=dict)
101
102     @classmethod
103     def from_initial_plan(
104         cls,
105         initial_plan_kwh: pd.Series,
106         *,
107         config: ProjectConfig = DEFAULT_CONFIG,
108     ) -> "RollingPlanBook":
109         expected = list(range(config.intervals_per_day))
110
111         if list(initial_plan_kwh.index) != expected:
112             raise ValueError(
113                 "0:00 初始计划 index 必须严格为 k=0~143。"
114             )
115
116         initial = pd.to_numeric(
117             initial_plan_kwh,
118             errors="raise",
119         ).astype(float)
120
121         if (initial < -1e-12).any():
122             raise ValueError("0:00 初始计划购电量不能为负。")
123
124         return cls(
125             original_plan_kwh=initial.copy(),
126             effective_plan_kwh=initial.copy(),
127             versions={0: initial.copy()},
128         )
129
130     def update_future(
131         self,
132         r: int,
133         future_plan_kwh: pd.Series,
134         *,
```

```
        config: ProjectConfig = DEFAULT_CONFIG,
    ) -> None:
        """
        只更新 k>=r, 严格冻结 k<r。
        """
        validate_update_k(r, config=config)

        expected_future = list(
            range(r, config.intervals_per_day)
        )

        if list(future_plan_kwh.index) != expected_future:
            raise ValueError(
                f"r={r} 更新结果 index 必须严格为 {r}~143。"
            )

        new_future = pd.to_numeric(
            future_plan_kwh,
            errors="raise",
        ).astype(float)

        if (new_future < -1e-12).any():
            raise ValueError(" 调整后正常购电量不能为负。")

        before_past = (
            self.effective_plan_kwh.loc[: r - 1].copy()
            if r > 0
            else pd.Series(dtype=float)
        )

        self.effective_plan_kwh.loc[r:] = (
            new_future.to_numpy(dtype=float)
        )

        if r > 0:
            after_past = self.effective_plan_kwh.loc[: r - 1]
            if not np.allclose(
                before_past.to_numpy(dtype=float),
                after_past.to_numpy(dtype=float),
                rtol=0.0,
                atol=0.0,
            ):
                raise AssertionError(
                    f"r={r} 更新错误地修改了 k<{r} 的过去计划。"
                )

        self.versions[r] = self.effective_plan_kwh.copy()

    def final_effective_plan(self) -> pd.Series:
        return self.effective_plan_kwh.copy()


@dataclass
class SettlementResult:
    """ 问题 3 单日费用结算结果。"""

    dG_plus_kwh: pd.Series
    dG_minus_kwh: pd.Series

    C_plan_yuan: float
    C_reg_yuan: float
    C_adj_yuan: float
    C_emg_yuan: float
    C_total_yuan: float

    identity_residual_yuan: float


def _validate_same_k(
```

```
    *series: pd.Series,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> None:
    expected = list(range(config.intervals_per_day))

    for s in series:
        if not isinstance(s, pd.Series):
            raise TypeError(" 结算输入必须全部为 pandas.Series。")
        if list(s.index) != expected:
            raise ValueError(" 结算输入 index 必须严格为 k=0~143。")
        if s.isna().any():
            raise ValueError(" 结算输入存在缺失值。")


def settle_question3_cost(
    *,
    G0_kwh: pd.Series,
    GF_kwh: pd.Series,
    price_yuan_per_kwh: pd.Series,
    G_emg_kwh: pd.Series,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> SettlementResult:
    """
    按最终唯一结算口径计算问题 3 费用。

    [RS-01]~[RS-07]
    """
    _validate_same_k(
        G0_kwh,
        GF_kwh,
        price_yuan_per_kwh,
        G_emg_kwh,
        config=config,
    )

    G0 = G0_kwh.astype(float)
    GF = GF_kwh.astype(float)
    c = price_yuan_per_kwh.astype(float)
    G_emg = G_emg_kwh.astype(float)

    if (G0 < -1e-12).any():
        raise ValueError("G0 不能为负。")
    if (GF < -1e-12).any():
        raise ValueError("GF 不能为负。")
    if (G_emg < -1e-12).any():
        raise ValueError("G_emg 不能为负。")

    # [RS-01]
    dG_plus = (GF - G0).clip(lower=0.0)
    dG_minus = (G0 - GF).clip(lower=0.0)

    # [RS-02]
    C_reg = float(
        (
            c
            * (
                GF
                + 0.5 * (dG_plus + dG_minus)
            )
        ).sum()
    )

    # [RS-03]
    C_plan = float((c * G0).sum())

    # [RS-04]
    C_adj = float(
        (
            c
```

```
            * (
                1.5 * dG_plus
                - 0.5 * dG_minus
            )
        ).sum()
    )

    # [RS-06]
    C_emg = float((5.0 * c * G_emg).sum())

    # [RS-07]
    C_total = C_reg + C_emg

    # [RS-05]
    identity_residual = abs(
        C_reg - (C_plan + C_adj)
    )

    return SettlementResult(
        dG_plus_kwh=dG_plus,
        dG_minus_kwh=dG_minus,
        C_plan_yuan=C_plan,
        C_reg_yuan=C_reg,
        C_adj_yuan=C_adj,
        C_emg_yuan=C_emg,
        C_total_yuan=C_total,
        identity_residual_yuan=identity_residual,
    )


def run_settlement_self_tests(
    *,
    c: float = 1.0,
    tolerance: float = 1e-12,
) -> pd.DataFrame:
    """
    自动测试 A/B/C。

    A. 下调: G0=100, GF=80 -> C_reg=90*c
    B. 上调: G0=100, GF=120 -> C_reg=130*c
    C. C_reg == C_plan + C_adj
    """
    if c <= 0:
        raise ValueError(" 测试电价 c 必须为正。")

    # A
    G0_a = 100.0
    GF_a = 80.0
    dplus_a = max(GF_a - G0_a, 0.0)
    dminus_a = max(G0_a - GF_a, 0.0)

    Creg_a = c * (
        GF_a + 0.5 * (dplus_a + dminus_a)
    )
    Cplan_a = c * G0_a
    Cadj_a = c * (
        1.5 * dplus_a - 0.5 * dminus_a
    )

    # B
    G0_b = 100.0
    GF_b = 120.0
    dplus_b = max(GF_b - G0_b, 0.0)
    dminus_b = max(G0_b - GF_b, 0.0)

    Creg_b = c * (
        GF_b + 0.5 * (dplus_b + dminus_b)
    )
    Cplan_b = c * G0_b
```

```
    Cadj_b = c * (
        1.5 * dplus_b - 0.5 * dminus_b
    )

    rows = [
        {
            "test": "A_down_adjustment",
            "passed": abs(Creg_a - 90.0 * c) <= tolerance,
            "actual": Creg_a,
            "expected": 90.0 * c,
        },
        {
            "test": "B_up_adjustment",
            "passed": abs(Creg_b - 130.0 * c) <= tolerance,
            "actual": Creg_b,
            "expected": 130.0 * c,
        },
        {
            "test": "C_identity_down",
            "passed": abs(
                Creg_a - (Cplan_a + Cadj_a)
            ) <= tolerance,
            "actual": Creg_a,
            "expected": Cplan_a + Cadj_a,
        },
        {
            "test": "C_identity_up",
            "passed": abs(
                Creg_b - (Cplan_b + Cadj_b)
            ) <= tolerance,
            "actual": Creg_b,
            "expected": Cplan_b + Cadj_b,
        },
    ]

    report = pd.DataFrame(rows)

    if not report["passed"].all():
        raise AssertionError(
            " 问题 3 结算自动测试失败: \n"
            + report.to_string(index=False)
        )

    return report


# 模块导入时自动执行 A/B/C，若公式被误改则立即失败。
SETTLEMENT_SELF_TEST_REPORT = run_settlement_self_tests()
```

文件：code/run_all.py

```
# -*- coding: utf-8 -*-
"""
run_all.py

正式复现入口：从附件 1-4 读取数据，依次求解 Q1、Q2、Q3、Q4-2、Q4-3，
并输出题目要求的五个 result*.xlsx。

示例：
    python code/run_all.py --data-dir ./data --output-dir ./results

说明：
- data-dir 中需包含 附件 1.xlsx ... 附件 4.xlsx；
- Q2-Q4 默认从 2025-01-01 开始顺序模拟，2025-02-01 起写入正式结果；
- 五个结果文件由 result_writer.write_all_results 统一写出并回读校验；
- 本脚本只串联正式模型，不启用 Oracle、CVaR、MILP 互斥或敏感性扩展。
"""
from __future__ import annotations

import argparse
import pickle
from pathlib import Path

from config import ProjectConfig
from data_loader import load_all
from question1_solver import Question1Solver
from question2_solver import Question2Solver
from question3_solver import Question3Solver
from question4_solver import Question4Solver
from result_writer import write_all_results


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="C 题 Q1-Q4 正式结果一键复现")
    parser.add_argument("--data-dir", default="data", help=" 附件 1-4 所在目录")
    parser.add_argument("--output-dir", default="results", help=" 五个 result 文件输出目录")
    parser.add_argument("--cache-dir", default="models", help=" 可选中间 PeriodResult 缓存目录")
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    data_dir = Path(args.data_dir)
    output_dir = Path(args.output_dir)
    cache_dir = Path(args.cache_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    cache_dir.mkdir(parents=True, exist_ok=True)

    config = ProjectConfig(data_dir=data_dir)
    bundle = load_all(config)

    print("[1/5] Q1", flush=True)
    q1 = Question1Solver(attachment1=bundle.attachment1, config=config).solve()

    print("[2/5] Q2", flush=True)
    q2_solver = Question2Solver(
        attachment1=bundle.attachment1,
        attachment2=bundle.attachment2,
        config=config,
    )
    q2 = q2_solver.solve_period()

    print("[3/5] Q3", flush=True)
    q3_solver = Question3Solver(
        attachment1=bundle.attachment1,
        attachment2=bundle.attachment2,
```

```
        attachment3=bundle.attachment3,
        config=config,
    )
    q3 = q3_solver.solve_period()

    print("[4/5] Q4-2", flush=True)
    q4_solver = Question4Solver(
        attachment1=bundle.attachment1,
        attachment2=bundle.attachment2,
        attachment3=bundle.attachment3,
        attachment4=bundle.attachment4,
        config=config,
    )
    q42 = q4_solver.solve_q4_2_period()

    print("[5/5] Q4-3", flush=True)
    q43 = q4_solver.solve_q4_3_period()

    # 保存可复核的 PeriodResult，供费用审计/结算口径分析复用。
    for name, obj in {
        "q2_period.pkl": q2,
        "q3_period.pkl": q3,
        "q4_2_period.pkl": q42,
        "q4_3_period.pkl": q43,
    }.items():
        with (cache_dir / name).open("wb") as f:
            pickle.dump(obj, f, protocol=pickle.HIGHEST_PROTOCOL)

    outputs = write_all_results(
        result1=q1,
        result2_period=q2,
        result3_period=q3,
        result4_2_period=q42,
        result4_3_period=q43,
        output_dir=output_dir,
        strict_max_three_events=False,
        config=config,
    )

    print("\n 正式结果已生成并回读校验：")
    for filename, (path, report) in outputs.items():
        passed = bool(report["passed"].all()) if "passed" in report.columns else True
        print(f"  {filename}: {path}  passed={passed}")


if __name__ == "__main__":
    main()
```

文件：code/result_writer.py

```
# -*- coding: utf-8 -*-
"""
result_writer.py
C 题 result1.xlsx / result2.xlsx / result3.xlsx /
result4-2.xlsx / result4-3.xlsx 统一结果写出器。

重要说明
--------
结果文件严格按 144 个物理业务槽位顺序写出：template_slot[k] = k。
正式输出表头统一采用真实物理区间标签，例如：
    k=0   -> 0:00-0:10
    k=60  -> 10:00-10:10
    k=143 -> 23:50-24:00

附件 1/2/4 的右端点标签只用于输入读取；若官方空白结果模板存在首槽
显示偏移，则写出时修正表头而不移动数值。任何 Excel 显示字符串都不参与
模型 k 的定位，因此不存在模型 k=144，也不会把次日 0:00-0:10 并入当天。

本模块使用 openpyxl 兼容层写出并回读校验结果文件，不从 Excel 表头文字
反向确定模型时间。
"""

from __future__ import annotations

import json
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, Iterable, Mapping, Optional, Sequence

import numpy as np
import pandas as pd

from _excel_backend import Blob, SpreadsheetFile, Workbook

from config import DEFAULT_CONFIG, ProjectConfig
from time_index import build_time_map, validate_time_map


# ============================================================================
# 模板固定结构
# ============================================================================

SHEET_PLAN = " 计划购电量"
SHEET_ADJUST = " 调整购电量"
SHEET_STORAGE = " 充放电量"
SHEET_EMERGENCY = " 紧急购电量"

RESULT_SHEETS: Dict[str, tuple[str, ...]] = {
    "result1.xlsx": (
        SHEET_PLAN,
        SHEET_STORAGE,
    ),
    "result2.xlsx": (
        SHEET_PLAN,
        SHEET_STORAGE,
        SHEET_EMERGENCY,
    ),
    "result3.xlsx": (
        SHEET_PLAN,
        SHEET_ADJUST,
        SHEET_STORAGE,
        SHEET_EMERGENCY,
    ),
    "result4-2.xlsx": (
        SHEET_PLAN,
```

```
        SHEET_STORAGE,
        SHEET_EMERGENCY,
    ),
    "result4-3.xlsx": (
        SHEET_PLAN,
        SHEET_ADJUST,
        SHEET_STORAGE,
        SHEET_EMERGENCY,
    ),
}

FORMAL_START_DATE = pd.Timestamp("2025-02-01")
FORMAL_END_DATE = pd.Timestamp("2025-12-31")
FORMAL_DATES = pd.date_range(
    FORMAL_START_DATE,
    FORMAL_END_DATE,
    freq="D",
)
FORMAL_DAY_COUNT = len(FORMAL_DATES)  # 334

REPORT_DATES: tuple[pd.Timestamp, ...] = (
    pd.Timestamp("2025-03-20"),
    pd.Timestamp("2025-06-21"),
    pd.Timestamp("2025-09-23"),
    pd.Timestamp("2025-12-21"),
)

# 论文表 1 必须只按模型 k 提取，不能搜索模板显示文字。
REPORT_INTERVAL_INDEX: tuple[int, ...] = (
    60,   # 10:00-10:10
    72,   # 12:00-12:10
    84,   # 14:00-14:10
    96,   # 16:00-16:10
    108,  # 18:00-18:10
    120,  # 20:00-20:10
)

FOUR_HOUR_BLOCKS: tuple[tuple[int, int, str], ...] = (
    (0, 24, "0:00-4:00"),
    (24, 48, "4:00-8:00"),
    (48, 72, "8:00-12:00"),
    (72, 96, "12:00-16:00"),
    (96, 120, "16:00-20:00"),
    (120, 144, "20:00-24:00"),
)



def _excel_col_name(index0: int) -> str:
    """0-based 列号转 Excel 列名。"""
    if index0 < 0:
        raise ValueError("index0 必须非负。")
    n = index0 + 1
    letters = []
    while n:
        n, rem = divmod(n - 1, 26)
        letters.append(chr(65 + rem))
    return "".join(reversed(letters))


def _range(sheet: Any, start_row0: int, start_col0: int, row_count: int, col_count: int):
    """artifact_tool 稳定 A1 范围适配器，避免依赖 get_range_by_indexes 实现差异。"""
    if row_count <= 0 or col_count <= 0:
        raise ValueError("row_count/col_count 必须为正。")
    start = f"{_excel_col_name(start_col0)}{start_row0 + 1}"
    end = (
        f"{_excel_col_name(start_col0 + col_count - 1)}"
        f"{start_row0 + row_count}"
    )
```

```
    return sheet.get_range(f"{start}:{end}")


# ============================================================================
# 公共检查 / 数据访问
# ============================================================================


class ResultWriterError(RuntimeError):
    """ 结果结构或输出模板不合法。"""


def _normalize_date(value) -> pd.Timestamp:
    return pd.Timestamp(value).normalize()


def _as_python_datetime(ts: pd.Timestamp) -> datetime:
    ts = pd.Timestamp(ts).normalize()
    return datetime(ts.year, ts.month, ts.day)


def _validate_series_144(
    series: pd.Series,
    *,
    name: str,
    config: ProjectConfig,
) -> pd.Series:
    if not isinstance(series, pd.Series):
        raise TypeError(f"{name} 必须为 pandas.Series。")

    expected = list(range(config.intervals_per_day))

    if list(series.index) != expected:
        raise ResultWriterError(
            f"{name}.index 必须严格为模型 k=0~143。"
            " 禁止通过 Excel 表头字符串重新定位。"
        )

    values = pd.to_numeric(series, errors="raise").astype(float)

    if values.isna().any():
        raise ResultWriterError(f"{name} 存在缺失值。")

    return values


def _validate_energy_145(
    series: pd.Series,
    *,
    name: str,
    config: ProjectConfig,
) -> pd.Series:
    if not isinstance(series, pd.Series):
        raise TypeError(f"{name} 必须为 pandas.Series。")

    expected = list(range(config.state_nodes_per_day))

    if list(series.index) != expected:
        raise ResultWriterError(
            f"{name}.index 必须严格为状态节点 j=0~144。"
        )

    values = pd.to_numeric(series, errors="raise").astype(float)

    if values.isna().any():
        raise ResultWriterError(f"{name} 存在缺失值。")

    return values

```

```

def _get_formal_results(period_result: Any) -> Mapping[pd.Timestamp, Any]:
    if not hasattr(period_result, "formal_results"):
        raise TypeError(
            " 多日结果必须提供 formal_results 属性。"
        )

    raw = period_result.formal_results
    if not isinstance(raw, Mapping):
        raise TypeError("formal_results 必须为日期-> 单日结果映射。")

    normalized = {
        _normalize_date(d): result
        for d, result in raw.items()
    }

    actual_dates = pd.DatetimeIndex(sorted(normalized))
    expected_dates = FORMAL_DATES

    if not actual_dates.equals(expected_dates):
        missing = expected_dates.difference(actual_dates)
        extra = actual_dates.difference(expected_dates)
        raise ResultWriterError(
            " 正式结果日期必须恰好为 2025-02-01~2025-12-31 "
            f" 共 {FORMAL_DAY_COUNT} 天。"
            f" 缺失 ={list(missing[:10])}; "
            f" 额外 ={list(extra[:10])}"
        )

    for required_date in REPORT_DATES:
        if required_date not in normalized:
            raise ResultWriterError(
                f" 缺少论文指定日期 {required_date.date()}。"
            )

    return normalized


def _unwrap_q4_base(day_result: Any) -> Any:
    """Q4 单日结果外层包含 base_result; Q1-Q3 直接返回自身。"""
    return getattr(day_result, "base_result", day_result)


def _get_plan_series(
    day_result: Any,
    *,
    mode: str,
    config: ProjectConfig,
) -> pd.Series:
    base = _unwrap_q4_base(day_result)

    if mode in {"q2", "q4-2"}:
        series = base.plan_purchase_kwh
    elif mode in {"q3", "q4-3"}:
        series = base.original_plan_kwh
    else:
        raise ValueError(f" 未知 mode={mode}")

    return _validate_series_144(
        series,
        name=f"{mode}.plan_purchase_kwh",
        config=config,
    )


def _get_adjusted_series(
    day_result: Any,
    *,
    mode: str,
```

```
    config: ProjectConfig,
) -> pd.Series:
    if mode not in {"q3", "q4-3"}:
        raise ValueError(" 只有 Q3/Q4-3 存在调整购电量。")

    base = _unwrap_q4_base(day_result)

    # 模板“调整购电量”写最终生效正常购电量 GF,
    # 而不是 dG_plus-dG_minus。
    return _validate_series_144(
        base.final_effective_plan_kwh,
        name=f"{mode}.final_effective_plan_kwh",
        config=config,
    )


def _get_charge_discharge_energy(
    day_result: Any,
    *,
    config: ProjectConfig,
) -> tuple[pd.Series, pd.Series, pd.Series]:
    base = _unwrap_q4_base(day_result)

    charge = _validate_series_144(
        base.actual_charge_kwh,
        name="actual_charge_kwh",
        config=config,
    )
    discharge = _validate_series_144(
        base.actual_discharge_kwh,
        name="actual_discharge_kwh",
        config=config,
    )
    energy = _validate_energy_145(
        base.energy_kwh,
        name="energy_kwh",
        config=config,
    )

    return charge, discharge, energy


def _get_emergency_series(
    day_result: Any,
    *,
    config: ProjectConfig,
) -> pd.Series:
    base = _unwrap_q4_base(day_result)
    return _validate_series_144(
        base.actual_emergency_kwh,
        name="actual_emergency_kwh",
        config=config,
    )


def _get_plan_cost(day_result: Any, *, mode: str) -> float:
    if mode == "q2":
        return float(day_result.plan_cost_yuan)
    if mode == "q3":
        return float(day_result.C_plan_yuan)
    if mode in {"q4-2", "q4-3"}:
        return float(day_result.C_plan_yuan)
    raise ValueError(f" 未知 mode={mode}")


def _get_regular_cost(day_result: Any, *, mode: str) -> float:
    """
    调整购电量 sheet 中的“全天购电费”:
    - Q3/Q4-3 使用最终正常购电及调整结算 C_reg;
```

```
    - 不包含紧急购电费用。
    """
    if mode in {"q3", "q4-3"}:
        return float(day_result.C_reg_yuan)
    raise ValueError(" 只有 Q3/Q4-3 存在 C_reg。")


# ============================================================================
# time_map / paper table / emergency interval
# ============================================================================


def _ordered_time_map(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.DataFrame:
    time_map = build_time_map(config)
    validate_time_map(time_map, config)

    ordered = time_map.sort_values(
        "template_slot"
    ).reset_index(drop=True)

    if ordered["template_slot"].tolist() != list(range(144)):
        raise ResultWriterError(
            "template_slot 必须严格为 0~143。"
        )

    if ordered["k"].tolist() != list(range(144)):
        raise ResultWriterError(
            "template_slot 顺序必须与模型 k 顺序一一对应。"
        )

    # 强制关键物理映射。
    checks = {
        0: ("0:00", "0:10", "0:10"),
        36: ("6:00", "6:10", "6:10"),
        60: ("10:00", "10:10", "10:10"),
        143: ("23:50", "24:00", "0:00+1"),
    }

    by_k = ordered.set_index("k")
    for k, expected in checks.items():
        got = (
            by_k.loc[k, "interval_start"],
            by_k.loc[k, "interval_end"],
            by_k.loc[k, "source_right_endpoint_label"],
        )
        if got != expected:
            raise ResultWriterError(
                f"time_map 关键映射错误: k={k}, "
                f"actual={got}, expected={expected}"
            )

    return ordered


def _series_in_template_slot_order(
    series: pd.Series,
    *,
    config: ProjectConfig,
) -> list[float]:
    """
    唯一合法的模板写入顺序。

    不是:
        根据 template_display_label 搜索时间。

    而是:
        model k
```

```
        -> time_map.template_slot
        -> 顺序写入 Excel 业务槽位。
    """
    values = _validate_series_144(
        series,
        name="template_series",
        config=config,
    )

    tm = _ordered_time_map(config)

    ordered = [
        float(values.loc[int(k)])
        for k in tm["k"]
    ]

    if len(ordered) != 144:
        raise AssertionError(" 每天必须恰好写入 144 个 10 分钟值。")

    # 额外防错: 第一个值必须是 model k=0。
    if not np.isclose(ordered[0], float(values.loc[0])):
        raise AssertionError("model k=0 在写入时发生丢失/错位。")

    # 最后一个值必须是 model k=143, 而不是不存在的 k=144。
    if not np.isclose(ordered[-1], float(values.loc[143])):
        raise AssertionError(" 最后槽位不是 model k=143。")

    return ordered


def extract_paper_table1(
    day_result: Any,
    *,
    mode: str,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.DataFrame:
    """
    论文表 1 提取器。

    只使用:
        REPORT_INTERVAL_INDEX
        time_map.paper_label

    绝不使用 template_display_label 搜索时间。
    """
    tm = _ordered_time_map(config).set_index("k")

    if mode in {"q2", "q4-2"}:
        series = _get_plan_series(
            day_result,
            mode=mode,
            config=config,
        )
    elif mode in {"q3", "q4-3"}:
        # 论文若展示最终生效购电, 可改为 _get_adjusted_series;
        # 当前按“计划购电量”表 1 默认提取 0:00 原计划 G0。
        series = _get_plan_series(
            day_result,
            mode=mode,
            config=config,
        )
    else:
        raise ValueError(f" 未知 mode={mode}")

    rows = []
    for k in REPORT_INTERVAL_INDEX:
        rows.append(
            {
                "k": k,
```

```
                "paper_label": tm.loc[k, "paper_label"],
                "purchase_kwh": float(series.loc[k]),
            }
        )

    frame = pd.DataFrame(rows)

    if frame.loc[frame["k"] == 60, "paper_label"].iloc[0] != "10:00-10:10":
        raise ResultWriterError(
            " 论文 10:00-10:10 必须来自 model k=60。"
        )

    return frame


def merge_emergency_intervals(
    emergency_kwh: pd.Series,
    *,
    tolerance_kwh: float = 1e-8,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> list[dict]:
    """
    把相邻的紧急购电物理区间自动合并。

    例如:
        k=78,79,80
    -> 13:00-13:30

    时间标签由 time_map.interval_start/end 生成,
    不使用模板显示标签。
    """
    q = _validate_series_144(
        emergency_kwh,
        name="emergency_kwh",
        config=config,
    )

    active = [
        k for k in range(config.intervals_per_day)
        if float(q.loc[k]) > tolerance_kwh
    ]

    if not active:
        return []

    runs: list[tuple[int, int]] = []
    start = active[0]
    prev = active[0]

    for k in active[1:]:
        if k == prev + 1:
            prev = k
            continue

        runs.append((start, prev))
        start = k
        prev = k

    runs.append((start, prev))

    tm = _ordered_time_map(config).set_index("k")

    events = []
    for start_k, end_k in runs:
        label = (
            f"{tm.loc[start_k, 'interval_start']}-"
            f"{tm.loc[end_k, 'interval_end']}"
        )

```

```
        amount = float(
            q.loc[start_k:end_k].sum()
        )

        events.append(
            {
                "start_k": start_k,
                "end_k": end_k,
                "paper_interval": label,
                "emergency_kwh": amount,
            }
        )

    return events


# ============================================================================
# artifact_tool workbook helpers
# ============================================================================


_HEADER_FORMAT = {
    "fill": "#D9EAF7",
    "font": {"bold": True},
    "horizontal_alignment": "center",
    "vertical_alignment": "center",
    "wrap_text": True,
}

_SUBHEADER_FORMAT = {
    "fill": "#EEF5FA",
    "font": {"bold": True},
    "horizontal_alignment": "center",
    "vertical_alignment": "center",
}


def _style_header(sheet: Any, row_count: int, col_count: int) -> None:
    header = _range(sheet,
        0, 0, 1, col_count
    )
    header.format = _HEADER_FORMAT

    body = _range(sheet,
        0, 0, row_count, col_count
    )
    body.format.vertical_alignment = "center"


def _create_plan_matrix_sheet(
    workbook: Workbook,
    *,
    sheet_name: str,
    formal_results: Mapping[pd.Timestamp, Any],
    mode: str,
    adjusted: bool,
    config: ProjectConfig,
) -> None:
    """
    result2/3/4 的全时段矩阵版计划/调整购电量。

    列:
        A 日期\时间
        B~? 144 个 template_slot
        最后两列 全天购电量 / 全天购电费
    """
    tm = _ordered_time_map(config)
    display_headers = tm["template_display_label"].tolist()

```

```
    header = (
        [" 日期\\时间"]
        + display_headers
        + [" 全天购电量", " 全天购电费"]
    )

    rows = [header]

    for d in FORMAL_DATES:
        day = formal_results[d]

        if adjusted:
            series = _get_adjusted_series(
                day,
                mode=mode,
                config=config,
            )
            total_fee = _get_regular_cost(
                day,
                mode=mode,
            )
        else:
            series = _get_plan_series(
                day,
                mode=mode,
                config=config,
            )
            total_fee = _get_plan_cost(
                day,
                mode=mode,
            )

        slot_values = _series_in_template_slot_order(
            series,
            config=config,
        )

        total_purchase = float(series.sum())

        # 全天购电量必须严格等于 sum(k=0..143)
        if abs(total_purchase - sum(slot_values)) > 1e-8:
            raise ResultWriterError(
                f"{d.date()} 全天购电量与 144 个 k 求和不一致。"
            )

        rows.append(
            [_as_python_datetime(d)]
            + slot_values
            + [total_purchase, float(total_fee)]
        )

    sheet = workbook.worksheets.add(sheet_name)
    _range(sheet,
        0, 0, len(rows), len(header)
    ).values = rows

    _style_header(
        sheet,
        row_count=len(rows),
        col_count=len(header),
    )

    sheet.freeze_panes.freeze_rows(1)
    sheet.freeze_panes.freeze_columns(1)

    _range(sheet,
        1, 0, FORMAL_DAY_COUNT, 1
    ).format.number_format = "yyyy-mm-dd"

```

```
    # 适度列宽，避免 144 列自动扩展失控。
    _range(sheet,
        0, 0, len(rows), 1
    ).format.column_width = 14

    _range(sheet,
        0, 1, len(rows), 144
    ).format.column_width = 12

    _range(sheet,
        0, 145, len(rows), 2
    ).format.column_width = 14


def _create_result1_plan_sheet(
    workbook: Workbook,
    *,
    result1: Any,
    config: ProjectConfig,
) -> None:
    tm = _ordered_time_map(config)
    q = _validate_series_144(
        result1.plan_purchase_kwh,
        name="result1.plan_purchase_kwh",
        config=config,
    )

    slot_values = _series_in_template_slot_order(
        q,
        config=config,
    )

    rows = [[" 时间段", " 购电量"]]

    for slot, value in enumerate(slot_values):
        # 只用 template_slot 顺序；显示文字不参与定位。
        rows.append(
            [
                tm.loc[slot, "template_display_label"],
                float(value),
            ]
        )

    sheet = workbook.worksheets.add(SHEET_PLAN)
    _range(sheet,
        0, 0, len(rows), 2
    ).values = rows

    _style_header(sheet, len(rows), 2)
    sheet.freeze_panes.freeze_rows(1)
    sheet.get_range("A1:A145").format.column_width = 20
    sheet.get_range("B1:B145").format.column_width = 14


def _aggregate_four_hour_blocks(
    charge: pd.Series,
    discharge: pd.Series,
) -> list[tuple[str, float, float]]:
    rows = []

    for start_k, end_k_exclusive, label in FOUR_HOUR_BLOCKS:
        charge_sum = float(
            charge.loc[
                start_k:end_k_exclusive - 1
            ].sum()
        )
        discharge_sum = float(
            discharge.loc[
                start_k:end_k_exclusive - 1
```

```
            ].sum()
        )
        rows.append(
            (label, charge_sum, discharge_sum)
        )

    return rows


def _create_result1_storage_sheet(
    workbook: Workbook,
    *,
    result1: Any,
    config: ProjectConfig,
) -> None:
    charge = _validate_series_144(
        result1.charge_kwh,
        name="result1.charge_kwh",
        config=config,
    )
    discharge = _validate_series_144(
        result1.discharge_kwh,
        name="result1.discharge_kwh",
        config=config,
    )
    energy = _validate_energy_145(
        result1.energy_kwh,
        name="result1.energy_kwh",
        config=config,
    )

    blocks = _aggregate_four_hour_blocks(
        charge,
        discharge,
    )

    rows = [[
        " 时间段",
        " 充电量",
        " 放电量",
        " 时刻",
        " 储电量",
    ]]

    for i, (label, ch, dis) in enumerate(blocks):
        if i == 0:
            time_label = "0:00"
            e = float(energy.loc[0])
        elif i == 1:
            time_label = "24:00"
            e = float(energy.loc[144])
        else:
            time_label = None
            e = None

        rows.append(
            [label, ch, dis, time_label, e]
        )

    sheet = workbook.worksheets.add(SHEET_STORAGE)
    _range(sheet,
        0, 0, len(rows), 5
    ).values = rows

    _style_header(sheet, len(rows), 5)
    sheet.get_range("A1:A7").format.column_width = 18
    sheet.get_range("B1:C7").format.column_width = 14
    sheet.get_range("D1:E7").format.column_width = 14

```

```
825
826 def _create_storage_sheet_multi(
827     workbook: Workbook,
828     *,
829     formal_results: Mapping[pd.Timestamp, Any],
830     config: ProjectConfig,
831 ) -> None:
832     rows = [[
833         " 日期",
834         " 时间段",
835         " 充电量",
836         " 放电量",
837         " 时刻",
838         " 储电量",
839     ]]
840
841     for d in FORMAL_DATES:
842         day = formal_results[d]
843         charge, discharge, energy = (
844             _get_charge_discharge_energy(
845                 day,
846                 config=config,
847             )
848         )
849         blocks = _aggregate_four_hour_blocks(
850             charge,
851             discharge,
852         )
853
854         for i, (label, ch, dis) in enumerate(blocks):
855             date_value = (
856                 _as_python_datetime(d)
857                 if i == 0
858                 else None
859             )
860
861             if i == 0:
862                 state_time = "0:00"
863                 state_energy = float(energy.loc[0])
864             elif i == 1:
865                 state_time = "24:00"
866                 state_energy = float(energy.loc[144])
867             else:
868                 state_time = None
869                 state_energy = None
870
871             rows.append(
872                 [
873                     date_value,
874                     label,
875                     ch,
876                     dis,
877                     state_time,
878                     state_energy,
879                 ]
880             )
881
882     sheet = workbook.worksheets.add(SHEET_STORAGE)
883     _range(sheet,
884         0, 0, len(rows), 6
885     ).values = rows
886
887     _style_header(sheet, len(rows), 6)
888     sheet.freeze_panes.freeze_rows(1)
889
890     _range(sheet,
891         1, 0, FORMAL_DAY_COUNT * 6, 1
892     ).format.number_format = "yyyy-mm-dd"
893
```

```
    _range(sheet, 0, 0, len(rows), 1).format.column_width = 14
    _range(sheet, 0, 1, len(rows), 1).format.column_width = 18
    _range(sheet, 0, 2, len(rows), 2).format.column_width = 14
    _range(sheet, 0, 4, len(rows), 2).format.column_width = 14


def _create_emergency_sheet(
    workbook: Workbook,
    *,
    formal_results: Mapping[pd.Timestamp, Any],
    config: ProjectConfig,
    strict_max_three_events: bool,
    tolerance_kwh: float,
) -> None:
    """
    紧急购电表按实际连续紧急购电区间逐条写出。

    当前模板中的“3 行”属于填写示例/预留展示，不作为
    每日最多 3 段的数学约束。正式默认 strict_max_three_events=False，
    因而当实际事件数超过 3 时动态扩展行数，绝不截断真实事件。

    strict_max_three_events=True 仅保留为人工模板审计模式：
    若显式启用且事件数>3，则立即报错，防止误以为 3 行是硬约束。
    """
    rows = [[" 日期", " 购电时间段", " 购电量"]]

    for d in FORMAL_DATES:
        day = formal_results[d]

        emergency = _get_emergency_series(
            day,
            config=config,
        )

        events = merge_emergency_intervals(
            emergency,
            tolerance_kwh=tolerance_kwh,
            config=config,
        )

        if strict_max_three_events and len(events) > 3:
            raise ResultWriterError(
                f"{d.date()} 合并后有 {len(events)} 个紧急购电区间，"
                " 超过模板每天预留 3 行。"
                " 为避免静默丢失，请检查模型或显式设置"
                "strict_max_three_events=False。"
            )

        n_rows = (
            max(3, len(events))
            if strict_max_three_events
            else max(1, len(events))
        )

        for i in range(n_rows):
            date_value = (
                _as_python_datetime(d)
                if i == 0
                else None
            )

            if i < len(events):
                interval_label = events[i][
                    "paper_interval"
                ]
                amount = float(
                    events[i]["emergency_kwh"]
                )
            else:
```

```
                interval_label = None
                amount = None

            rows.append(
                [date_value, interval_label, amount]
            )

    sheet = workbook.worksheets.add(SHEET_EMERGENCY)
    _range(sheet,
        0, 0, len(rows), 3
    ).values = rows

    _style_header(sheet, len(rows), 3)
    sheet.freeze_panes.freeze_rows(1)

    _range(sheet,
        1, 0, len(rows) - 1, 1
    ).format.number_format = "yyyy-mm-dd"

    _range(sheet, 0, 0, len(rows), 1).format.column_width = 14
    _range(sheet, 0, 1, len(rows), 1).format.column_width = 22
    _range(sheet, 0, 2, len(rows), 1).format.column_width = 14


# ============================================================================
# 写出后重新读取校验
# ============================================================================


def _sheet_names_from_imported_workbook(
    workbook: Any,
) -> list[str]:
    inspected = workbook.inspect(
        {
            "kind": "sheet",
            "include": "id,name",
        }
    ).ndjson

    names = []
    for line in inspected.splitlines():
        line = line.strip()
        if not line:
            continue
        record = json.loads(line)
        names.append(record["name"])

    return names


def _validate_plan_sheet_after_reload(
    workbook: Any,
    *,
    sheet_name: str,
    formal_results: Optional[
        Mapping[pd.Timestamp, Any]
    ],
    result1: Optional[Any],
    mode: Optional[str],
    adjusted: bool,
    config: ProjectConfig,
) -> list[dict]:
    checks = []
    tm = _ordered_time_map(config)

    sheet = workbook.worksheets.get_item(sheet_name)

    if result1 is not None:
        values = _range(sheet,
```

```
            0, 0, 145, 2
        ).values

        checks.append(
            {
                "item": f"{sheet_name}:result1_row_count",
                "passed": len(values) == 145,
                "detail": f"rows={len(values)}",
            }
        )

        # 144 显示标签严格来自 template_slot 顺序。
        headers = [row[0] for row in values[1:]]
        expected_headers = (
            tm["template_display_label"].tolist()
        )
        checks.append(
            {
                "item": f"{sheet_name}:template_slot_headers",
                "passed": headers == expected_headers,
                "detail": " 显示标签仅按 template_slot 顺序输出。",
            }
        )

        expected_q = _validate_series_144(
            result1.plan_purchase_kwh,
            name="result1.plan_purchase_kwh",
            config=config,
        )

        written_q = [
            float(row[1]) for row in values[1:]
        ]

        checks.append(
            {
                "item": f"{sheet_name}:k0_not_missing",
                "passed": np.isclose(
                    written_q[0],
                    float(expected_q.loc[0]),
                ),
                "detail": (
                    " 第一业务槽位必须写 model k=0, "
                    " 不能从 k=1 开始。"
                ),
            }
        )

        checks.append(
            {
                "item": f"{sheet_name}:last_is_k143",
                "passed": np.isclose(
                    written_q[-1],
                    float(expected_q.loc[143]),
                ),
                "detail": (
                    " 最后业务槽位必须写 model k=143, "
                    " 不能写不存在的次日 k=144。"
                ),
            }
        )

        return checks

    if formal_results is None or mode is None:
        raise ValueError(" 多日计划表校验缺少 formal_results/mode。")

    # Header + 334 dates, 147 columns.
    values = _range(sheet,
```

```
        0, 0, FORMAL_DAY_COUNT + 1, 147
    ).values

    checks.append(
        {
            "item": f"{sheet_name}:shape",
            "passed": (
                len(values) == FORMAL_DAY_COUNT + 1
                and all(len(row) == 147 for row in values)
            ),
            "detail": (
                f"expected=335x147; "
                f"actual_rows={len(values)}"
            ),
        }
    )

    header = values[0]
    expected_slot_headers = (
        tm["template_display_label"].tolist()
    )

    checks.append(
        {
            "item": f"{sheet_name}:slot_header_order",
            "passed": (
                header[1:145] == expected_slot_headers
            ),
            "detail": (
                "Excel 显示表头只验证 template_slot 顺序, "
                " 不参与模型 k 定位。"
            ),
        }
    )

    first_day = formal_results[FORMAL_DATES[0]]

    if adjusted:
        expected_first_series = _get_adjusted_series(
            first_day,
            mode=mode,
            config=config,
        )
    else:
        expected_first_series = _get_plan_series(
            first_day,
            mode=mode,
            config=config,
        )

    # B 列对应 template_slot=0 -> model k=0
    written_first_k0 = float(values[1][1])
    checks.append(
        {
            "item": f"{sheet_name}:k0_not_missing",
            "passed": np.isclose(
                written_first_k0,
                float(expected_first_series.loc[0]),
            ),
            "detail": (
                " 首个业务单元格必须是 model k=0 "
                "(physical 0:00-0:10)。"
            ),
        }
    )

    # 第 144 个槽位对应 k=143, 不是下一天 0:00-0:10。
    written_first_k143 = float(values[1][144])
    checks.append(
```

```
        {
            "item": f"{sheet_name}:no_next_day_k144",
            "passed": np.isclose(
                written_first_k143,
                float(expected_first_series.loc[143]),
            ),
            "detail": (
                " 最后业务槽位必须显示并写入 model k=143 "
                "(physical 23:50-24:00)。"
            ),
        }
    )

    # 全量检查每天全天购电量 =144 个槽位之和。
    total_ok = True
    max_residual = 0.0

    for row_index, d in enumerate(FORMAL_DATES, start=1):
        slot_values = np.array(
            [
                float(x)
                for x in values[row_index][1:145]
            ],
            dtype=float,
        )
        stored_total = float(values[row_index][145])
        residual = abs(
            stored_total - float(slot_values.sum())
        )
        max_residual = max(max_residual, residual)
        if residual > 1e-7:
            total_ok = False
            break

    checks.append(
        {
            "item": f"{sheet_name}:daily_total_equals_sum_k0_143",
            "passed": total_ok,
            "detail": (
                f"max_residual={max_residual:.12g} kWh"
            ),
        }
    )

    return checks


def validate_written_result(
    path: str | Path,
    *,
    filename: str,
    config: ProjectConfig = DEFAULT_CONFIG,
    formal_results: Optional[
        Mapping[pd.Timestamp, Any]
    ] = None,
    result1: Optional[Any] = None,
    mode: Optional[str] = None,
) -> pd.DataFrame:
    """
    导出后重新读取 xlsx, 执行结构校验。

    校验包括:
    - sheet 名称与数量;
    - 日期范围/数量;
    - 144 个 template_slot;
    - k=0 未漏;
    - 最后槽位为 k=143 而非次日 k=144;
    - 每日总购电量 =sum(k=0..143);
    - 充放电表的 0:00/24:00 SOC 结构;
```

```
    - 紧急购电表结构;
    - 论文 k=60 物理标签为 10:00-10:10。
    """
    path = Path(path)

    if filename not in RESULT_SHEETS:
        raise ValueError(f" 未知结果文件名: {filename}")

    workbook = SpreadsheetFile.import_xlsx(
        Blob.load(str(path))
    )

    rows = []

    actual_sheet_names = (
        _sheet_names_from_imported_workbook(workbook)
    )
    expected_sheet_names = list(
        RESULT_SHEETS[filename]
    )

    rows.append(
        {
            "item": "sheet_names",
            "passed": actual_sheet_names
            == expected_sheet_names,
            "detail": (
                f"actual={actual_sheet_names}; "
                f"expected={expected_sheet_names}"
            ),
        }
    )

    # time_map 本体防错
    tm = _ordered_time_map(config).set_index("k")

    rows.append(
        {
            "item": "paper_k60",
            "passed": (
                tm.loc[60, "paper_label"]
                == "10:00-10:10"
            ),
            "detail": (
                " 论文 10:00-10:10 必须来自 model k=60。"
            ),
        }
    )

    rows.append(
        {
            "item": "physical_k0",
            "passed": (
                tm.loc[0, "paper_label"] == "0:00-0:10"
                and tm.loc[
                    0, "source_right_endpoint_label"
                ] == "0:10"
            ),
            "detail": "k=0 物理区间/右端点映射。",
        }
    )

    rows.append(
        {
            "item": "physical_k143",
            "passed": (
                tm.loc[143, "paper_label"]
                == "23:50-24:00"
                and tm.loc[
```

```
                    143, "source_right_endpoint_label"
                ] == "0:00+1"
            ),
            "detail": "k=143 为当天最后物理区间。",
        }
    )

    if filename == "result1.xlsx":
        rows.extend(
            _validate_plan_sheet_after_reload(
                workbook,
                sheet_name=SHEET_PLAN,
                formal_results=None,
                result1=result1,
                mode=None,
                adjusted=False,
                config=config,
            )
        )

        storage = workbook.worksheets.get_item(
            SHEET_STORAGE
        )
        storage_values = _range(storage,
            0, 0, 7, 5
        ).values

        rows.append(
            {
                "item": "result1_storage_rows",
                "passed": len(storage_values) == 7,
                "detail": "header+6 个 4 小时区间。",
            }
        )

        rows.append(
            {
                "item": "result1_soc_0_24",
                "passed": (
                    storage_values[1][3] == "0:00"
                    and storage_values[2][3] == "24:00"
                ),
                "detail": " 只在模板指定位置写 0:00/24:00 储电量。",
            }
        )
    else:
        adjusted = filename in {
            "result3.xlsx",
            "result4-3.xlsx",
        }

        rows.extend(
            _validate_plan_sheet_after_reload(
                workbook,
                sheet_name=SHEET_PLAN,
                formal_results=formal_results,
                result1=None,
                mode=mode,
                adjusted=False,
                config=config,
            )
        )

        if adjusted:
            rows.extend(
                _validate_plan_sheet_after_reload(
                    workbook,
                    sheet_name=SHEET_ADJUST,
                    formal_results=formal_results,
```

```
                result1=None,
                mode=mode,
                adjusted=True,
                config=config,
            )
        )

    storage = workbook.worksheets.get_item(
        SHEET_STORAGE
    )

    expected_storage_rows = (
        1 + FORMAL_DAY_COUNT * 6
    )

    storage_values = _range(storage,
        0, 0, expected_storage_rows, 6
    ).values

    rows.append(
        {
            "item": "storage_row_count",
            "passed": (
                len(storage_values)
                == expected_storage_rows
            ),
            "detail": (
                f"header+{FORMAL_DAY_COUNT}*6="
                f"{expected_storage_rows}"
            ),
        }
    )

    # 每天第 1/2 行必须分别对应 0:00/24:00。
    soc_format_ok = True
    for day_i in range(FORMAL_DAY_COUNT):
        base_row = 1 + day_i * 6
        if (
            storage_values[base_row][4] != "0:00"
            or storage_values[base_row + 1][4]
            != "24:00"
        ):
            soc_format_ok = False
            break

    rows.append(
        {
            "item": "storage_soc_0_24_format",
            "passed": soc_format_ok,
            "detail": " 每个日期只写 0:00 和 24:00 储电量。",
        }
    )

    emergency = workbook.worksheets.get_item(
        SHEET_EMERGENCY
    )

    emergency_header = emergency.get_range(
        "A1:C1"
    ).values[0]

    rows.append(
        {
            "item": "emergency_header",
            "passed": emergency_header
            == [" 日期", " 购电时间段", " 购电量"],
            "detail": f"header={emergency_header}",
        }
    )
```

```
    report = pd.DataFrame(rows)

    failed = report.loc[~report["passed"]]
    if not failed.empty:
        raise ResultWriterError(
            f"{filename} 写出后结构校验失败: \n"
            + failed.to_string(index=False)
        )

    return report


# ============================================================================
# 五个文件写出
# ============================================================================


def write_result1(
    result1: Any,
    output_dir: str | Path,
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> tuple[Path, pd.DataFrame]:
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    workbook = Workbook.create()

    _create_result1_plan_sheet(
        workbook,
        result1=result1,
        config=config,
    )
    _create_result1_storage_sheet(
        workbook,
        result1=result1,
        config=config,
    )

    path = output_dir / "result1.xlsx"
    SpreadsheetFile.export_xlsx(workbook).save(
        str(path)
    )

    report = validate_written_result(
        path,
        filename="result1.xlsx",
        result1=result1,
        config=config,
    )

    return path, report


def _write_multi_day_result(
    *,
    filename: str,
    period_result: Any,
    mode: str,
    output_dir: str | Path,
    include_adjustment: bool,
    strict_max_three_events: bool,
    emergency_tolerance_kwh: float,
    config: ProjectConfig,
) -> tuple[Path, pd.DataFrame]:
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

```

```
    formal_results = _get_formal_results(
        period_result
    )

    workbook = Workbook.create()

    _create_plan_matrix_sheet(
        workbook,
        sheet_name=SHEET_PLAN,
        formal_results=formal_results,
        mode=mode,
        adjusted=False,
        config=config,
    )

    if include_adjustment:
        _create_plan_matrix_sheet(
            workbook,
            sheet_name=SHEET_ADJUST,
            formal_results=formal_results,
            mode=mode,
            adjusted=True,
            config=config,
        )

    _create_storage_sheet_multi(
        workbook,
        formal_results=formal_results,
        config=config,
    )

    _create_emergency_sheet(
        workbook,
        formal_results=formal_results,
        config=config,
        strict_max_three_events=(
            strict_max_three_events
        ),
        tolerance_kwh=emergency_tolerance_kwh,
    )

    path = output_dir / filename
    SpreadsheetFile.export_xlsx(workbook).save(
        str(path)
    )

    report = validate_written_result(
        path,
        filename=filename,
        formal_results=formal_results,
        mode=mode,
        config=config,
    )

    return path, report


def write_result2(
    period_result: Any,
    output_dir: str | Path,
    *,
    strict_max_three_events: bool = False,
    emergency_tolerance_kwh: float = 1e-8,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> tuple[Path, pd.DataFrame]:
    return _write_multi_day_result(
        filename="result2.xlsx",
        period_result=period_result,
        mode="q2",
```

```
        output_dir=output_dir,
        include_adjustment=False,
        strict_max_three_events=(
            strict_max_three_events
        ),
        emergency_tolerance_kwh=(
            emergency_tolerance_kwh
        ),
        config=config,
    )


def write_result3(
    period_result: Any,
    output_dir: str | Path,
    *,
    strict_max_three_events: bool = False,
    emergency_tolerance_kwh: float = 1e-8,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> tuple[Path, pd.DataFrame]:
    return _write_multi_day_result(
        filename="result3.xlsx",
        period_result=period_result,
        mode="q3",
        output_dir=output_dir,
        include_adjustment=True,
        strict_max_three_events=(
            strict_max_three_events
        ),
        emergency_tolerance_kwh=(
            emergency_tolerance_kwh
        ),
        config=config,
    )


def write_result4_2(
    period_result: Any,
    output_dir: str | Path,
    *,
    strict_max_three_events: bool = False,
    emergency_tolerance_kwh: float = 1e-8,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> tuple[Path, pd.DataFrame]:
    return _write_multi_day_result(
        filename="result4-2.xlsx",
        period_result=period_result,
        mode="q4-2",
        output_dir=output_dir,
        include_adjustment=False,
        strict_max_three_events=(
            strict_max_three_events
        ),
        emergency_tolerance_kwh=(
            emergency_tolerance_kwh
        ),
        config=config,
    )


def write_result4_3(
    period_result: Any,
    output_dir: str | Path,
    *,
    strict_max_three_events: bool = False,
    emergency_tolerance_kwh: float = 1e-8,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> tuple[Path, pd.DataFrame]:
    return _write_multi_day_result(
```

```
        filename="result4-3.xlsx",
        period_result=period_result,
        mode="q4-3",
        output_dir=output_dir,
        include_adjustment=True,
        strict_max_three_events=(
            strict_max_three_events
        ),
        emergency_tolerance_kwh=(
            emergency_tolerance_kwh
        ),
        config=config,
    )


def write_all_results(
    *,
    result1: Any,
    result2_period: Any,
    result3_period: Any,
    result4_2_period: Any,
    result4_3_period: Any,
    output_dir: str | Path,
    strict_max_three_events: bool = False,
    emergency_tolerance_kwh: float = 1e-8,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> Dict[str, tuple[Path, pd.DataFrame]]:
    """
    一次写出五个题目要求的结果文件，并全部执行导出后回读校验。
    """
    outputs: Dict[
        str,
        tuple[Path, pd.DataFrame],
    ] = {}

    outputs["result1.xlsx"] = write_result1(
        result1,
        output_dir,
        config=config,
    )

    outputs["result2.xlsx"] = write_result2(
        result2_period,
        output_dir,
        strict_max_three_events=(
            strict_max_three_events
        ),
        emergency_tolerance_kwh=(
            emergency_tolerance_kwh
        ),
        config=config,
    )

    outputs["result3.xlsx"] = write_result3(
        result3_period,
        output_dir,
        strict_max_three_events=(
            strict_max_three_events
        ),
        emergency_tolerance_kwh=(
            emergency_tolerance_kwh
        ),
        config=config,
    )

    outputs["result4-2.xlsx"] = write_result4_2(
        result4_2_period,
        output_dir,
        strict_max_three_events=(
```

```
            strict_max_three_events
        ),
        emergency_tolerance_kwh=(
            emergency_tolerance_kwh
        ),
        config=config,
    )

    outputs["result4-3.xlsx"] = write_result4_3(
        result4_3_period,
        output_dir,
        strict_max_three_events=(
            strict_max_three_events
        ),
        emergency_tolerance_kwh=(
            emergency_tolerance_kwh
        ),
        config=config,
    )

    return outputs
```

文件：code/review_audit.py

```
# -*- coding: utf-8 -*-
"""
review_audit.py
评审意见对应的可复现审计脚本。

用途：
1. 检查 Q3/Q4-3 的附件 3 发布时间映射是否严格为 0/6/12/18 点；
2. 检查物理 k、业务槽位和正式输出时间标签是否一致；
3. 检查 result_writer 默认不会把“3 行示例”误当成每日最多 3 段；
4. 可选：核对官方模板与正式结果的工作表结构，并验证结果时间表头已修正为物理区间。

运行：
    python review_audit.py
    python review_audit.py --template-dir data/template --result-dir output/results
"""

from __future__ import annotations

import argparse
import inspect
from pathlib import Path

import pandas as pd

from config import DEFAULT_CONFIG
from rolling_scheduler import UPDATE_K, run_settlement_self_tests
from time_index import build_time_map
import result_writer


def check_issue_mapping() -> list[dict]:
    rows = []
    date = pd.Timestamp("2025-01-01")
    for r, hour in zip(UPDATE_K, (0, 6, 12, 18)):
        issue = date + pd.Timedelta(
            minutes=int(r) * int(DEFAULT_CONFIG.dt_minutes)
        )
        expected = date + pd.Timedelta(hours=hour)
        rows.append(
            {
                "item": f"issue_r_{r}",
                "passed": issue == expected,
                "detail": f"{issue} == {expected}",
            }
        )
    return rows


def check_time_map() -> list[dict]:
    tm = build_time_map(DEFAULT_CONFIG).set_index("k")
    expected = {
        0: ("0:00-0:10", "0:00-0:10"),
        41: ("6:50-7:00", "6:50-7:00"),
        60: ("10:00-10:10", "10:00-10:10"),
        143: ("23:50-24:00", "23:50-24:00"),
    }
    rows = []
    for k, (physical, template) in expected.items():
        got_physical = tm.loc[k, "paper_label"]
        got_template = tm.loc[k, "template_display_label"]
        rows.append(
            {
                "item": f"time_k_{k}",
                "passed": (
                    got_physical == physical
```

```
                    and got_template == template
                    and int(tm.loc[k, "template_slot"]) == k
                ),
                "detail": (
                    f"physical={got_physical}; "
                    f"template={got_template}; "
                    f"slot={tm.loc[k, 'template_slot']}"
                ),
            }
        )
    return rows


def check_writer_defaults() -> list[dict]:
    funcs = [
        result_writer.write_result2,
        result_writer.write_result3,
        result_writer.write_result4_2,
        result_writer.write_result4_3,
        result_writer.write_all_results,
    ]
    rows = []
    for func in funcs:
        default = inspect.signature(func).parameters[
            "strict_max_three_events"
        ].default
        rows.append(
            {
                "item": f"{func.__name__}:strict_default_false",
                "passed": default is False,
                "detail": f"default={default!r}",
            }
        )
    return rows


def compare_template_headers(
    template_dir: Path,
    result_dir: Path,
) -> list[dict]:
    try:
        from openpyxl import load_workbook
    except ImportError as exc:
        raise RuntimeError(
            " 模板逐列核验需要 openpyxl。"
        ) from exc

    rows = []
    for filename in (
        "result2.xlsx",
        "result3.xlsx",
        "result4-2.xlsx",
        "result4-3.xlsx",
    ):
        template_path = template_dir / filename
        result_path = result_dir / filename
        if not template_path.exists() or not result_path.exists():
            rows.append(
                {
                    "item": f"{filename}:files_exist",
                    "passed": False,
                    "detail": (
                        f"template={template_path.exists()}, "
                        f"result={result_path.exists()}"
                    ),
                }
            )
            continue

```

```
        twb = load_workbook(
            template_path,
            read_only=True,
            data_only=False,
        )
        rwb = load_workbook(
            result_path,
            read_only=True,
            data_only=False,
        )

        rows.append(
            {
                "item": f"{filename}:sheet_names",
                "passed": twb.sheetnames == rwb.sheetnames,
                "detail": (
                    f"template={twb.sheetnames}; "
                    f"result={rwb.sheetnames}"
                ),
            }
        )

        for sheet_name in (" 计划购电量", " 调整购电量"):
            if sheet_name not in twb.sheetnames:
                continue
            tws = twb[sheet_name]
            rws = rwb[sheet_name]

            template_header = [
                tws.cell(1, c).value
                for c in range(1, tws.max_column + 1)
            ]
            result_header = [
                rws.cell(1, c).value
                for c in range(1, tws.max_column + 1)
            ]
            # 官方空白模板的时间显示文字可能存在 10 min 偏移;
            # 正式结果保留相同槽位数量和非时间字段，但时间表头统一
            # 改写为真实物理区间，不要求与异常显示文字逐字相同。
            corrected = build_time_map(DEFAULT_CONFIG)[
                "template_display_label"
            ].tolist()
            same_shape = tws.max_column == rws.max_column
            same_non_time_tail = (
                template_header[145:] == result_header[145:]
                if len(template_header) >= 147 and len(result_header) >= 147
                else True
            )
            rows.append(
                {
                    "item": f"{filename}:{sheet_name}:structure",
                    "passed": bool(same_shape and same_non_time_tail),
                    "detail": (
                        f"columns(template/result)={tws.max_column}/{rws.max_column}; "
                        f"tail_match={same_non_time_tail}"
                    ),
                }
            )
            rows.append(
                {
                    "item": f"{filename}:{sheet_name}:physical_time_headers",
                    "passed": result_header[1:145] == corrected,
                    "detail": (
                        f"result_first={result_header[1]!r}; "
                        f"result_k60={result_header[61]!r}; "
                        f"result_last={result_header[144]!r}"
                    ),
                }
            )
```

```
    return rows


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--template-dir",
        type=Path,
        default=None,
    )
    parser.add_argument(
        "--result-dir",
        type=Path,
        default=None,
    )
    parser.add_argument(
        "--csv",
        type=Path,
        default=Path("output/diagnostics/review_audit.csv"),
    )
    args = parser.parse_args()

    rows = []
    rows += check_issue_mapping()
    rows += check_time_map()
    rows += check_writer_defaults()

    settlement = run_settlement_self_tests()
    for _, row in settlement.iterrows():
        rows.append(
            {
                "item": f"settlement:{row['test']}",
                "passed": bool(row["passed"]),
                "detail": (
                    f"actual={row['actual']}; "
                    f"expected={row['expected']}"
                ),
            }
        )

    if args.template_dir is not None and args.result_dir is not None:
        rows += compare_template_headers(
            args.template_dir,
            args.result_dir,
        )

    report = pd.DataFrame(rows)
    args.csv.parent.mkdir(parents=True, exist_ok=True)
    report.to_csv(args.csv, index=False, encoding="utf-8-sig")

    print(report.to_string(index=False))
    if not report["passed"].all():
        raise SystemExit("REVIEW AUDIT FAILED")
    print("REVIEW AUDIT PASSED")


if __name__ == "__main__":
    main()
```

文件：code/analysis_cost_summary.py

```python
# -*- coding: utf-8 -*-
"""
analysis_cost_summary.py
从已经完成的 period.pkl 直接汇总 Q2/Q3/Q4 的真实费用分解。

本脚本不重新优化，只读取正式运行结果对象中的逐日费用字段，
因此不会根据 Excel 长表反推紧急购电费用。

运行:
    python analysis_cost_summary.py \
        --model-dir models \
        --out output/diagnostics/cost_summary.csv
"""

from __future__ import annotations

import argparse
import pickle
from pathlib import Path

import pandas as pd


def _load(path: Path):
    with path.open("rb") as f:
        return pickle.load(f)


def summarize_q2(period) -> dict:
    days = list(period.formal_results.values())
    regular = sum(float(d.plan_cost_yuan) for d in days)
    emergency = sum(float(d.emergency_cost_yuan) for d in days)
    total = sum(float(d.total_cost_yuan) for d in days)
    return {
        "方案": "Q2",
        "正常购电结算费用_元": regular,
        "紧急购电费用_元": emergency,
        "总运行费用_元": total,
    }


def summarize_q3_like(name: str, period) -> dict:
    days = list(period.formal_results.values())
    regular = sum(float(d.C_reg_yuan) for d in days)
    emergency = sum(float(d.C_emg_yuan) for d in days)
    total = sum(float(d.C_total_yuan) for d in days)
    plan = sum(float(d.C_plan_yuan) for d in days)
    adjustment = sum(float(d.C_adj_yuan) for d in days)
    return {
        "方案": name,
        "原计划费用_元": plan,
        "调整费用_元": adjustment,
        "正常购电结算费用_元": regular,
        "紧急购电费用_元": emergency,
        "总运行费用_元": total,
    }


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--model-dir", type=Path, default=Path("models"))
    parser.add_argument(
        "--out",
        type=Path,
        default=Path("output/diagnostics/cost_summary.csv"),
```

```
    )
    args = parser.parse_args()

    q2 = _load(args.model_dir / "q2_period.pkl")
    q3 = _load(args.model_dir / "q3_period.pkl")
    q42 = _load(args.model_dir / "q4_2_period.pkl")
    q43 = _load(args.model_dir / "q4_3_period.pkl")

    rows = [
        summarize_q2(q2),
        summarize_q3_like("Q3", q3),
        summarize_q3_like("Q4-2", q42),
        summarize_q3_like("Q4-3", q43),
    ]
    frame = pd.DataFrame(rows)
    args.out.parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(args.out, index=False, encoding="utf-8-sig")
    print(frame.to_string(index=False))


if __name__ == "__main__":
    main()
```

文件：code/analysis_settlement_sensitivity.py

```python
# -*- coding: utf-8 -*-
"""
analysis_settlement_sensitivity.py
Q3/Q4-3 调整费用结算口径敏感性。

目的：
比较同一组已经求得的滚动计划版本在两种“计费解释”下的费用：
A. 正式主假设：最终计划 GF 相对 0:00 原计划 G0 一次净结算；
B. 逐次结算：6:00/12:00/18:00 每次新计划相对上一版有效计划结算。

注意：
本脚本属于“固定计划版本的结算口径敏感性”，不重新优化滚动计划。
若要研究不同结算规则对最优策略本身的影响，应另行修改目标函数并
重新运行 Q3/Q4-3。论文中必须明确区分这两层含义。

运行：
    python analysis_settlement_sensitivity.py \
        --model-dir models \
        --data-dir data \
        --out output/diagnostics/settlement_sensitivity.csv
"""

from __future__ import annotations

import argparse
import pickle
from pathlib import Path

import pandas as pd

from config import ProjectConfig
from data_loader import load_attachment1


def _load(path: Path):
    with path.open("rb") as f:
        return pickle.load(f)


def sequential_regular_cost(
    *,
    original_plan: pd.Series,
    plan_versions: dict[int, pd.Series],
    price: pd.Series,
) -> tuple[float, float, float]:
    """
    返回 (C_plan, C_adj_seq, C_reg_seq)。

    对每次 r>0 更新，仅对尚未执行的 k>=r 与上一版计划之间的差额收费。
    """
    G0 = original_plan.astype(float).copy()
    current = G0.copy()
    price = price.astype(float)

    expected = list(range(144))
    for series in (G0, price):
        if list(series.index) != expected:
            raise ValueError(" 结算输入必须严格使用 k=0..143。")

    C_plan = float((price * G0).sum())
    C_adj_seq = 0.0

    for r in sorted(int(x) for x in plan_versions):
        if r == 0:
            continue
```

```
        if r not in {36, 72, 108}:
            raise ValueError(f" 发现非标准更新时间 r={r}")

        new = plan_versions[r].astype(float)
        if list(new.index) != expected:
            raise ValueError("plan_versions 必须保存完整 144 点快照。")

        if r > 0 and not new.loc[: r - 1].equals(current.loc[: r - 1]):
            # 允许极小浮点误差，使用差值再判断
            max_residual = float(
                (new.loc[: r - 1] - current.loc[: r - 1]).abs().max()
            )
            if max_residual > 1e-8:
                raise AssertionError(
                    f"r={r} 计划版本修改了已执行区间，"
                    f"max_residual={max_residual}"
                )

        idx = list(range(r, 144))
        delta = new.loc[idx] - current.loc[idx]
        up = delta.clip(lower=0.0)
        down = (-delta).clip(lower=0.0)

        C_adj_seq += float(
            (
                price.loc[idx]
                * (1.5 * up - 0.5 * down)
            ).sum()
        )
        current.loc[idx] = new.loc[idx]

    return C_plan, C_adj_seq, C_plan + C_adj_seq


def summarize_q3(period, fixed_price: pd.Series) -> dict:
    net_regular = 0.0
    sequential_regular = 0.0
    emergency = 0.0

    for day in period.formal_results.values():
        _, _, seq_reg = sequential_regular_cost(
            original_plan=day.original_plan_kwh,
            plan_versions=day.plan_versions,
            price=fixed_price,
        )
        net_regular += float(day.C_reg_yuan)
        sequential_regular += seq_reg
        emergency += float(day.C_emg_yuan)

    return {
        " 方案": "Q3",
        " 净结算正常购电费用 _ 元": net_regular,
        " 逐次结算正常购电费用 _ 元": sequential_regular,
        " 逐次相对净结算增量 _ 元": sequential_regular - net_regular,
        " 逐次相对净结算增幅 _pct": (
            sequential_regular / net_regular - 1.0
        ) * 100.0,
        " 紧急购电费用 _ 元": emergency,
        " 净结算总运行费用 _ 元": net_regular + emergency,
        " 逐次结算总运行费用 _ 元": sequential_regular + emergency,
    }


def summarize_q43(period) -> dict:
    net_regular = 0.0
    sequential_regular = 0.0
    emergency = 0.0

    for day4 in period.formal_results.values():
```

```
        base = day4.base_result
        _, _, seq_reg = sequential_regular_cost(
            original_plan=base.original_plan_kwh,
            plan_versions=base.plan_versions,
            price=day4.actual_price_yuan_per_kwh,
        )
        net_regular += float(day4.C_reg_yuan)
        sequential_regular += seq_reg
        emergency += float(day4.C_emg_yuan)

    return {
        " 方案": "Q4-3",
        " 净结算正常购电费用 _ 元": net_regular,
        " 逐次结算正常购电费用 _ 元": sequential_regular,
        " 逐次相对净结算增量 _ 元": sequential_regular - net_regular,
        " 逐次相对净结算增幅 _pct": (
            sequential_regular / net_regular - 1.0
        ) * 100.0,
        " 紧急购电费用 _ 元": emergency,
        " 净结算总运行费用 _ 元": net_regular + emergency,
        " 逐次结算总运行费用 _ 元": sequential_regular + emergency,
    }


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--model-dir", type=Path, default=Path("models"))
    parser.add_argument("--data-dir", type=Path, default=Path("data"))
    parser.add_argument(
        "--out",
        type=Path,
        default=Path(
            "output/diagnostics/settlement_sensitivity.csv"
        ),
    )
    args = parser.parse_args()

    config = ProjectConfig(data_dir=args.data_dir)
    attachment1 = load_attachment1(config)
    fixed_price = (
        attachment1.aligned["price_yuan_per_kwh"]
        .astype(float)
        .copy()
    )
    fixed_price.index = range(144)

    q3 = _load(args.model_dir / "q3_period.pkl")
    q43 = _load(args.model_dir / "q4_3_period.pkl")

    frame = pd.DataFrame(
        [
            summarize_q3(q3, fixed_price),
            summarize_q43(q43),
        ]
    )
    args.out.parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(args.out, index=False, encoding="utf-8-sig")
    print(frame.to_string(index=False))


if __name__ == "__main__":
    main()
```

文件：code/gurobipy.py

```
# -*- coding: utf-8 -*-
"""
gurobipy.py
gurobipy 兼容层（HiGHS 后端）。

本项目原先依赖 Gurobi，但 Gurobi 许可证受限/被禁止。
本模块用开源求解器 HiGHS（经 scipy.optimize.milp 调用）实现了
src/ 下各求解器实际用到的 gurobipy 子集，从而无需改动求解器的
业务逻辑即可替换求解器。

覆盖的接口（仅这些，与 model_base/question1/2/3 的用法一一对应）：
    gp.Model / gp.quicksum / gp.LinExpr / gp.tupledict
    gp.Var / gp.Constr / GRB 常量
    model.addVar / addVars / addConstr / setObjective / optimize
    model.Params.OutputFlag / model.Status / model.ObjVal / var.X

不支持的 gurobipy 高级特性（本项目未使用）会抛出明确的 TypeError/ValueError。
"""

from __future__ import annotations

import itertools
from typing import Any, Iterable, Optional


# ============================================================================
# 常量
# ============================================================================

class GRB:
    """gurobipy.GRB 的常量子集。"""

    MINIMIZE = 1
    MAXIMIZE = -1

    CONTINUOUS = "C"
    BINARY = "B"
    INTEGER = "I"

    OPTIMAL = 2
    INFEASIBLE = 3
    INF_OR_UNBD = 4
    UNBOUNDED = 5
    TIME_LIMIT = 9
    SUBOPTIMAL = 13

    INFINITY = float("inf")

    EQUAL = "="
    LESS_EQUAL = "<"
    GREATER_EQUAL = ">"


# ============================================================================
# 线性表达式
# ============================================================================

def _expr_parts(x: Any) -> tuple[dict[int, float], float]:
    """ 把 Var / LinExpr / 数字 拆成（系数表，常数）。"""
    if isinstance(x, LinExpr):
        return x._coeffs, x._constant
    if isinstance(x, (int, float)):
        return {}, float(x)
    raise TypeError(f" 不支持的操作数类型：{type(x)!r}")

```

```

def _as_lin(x: Any) -> "LinExpr":
    """ 把 Var / 数字 归一为 LinExpr。"""
    if isinstance(x, LinExpr):
        return x
    if isinstance(x, (int, float)):
        e = LinExpr()
        e._constant = float(x)
        return e
    raise TypeError(f" 不支持的操作数类型: {type(x)!r}")


def _new_expr(coeffs: dict[int, float], constant: float) -> "LinExpr":
    e = LinExpr()
    e._coeffs = {
        k: float(v)
        for k, v in coeffs.items()
        if v != 0.0
    }
    e._constant = float(constant)
    return e


class LinExpr:
    """ 线性表达式: coeffs · x + constant。"""

    def __init__(self, constant: float = 0.0):
        self._coeffs: dict[int, float] = {}
        self._constant: float = float(constant)

    # -- 算术 ------------------------------------------------------------
    def __add__(self, other: Any) -> "LinExpr":
        c2, k2 = _expr_parts(other)
        c = dict(self._coeffs)
        for i, v in c2.items():
            c[i] = c.get(i, 0.0) + v
        return _new_expr(c, self._constant + k2)

    def __radd__(self, other: Any) -> "LinExpr":
        return self.__add__(other)

    def __sub__(self, other: Any) -> "LinExpr":
        c2, k2 = _expr_parts(other)
        c = dict(self._coeffs)
        for i, v in c2.items():
            c[i] = c.get(i, 0.0) - v
        return _new_expr(c, self._constant - k2)

    def __rsub__(self, other: Any) -> "LinExpr":
        c2, k2 = _expr_parts(other)
        c = {i: -v for i, v in self._coeffs.items()}
        for i, v in c2.items():
            c[i] = c.get(i, 0.0) + v
        return _new_expr(c, k2 - self._constant)

    def __mul__(self, other: Any) -> "LinExpr":
        if isinstance(other, (int, float)):
            s = float(other)
            return _new_expr(
                {i: v * s for i, v in self._coeffs.items()},
                self._constant * s,
            )
        raise TypeError(" 不支持非线性项 (变量 × 变量)。")

    def __rmul__(self, other: Any) -> "LinExpr":
        return self.__mul__(other)

    def __truediv__(self, other: Any) -> "LinExpr":
        if isinstance(other, (int, float)):
```

```
            return self.__mul__(1.0 / float(other))
        raise TypeError(" 仅支持线性表达式除以标量。")

    def __neg__(self) -> "LinExpr":
        return _new_expr(
            {i: -v for i, v in self._coeffs.items()},
            -self._constant,
        )

    # -- 比较（生成临时约束，交给 model.addConstr 消费） ---------------------
    def __eq__(self, other: Any) -> "_TempConstr":  # type: ignore[override]
        return _TempConstr(self, _as_lin(other), GRB.EQUAL)

    def __le__(self, other: Any) -> "_TempConstr":
        return _TempConstr(self, _as_lin(other), GRB.LESS_EQUAL)

    def __ge__(self, other: Any) -> "_TempConstr":
        return _TempConstr(self, _as_lin(other), GRB.GREATER_EQUAL)

    def __hash__(self) -> int:
        return id(self)


class Var(LinExpr):
    """ 单个决策变量（本身就是含一项系数的线性表达式）。"""

    def __init__(
        self,
        model: "Model",
        index: int,
        lb: Optional[float],
        ub: Optional[float],
        vtype: str,
        name: str = "",
    ):
        super().__init__(0.0)
        self._coeffs = {index: 1.0}
        self._model = model
        self._index = index
        self._lb = lb
        self._ub = ub
        self._vtype = vtype
        self._name = name
        self.X: float = 0.0

    def __repr__(self) -> str:
        return f"<Var {self._name or self._index}>"


class Constr:
    """ 约束占位对象（本项目仅存储，不读取其属性）。"""

    def __init__(self, constr: "_TempConstr", name: str = ""):
        self._constr = constr
        self.ConstrName = name


class _TempConstr:
    """ 由比较运算临时生成的约束：lhs [sense] rhs。"""

    def __init__(self, lhs: LinExpr, rhs: LinExpr, sense: str):
        self.lhs = _as_lin(lhs)
        self.rhs = _as_lin(rhs)
        self.sense = sense


# ============================================================================
# tupledict
# ============================================================================
```

```
204
205 class tupledict(dict):
206     """gurobipy.tupledict 的极简替代 (本项目中只当普通 dict 使用)。"""
207
208     def sum(self) -> LinExpr:
209         total = LinExpr(0.0)
210         for v in self.values():
211             total = total + v
212         return total
213
214     def prod(self, coeffs: Any) -> LinExpr:
215         total = LinExpr(0.0)
216         for key, v in self.items():
217             total = total + coeffs[key] * v
218         return total
219
220
221 # ============================================================================
222 # quicksum
223 # ============================================================================
224
225 def quicksum(data: Iterable[Any]) -> LinExpr:
226     """ 把可迭代对象中的各项累加为线性表达式。"""
227     total = LinExpr(0.0)
228     for term in data:
229         if isinstance(term, (LinExpr, int, float)):
230             total = total + term
231         elif isinstance(term, tuple) and len(term) == 2:
232             total = total + term[0] * term[1]
233         else:
234             raise TypeError(f"quicksum 无法处理项: {type(term)!r}")
235     return total
236
237
238 # ============================================================================
239 # 模型
240 # ============================================================================
241
242 class _Params:
243     def __init__(self):
244         self.OutputFlag: int = 1
245         self.TimeLimit: Optional[float] = None
246         self.MIPGap: Optional[float] = None
247         self.Threads: int = 0
248
249
250 class Model:
251     """gurobipy.Model 的极简替代, 底层调用 HiGHS。"""
252
253     def __init__(self, name: str = ""):
254         self.ModelName = name
255         self.Params = _Params()
256         self.Status: Optional[int] = None
257         self.ObjVal: Optional[float] = None
258
259         self._vars: list[Var] = []
260         self._constrs: list[_TempConstr] = []
261         self._objective: Optional[LinExpr] = None
262         self._objective_sense: int = GRB.MINIMIZE
263         self._optimized: bool = False
264
265     # -- 变量 ----------------------------------------------------------------
266     def addVar(
267         self,
268         lb: Optional[float] = 0.0,
269         ub: Optional[float] = None,
270         vtype: str = GRB.CONTINUOUS,
271         name: str = "",
272         obj: float = 0.0,
```

```
    ) -> Var:
        if vtype == GRB.BINARY:
            lb = 0.0 if lb is None else lb
            ub = 1.0 if ub is None else ub
        else:
            lb = 0.0 if lb is None else lb
            ub = GRB.INFINITY if ub is None else ub

        idx = len(self._vars)
        v = Var(self, idx, float(lb), float(ub), vtype, name)
        self._vars.append(v)

        if obj != 0.0:
            if self._objective is None:
                self._objective = LinExpr(0.0)
            self._objective = self._objective + obj * v

        self._optimized = False
        return v

    def addVars(
        self,
        *keys: Any,
        lb: Optional[float] = 0.0,
        ub: Optional[float] = None,
        vtype: str = GRB.CONTINUOUS,
        name: str = "",
        obj: Any = 0.0,
    ) -> tupledict:
        result = tupledict()
        if len(keys) == 1:
            for k in keys[0]:
                result[k] = self.addVar(lb=lb, ub=ub, vtype=vtype, name=name)
        else:
            for combo in itertools.product(*keys):
                key = combo[0] if len(combo) == 1 else combo
                result[key] = self.addVar(lb=lb, ub=ub, vtype=vtype, name=name)
        return result

    # -- 约束 ------------------------------------------------------------
    def addConstr(
        self,
        lhs: Any,
        sense: Optional[str] = None,
        rhs: Any = None,
        name: str = "",
    ) -> Constr:
        if isinstance(lhs, _TempConstr):
            c = lhs
        else:
            if sense is None:
                raise TypeError("addConstr 需要比较表达式或显式给出 sense/rhs。")
            c = _TempConstr(_as_lin(lhs), _as_lin(rhs), sense)

        self._constrs.append(c)
        self._optimized = False
        return Constr(c, name)

    # -- 目标 ------------------------------------------------------------
    def setObjective(self, expr: Any, sense: Optional[int] = None) -> None:
        self._objective = _as_lin(expr)
        if sense is not None:
            self._objective_sense = sense
        self._optimized = False

    # -- 求解 ------------------------------------------------------------
    def update(self) -> None:
        return None

```

```
342     def optimize(self) -> None:
343         self._solve_highs()
344         self._optimized = True
345 
346     def _solve_highs(self) -> None:
347         import numpy as np
348         from scipy.optimize import Bounds, LinearConstraint, milp
349         from scipy.sparse import coo_matrix
350 
351         n = len(self._vars)
352         if n == 0:
353             self.Status = GRB.OPTIMAL
354             self.ObjVal = 0.0
355             return
356 
357         # 目标向量
358         c = np.zeros(n, dtype=float)
359         obj_const = 0.0
360         if self._objective is not None:
361             for idx, coeff in self._objective._coeffs.items():
362                 c[idx] = coeff
363             obj_const = self._objective._constant
364         if self._objective_sense == GRB.MAXIMIZE:
365             c = -c
366             obj_const = -obj_const
367 
368         # 变量边界与整数性
369         lb = np.empty(n, dtype=float)
370         ub = np.empty(n, dtype=float)
371         has_integer = any(
372             v._vtype in (GRB.BINARY, GRB.INTEGER) for v in self._vars
373         )
374         integrality = None
375         if has_integer:
376             integrality = np.zeros(n, dtype=np.uint8)
377             for i, v in enumerate(self._vars):
378                 integrality[i] = (
379                     1 if v._vtype in (GRB.BINARY, GRB.INTEGER) else 0
380                 )
381         for i, v in enumerate(self._vars):
382             lb[i] = v._lb if v._lb is not None else -np.inf
383             ub[i] = v._ub if v._ub is not None else np.inf
384 
385         # 约束：统一为 (lhs - rhs) op 0
386         rows: list[int] = []
387         cols: list[int] = []
388         data: list[float] = []
389         c_lb: list[float] = []
390         c_ub: list[float] = []
391 
392         for ci, constr in enumerate(self._constrs):
393             coeffs: dict[int, float] = {}
394             for idx, val in constr.lhs._coeffs.items():
395                 coeffs[idx] = coeffs.get(idx, 0.0) + val
396             for idx, val in constr.rhs._coeffs.items():
397                 coeffs[idx] = coeffs.get(idx, 0.0) - val
398 
399             b = constr.rhs._constant - constr.lhs._constant
400 
401             for idx, val in coeffs.items():
402                 if val != 0.0:
403                     rows.append(ci)
404                     cols.append(idx)
405                     data.append(val)
406 
407             if constr.sense == GRB.LESS_EQUAL:
408                 c_lb.append(-np.inf)
409                 c_ub.append(b)
410             elif constr.sense == GRB.GREATER_EQUAL:
```

```
                c_lb.append(b)
                c_ub.append(np.inf)
            else:  # EQUAL
                c_lb.append(b)
                c_ub.append(b)

        constraints = None
        if self._constrs:
            A = coo_matrix(
                (data, (rows, cols)),
                shape=(len(self._constrs), n),
            ).tocsr()
            constraints = LinearConstraint(
                A,
                np.array(c_lb, dtype=float),
                np.array(c_ub, dtype=float),
            )

        bounds = Bounds(lb, ub)

        options = {"disp": self.Params.OutputFlag > 0}
        if self.Params.TimeLimit is not None:
            options["time_limit"] = float(self.Params.TimeLimit)
        if self.Params.MIPGap is not None:
            options["mip_rel_gap"] = float(self.Params.MIPGap)

        res = milp(
            c,
            integrality=integrality,
            bounds=bounds,
            constraints=constraints,
            options=options,
        )

        if res.status == 0:
            self.Status = GRB.OPTIMAL
        elif res.status == 2:
            self.Status = GRB.INFEASIBLE
        elif res.status == 3:
            self.Status = GRB.UNBOUNDED
        else:
            self.Status = int(res.status)

        if res.status == 0:
            x = res.x
            for i, v in enumerate(self._vars):
                xi = x[i]
                v.X = float(xi) if not np.isnan(xi) else 0.0
            self.ObjVal = float(res.fun) + obj_const
        else:
            for v in self._vars:
                v.X = 0.0
            self.ObjVal = None

    # -- 查询（供调试，本项目未直接使用） ------------------------------------
    @property
    def NumVars(self) -> int:
        return len(self._vars)

    @property
    def NumConstrs(self) -> int:
        return len(self._constrs)

    def getVars(self) -> list[Var]:
        return list(self._vars)
```

文件：code/_excel_backend.py

```
# -*- coding: utf-8 -*-
"""
_excel_backend.py
openpyxl 实现的 artifact_tool 兼容适配层。

仅覆盖 src/result_writer.py 实际用到的 artifact_tool 接口：
    Workbook / SpreadsheetFile / Blob / Range / FreezePanes

目的是在不改动 result_writer.py 业务逻辑的前提下，用 openpyxl 完成
xlsx 的写出与回读校验。
"""

from __future__ import annotations

import json
import re
from typing import Any, Iterator

from openpyxl import Workbook as _PyWorkbook
from openpyxl import load_workbook
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter


_RANGE_RE = re.compile(
    r"^\s*([A-Za-z]+)(\d+)\s*:\s*([A-Za-z]+)(\d+)\s*$"
)


def _col_index(letters: str) -> int:
    n = 0
    for ch in letters.upper():
        n = n * 26 + (ord(ch) - ord("A") + 1)
    return n - 1


def _to_argb(color: Any) -> str | None:
    if color is None:
        return None
    color = str(color).strip()
    if color.startswith("#"):
        color = color[1:]
    if len(color) == 6:
        color = "FF" + color
    return color


class Blob:
    """ 兼容 artifact_tool.Blob 的最小实现。"""

    def __init__(self, path: str):
        self.path = path

    @staticmethod
    def load(path: str) -> "Blob":
        return Blob(path)


class _FormatProxy:
    def __init__(self, rng: "_Range"):
        object.__setattr__(self, "_rng", rng)

    def __setattr__(self, name: str, value: Any) -> None:
        if name == "_rng":
            object.__setattr__(self, name, value)
```

```
            return

        rng = self._rng
        if name == "column_width":
            rng._apply_column_width(value)
        else:
            rng._apply_cell_style(name, value)


class _Range:
    def __init__(
        self,
        ws: Any,
        start_row0: int,
        start_col0: int,
        row_count: int,
        col_count: int,
    ):
        self._ws = ws
        self._start_row0 = start_row0
        self._start_col0 = start_col0
        self._row_count = row_count
        self._col_count = col_count
        self._format_proxy = _FormatProxy(self)

    def _cells(self) -> Iterator[Any]:
        for i in range(self._row_count):
            for j in range(self._col_count):
                yield self._ws.cell(
                    row=self._start_row0 + 1 + i,
                    column=self._start_col0 + 1 + j,
                )

    @property
    def values(self) -> list[list[Any]]:
        out: list[list[Any]] = []
        for i in range(self._row_count):
            row: list[Any] = []
            for j in range(self._col_count):
                row.append(
                    self._ws.cell(
                        row=self._start_row0 + 1 + i,
                        column=self._start_col0 + 1 + j,
                    ).value
                )
            out.append(row)
        return out

    @values.setter
    def values(self, rows: list[list[Any]]) -> None:
        for i, row in enumerate(rows):
            for j, value in enumerate(row):
                self._ws.cell(
                    row=self._start_row0 + 1 + i,
                    column=self._start_col0 + 1 + j,
                    value=value,
                )

    @property
    def format(self) -> _FormatProxy:
        return self._format_proxy

    @format.setter
    def format(self, fmt_dict: dict) -> None:
        for key, value in fmt_dict.items():
            self._apply_cell_style(key, value)

    def _apply_cell_style(self, key: str, value: Any) -> None:
        for cell in self._cells():
```

```
            if key == "fill":
                color = _to_argb(value)
                cell.fill = PatternFill(
                    start_color=color,
                    end_color=color,
                    fill_type="solid",
                )
            elif key == "font":
                cell.font = (
                    Font(**value)
                    if isinstance(value, dict)
                    else value
                )
            elif key in (
                "horizontal_alignment",
                "vertical_alignment",
                "wrap_text",
            ):
                a = cell.alignment
                kwargs: dict[str, Any] = {}
                if a is not None:
                    kwargs["horizontal"] = a.horizontal
                    kwargs["vertical"] = a.vertical
                    kwargs["wrap_text"] = a.wrap_text

                if key == "horizontal_alignment":
                    kwargs["horizontal"] = value
                elif key == "vertical_alignment":
                    kwargs["vertical"] = value
                else:  # wrap_text
                    kwargs["wrap_text"] = value

                cell.alignment = Alignment(**kwargs)
            elif key == "number_format":
                cell.number_format = value
            else:
                raise ValueError(f" 不支持的格式键: {key}")

    def _apply_column_width(self, value: Any) -> None:
        for j in range(self._col_count):
            letter = get_column_letter(self._start_col0 + 1 + j)
            self._ws.column_dimensions[letter].width = value


class _FreezePanes:
    def __init__(self, ws: Any):
        self._ws = ws
        self._rows = 0
        self._cols = 0

    def freeze_rows(self, n: int) -> None:
        self._rows = n
        self._apply()

    def freeze_columns(self, n: int) -> None:
        self._cols = n
        self._apply()

    def _apply(self) -> None:
        if self._rows or self._cols:
            self._ws.freeze_panes = (
                f"{get_column_letter(self._cols + 1)}"
                f"{self._rows + 1}"
            )
        else:
            self._ws.freeze_panes = None


class _Sheet:
```

```
    def __init__(self, ws: Any):
        self._ws = ws
        self.freeze_panes = _FreezePanes(ws)

    def get_range(self, a1: str) -> _Range:
        m = _RANGE_RE.match(a1)
        if not m:
            raise ValueError(f" 非法 A1 范围: {a1}")

        c1, r1, c2, r2 = m.groups()
        start_col0 = _col_index(c1)
        end_col0 = _col_index(c2)
        start_row0 = int(r1) - 1
        end_row0 = int(r2) - 1

        row_count = end_row0 - start_row0 + 1
        col_count = end_col0 - start_col0 + 1

        if row_count <= 0 or col_count <= 0:
            raise ValueError(f" 非法 A1 范围: {a1}")

        return _Range(
            self._ws,
            start_row0,
            start_col0,
            row_count,
            col_count,
        )


class _WorksheetCollection:
    def __init__(self, wb: Any):
        self._wb = wb

    def add(self, name: str) -> _Sheet:
        if name in self._wb.sheetnames:
            ws = self._wb[name]
        else:
            ws = self._wb.create_sheet(title=name)
        return _Sheet(ws)

    def get_item(self, name: str) -> _Sheet:
        return _Sheet(self._wb[name])


class _InspectResult:
    def __init__(self, ndjson: str):
        self.ndjson = ndjson


class Workbook:
    def __init__(self, wb: Any = None):
        self._wb = wb if wb is not None else _PyWorkbook()
        self.worksheets = _WorksheetCollection(self._wb)

    @staticmethod
    def create() -> "Workbook":
        wb = _PyWorkbook()
        default = wb.active
        if default is not None:
            wb.remove(default)
        return Workbook(wb)

    def inspect(self, spec: dict) -> _InspectResult:
        names = [ws.title for ws in self._wb.worksheets]
        lines = [
            json.dumps({"id": i, "name": name}, ensure_ascii=False)
            for i, name in enumerate(names)
        ]
```

```
        return _InspectResult("\n".join(lines))


class _ExportResult:
    def __init__(self, workbook: "Workbook"):
        self._workbook = workbook

    def save(self, path: str) -> None:
        self._workbook._wb.save(path)


class SpreadsheetFile:
    @staticmethod
    def export_xlsx(workbook: "Workbook") -> _ExportResult:
        return _ExportResult(workbook)

    @staticmethod
    def import_xlsx(blob: Blob) -> "Workbook":
        wb = load_workbook(blob.path)
        return Workbook(wb)
```

文件：code/visualize.py

```python
# -*- coding: utf-8 -*-
"""
visualize.py
阶段 24：可视化模块

设计目标
--------
1. 严格复用已经确定的变量命名、时间索引和单位；
2. 图题、坐标轴和图例统一使用英文，避免中文字体依赖；
3. 每张图单独成图，不使用 seaborn，不使用 subplot；
4. 所有时间轴均基于 model k=0,...,143，不允许用 Excel 表头字符串反推；
5. 默认输出到 output/figures 目录；
6. 量纲明确：负荷/光伏可显示为 kW，购电/充放电可显示为 kWh per interval，
   电价为 CNY/kWh，储电量为 kWh。

推荐正文重点图
--------------
Figure 1: Typical day profiles (load, PV, price)
Figure 2: Q1 operation (purchase, charge, discharge, energy)
Figure 3: Q2 forecast vs actual
Figure 4: Emergency purchase time distribution
Figure 5: Q3 PV forecast updates approaching actual PV
Figure 6: Total cost comparison among update strategies
Figure 7: Q4 storage response under dynamic price
Figure 8: Sensitivity analysis
"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any, Dict, Iterable, Mapping, Optional, Sequence

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from config import DEFAULT_CONFIG, ProjectConfig
from time_index import build_time_map, validate_time_map


# ============================================================================
# 路径与公共工具
# ============================================================================


@dataclass
class VisualizationPaths:
    project_root: Path
    output_root: Path
    figure_root: Path
    table_root: Path
    result_root: Path
    log_root: Path


def ensure_project_output_dirs(
    project_root: str | Path = ".",
) -> VisualizationPaths:
    """
    创建并返回标准输出目录。

    目录结构：
        project_root/
            output/
```

```
                figures/
                tables/
                results/
                logs/
    """
    project_root = Path(project_root)
    output_root = project_root / "output"
    figure_root = output_root / "figures"
    table_root = output_root / "tables"
    result_root = output_root / "results"
    log_root = output_root / "logs"

    for p in [
        output_root,
        figure_root,
        table_root,
        result_root,
        log_root,
    ]:
        p.mkdir(parents=True, exist_ok=True)

    return VisualizationPaths(
        project_root=project_root,
        output_root=output_root,
        figure_root=figure_root,
        table_root=table_root,
        result_root=result_root,
        log_root=log_root,
    )


def _ordered_time_map(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.DataFrame:
    tm = build_time_map(config)
    validate_time_map(tm, config)
    tm = tm.sort_values("k").reset_index(drop=True)
    return tm


def _midpoint_hours(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> np.ndarray:
    """
    144 个物理 10 分钟区间的中点小时坐标:
        x_k = (k + 0.5) * 10 / 60
    """
    return (
        np.arange(config.intervals_per_day, dtype=float) + 0.5
    ) * (config.dt_minutes / 60.0)


def _boundary_hours(
    config: ProjectConfig = DEFAULT_CONFIG,
) -> np.ndarray:
    """
    145 个状态边界的小时坐标:
        x_j = j * 10 / 60
    """
    return np.arange(
        config.state_nodes_per_day, dtype=float
    ) * (config.dt_minutes / 60.0)


def _series_144(
    series: pd.Series,
    *,
    name: str,
    config: ProjectConfig,
```

```
) -> pd.Series:
    if not isinstance(series, pd.Series):
        raise TypeError(f"{name} 必须为 pandas.Series。")
    expected = list(range(config.intervals_per_day))
    if list(series.index) != expected:
        raise ValueError(
            f"{name}.index 必须严格为 model k=0~143。"
        )
    values = pd.to_numeric(series, errors="raise").astype(float)
    if values.isna().any():
        raise ValueError(f"{name} 存在缺失值。")
    return values


def _series_145(
    series: pd.Series,
    *,
    name: str,
    config: ProjectConfig,
) -> pd.Series:
    if not isinstance(series, pd.Series):
        raise TypeError(f"{name} 必须为 pandas.Series。")
    expected = list(range(config.state_nodes_per_day))
    if list(series.index) != expected:
        raise ValueError(
            f"{name}.index 必须严格为 state j=0~144。"
        )
    values = pd.to_numeric(series, errors="raise").astype(float)
    if values.isna().any():
        raise ValueError(f"{name} 存在缺失值。")
    return values


def interval_kwh_to_kw(
    series_kwh: pd.Series,
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Series:
    """
    10 分钟区间电量 (kWh) -> 区间平均功率 (kW)
    """
    s = _series_144(
        series_kwh,
        name="series_kwh",
        config=config,
    )
    return s / (config.dt_minutes / 60.0)


def get_actual_day_load_kwh(
    attachment2,
    date,
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Series:
    d = pd.Timestamp(date).normalize()
    s = (
        attachment2.load_kw.loc[d]
        .astype(float)
        .copy()
        * (config.dt_minutes / 60.0)
    )
    s.index = list(range(config.intervals_per_day))
    return _series_144(s, name="actual_load_kwh", config=config)


def get_actual_day_pv_kwh(
    attachment2,
    date,
```

```
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Series:
    d = pd.Timestamp(date).normalize()
    s = (
        attachment2.pv_kw.loc[d]
        .astype(float)
        .copy()
        * (config.dt_minutes / 60.0)
    )
    s.index = list(range(config.intervals_per_day))
    return _series_144(s, name="actual_pv_kwh", config=config)


def get_actual_day_price(
    attachment4,
    date,
    *,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Series:
    d = pd.Timestamp(date).normalize()
    s = attachment4.price.loc[d].astype(float).copy()
    s.index = list(range(config.intervals_per_day))
    return _series_144(s, name="actual_price", config=config)


def _save_figure(
    fig,
    output_path: str | Path | None,
) -> Optional[Path]:
    if output_path is None:
        return None
    output_path = Path(output_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(
        output_path,
        dpi=300,
        bbox_inches="tight",
    )
    return output_path


# ============================================================================
# Figure 1: 典型日负荷-光伏-电价
# ============================================================================


def plot_figure1_typical_day_profiles(
    *,
    date,
    load_kwh: pd.Series,
    pv_kwh: pd.Series,
    price_yuan_per_kwh: pd.Series,
    output_path: str | Path | None = None,
    config: ProjectConfig = DEFAULT_CONFIG,
):
    """
    Figure 1
    Typical day:
        load, PV, and electricity price.

    负荷/光伏用区间平均功率 (kW);
    电价用 CNY/kWh, 采用右轴。
    """
    d = pd.Timestamp(date).normalize()
    load_kwh = _series_144(load_kwh, name="load_kwh", config=config)
    pv_kwh = _series_144(pv_kwh, name="pv_kwh", config=config)
    price = _series_144(
        price_yuan_per_kwh,
```

```
        name="price_yuan_per_kwh",
        config=config,
    )

    x = _midpoint_hours(config)
    load_kw = interval_kwh_to_kw(load_kwh, config=config)
    pv_kw = interval_kwh_to_kw(pv_kwh, config=config)

    fig, ax1 = plt.subplots(figsize=(10, 5))
    ax1.plot(x, load_kw.to_numpy(), label="Load power")
    ax1.plot(x, pv_kw.to_numpy(), label="PV power")
    ax1.set_xlabel("Time of day (hour)")
    ax1.set_ylabel("Power (kW)")
    ax1.set_title(
        f"Figure 1. Typical Day Profiles - {d.date()}"
    )
    ax1.set_xlim(0, 24)
    ax1.grid(True, alpha=0.25)

    ax2 = ax1.twinx()
    ax2.plot(
        x,
        price.to_numpy(),
        label="Electricity price",
    )
    ax2.set_ylabel("Price (CNY/kWh)")

    handles1, labels1 = ax1.get_legend_handles_labels()
    handles2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(
        handles1 + handles2,
        labels1 + labels2,
        loc="upper right",
    )

    fig.tight_layout()
    _save_figure(fig, output_path)
    return fig, ax1, ax2


# ============================================================================
# Figure 2: 问题 1 购电-充放电-SOC
# ============================================================================


def plot_figure2_q1_operation(
    result1,
    *,
    output_path: str | Path | None = None,
    config: ProjectConfig = DEFAULT_CONFIG,
):
    """
    Figure 2
    Q1 operation:
        purchase, charge, discharge, and battery energy.
    """
    purchase = _series_144(
        result1.plan_purchase_kwh,
        name="result1.plan_purchase_kwh",
        config=config,
    )
    charge = _series_144(
        result1.charge_kwh,
        name="result1.charge_kwh",
        config=config,
    )
    discharge = _series_144(
        result1.discharge_kwh,
        name="result1.discharge_kwh",
```

```
        config=config,
    )
    energy = _series_145(
        result1.energy_kwh,
        name="result1.energy_kwh",
        config=config,
    )

    x = _midpoint_hours(config)
    x_state = _boundary_hours(config)

    fig, ax1 = plt.subplots(figsize=(10, 5))
    ax1.plot(x, purchase.to_numpy(), label="Grid purchase")
    ax1.plot(x, charge.to_numpy(), label="Charge")
    ax1.plot(x, discharge.to_numpy(), label="Discharge")
    ax1.set_xlabel("Time of day (hour)")
    ax1.set_ylabel("Energy per 10-min interval (kWh)")
    ax1.set_title("Figure 2. Q1 Purchase-Charge-Discharge Schedule")
    ax1.set_xlim(0, 24)
    ax1.grid(True, alpha=0.25)

    ax2 = ax1.twinx()
    ax2.plot(x_state, energy.to_numpy(), label="Battery energy")
    ax2.set_ylabel("Battery energy (kWh)")

    handles1, labels1 = ax1.get_legend_handles_labels()
    handles2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(
        handles1 + handles2,
        labels1 + labels2,
        loc="upper right",
    )

    fig.tight_layout()
    _save_figure(fig, output_path)
    return fig, ax1, ax2


# ============================================================================
# Figure 3: 问题 2 预测 vs 实际
# ============================================================================


def plot_figure3_q2_forecast_vs_actual(
    q2_day_result,
    *,
    actual_load_kwh: pd.Series,
    actual_pv_kwh: pd.Series,
    output_path: str | Path | None = None,
    config: ProjectConfig = DEFAULT_CONFIG,
):
    """
    Figure 3
    Q2:
        day-ahead forecast vs actual values.

    同一张图中绘制:
        load forecast / actual
        PV forecast / actual
    单位统一为每 10 分钟电量 (kWh)。
    """
    load_fc = _series_144(
        q2_day_result.load_forecast_kwh,
        name="q2_day_result.load_forecast_kwh",
        config=config,
    )
    pv_fc = _series_144(
        q2_day_result.pv_forecast_kwh,
        name="q2_day_result.pv_forecast_kwh",
```

```
        config=config,
    )
    actual_load_kwh = _series_144(
        actual_load_kwh,
        name="actual_load_kwh",
        config=config,
    )
    actual_pv_kwh = _series_144(
        actual_pv_kwh,
        name="actual_pv_kwh",
        config=config,
    )

    x = _midpoint_hours(config)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.plot(x, load_fc.to_numpy(), label="Load forecast")
    ax.plot(x, actual_load_kwh.to_numpy(), label="Load actual")
    ax.plot(x, pv_fc.to_numpy(), label="PV forecast")
    ax.plot(x, actual_pv_kwh.to_numpy(), label="PV actual")

    ax.set_xlabel("Time of day (hour)")
    ax.set_ylabel("Energy per 10-min interval (kWh)")
    ax.set_title(
        f"Figure 3. Q2 Forecast vs Actual - "
        f"{pd.Timestamp(q2_day_result.date).date()}"
    )
    ax.set_xlim(0, 24)
    ax.grid(True, alpha=0.25)
    ax.legend(loc="upper right")

    fig.tight_layout()
    _save_figure(fig, output_path)
    return fig, ax


# ============================================================================
# Figure 4: 紧急购电时间分布
# ============================================================================


def plot_figure4_emergency_distribution(
    period_result,
    *,
    output_path: str | Path | None = None,
    config: ProjectConfig = DEFAULT_CONFIG,
):
    """
    Figure 4
    Emergency purchase time distribution.

    统计所有正式日期 (2025-02-01~2025-12-31) 在每个 model k 上的
    紧急购电量总和, 展示其日内时段分布。
    """
    if not hasattr(period_result, "formal_results"):
        raise TypeError("period_result 必须具有 formal_results。")

    total = pd.Series(
        np.zeros(config.intervals_per_day, dtype=float),
        index=list(range(config.intervals_per_day)),
        dtype=float,
    )

    for _, day_result in period_result.formal_results.items():
        base = getattr(day_result, "base_result", day_result)
        s = _series_144(
            base.actual_emergency_kwh,
            name="actual_emergency_kwh",
            config=config,
```

```
        )
        total = total + s

    x = _midpoint_hours(config)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.bar(x, total.to_numpy(), width=(config.dt_minutes / 60.0) * 0.9)
    ax.set_xlabel("Time of day (hour)")
    ax.set_ylabel("Total emergency purchase (kWh)")
    ax.set_title("Figure 4. Emergency Purchase Time Distribution")
    ax.set_xlim(0, 24)
    ax.grid(True, axis="y", alpha=0.25)

    fig.tight_layout()
    _save_figure(fig, output_path)
    return fig, ax


# ============================================================================
# Figure 5: 问题 3 0/6/12/18 预测逐步逼近实际
# ============================================================================


def _extract_q3_parser_forecast_kw(
    update_record,
    *,
    target_date,
    config: ProjectConfig = DEFAULT_CONFIG,
) -> pd.Series:
    """
    从 MPCUpdateRecord.parser_audit 重建当前自然日未来 10 分钟 PV 预测平均功率 (kW)。

    当前 question3_solver 保存的 parser_audit 来自附件 3 整点锚点:
        lead_hour
        target_datetime
        forecast_kw
        source

    因此这里不把整点锚点直接当作 10 分钟区间值, 而是与正式模型保持一致:

    1. 以整点锚点做线性插值;
    2. 对每个 10 分钟物理区间取两个边界预测功率的平均值;
    3. 只返回该发布时刻以后、仍属于 target_date 的 model k。

    该函数只服务可视化, 不改变任何优化结果。
    """
    if not hasattr(update_record, "parser_audit"):
        raise AttributeError("update_record 缺少 parser_audit。")

    audit = update_record.parser_audit.copy()
    required = {"target_datetime", "forecast_kw"}

    if audit.empty:
        return pd.Series(dtype=float)

    if not required.issubset(audit.columns):
        raise ValueError(
            "parser_audit 必须至少包含 target_datetime 和 forecast_kw。"
        )

    audit["target_datetime"] = pd.to_datetime(
        audit["target_datetime"]
    )
    audit["forecast_kw"] = pd.to_numeric(
        audit["forecast_kw"],
        errors="raise",
    ).astype(float)

    audit = audit.sort_values("target_datetime")
```

```
    anchor_t = pd.DatetimeIndex(
        audit["target_datetime"]
    )
    anchor_y = audit["forecast_kw"].to_numpy(dtype=float)

    if len(anchor_t) < 2:
        raise ValueError(" 附件 3 可视化至少需要两个预测锚点。")

    d = pd.Timestamp(target_date).normalize()
    issue = pd.Timestamp(update_record.issue_datetime)

    start_k = int(
        (
            issue - d
        ).total_seconds()
        // (config.dt_minutes * 60)
    )
    start_k = max(0, min(config.intervals_per_day, start_k))

    anchor_ns = anchor_t.asi8.astype(float)

    result = {}

    for k in range(start_k, config.intervals_per_day):
        t0 = d + pd.Timedelta(
            minutes=k * config.dt_minutes
        )
        t1 = t0 + pd.Timedelta(
            minutes=config.dt_minutes
        )

        # 发布版本必须覆盖该区间两个边界。
        if t0 < anchor_t[0] or t1 > anchor_t[-1]:
            continue

        b = np.array(
            [
                pd.Timestamp(t0).value,
                pd.Timestamp(t1).value,
            ],
            dtype=float,
        )

        p0, p1 = np.interp(
            b,
            anchor_ns,
            anchor_y,
        )

        result[k] = 0.5 * (
            float(p0) + float(p1)
        )

    return pd.Series(
        result,
        dtype=float,
        name="pv_forecast_kw",
    ).sort_index()


def plot_figure5_q3_pv_update_progression(
    q3_day_result,
    *,
    actual_pv_kwh: pd.Series,
    output_path: str | Path | None = None,
    config: ProjectConfig = DEFAULT_CONFIG,
):
    """
    Figure 5
```

```
    Q3:
        PV forecasts issued at 0:00, 6:00, 12:00, 18:00
        progressively approach actual PV.

    实际值使用 actual_pv_kwh，并转换为 kW 显示。
    预测曲线从对应更新时间 r 开始绘制；r 之前置为 NaN。
    """
    actual_pv_kwh = _series_144(
        actual_pv_kwh,
        name="actual_pv_kwh",
        config=config,
    )
    actual_pv_kw = interval_kwh_to_kw(
        actual_pv_kwh,
        config=config,
    )

    x = _midpoint_hours(config)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.plot(x, actual_pv_kw.to_numpy(), label="Actual PV")

    update_records = getattr(
        q3_day_result,
        "update_records",
        {},
    )
    if not isinstance(update_records, Mapping):
        raise TypeError("q3_day_result.update_records 必须为映射。")

    for r in sorted(update_records):
        record = update_records[r]
        forecast_kw = _extract_q3_parser_forecast_kw(
            record,
            target_date=q3_day_result.date,
            config=config,
        )

        values = np.full(
            config.intervals_per_day,
            np.nan,
            dtype=float,
        )
        for k, v in forecast_kw.items():
            if 0 <= int(k) <= 143:
                values[int(k)] = float(v)

        label = f"Forecast issued at {int(r/6):02d}:00"
        ax.plot(x, values, label=label)

    ax.set_xlabel("Time of day (hour)")
    ax.set_ylabel("PV power (kW)")
    ax.set_title(
        f"Figure 5. Q3 PV Forecast Update Progression - "
        f"{pd.Timestamp(q3_day_result.date).date()}"
    )
    ax.set_xlim(0, 24)
    ax.grid(True, alpha=0.25)
    ax.legend(loc="upper right")

    fig.tight_layout()
    _save_figure(fig, output_path)
    return fig, ax


# ============================================================================
# Figure 6：不同更新策略总成本比较
# ============================================================================

```

```
687
688 def plot_figure6_strategy_cost_comparison(
689     strategy_cost,
690     *,
691     strategy_col: str = "strategy",
692     cost_col: str = "total_cost_yuan",
693     output_path: str | Path | None = None,
694 ):
695     """
696     Figure 6
697     Total cost comparison under different update strategies.
698
699     输入可以是:
700     1. DataFrame, 包含 strategy_col 和 cost_col;
701     2. dict[str, float]
702     """
703     if isinstance(strategy_cost, dict):
704         df = pd.DataFrame(
705             {
706                 strategy_col: list(strategy_cost.keys()),
707                 cost_col: list(strategy_cost.values()),
708             }
709         )
710     else:
711         df = pd.DataFrame(strategy_cost).copy()
712
713     if strategy_col not in df.columns or cost_col not in df.columns:
714         raise ValueError(
715             f" 输入必须包含列 {strategy_col} 和 {cost_col}。"
716         )
717
718     labels = df[strategy_col].astype(str).tolist()
719     values = pd.to_numeric(
720         df[cost_col],
721         errors="raise",
722     ).astype(float)
723
724     x = np.arange(len(df), dtype=float)
725
726     fig, ax = plt.subplots(figsize=(9, 5))
727     ax.bar(x, values.to_numpy())
728     ax.set_xticks(x)
729     ax.set_xticklabels(labels)
730     ax.set_xlabel("Update strategy")
731     ax.set_ylabel("Total cost (CNY)")
732     ax.set_title(
733         "Figure 6. Total Cost Comparison among Update Strategies"
734     )
735     ax.grid(True, axis="y", alpha=0.25)
736
737     fig.tight_layout()
738     _save_figure(fig, output_path)
739     return fig, ax
740
741
742 # ============================================================================
743 # Figure 7: 问题 4 动态电价下低价充高价放
744 # ============================================================================
745
746
747 def plot_figure7_q4_dynamic_price_response(
748     q4_day_result,
749     *,
750     output_path: str | Path | None = None,
751     config: ProjectConfig = DEFAULT_CONFIG,
752 ):
753     """
754     Figure 7
755     Q4:
```

```
        storage response under dynamic electricity price.

    左轴:
        actual charge / actual discharge (kWh per interval)
    右轴:
        actual real-time electricity price (CNY/kWh)
    """
    base = getattr(q4_day_result, "base_result", q4_day_result)

    charge = _series_144(
        base.actual_charge_kwh,
        name="actual_charge_kwh",
        config=config,
    )
    discharge = _series_144(
        base.actual_discharge_kwh,
        name="actual_discharge_kwh",
        config=config,
    )

    if not hasattr(q4_day_result, "actual_price_yuan_per_kwh"):
        raise AttributeError(
            "q4_day_result 必须包含 actual_price_yuan_per_kwh。"
        )

    price = _series_144(
        q4_day_result.actual_price_yuan_per_kwh,
        name="actual_price_yuan_per_kwh",
        config=config,
    )

    x = _midpoint_hours(config)

    fig, ax1 = plt.subplots(figsize=(10, 5))
    ax1.plot(x, charge.to_numpy(), label="Charge")
    ax1.plot(x, discharge.to_numpy(), label="Discharge")
    ax1.set_xlabel("Time of day (hour)")
    ax1.set_ylabel("Energy per 10-min interval (kWh)")
    ax1.set_title(
        f"Figure 7. Q4 Storage Response under Dynamic Price - "
        f"{pd.Timestamp(q4_day_result.date).date()}"
    )
    ax1.set_xlim(0, 24)
    ax1.grid(True, alpha=0.25)

    ax2 = ax1.twinx()
    ax2.plot(
        x,
        price.to_numpy(),
        label="Real-time price",
    )
    ax2.set_ylabel("Price (CNY/kWh)")

    handles1, labels1 = ax1.get_legend_handles_labels()
    handles2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(
        handles1 + handles2,
        labels1 + labels2,
        loc="upper right",
    )

    fig.tight_layout()
    _save_figure(fig, output_path)
    return fig, ax1, ax2


# ============================================================================
# Figure 8: 敏感性分析
# ============================================================================
```

```
825
826
827 def plot_figure8_sensitivity_analysis(
828     x_values,
829     y_values,
830     *,
831     xlabel: str,
832     ylabel: str,
833     title: str = "Figure 8. Sensitivity Analysis",
834     output_path: str | Path | None = None,
835 ):
836     """
837     Figure 8
838     Generic one-factor sensitivity analysis.
839     """
840     x = np.asarray(x_values, dtype=float)
841     y = np.asarray(y_values, dtype=float)
842
843     if x.ndim != 1 or y.ndim != 1 or len(x) != len(y):
844         raise ValueError("x_values 与 y_values 必须为等长一维数组。")
845
846     fig, ax = plt.subplots(figsize=(9, 5))
847     ax.plot(x, y, marker="o")
848     ax.set_xlabel(xlabel)
849     ax.set_ylabel(ylabel)
850     ax.set_title(title)
851     ax.grid(True, alpha=0.25)
852
853     fig.tight_layout()
854     _save_figure(fig, output_path)
855     return fig, ax
856
857
858 # ============================================================================
859 # 批量生成包装器
860 # ============================================================================
861
862
863 def generate_phase24_figures(
864     *,
865     project_root: str | Path = ".",
866     typical_day_date=None,
867     typical_day_load_kwh: Optional[pd.Series] = None,
868     typical_day_pv_kwh: Optional[pd.Series] = None,
869     typical_day_price_yuan_per_kwh: Optional[pd.Series] = None,
870     q1_result=None,
871     q2_day_result=None,
872     q2_actual_load_kwh: Optional[pd.Series] = None,
873     q2_actual_pv_kwh: Optional[pd.Series] = None,
874     q2_or_q3_or_q4_period_result=None,
875     q3_day_result=None,
876     q3_actual_pv_kwh: Optional[pd.Series] = None,
877     strategy_cost=None,
878     q4_day_result=None,
879     sensitivity_x=None,
880     sensitivity_y=None,
881     sensitivity_xlabel: str = "Parameter",
882     sensitivity_ylabel: str = "Response",
883     sensitivity_title: str = "Figure 8. Sensitivity Analysis",
884     config: ProjectConfig = DEFAULT_CONFIG,
885 ) -> Dict[str, Path]:
886     """
887     按可用输入批量生成阶段 24 正文重点图。
888
889     返回:
890         {figure_name: saved_path}
891     """
892     paths = ensure_project_output_dirs(project_root)
893     saved: Dict[str, Path] = {}
```

```
894
895     if (
896         typical_day_date is not None
897         and typical_day_load_kwh is not None
898         and typical_day_pv_kwh is not None
899         and typical_day_price_yuan_per_kwh is not None
900     ):
901         p = paths.figure_root / "fig1_typical_day_profiles.png"
902         plot_figure1_typical_day_profiles(
903             date=typical_day_date,
904             load_kwh=typical_day_load_kwh,
905             pv_kwh=typical_day_pv_kwh,
906             price_yuan_per_kwh=typical_day_price_yuan_per_kwh,
907             output_path=p,
908             config=config,
909         )
910         saved["figure1"] = p
911
912     if q1_result is not None:
913         p = paths.figure_root / "fig2_q1_operation.png"
914         plot_figure2_q1_operation(
915             q1_result,
916             output_path=p,
917             config=config,
918         )
919         saved["figure2"] = p
920
921     if (
922         q2_day_result is not None
923         and q2_actual_load_kwh is not None
924         and q2_actual_pv_kwh is not None
925     ):
926         p = paths.figure_root / "fig3_q2_forecast_vs_actual.png"
927         plot_figure3_q2_forecast_vs_actual(
928             q2_day_result,
929             actual_load_kwh=q2_actual_load_kwh,
930             actual_pv_kwh=q2_actual_pv_kwh,
931             output_path=p,
932             config=config,
933         )
934         saved["figure3"] = p
935
936     if q2_or_q3_or_q4_period_result is not None:
937         p = paths.figure_root / "fig4_emergency_distribution.png"
938         plot_figure4_emergency_distribution(
939             q2_or_q3_or_q4_period_result,
940             output_path=p,
941             config=config,
942         )
943         saved["figure4"] = p
944
945     if q3_day_result is not None and q3_actual_pv_kwh is not None:
946         p = paths.figure_root / "fig5_q3_pv_update_progression.png"
947         plot_figure5_q3_pv_update_progression(
948             q3_day_result,
949             actual_pv_kwh=q3_actual_pv_kwh,
950             output_path=p,
951             config=config,
952         )
953         saved["figure5"] = p
954
955     if strategy_cost is not None:
956         p = paths.figure_root / "fig6_strategy_cost_comparison.png"
957         plot_figure6_strategy_cost_comparison(
958             strategy_cost,
959             output_path=p,
960         )
961         saved["figure6"] = p
962
```

```
    if q4_day_result is not None:
        p = paths.figure_root / "fig7_q4_dynamic_price_response.png"
        plot_figure7_q4_dynamic_price_response(
            q4_day_result,
            output_path=p,
            config=config,
        )
        saved["figure7"] = p

    if sensitivity_x is not None and sensitivity_y is not None:
        p = paths.figure_root / "fig8_sensitivity_analysis.png"
        plot_figure8_sensitivity_analysis(
            sensitivity_x,
            sensitivity_y,
            xlabel=sensitivity_xlabel,
            ylabel=sensitivity_ylabel,
            title=sensitivity_title,
            output_path=p,
        )
        saved["figure8"] = p

    return saved
```

文件：code/make_figures.py

```
# -*- coding: utf-8 -*-
from pathlib import Path
import re
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.dates import MonthLocator, DateFormatter
from matplotlib.ticker import MaxNLocator

ROOT = Path(__file__).parent
DATA = ROOT / 'data'
RES = ROOT / 'results'
OUT = ROOT / 'figures'
OUT.mkdir(exist_ok=True)

# 与论文宋体体系一致：优先使用系统宋体；服务器无 SimSun 时退回 FandolSong/Noto Serif CJK。
def choose_song_font():
    candidates = ['SimSun', '宋体', 'Songti SC', 'STSong', 'FandolSong', 'Noto Serif CJK SC']
    for name in candidates:
        try:
            font_manager.findfont(name, fallback_to_default=False)
            return name
        except Exception:
            pass
    local_candidates = [
        ROOT / 'fonts' / 'simsun.ttc',
        Path('/usr/share/texlive/texmf-dist/fonts/opentype/public/fandol/FandolSong-Regular.ot
        ↪ f'),
    ]
    for fp in local_candidates:
        if fp.exists():
            font_manager.fontManager.addfont(str(fp))
            return font_manager.FontProperties(fname=str(fp)).get_name()
    return 'serif'

FONT = choose_song_font()

mpl.rcParams.update({
    'font.family': FONT,
    'axes.unicode_minus': False,
    'font.size': 8.5,
    'axes.labelsize': 8.5,
    'axes.titlesize': 9.5,
    'legend.fontsize': 7.5,
    'xtick.labelsize': 7.5,
    'ytick.labelsize': 7.5,
    'axes.linewidth': 0.75,
    'lines.linewidth': 1.35,
    'figure.facecolor': 'white',
    'axes.facecolor': 'white',
    'savefig.facecolor': 'white',
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
    'svg.fonttype': 'none',
})

# 用户推荐配色表中的主力色
C = {
    'deep': '#3F3770',
    'blue': '#036EAD',
    'midblue': '#3573A1',
    'lightblue': '#A3D8F4',
    'orange': '#F3962F',
```

```
    'coral': '#F57065',
    'rose': '#D6456C',
    'slate': '#8798C9',
    'gray': '#666666',
    'lightgray': '#D9D9D9',
    'green': '#4E8B57',
}

def style(ax, grid=True):
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.tick_params(direction='out', length=3, width=.7)
    if grid:
        ax.grid(axis='y', color=C['lightgray'], lw=.5, alpha=.55, zorder=0)

def panel(ax, label):
    ax.text(-0.10, 1.04, label, transform=ax.transAxes, fontweight='bold', fontsize=10,
    ↪  va='bottom')

def save(fig, name):
    fig.savefig(OUT/f'{name}.pdf', bbox_inches='tight', pad_inches=.03)
    fig.savefig(OUT/f'{name}.png', dpi=450, bbox_inches='tight', pad_inches=.03)
    fig.savefig(OUT/f'{name}.svg', bbox_inches='tight', pad_inches=.03)
    plt.close(fig)

def time_x(): return (np.arange(144)+0.5)/6

def read_wide(path, sheet, date):
    df=pd.read_excel(path,sheet_name=sheet)
    ds=pd.to_datetime(df.iloc[:,0],errors='coerce').dt.normalize()
    target=pd.Timestamp(date).normalize()
    row=df.loc[ds==target].iloc[0,1:145]
    return pd.to_numeric(row,errors='coerce').to_numpy(float)

def read_result_matrix(path, sheet):
    df=pd.read_excel(path,sheet_name=sheet)
    dates=pd.to_datetime(df.iloc[:,0],errors='coerce').dt.normalize()
    valid=dates.notna()
    mat=df.loc[valid,df.columns[1:145]].apply(pd.to_numeric,errors='coerce').to_numpy(float)
    return pd.DatetimeIndex(dates[valid]),mat,df.loc[valid].reset_index(drop=True)

def read_result_day(path, sheet, date):
    dates,mat,_=read_result_matrix(path,sheet)
    idx=np.where(dates==pd.Timestamp(date).normalize())[0]
    if len(idx)!=1: raise ValueError(date)
    return mat[idx[0]]

def daily_emergency(path):
    # 完整日期由计划购电量表给出；无紧急购电的日期补 0
    all_dates=pd.to_datetime(pd.read_excel(path,sheet_name='计划购电量').iloc[:,0],errors='coe」
    ↪  rce').dropna().dt.normalize()
    em=pd.read_excel(path,sheet_name='紧急购电量').iloc[:,:3].copy()
    em.columns=['日期','时段','购电量']
    em['日期']=pd.to_datetime(em['日期'].ffill(),errors='coerce').dt.normalize()
    em['购电量']=pd.to_numeric(em['购电量'],errors='coerce')
    em=em.dropna(subset=['日期','购电量'])
    g=em.groupby('日期').agg(紧急购电量 =('购电量','sum'), 区间块数 =('购电量','size'))
    g=g.reindex(pd.DatetimeIndex(all_dates),fill_value=0)
    g.index.name='日期'
    return g

# ---------- 图：基础数据特征 ----------
att1=pd.read_excel(DATA/'附件 1.xlsx')
price=pd.to_numeric(att1.iloc[:144,1],errors='coerce').to_numpy(float)
load_kw=pd.to_numeric(att1.iloc[:144,2],errors='coerce').to_numpy(float)
pv_kw=pd.to_numeric(att1.iloc[:144,3],errors='coerce').to_numpy(float)
x=time_x()
fig,axs=plt.subplots(1,3,figsize=(7.1,2.55),gridspec_kw={'wspace':.30})
```

```
for a in axs: style(a); a.set_xlim(0,24); a.set_xticks([0,6,12,18,24]); a.set_xlabel('时刻 / h')
axs[0].plot(x,load_kw,color=C['blue']); axs[0].set_ylabel('功率 / kW'); axs[0].set_title('负荷'); panel(axs[0],'(a)')
axs[1].plot(x,pv_kw,color=C['orange']); axs[1].set_ylabel('功率 / kW'); axs[1].set_title('光伏预测'); panel(axs[1],'(b)')
axs[2].step(x,price,where='mid',color=C['deep'],ls='--'); axs[2].set_ylabel('电价 / (元 · kWh$^{-1}$)'); axs[2].set_title('分时电价'); panel(axs[2],'(c)')
save(fig,'data_profiles')

# ---------- Q1 ----------
q1=pd.read_excel(RES/'result1.xlsx',sheet_name='计划购电量')
qbuy=pd.to_numeric(q1.iloc[:144,1],errors='coerce').to_numpy(float)
load_kwh=load_kw/6; pv_kwh=pv_kw/6
surplus=qbuy+pv_kwh-load_kwh
qch=np.clip(surplus,0,None); qdis=np.clip(-surplus,0,None)
soc=np.empty(145); soc[0]=6000
for k in range(144): soc[k+1]=soc[k]+.9*qch[k]-qdis[k]/.9
fig,axs=plt.subplots(2,1,figsize=(7.1,4.6),gridspec_kw={'hspace':.33})
style(axs[0]); panel(axs[0],'(a)')
axs[0].bar(x,qch,width=.145,color=C['lightblue'],edgecolor='none',label='充电量',zorder=2)
axs[0].bar(x,-qdis,width=.145,color=C['coral'],edgecolor='none',label='放电量',zorder=2)
axs[0].plot(x,qbuy,color=C['blue'],lw=1.25,label='正常购电量',zorder=3)
axs[0].axhline(0,color='#444',lw=.6); axs[0].set_xlim(0,24); axs[0].set_xticks([0,4,8,12,16,20,24]); axs[0].set_ylabel('区间电量 / kWh'); axs[0].set_xlabel('时刻 / h')
axs[0].legend(ncol=3,frameon=False,loc='upper center')
style(axs[1]); panel(axs[1],'(b)')
xb=np.arange(145)/6
axs[1].fill_between(xb,1200,10800,color=C['lightblue'],alpha=.18,label='允许区间')
axs[1].plot(xb,soc,color=C['deep'],lw=1.5,label='SOC')
axs[1].axhline(1200,color=C['gray'],ls='--',lw=.8); axs[1].axhline(10800,color=C['gray'],ls='--',lw=.8)
axs[1].set_xlim(0,24); axs[1].set_ylim(0,12000); axs[1].set_xticks([0,4,8,12,16,20,24]); axs[1].set_ylabel('储能电量 / kWh'); axs[1].set_xlabel('时刻 / h'); axs[1].legend(frameon=False,loc='upper right')
save(fig,'q1_dispatch')

# ---------- Q2 annual risk ----------
q2e=daily_emergency(RES/'result2.xlsx')
fig,axs=plt.subplots(2,2,figsize=(7.1,5.1),gridspec_kw={'hspace':.42,'wspace':.32})
for a,l in zip(axs.flat,['(a)','(b)','(c)','(d)']): style(a); panel(a,l)
axs[0,0].plot(q2e.index,q2e['紧急购电量']/1000,color=C['blue'],lw=.9)
axs[0,0].set_ylabel('日紧急购电量 / MWh'); axs[0,0].xaxis.set_major_locator(MonthLocator(interval=2)); axs[0,0].xaxis.set_major_formatter(DateFormatter('%m月'))
vals=np.sort(q2e['紧急购电量'].to_numpy(float)/1000); yy=np.arange(1,len(vals)+1)/len(vals)
axs[0,1].plot(vals,yy,color=C['rose']); axs[0,1].set_xlabel('日紧急购电量 / MWh'); axs[0,1].set_ylabel('经验累积概率'); axs[0,1].set_ylim(0,1.02)
axs[1,0].scatter(q2e['区间块数'],q2e['紧急购电量']/1000,s=11,alpha=.45,color=C['midblue'],edgecolors='none')
axs[1,0].set_xlabel('日紧急购电区间块数 / 个'); axs[1,0].set_ylabel('日紧急购电量 / MWh')
monthly=(q2e['紧急购电量']/1000).groupby(q2e.index.month).sum()
axs[1,1].bar(monthly.index,monthly.values,color=C['slate'],width=.72); axs[1,1].set_xlabel('月份'); axs[1,1].set_ylabel('月紧急购电量 / MWh'); axs[1,1].set_xticks(range(2,13,2))
save(fig,'q2_risk')

# ---------- Q3 ----------
d3,p0,df0 = None,None,None
dates3,p3,dfp=read_result_matrix(RES/'result3.xlsx','计划购电量')
dates3f,f3,dff=read_result_matrix(RES/'result3.xlsx','调整购电量')
delta=f3-p3
absday=np.nansum(np.abs(delta),axis=1)
rep_idx=int(np.nanargmax(absday)); rep_date=dates3[rep_idx]
q2daily=daily_emergency(RES/'result2.xlsx'); q3daily=daily_emergency(RES/'result3.xlsx')
fig=plt.figure(figsize=(7.1,6.15))
gs=fig.add_gridspec(2,2,height_ratios=[.9,1.1],hspace=.38,wspace=.33)
```

```
a=fig.add_subplot(gs[0,0]); style(a); panel(a,'(a)')
a.plot(x,p3[rep_idx],color=C['gray'],ls='--',label='0:00 原计划')
a.plot(x,f3[rep_idx],color=C['blue'],label='最终有效计划')
a.fill_between(x,p3[rep_idx],f3[rep_idx],color=C['lightblue'],alpha=.35)
a.set_xlim(0,24); a.set_xticks([0,6,12,18,24]); a.set_xlabel('时刻 / h'); a.set_ylabel('购电量 / (kWh/10 min)'); a.legend(frameon=False)
a.text(.02,.93,f'代表日：{rep_date:%m 月%d 日}',transform=a.transAxes,fontsize=7.2)
a=fig.add_subplot(gs[0,1]); style(a); panel(a,'(b)')
a.plot(dates3,absday/1000,color=C['orange'],lw=.85); a.set_ylabel('日绝对调整量 / MWh'); a.xaxis.set_major_locator(MonthLocator(interval=2)); a.xaxis.set_major_formatter(DateFormatter('%m月'))
a=fig.add_subplot(gs[1,0]); panel(a,'(c)')
finite=np.abs(delta[np.isfinite(delta)]); vmax=np.percentile(finite,98) if len(finite) else 1
cmap=mpl.colors.LinearSegmentedColormap.from_list('adj',[C['blue'],'white',C['rose']])
im=a.imshow(delta,aspect='auto',interpolation='nearest',cmap=cmap,vmin=-vmax,vmax=vmax,origin='upper')
a.set_xticks(np.arange(0,145,24)); a.set_xticklabels(['0','4','8','12','16','20','24']); a.set_xlabel('时刻 / h'); a.set_ylabel('月份')
pos=[]; lab=[]; seen=set()
for i,d in enumerate(dates3):
    if d.month not in seen: seen.add(d.month); pos.append(i); lab.append(f'{d.month} 月')
a.set_yticks(pos); a.set_yticklabels(lab); a.spines['top'].set_visible(False); a.spines['right'].set_visible(False)
cb=fig.colorbar(im,ax=a,pad=.02,fraction=.05); cb.set_label('调整量 / kWh')
a=fig.add_subplot(gs[1,1]); style(a); panel(a,'(d)')
pair=pd.DataFrame({'Q2':q2daily['紧急购电量']/1000,'Q3':q3daily['紧急购电量']/1000}).dropna(); m=max(pair.max())
a.scatter(pair['Q2'],pair['Q3'],s=10,alpha=.42,color=C['slate'],edgecolors='none'); a.plot([0,m],[0,m],ls='--',color=C['gray'],lw=.8)
a.set_xlabel('Q2 日紧急购电量 / MWh'); a.set_ylabel('Q3 日紧急购电量 / MWh'); a.set_xlim(0,m*1.03); a.set_ylim(0,m*1.03)
save(fig,'q3_update')

# ---------- Q4 ----------
dates42,q42,df42=read_result_matrix(RES/'result4-2.xlsx','计划购电量')
dates43,q43,df43=read_result_matrix(RES/'result4-3.xlsx','调整购电量')
rep_date=pd.Timestamp('2025-08-25')
i42=np.where(dates42==rep_date)[0][0]; i43=np.where(dates43==rep_date)[0][0]
price4=read_wide(DATA/'附件 4.xlsx','Sheet1',rep_date)
q4e2=daily_emergency(RES/'result4-2.xlsx'); q4e3=daily_emergency(RES/'result4-3.xlsx')
summary=[]
for name,path,sheet in [('Q2',RES/'result2.xlsx','计划购电量'),('Q3',RES/'result3.xlsx','调整购电量'),('Q4-2',RES/'result4-2.xlsx','计划购电量'),('Q4-3',RES/'result4-3.xlsx','调整购电量')]:
    df=pd.read_excel(path,sheet_name=sheet); cost=pd.to_numeric(df['全天购电费'],errors='coerce').sum()/1e4
    em=daily_emergency(path)['紧急购电量'].sum()/1000
    summary.append((name,cost,em))
sumdf=pd.DataFrame(summary,columns=['方案','正常购电费用','紧急购电量'])
fig=plt.figure(figsize=(7.1,5.55)); gs=fig.add_gridspec(2,2,hspace=.42,wspace=.34)
a=fig.add_subplot(gs[0,:]); style(a); panel(a,'(a)')
a.plot(x,q42[i42],color=C['midblue'],label='Q4-2 正常购电')
a.plot(x,q43[i43],color=C['rose'],label='Q4-3 最终正常购电')
a.set_xlim(0,24); a.set_xticks([0,4,8,12,16,20,24]); a.set_xlabel('时刻 / h'); a.set_ylabel('购电量 / (kWh/10 min)')
a2=a.twinx(); a2.plot(x,price4,color=C['deep'],ls='--',lw=1.05,label='事后真实动态电价'); a2.set_ylabel('电价 / (元 · kWh$^{-1}$)'); a2.spines['top'].set_visible(False)
h1,l1=a.get_legend_handles_labels(); h2,l2=a2.get_legend_handles_labels(); a.legend(h1+h2,l1+l2,frameon=False,ncol=3,loc='upper center')
a=fig.add_subplot(gs[1,0]); style(a); panel(a,'(b)')
xx=np.arange(4); a.bar(xx,sumdf['正常购电费用'],color=[C['midblue'],C['orange'],C['slate'],C['rose']],width=.65); a.set_xticks(xx); a.set_xticklabels(sumdf['方案']); a.set_ylabel('正常购电费用 / 万元')
for xi,v in zip(xx,sumdf['正常购电费用']): a.text(xi,v+8,f'{v:.1f}',ha='center',va='bottom',fontsize=6.8)
a=fig.add_subplot(gs[1,1]); style(a); panel(a,'(c)')
a.bar(xx,sumdf['紧急购电量'],color=[C['midblue'],C['orange'],C['slate'],C['rose']],width=.65); a.set_xticks(xx); a.set_xticklabels(sumdf['方案']); a.set_ylabel('紧急购电量 / MWh')
```

```
for xi,v in zip(xx,sumdf['紧急购电量']):
    a.text(xi,v+3,f'{v:.1f}',ha='center',va='bottom',fontsize=6.8)
save(fig,'q4_dynamic')

print('Figures rebuilt from final input/result files using', FONT)
```